合肥大學
HEFEI UNIVERSITY

Programming with Python

42. Klassen: Kapselung

Thomas Weise (汤卫思)
tweise@hfuu.edu.cn · https://thomasweise.github.io

School of Artificial Intelligence and Big Data
Hefei University
Hefei, Anhui, China

人工智能与大数据学院
合肥大学
中国安徽省合肥市

Version: 2026-11-05

Programming with Python



Dies ist ein Kurs über das Programmieren mit der Programmiersprache Python an der Universität Hefei (合肥大学).

Die Webseite mit dem Lehrmaterial dieses Kurses ist https://thomasweise.github.io/programmingWithPython (siehe auch den QR-Kode unten rechts). Dort können Sie das Kursbuch (in Englisch) und diese Slides finden. Das Repository mit den Beispielprogrammen in Python finden Sie unter https://github.com/thomasWeise/programmingWithPythonCode.



Outline

1. Einleitung
2. Beispiel
3. Zusammenfassung



Einleitung



Einleitung



- Oftmals wollen wir, dass unsere Objekte unveränderlich sind.

Einleitung



- Oftmals wollen wir, dass unsere Objekte unveränderlich sind.
- Das geht natürlich nicht immer.

Einleitung



- Oftmals wollen wir, dass unsere Objekte unveränderlich sind.
- Das geht natürlich nicht immer.
- Es gibt auch viele Situationen, wo wir wollen, dass sich die Attribute eines Objekts ändern.

Einleitung



- Oftmals wollen wir, dass unsere Objekte unveränderlich sind.
- Das geht natürlich nicht immer.
- Es gibt auch viele Situationen, wo wir wollen, dass sich die Attribute eines Objekts ändern.
- Die Frage ist dann *Wie sollte man den Zustand eines Objekts ändern?*

Einleitung



- Oftmals wollen wir, dass unsere Objekte unveränderlich sind.
- Das geht natürlich nicht immer.
- Es gibt auch viele Situationen, wo wir wollen, dass sich die Attribute eines Objekts ändern.
- Die Frage ist dann *Wie sollte man den Zustand eines Objekts ändern?*
- Die große Mehrheit unsere Klassen werden mehr als ein Attribut haben.

Einleitung



- Oftmals wollen wir, dass unsere Objekte unveränderlich sind.
- Das geht natürlich nicht immer.
- Es gibt auch viele Situationen, wo wir wollen, dass sich die Attribute eines Objekts ändern.
- Die Frage ist dann *Wie sollte man den Zustand eines Objekts ändern?*
- Die große Mehrheit unsere Klassen werden mehr als ein Attribut haben.
- Alle Attribute eines Objekts zusammen formen den Zustand des Objekts und bilden eine semantische Einheit.

Einleitung



- Oftmals wollen wir, dass unsere Objekte unveränderlich sind.
- Das geht natürlich nicht immer.
- Es gibt auch viele Situationen, wo wir wollen, dass sich die Attribute eines Objekts ändern.
- Die Frage ist dann *Wie sollte man den Zustand eines Objekts ändern?*
- Die große Mehrheit unsere Klassen werden mehr als ein Attribut haben.
- Alle Attribute eines Objekts zusammen formen den Zustand des Objekts und bilden eine semantische Einheit.
- Wenn ein Objekt mehrere Attribute hat, dann stehen diese Attribute wahrscheinlich miteinander in einer Beziehung und ihre Werte hängen irgendwie zusammen.

Einleitung



- Oftmals wollen wir, dass unsere Objekte unveränderlich sind.
- Das geht natürlich nicht immer.
- Es gibt auch viele Situationen, wo wir wollen, dass sich die Attribute eines Objekts ändern.
- Die Frage ist dann *Wie sollte man den Zustand eines Objekts ändern?*
- Die große Mehrheit unsere Klassen werden mehr als ein Attribut haben.
- Alle Attribute eines Objekts zusammen formen den Zustand des Objekts und bilden eine semantische Einheit.
- Wenn ein Objekt mehrere Attribute hat, dann stehen diese Attribute wahrscheinlich miteinander in einer Beziehung und ihre Werte hängen irgendwie zusammen.
- Wir wollen dann nicht, dass irgendjemand diese Werte beliebig veränden kann.

Einleitung



- Oftmals wollen wir, dass unsere Objekte unveränderlich sind.
- Das geht natürlich nicht immer.
- Es gibt auch viele Situationen, wo wir wollen, dass sich die Attribute eines Objekts ändern.
- Die Frage ist dann *Wie sollte man den Zustand eines Objekts ändern?*
- Die große Mehrheit unsere Klassen werden mehr als ein Attribut haben.
- Alle Attribute eines Objekts zusammen formen den Zustand des Objekts und bilden eine semantische Einheit.
- Wenn ein Objekt mehrere Attribute hat, dann stehen diese Attribute wahrscheinlich miteinander in einer Beziehung und ihre Werte hängen irgendwie zusammen.
- Wir wollen dann nicht, dass irgendjemand diese Werte beliebig veränden kann.
- Oftmals wollen wir veränderliche Objekte so entwickeln, dass auf die Werte ihrer veränderlichen Attribute nur über Method zugegriffen werden kann.

Einleitung



- Oftmals wollen wir, dass unsere Objekte unveränderlich sind.
- Das geht natürlich nicht immer.
- Es gibt auch viele Situationen, wo wir wollen, dass sich die Attribute eines Objekts ändern.
- Die Frage ist dann *Wie sollte man den Zustand eines Objekts ändern?*
- Die große Mehrheit unsere Klassen werden mehr als ein Attribut haben.
- Alle Attribute eines Objekts zusammen formen den Zustand des Objekts und bilden eine semantische Einheit.
- Wenn ein Objekt mehrere Attribute hat, dann stehen diese Attribute wahrscheinlich miteinander in einer Beziehung und ihre Werte hängen irgendwie zusammen.
- Wir wollen dann nicht, dass irgendjemand diese Werte beliebig veränden kann.
- Oftmals wollen wir veränderliche Objekte so entwickeln, dass auf die Werte ihrer veränderlichen Attribute nur über Method zugegriffen werden kann.
- Die Methoden und Attribute einer Klasse formen ja auch eine semantische Einheit.

Einleitung



- Das geht natürlich nicht immer.
- Es gibt auch viele Situationen, wo wir wollen, dass sich die Attribute eines Objekts ändern.
- Die Frage ist dann *Wie sollte man den Zustand eines Objekts ändern?*
- Die große Mehrheit unsere Klassen werden mehr als ein Attribut haben.
- Alle Attribute eines Objekts zusammen formen den Zustand des Objekts und bilden eine semantische Einheit.
- Wenn ein Objekt mehrere Attribute hat, dann stehen diese Attribute wahrscheinlich miteinander in einer Beziehung und ihre Werte hängen irgendwie zusammen.
- Wir wollen dann nicht, dass irgendjemand diese Werte beliebig veränden kann.
- Oftmals wollen wir veränderliche Objekte so entwickeln, dass auf die Werte ihrer veränderlichen Attribute nur über Method zugegriffen werden kann.
- Die Methoden und Attribute einer Klasse formen ja auch eine semantische Einheit.
- Die Methoden werden von den selben Programmierern geschrieben, die die Attribute designed haben.

Einleitung



- Es gibt auch viele Situationen, wo wir wollen, dass sich die Attribute eines Objekts ändern.
- Die Frage ist dann *Wie sollte man den Zustand eines Objekts ändern?*
- Die große Mehrheit unsere Klassen werden mehr als ein Attribut haben.
- Alle Attribute eines Objekts zusammen formen den Zustand des Objekts und bilden eine semantische Einheit.
- Wenn ein Objekt mehrere Attribute hat, dann stehen diese Attribute wahrscheinlich miteinander in einer Beziehung und ihre Werte hängen irgendwie zusammen.
- Wir wollen dann nicht, dass irgendjemand diese Werte beliebig verändern kann.
- Oftmals wollen wir veränderliche Objekte so entwickeln, dass auf die Werte ihrer veränderlichen Attribute nur über Method zugegriffen werden kann.
- Die Methoden und Attribute einer Klasse formen ja auch eine semantische Einheit.
- Die Methoden werden von den selben Programmierern geschrieben, die die Attribute designed haben.
- Diese Programmierer wissen genau, wie die Attribute in einer vernünftigen und konsistenten Art verändert werden können.

Beispiel



Einleitung zum Beispiel

- Wir schauen uns nun ein Szenario an.

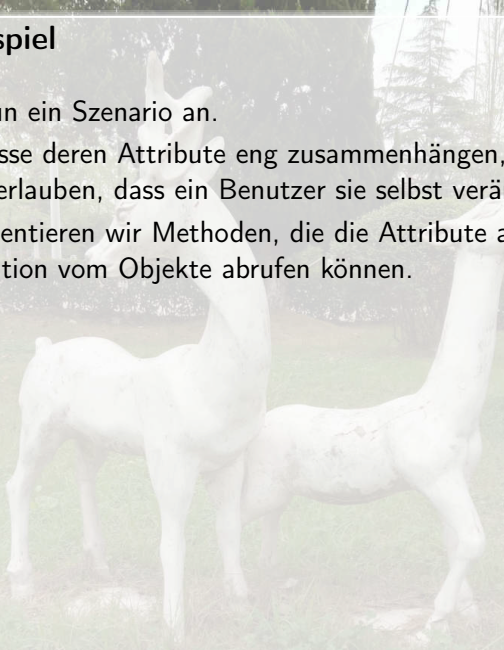


Einleitung zum Beispiel



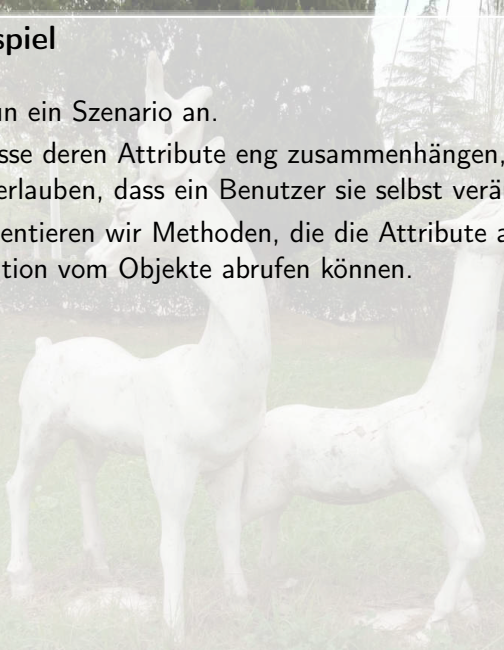
- Wir schauen uns nun ein Szenario an.
- Wir haben eine Klasse deren Attribute eng zusammenhängen, so dass es gar keinen Sinn ergeben würde, zu erlauben, dass ein Benutzer sie selbst verändern könnte.

Einleitung zum Beispiel



- Wir schauen uns nun ein Szenario an.
- Wir haben eine Klasse deren Attribute eng zusammenhängen, so dass es gar keinen Sinn ergeben würde, zu erlauben, dass ein Benutzer sie selbst verändern könnte.
- Stattdessen implementieren wir Methoden, die die Attribute auf eine konsistente Art ändern und Information vom Objekte abrufen können.

Einleitung zum Beispiel



- Wir schauen uns nun ein Szenario an.
- Wir haben eine Klasse deren Attribute eng zusammenhängen, so dass es gar keinen Sinn ergeben würde, zu erlauben, dass ein Benutzer sie selbst verändern könnte.
- Stattdessen implementieren wir Methoden, die die Attribute auf eine konsistente Art ändern und Information vom Objekte abrufen können.

Einleitung zum Beispiel



- Wir schauen uns nun ein Szenario an.
- Wir haben eine Klasse deren Attribute eng zusammenhängen, so dass es gar keinen Sinn ergeben würde, zu erlauben, dass ein Benutzer sie selbst verändern könnte.
- Stattdessen implementieren wir Methoden, die die Attribute auf eine konsistente Art ändern und Information vom Objekte abrufen können.
- Natürlich nehmen wir als Beispiel wieder einen mathematischen Algorithmus.

Einleitung zum Beispiel



- Wir schauen uns nun ein Szenario an.
- Wir haben eine Klasse deren Attribute eng zusammenhängen, so dass es gar keinen Sinn ergeben würde, zu erlauben, dass ein Benutzer sie selbst verändern könnte.
- Stattdessen implementieren wir Methoden, die die Attribute auf eine konsistente Art ändern und Information vom Objekte abrufen können.
- Natürlich nehmen wir als Beispiel wieder einen mathematischen Algorithmus.
- Wir haben ja schon einige interessante mathematische Algorithmen implementiert.

Einleitung zum Beispiel



- Wir schauen uns nun ein Szenario an.
- Wir haben eine Klasse deren Attribute eng zusammenhängen, so dass es gar keinen Sinn ergeben würde, zu erlauben, dass ein Benutzer sie selbst verändern könnte.
- Stattdessen implementieren wir Methoden, die die Attribute auf eine konsistente Art ändern und Information vom Objekte abrufen können.
- Natürlich nehmen wir als Beispiel wieder einen mathematischen Algorithmus.
- Wir haben ja schon einige interessante mathematische Algorithmen implementiertz. B.:
  1. die Methode von LIU Hui (刘徽) zum annähern von π in Einheit 13.

Einleitung zum Beispiel



- Wir schauen uns nun ein Szenario an.
- Wir haben eine Klasse deren Attribute eng zusammenhängen, so dass es gar keinen Sinn ergeben würde, zu erlauben, dass ein Benutzer sie selbst verändern könnte.
- Stattdessen implementieren wir Methoden, die die Attribute auf eine konsistente Art ändern und Information vom Objekte abrufen können.
- Natürlich nehmen wir als Beispiel wieder einen mathematischen Algorithmus.
- Wir haben ja schon einige interessante mathematische Algorithmen implementiertz. B.:
  1. die Methode von LIU Hui (刘徽) zum annähern von π in Einheit 13,
  2. Heron's Methode zum annähern der Quadratwurzel in Einheit 25.

Einleitung zum Beispiel



- Wir schauen uns nun ein Szenario an.
- Wir haben eine Klasse deren Attribute eng zusammenhängen, so dass es gar keinen Sinn ergeben würde, zu erlauben, dass ein Benutzer sie selbst verändern könnte.
- Stattdessen implementieren wir Methoden, die die Attribute auf eine konsistente Art ändern und Information vom Objekte abrufen können.
- Natürlich nehmen wir als Beispiel wieder einen mathematischen Algorithmus.
- Wir haben ja schon einige interessante mathematische Algorithmen implementiertz. B.:
  1. die Methode von LIU Hui (刘徽) zum annähern von π in Einheit 13,
  2. Heron's Methode zum annähern der Quadratwurzel in Einheit 25 und
  3. Euclids Algorithmus zum berechnen des größten gemeinsamen Teilers in Einheit 26.

Einleitung zum Beispiel



- Wir schauen uns nun ein Szenario an.
- Wir haben eine Klasse deren Attribute eng zusammenhängen, so dass es gar keinen Sinn ergeben würde, zu erlauben, dass ein Benutzer sie selbst verändern könnte.
- Stattdessen implementieren wir Methoden, die die Attribute auf eine konsistente Art ändern und Information vom Objekte abrufen können.
- Natürlich nehmen wir als Beispiel wieder einen mathematischen Algorithmus.
- Wir haben ja schon einige interessante mathematische Algorithmen implementiertz. B.:
  1. die Methode von LIU Hui (刘徽) zum annähern von π in Einheit 13,
  2. Heron's Methode zum annähern der Quadratwurzel in Einheit 25 und
  3. Euclids Algorithmus zum berechnen des größten gemeinsamen Teilers in Einheit 26.
- Diese Algorithmen kennen Sie wahrscheinlich aus der Schule.

Einleitung zum Beispiel



- Wir schauen uns nun ein Szenario an.
- Wir haben eine Klasse deren Attribute eng zusammenhängen, so dass es gar keinen Sinn ergeben würde, zu erlauben, dass ein Benutzer sie selbst verändern könnte.
- Stattdessen implementieren wir Methoden, die die Attribute auf eine konsistente Art ändern und Information vom Objekte abrufen können.
- Natürlich nehmen wir als Beispiel wieder einen mathematischen Algorithmus.
- Wir haben ja schon einige interessante mathematische Algorithmen implementiertz. B.:
  1. die Methode von LIU Hui (刘徽) zum annähern von π in Einheit 13,
  2. Heron's Methode zum annähern der Quadratwurzel in Einheit 25 und
  3. Euclids Algorithmus zum berechnen des größten gemeinsamen Teilers in Einheit 26.
- Diese Algorithmen kennen Sie wahrscheinlich aus der Schule.
- Implementieren wir nun einen Algorithmus, von dem Sie wahrscheinlich noch *nie* gehört haben.

Einleitung zum Beispiel



- Wir schauen uns nun ein Szenario an.
- Wir haben eine Klasse deren Attribute eng zusammenhängen, so dass es gar keinen Sinn ergeben würde, zu erlauben, dass ein Benutzer sie selbst verändern könnte.
- Stattdessen implementieren wir Methoden, die die Attribute auf eine konsistente Art ändern und Information vom Objekte abrufen können.
- Natürlich nehmen wir als Beispiel wieder einen mathematischen Algorithmus.
- Wir haben ja schon einige interessante mathematische Algorithmen implementiertz. B.:
  1. die Methode von LIU Hui (刘徽) zum annähern von π in Einheit 13,
  2. Heron's Methode zum annähern der Quadratwurzel in Einheit 25 und
  3. Euclids Algorithmus zum berechnen des größten gemeinsamen Teilers in Einheit 26.
- Diese Algorithmen kennen Sie wahrscheinlich aus der Schule.
- Implementieren wir nun einen Algorithmus, von dem Sie wahrscheinlich noch *nie* gehört haben.
- Eine praktische Methode, um die Beschränkungen des Datentyps `float` zu umgehen, die wir vor langer Zeit in Einheit 8 diskutiert haben.

Grenzen des Datentyps `float`

- Wir wissen, dass der Datentyp `float` Zahlen auf 15 bis 16 Ziffern genau darstellen kann.

```
Python 3.14.10 (main, Oct 27 2026, 09:35:01) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
# Python's float supports 15 to 16 digits of precision.
>>> 1e15 + 1          # This works and will give us 1_000_000_000_000_001
1000000000000001.0
>>> 1e16 + 1          # This gives us 1e16, since we would need 17 digits
1e+16

# While 1e16 + 1 cannot be represented exactly with a `float`, one
# may wonder what happens if the computation produces results that
# can be represented exactly ... but has intermediate steps that
# would exceed the range that can be covered by a `float`.
>>> 1e16 + 1 - 1e16  # Since 1e16 + 1 == 1e16, this does yield 0.
0.0

# The problems with the range of `float` can also occur if we have
# two very large numbers ... as long as one is much larger than the
# other.
>>> 1e18 + 1e36       # 1_000_000_000_000_000_001 * 1e18 needs 19 digits
1e+36

# We here can again observe the problem with intermediate results.
# The final result may fit well into a `float`, but if precision is
# lost in intermediate steps, the final result can be off quite a
# bit.
>>> 1e18 + 1e36 - 1e36  # Since 1e18 + 1e36 == 1e36, this gives us 0.
0.0
>>> 1e18 + 1 + 1e36 - 1e36 - 1e18  # The exact result would be 1, but...
-1e+18
```

Grenzen des Datentyps `float`

- Wir wissen, dass der Datentyp `float` Zahlen auf 15 bis 16 Ziffern genau darstellen kann.
- Wenn wir `1` zu 10^{15} = `1e15` addieren, dann ist das korrekte Ergebnis `1000000000000001.0`.

```
Python 3.14.10 (main, Oct 27 2026, 09:35:01) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
# Python's float supports 15 to 16 digits of precision.
>>> 1e15 + 1          # This works and will give us 1_000_000_000_000_001
1000000000000001.0
>>> 1e16 + 1          # This gives us 1e16, since we would need 17 digits
1e+16

# While 1e16 + 1 cannot be represented exactly with a `float`, one
# may wonder what happens if the computation produces results that
# can be represented exactly ... but has intermediate steps that
# would exceed the range that can be covered by a `float`.
>>> 1e16 + 1 - 1e16  # Since 1e16 + 1 == 1e16, this does yield 0.
0.0

# The problems with the range of `float` can also occur if we have
# two very large numbers ... as long as one is much larger than the
# other.
>>> 1e18 + 1e36       # 1_000_000_000_000_000_001 * 1e18 needs 19 digits
1e+36

# We here can again observe the problem with intermediate results.
# The final result may fit well into a `float`, but if precision is
# lost in intermediate steps, the final result can be off quite a
# bit.
>>> 1e18 + 1e36 - 1e36  # Since 1e18 + 1e36 == 1e36, this gives us 0.
0.0
>>> 1e18 + 1 + 1e36 - 1e36 - 1e18  # The exact result would be 1, but...
-1e+18
```

Grenzen des Datentyps `float`

- Wir wissen, dass der Datentyp `float` Zahlen auf 15 bis 16 Ziffern genau darstellen kann.
- Wenn wir `1` zu 10^{15} = `1e15` addieren, dann ist das korrekte Ergebnis `1000000000000001.0`.
- Wie Sie selbst zählen können, finden wir 16 Ziffern vor dem Dezimalpunkt, zwei `1`en und 14 `0`en.

```
Python 3.14.10 (main, Oct 27 2026, 09:35:01) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
# Python's float supports 15 to 16 digits of precision.
>>> 1e15 + 1          # This works and will give us 1_000_000_000_000_001
1000000000000001.0
>>> 1e16 + 1          # This gives us 1e16, since we would need 17 digits
1e+16

# While 1e16 + 1 cannot be represented exactly with a `float`, one
# may wonder what happens if the computation produces results that
# can be represented exactly ... but has intermediate steps that
# would exceed the range that can be covered by a `float`.
>>> 1e16 + 1 - 1e16  # Since 1e16 + 1 == 1e16, this does yield 0.
0.0

# The problems with the range of `float` can also occur if we have
# two very large numbers ... as long as one is much larger than the
# other.
>>> 1e18 + 1e36       # 1_000_000_000_000_000_001 * 1e18 needs 19 digits
1e+36

# We here can again observe the problem with intermediate results.
# The final result may fit well into a `float`, but if precision is
# lost in intermediate steps, the final result can be off quite a
# bit.
>>> 1e18 + 1e36 - 1e36  # Since 1e18 + 1e36 == 1e36, this gives us 0.
0.0
>>> 1e18 + 1 + 1e36 - 1e36 - 1e18  # The exact result would be 1, but...
-1e+18
```

Grenzen des Datentyps `float`

- Wir wissen, dass der Datentyp `float` Zahlen auf 15 bis 16 Ziffern genau darstellen kann.
- Wenn wir `1` zu 10^{15} = `1e15` addieren, dann ist das korrekte Ergebnis `1000000000000001.0`.
- Wie Sie selbst zählen können, finden wir 16 Ziffern vor dem Dezimalpunkt, zwei `1`en und 14 `0`en.
- Daher würde die Addition von `1` zu 10^{16} = `1e16` 17 Ziffern erfordern.

```
Python 3.14.10 (main, Oct 27 2026, 09:35:01) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
# Python's float supports 15 to 16 digits of precision.
>>> 1e15 + 1          # This works and will give us 1_000_000_000_000_001
1000000000000001.0
>>> 1e16 + 1          # This gives us 1e16, since we would need 17 digits
1e+16

# While 1e16 + 1 cannot be represented exactly with a `float`, one
# may wonder what happens if the computation produces results that
# can be represented exactly ... but has intermediate steps that
# would exceed the range that can be covered by a `float`.
>>> 1e16 + 1 - 1e16  # Since 1e16 + 1 == 1e16, this does yield 0.
0.0

# The problems with the range of `float` can also occur if we have
# two very large numbers ... as long as one is much larger than the
# other.
>>> 1e18 + 1e36       # 1_000_000_000_000_000_001 * 1e18 needs 19 digits
1e+36

# We here can again observe the problem with intermediate results.
# The final result may fit well into a `float`, but if precision is
# lost in intermediate steps, the final result can be off quite a
# bit.
>>> 1e18 + 1e36 - 1e36  # Since 1e18 + 1e36 == 1e36, this gives us 0.
0.0
>>> 1e18 + 1 + 1e36 - 1e36 - 1e18  # The exact result would be 1, but...
-1e+18
```

Grenzen des Datentyps `float`

- Wir wissen, dass der Datentyp `float` Zahlen auf 15 bis 16 Ziffern genau darstellen kann.
- Wenn wir `1` zu 10^{15} = `1e15` addieren, dann ist das korrekte Ergebnis `1000000000000001.0`.
- Wie Sie selbst zählen können, finden wir 16 Ziffern vor dem Dezimalpunkt, zwei `1`en und 14 `0`en.
- Daher würde die Addition von `1` zu 10^{16} = `1e16` 17 Ziffern erfordern.
- Es überschreitet die Kapazität des Datentyps `float`s.

```
Python 3.14.10 (main, Oct 27 2026, 09:35:01) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
# Python's float supports 15 to 16 digits of precision.
>>> 1e15 + 1          # This works and will give us 1_000_000_000_000_001
1000000000000001.0
>>> 1e16 + 1          # This gives us 1e16, since we would need 17 digits
1e+16

# While 1e16 + 1 cannot be represented exactly with a `float`, one
# may wonder what happens if the computation produces results that
# can be represented exactly ... but has intermediate steps that
# would exceed the range that can be covered by a `float`.
>>> 1e16 + 1 - 1e16  # Since 1e16 + 1 == 1e16, this does yield 0.
0.0

# The problems with the range of `float` can also occur if we have
# two very large numbers ... as long as one is much larger than the
# other.
>>> 1e18 + 1e36       # 1_000_000_000_000_000_001 * 1e18 needs 19 digits
1e+36

# We here can again observe the problem with intermediate results.
# The final result may fit well into a `float`, but if precision is
# lost in intermediate steps, the final result can be off quite a
# bit.
>>> 1e18 + 1e36 - 1e36  # Since 1e18 + 1e36 == 1e36, this gives us 0.
0.0
>>> 1e18 + 1 + 1e36 - 1e36 - 1e18  # The exact result would be 1, but...
-1e+18
```

Grenzen des Datentyps `float`

- Wenn wir `1` zu 10^{15} = `1e15` addieren, dann ist das korrekte Ergebnis `1000000000000001.0`.
- Wie Sie selbst zählen können, finden wir 16 Ziffern vor dem Dezimalpunkt, zwei `1`en und 14 `0`en.
- Daher würde die Addition von `1` zu 10^{16} = `1e16` 17 Ziffern erfordern.
- Es überschreitet die Kapazität des Datentyps `float`s.
- Daher geht die niedrigstwertige Ziffer „verloren“.

```
Python 3.14.10 (main, Oct 27 2026, 09:35:01) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
# Python's float supports 15 to 16 digits of precision.
>>> 1e15 + 1          # This works and will give us 1_000_000_000_000_001
1000000000000001.0
>>> 1e16 + 1          # This gives us 1e16, since we would need 17 digits
1e+16

# While 1e16 + 1 cannot be represented exactly with a `float`, one
# may wonder what happens if the computation produces results that
# can be represented exactly ... but has intermediate steps that
# would exceed the range that can be covered by a `float`.
>>> 1e16 + 1 - 1e16  # Since 1e16 + 1 == 1e16, this does yield 0.
0.0

# The problems with the range of `float` can also occur if we have
# two very large numbers ... as long as one is much larger than the
# other.
>>> 1e18 + 1e36       # 1_000_000_000_000_000_001 * 1e18 needs 19 digits
1e+36

# We here can again observe the problem with intermediate results.
# The final result may fit well into a `float`, but if precision is
# lost in intermediate steps, the final result can be off quite a
# bit.
>>> 1e18 + 1e36 - 1e36  # Since 1e18 + 1e36 == 1e36, this gives us 0.
0.0
>>> 1e18 + 1 + 1e36 - 1e36 - 1e18  # The exact result would be 1, but...
-1e+18
```

Grenzen des Datentyps `float`

- Wie Sie selbst zählen können, finden wir 16 Ziffern vor dem Dezimalpunkt, zwei `1`en und 14 `0`en.
- Daher würde die Addition von `1` zu 10^{16} = `1e16` 17 Ziffern erfordern.
- Es überschreitet die Kapazität des Datentyps `floats`.
- Daher geht die niedrigstwertige Ziffer „verloren“.
- Das Ergebnis von `1e16 + 1`, berechnet mit `floats`, ist immer noch `1e16`.

```
Python 3.14.10 (main, Oct 27 2026, 09:35:01) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
# Python's float supports 15 to 16 digits of precision.
>>> 1e15 + 1          # This works and will give us 1_000_000_000_000_001
1000000000000001.0
>>> 1e16 + 1          # This gives us 1e16, since we would need 17 digits
1e+16

# While 1e16 + 1 cannot be represented exactly with a `float`, one
# may wonder what happens if the computation produces results that
# can be represented exactly ... but has intermediate steps that
# would exceed the range that can be covered by a `float`.
>>> 1e16 + 1 - 1e16  # Since 1e16 + 1 == 1e16, this does yield 0.
0.0

# The problems with the range of `float` can also occur if we have
# two very large numbers ... as long as one is much larger than the
# other.
>>> 1e18 + 1e36       # 1_000_000_000_000_000_001 * 1e18 needs 19 digits
1e+36

# We here can again observe the problem with intermediate results.
# The final result may fit well into a `float`, but if precision is
# lost in intermediate steps, the final result can be off quite a
# bit.
>>> 1e18 + 1e36 - 1e36  # Since 1e18 + 1e36 == 1e36, this gives us 0.
0.0
>>> 1e18 + 1 + 1e36 - 1e36 - 1e18  # The exact result would be 1, but...
-1e+18
```

Grenzen des Datentyps `float`

- Daher würde die Addition von `1` zu 10^{16} = `1e16` 17 Ziffern erfordern.
- Es überschreitet die Kapazität des Datentyps `floats`.
- Daher geht die niedrigstwertige Ziffer „verloren“.
- Das Ergebnis von `1e16 + 1`, berechnet mit `floats`, ist immer noch `1e16`.
- Es ist nicht möglich, die Zahl $1 + 10^{16}$ exakt mit den 64 Bit Double Precision Floating Point Numbers[115,145,150] die Python bietet, darzustellen.

```
Python 3.14.10 (main, Oct 27 2026, 09:35:01) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
# Python's float supports 15 to 16 digits of precision.
>>> 1e15 + 1          # This works and will give us 1_000_000_000_000_001
1000000000000001.0
>>> 1e16 + 1          # This gives us 1e16, since we would need 17 digits
1e+16

# While 1e16 + 1 cannot be represented exactly with a `float`, one
# may wonder what happens if the computation produces results that
# can be represented exactly ... but has intermediate steps that
# would exceed the range that can be covered by a `float`.
>>> 1e16 + 1 - 1e16  # Since 1e16 + 1 == 1e16, this does yield 0.
0.0

# The problems with the range of `float` can also occur if we have
# two very large numbers ... as long as one is much larger than the
# other.
>>> 1e18 + 1e36       # 1_000_000_000_000_000_001 * 1e18 needs 19 digits
1e+36

# We here can again observe the problem with intermediate results.
# The final result may fit well into a `float`, but if precision is
# lost in intermediate steps, the final result can be off quite a
# bit.
>>> 1e18 + 1e36 - 1e36  # Since 1e18 + 1e36 == 1e36, this gives us 0.
0.0
>>> 1e18 + 1 + 1e36 - 1e36 - 1e18  # The exact result would be 1, but...
-1e+18
```

Grenzen des Datentyps `float`

- Es überschreitet die Kapazität des Datentyps `floats`.
- Daher geht die niedrigstwertige Ziffer „verloren“.
- Das Ergebnis von `1e16 + 1`, berechnet mit `floats`, ist immer noch `1e16`.
- Es ist nicht möglich, die Zahl $1 + 10^{16}$ exakt mit den 64 Bit Double Precision Floating Point Numbers[115,145,150] die Python bietet, darzustellen.
- Normalerweise ist das auch OK.

```
Python 3.14.10 (main, Oct 27 2026, 09:35:01) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
# Python's float supports 15 to 16 digits of precision.
>>> 1e15 + 1          # This works and will give us 1_000_000_000_000_001
1000000000000001.0
>>> 1e16 + 1          # This gives us 1e16, since we would need 17 digits
1e+16

# While 1e16 + 1 cannot be represented exactly with a `float`, one
# may wonder what happens if the computation produces results that
# can be represented exactly ... but has intermediate steps that
# would exceed the range that can be covered by a `float`.
>>> 1e16 + 1 - 1e16  # Since 1e16 + 1 == 1e16, this does yield 0.
0.0

# The problems with the range of `float` can also occur if we have
# two very large numbers ... as long as one is much larger than the
# other.
>>> 1e18 + 1e36       # 1_000_000_000_000_000_001 * 1e18 needs 19 digits
1e+36

# We here can again observe the problem with intermediate results.
# The final result may fit well into a `float`, but if precision is
# lost in intermediate steps, the final result can be off quite a
# bit.
>>> 1e18 + 1e36 - 1e36  # Since 1e18 + 1e36 == 1e36, this gives us 0.
0.0
>>> 1e18 + 1 + 1e36 - 1e36 - 1e18  # The exact result would be 1, but...
-1e+18
```

Grenzen des Datentyps `float`

- Daher geht die niedrigstwertige Ziffer „verloren“.
- Das Ergebnis von `1e16 + 1`, berechnet mit `floats`, ist immer noch `1e16`.
- Es ist nicht möglich, die Zahl $1 + 10^{16}$ exakt mit den 64 Bit Double Precision Floating Point Numbers[115,145,150] die Python bietet, darzustellen.
- Normalerweise ist das auch OK.
- Es gibt sehr wenige Anwendungen, bei denen wir wirklich mehr als 15 Ziffern Genauigkeit brauchen.

```
Python 3.14.10 (main, Oct 27 2026, 09:35:01) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
# Python's float supports 15 to 16 digits of precision.
>>> 1e15 + 1          # This works and will give us 1_000_000_000_000_001
1000000000000001.0
>>> 1e16 + 1          # This gives us 1e16, since we would need 17 digits
1e+16

# While 1e16 + 1 cannot be represented exactly with a `float`, one
# may wonder what happens if the computation produces results that
# can be represented exactly ... but has intermediate steps that
# would exceed the range that can be covered by a `float`.
>>> 1e16 + 1 - 1e16  # Since 1e16 + 1 == 1e16, this does yield 0.
0.0

# The problems with the range of `float` can also occur if we have
# two very large numbers ... as long as one is much larger than the
# other.
>>> 1e18 + 1e36       # 1_000_000_000_000_000_001 * 1e18 needs 19 digits
1e+36

# We here can again observe the problem with intermediate results.
# The final result may fit well into a `float`, but if precision is
# lost in intermediate steps, the final result can be off quite a
# bit.
>>> 1e18 + 1e36 - 1e36  # Since 1e18 + 1e36 == 1e36, this gives us 0.
0.0
>>> 1e18 + 1 + 1e36 - 1e36 - 1e18  # The exact result would be 1, but...
-1e+18
```

Grenzen des Datentyps `float`

- Das Ergebnis von `1e16 + 1`, berechnet mit `floats`, ist immer noch `1e16`.
- Es ist nicht möglich, die Zahl $1 + 10^{16}$ exakt mit den 64 Bit Double Precision Floating Point Numbers[115,145,150] die Python bietet, darzustellen.
- Normalerweise ist das auch OK.
- Es gibt sehr wenige Anwendungen, bei denen wir wirklich mehr als 15 Ziffern Genauigkeit brauchen.
- Machen wir mit dem Beispiel weiter.

```
Python 3.14.10 (main, Oct 27 2026, 09:35:01) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
# Python's float supports 15 to 16 digits of precision.
>>> 1e15 + 1          # This works and will give us 1_000_000_000_000_001
1000000000000001.0
>>> 1e16 + 1          # This gives us 1e16, since we would need 17 digits
1e+16

# While 1e16 + 1 cannot be represented exactly with a `float`, one
# may wonder what happens if the computation produces results that
# can be represented exactly ... but has intermediate steps that
# would exceed the range that can be covered by a `float`.
>>> 1e16 + 1 - 1e16  # Since 1e16 + 1 == 1e16, this does yield 0.
0.0

# The problems with the range of `float` can also occur if we have
# two very large numbers ... as long as one is much larger than the
# other.
>>> 1e18 + 1e36       # 1_000_000_000_000_000_001 * 1e18 needs 19 digits
1e+36

# We here can again observe the problem with intermediate results.
# The final result may fit well into a `float`, but if precision is
# lost in intermediate steps, the final result can be off quite a
# bit.
>>> 1e18 + 1e36 - 1e36  # Since 1e18 + 1e36 == 1e36, this gives us 0.
0.0
>>> 1e18 + 1 + 1e36 - 1e36 - 1e18  # The exact result would be 1, but...
-1e+18
```

Grenzen des Datentyps `float`

- Es ist nicht möglich, die Zahl $1 + 10^{16}$ exakt mit den 64 Bit Double Precision Floating Point Numbers[115,145,150] die Python bietet, darzustellen.
- Normalerweise ist das auch OK.
- Es gibt sehr wenige Anwendungen, bei denen wir wirklich mehr als 15 Ziffern Genauigkeit brauchen.
- Machen wir mit dem Beispiel weiter.
- Was passiert, wenn wir $10^{16} + 1$ berechnen, und dann 10^{16} von der Summe abziehen?

```
Python 3.14.10 (main, Oct 27 2026, 09:35:01) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
# Python's float supports 15 to 16 digits of precision.
>>> 1e15 + 1          # This works and will give us 1_000_000_000_000_001
1000000000000001.0
>>> 1e16 + 1          # This gives us 1e16, since we would need 17 digits
1e+16

# While 1e16 + 1 cannot be represented exactly with a `float`, one
# may wonder what happens if the computation produces results that
# can be represented exactly ... but has intermediate steps that
# would exceed the range that can be covered by a `float`.
>>> 1e16 + 1 - 1e16  # Since 1e16 + 1 == 1e16, this does yield 0.
0.0

# The problems with the range of `float` can also occur if we have
# two very large numbers ... as long as one is much larger than the
# other.
>>> 1e18 + 1e36       # 1_000_000_000_000_000_001 * 1e18 needs 19 digits
1e+36

# We here can again observe the problem with intermediate results.
# The final result may fit well into a `float`, but if precision is
# lost in intermediate steps, the final result can be off quite a
# bit.
>>> 1e18 + 1e36 - 1e36  # Since 1e18 + 1e36 == 1e36, this gives us 0.
0.0
>>> 1e18 + 1 + 1e36 - 1e36 - 1e18  # The exact result would be 1, but...
-1e+18
```

Grenzen des Datentyps `float`

- Normalerweise ist das auch OK.
- Es gibt sehr wenige Anwendungen, bei denen wir wirklich mehr als 15 Ziffern Genauigkeit brauchen.
- Machen wir mit dem Beispiel weiter.
- Was passiert, wenn wir $10^{16} + 1$ berechnen, und dann 10^{16} von der Summe abziehen?
- Offensichtlich, in einer idealen Welt, wäre das Ergbenis `1.0`.

```
Python 3.14.10 (main, Oct 27 2026, 09:35:01) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
# Python's float supports 15 to 16 digits of precision.
>>> 1e15 + 1          # This works and will give us 1_000_000_000_000_001
1000000000000001.0
>>> 1e16 + 1          # This gives us 1e16, since we would need 17 digits
1e+16

# While 1e16 + 1 cannot be represented exactly with a `float`, one
# may wonder what happens if the computation produces results that
# can be represented exactly ... but has intermediate steps that
# would exceed the range that can be covered by a `float`.
>>> 1e16 + 1 - 1e16  # Since 1e16 + 1 == 1e16, this does yield 0.
0.0

# The problems with the range of `float` can also occur if we have
# two very large numbers ... as long as one is much larger than the
# other.
>>> 1e18 + 1e36       # 1_000_000_000_000_000_001 * 1e18 needs 19 digits
1e+36

# We here can again observe the problem with intermediate results.
# The final result may fit well into a `float`, but if precision is
# lost in intermediate steps, the final result can be off quite a
# bit.
>>> 1e18 + 1e36 - 1e36  # Since 1e18 + 1e36 == 1e36, this gives us 0.
0.0
>>> 1e18 + 1 + 1e36 - 1e36 - 1e18  # The exact result would be 1, but...
-1e+18
```

Grenzen des Datentyps `float`

- Es gibt sehr wenige Anwendungen, bei denen wir wirklich mehr als 15 Ziffern Genauigkeit brauchen.
- Machen wir mit dem Beispiel weiter.
- Was passiert, wenn wir $10^{16} + 1$ berechnen, und dann 10^{16} von der Summe abziehen?
- Offensichtlich, in einer idealen Welt, wäre das Ergbenis `1.0`.
- Nun wäre `1e16 + 1` ja `10_000_000_000_000_001.0`, was nicht in einen `float` passt.

```
Python 3.14.10 (main, Oct 27 2026, 09:35:01) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
# Python's float supports 15 to 16 digits of precision.
>>> 1e15 + 1          # This works and will give us 1_000_000_000_000_001
1000000000000001.0
>>> 1e16 + 1          # This gives us 1e16, since we would need 17 digits
1e+16

# While 1e16 + 1 cannot be represented exactly with a `float`, one
# may wonder what happens if the computation produces results that
# can be represented exactly ... but has intermediate steps that
# would exceed the range that can be covered by a `float`.
>>> 1e16 + 1 - 1e16  # Since 1e16 + 1 == 1e16, this does yield 0.
0.0

# The problems with the range of `float` can also occur if we have
# two very large numbers ... as long as one is much larger than the
# other.
>>> 1e18 + 1e36       # 1_000_000_000_000_000_001 * 1e18 needs 19 digits
1e+36

# We here can again observe the problem with intermediate results.
# The final result may fit well into a `float`, but if precision is
# lost in intermediate steps, the final result can be off quite a
# bit.
>>> 1e18 + 1e36 - 1e36  # Since 1e18 + 1e36 == 1e36, this gives us 0.
0.0
>>> 1e18 + 1 + 1e36 - 1e36 - 1e18  # The exact result would be 1, but...
-1e+18
```

Grenzen des Datentyps `float`

- Machen wir mit dem Beispiel weiter.
- Was passiert, wenn wir $10^{16} + 1$ berechnen, und dann 10^{16} von der Summe abziehen?
- Offensichtlich, in einer idealen Welt, wäre das Ergbenis `1.0`.
- Nun wäre `1e16 + 1` ja `10_000_000_000_000_001.0`, was nicht in einen `float` passt.
- `1.0` ist aber eine Zahl, die wir sehr wohl genau als `float` darstellen können.

```
Python 3.14.10 (main, Oct 27 2026, 09:35:01) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
# Python's float supports 15 to 16 digits of precision.
>>> 1e15 + 1          # This works and will give us 1_000_000_000_000_001
1000000000000001.0
>>> 1e16 + 1          # This gives us 1e16, since we would need 17 digits
1e+16

# While 1e16 + 1 cannot be represented exactly with a `float`, one
# may wonder what happens if the computation produces results that
# can be represented exactly ... but has intermediate steps that
# would exceed the range that can be covered by a `float`.
>>> 1e16 + 1 - 1e16  # Since 1e16 + 1 == 1e16, this does yield 0.
0.0

# The problems with the range of `float` can also occur if we have
# two very large numbers ... as long as one is much larger than the
# other.
>>> 1e18 + 1e36       # 1_000_000_000_000_000_001 * 1e18 needs 19 digits
1e+36

# We here can again observe the problem with intermediate results.
# The final result may fit well into a `float`, but if precision is
# lost in intermediate steps, the final result can be off quite a
# bit.
>>> 1e18 + 1e36 - 1e36  # Since 1e18 + 1e36 == 1e36, this gives us 0.
0.0
>>> 1e18 + 1 + 1e36 - 1e36 - 1e18  # The exact result would be 1, but...
-1e+18
```

Grenzen des Datentyps `float`

- Was passiert, wenn wir $10^{16} + 1$ berechnen, und dann 10^{16} von der Summe abziehen?
- Offensichtlich, in einer idealen Welt, wäre das Ergbenis `1.0`.
- Nun wäre `1e16 + 1` ja `10_000_000_000_000_001.0`, was nicht in einen `float` passt.
- `1.0` ist aber eine Zahl, die wir sehr wohl genau als `float` darstellen können.
- Das wirkliche Ergebnis der Berechnung in Python ist jedoch `0.0`.

```
Python 3.14.10 (main, Oct 27 2026, 09:35:01) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
# Python's float supports 15 to 16 digits of precision.
>>> 1e15 + 1          # This works and will give us 1_000_000_000_000_001
1000000000000001.0
>>> 1e16 + 1          # This gives us 1e16, since we would need 17 digits
1e+16

# While 1e16 + 1 cannot be represented exactly with a `float`, one
# may wonder what happens if the computation produces results that
# can be represented exactly ... but has intermediate steps that
# would exceed the range that can be covered by a `float`.
>>> 1e16 + 1 - 1e16  # Since 1e16 + 1 == 1e16, this does yield 0.
0.0

# The problems with the range of `float` can also occur if we have
# two very large numbers ... as long as one is much larger than the
# other.
>>> 1e18 + 1e36       # 1_000_000_000_000_000_001 * 1e18 needs 19 digits
1e+36

# We here can again observe the problem with intermediate results.
# The final result may fit well into a `float`, but if precision is
# lost in intermediate steps, the final result can be off quite a
# bit.
>>> 1e18 + 1e36 - 1e36  # Since 1e18 + 1e36 == 1e36, this gives us 0.
0.0
>>> 1e18 + 1 + 1e36 - 1e36 - 1e18  # The exact result would be 1, but...
-1e+18
```

Grenzen des Datentyps `float`

- Offensichtlich, in einer idealen Welt, wäre das Ergbenis `1.0`.
- Nun wäre `1e16 + 1` ja `10_000_000_000_000_001.0`, was nicht in einen `float` passt.
- `1.0` ist aber eine Zahl, die wir sehr wohl genau als `float` darstellen können.
- Das wirkliche Ergebnis der Berechnung in Python ist jedoch `0.0`.
- Der Grund ist, dass das Zwischenergebnis `1e16 + 1 == 1e16` und dann `1e16 - 1e16 == 0` erfolgt.

```
Python 3.14.10 (main, Oct 27 2026, 09:35:01) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
# Python's float supports 15 to 16 digits of precision.
>>> 1e15 + 1          # This works and will give us 1_000_000_000_000_001
1000000000000001.0
>>> 1e16 + 1          # This gives us 1e16, since we would need 17 digits
1e+16

# While 1e16 + 1 cannot be represented exactly with a `float`, one
# may wonder what happens if the computation produces results that
# can be represented exactly ... but has intermediate steps that
# would exceed the range that can be covered by a `float`.
>>> 1e16 + 1 - 1e16  # Since 1e16 + 1 == 1e16, this does yield 0.
0.0

# The problems with the range of `float` can also occur if we have
# two very large numbers ... as long as one is much larger than the
# other.
>>> 1e18 + 1e36       # 1_000_000_000_000_000_001 * 1e18 needs 19 digits
1e+36

# We here can again observe the problem with intermediate results.
# The final result may fit well into a `float`, but if precision is
# lost in intermediate steps, the final result can be off quite a
# bit.
>>> 1e18 + 1e36 - 1e36  # Since 1e18 + 1e36 == 1e36, this gives us 0.
0.0
>>> 1e18 + 1 + 1e36 - 1e36 - 1e18  # The exact result would be 1, but...
-1e+18
```

Grenzen des Datentyps `float`

- `1.0` ist aber eine Zahl, die wir sehr wohl genau als `float` darstellen können.
- Das wirkliche Ergebnis der Berechnung in Python ist jedoch `0.0`.
- Der Grund ist, dass das Zwischenergebnis `1e16 + 1 == 1e16` und dann `1e16 - 1e16 == 0` erfolgt.
- Ähnliches passiert, wenn wir `1e18 + 1 + 1e36 - 1e36 - 1e18` berechnen, was dann `-1e18` ergibt, wohingegen das „korrekte“ Ergebnis wieder `1.0` wäre.

```
Python 3.14.10 (main, Oct 27 2026, 09:35:01) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
# Python's float supports 15 to 16 digits of precision.
>>> 1e15 + 1          # This works and will give us 1_000_000_000_000_001
1000000000000001.0
>>> 1e16 + 1          # This gives us 1e16, since we would need 17 digits
1e+16

# While 1e16 + 1 cannot be represented exactly with a `float`, one
# may wonder what happens if the computation produces results that
# can be represented exactly ... but has intermediate steps that
# would exceed the range that can be covered by a `float`.
>>> 1e16 + 1 - 1e16  # Since 1e16 + 1 == 1e16, this does yield 0.
0.0

# The problems with the range of `float` can also occur if we have
# two very large numbers ... as long as one is much larger than the
# other.
>>> 1e18 + 1e36       # 1_000_000_000_000_000_001 * 1e18 needs 19 digits
1e+36

# We here can again observe the problem with intermediate results.
# The final result may fit well into a `float`, but if precision is
# lost in intermediate steps, the final result can be off quite a
# bit.
>>> 1e18 + 1e36 - 1e36  # Since 1e18 + 1e36 == 1e36, this gives us 0.
0.0
>>> 1e18 + 1 + 1e36 - 1e36 - 1e18  # The exact result would be 1, but...
-1e+18
```

Grenzen des Datentyps `float`

- `1.0` ist aber eine Zahl, die wir sehr wohl genau als `float` darstellen können.
- Das wirkliche Ergebnis der Berechnung in Python ist jedoch `0.0`.
- Der Grund ist, dass das Zwischenergebnis `1e16 + 1 == 1e16` und dann `1e16 - 1e16 == 0` erfolgt.
- Ähnliches passiert, wenn wir `1e18 + 1 + 1e36 - 1e36 - 1e18` berechnen, was dann `-1e18` ergibt, wohingegen das „korrekte“ Ergebnis wieder `1.0` wäre.
- Der Grund ist dass zuerst `1e18 + 1 == 1e18` berechnet wird.

```
Python 3.14.10 (main, Oct 27 2026, 09:35:01) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
# Python's float supports 15 to 16 digits of precision.
>>> 1e15 + 1          # This works and will give us 1_000_000_000_000_001
1000000000000001.0
>>> 1e16 + 1          # This gives us 1e16, since we would need 17 digits
1e+16

# While 1e16 + 1 cannot be represented exactly with a `float`, one
# may wonder what happens if the computation produces results that
# can be represented exactly ... but has intermediate steps that
# would exceed the range that can be covered by a `float`.
>>> 1e16 + 1 - 1e16  # Since 1e16 + 1 == 1e16, this does yield 0.
0.0

# The problems with the range of `float` can also occur if we have
# two very large numbers ... as long as one is much larger than the
# other.
>>> 1e18 + 1e36       # 1_000_000_000_000_000_001 * 1e18 needs 19 digits
1e+36

# We here can again observe the problem with intermediate results.
# The final result may fit well into a `float`, but if precision is
# lost in intermediate steps, the final result can be off quite a
# bit.
>>> 1e18 + 1e36 - 1e36  # Since 1e18 + 1e36 == 1e36, this gives us 0.
0.0
>>> 1e18 + 1 + 1e36 - 1e36 - 1e18  # The exact result would be 1, but...
-1e+18
```

Grenzen des Datentyps `float`

- Das wirkliche Ergebnis der Berechnung in Python ist jedoch `0.0`.
- Der Grund ist, dass das Zwischenergebnis `1e16 + 1 == 1e16` und dann `1e16 - 1e16 == 0` erfolgt.
- Ähnliches passiert, wenn wir `1e18 + 1 + 1e36 - 1e36 - 1e18` berechnen, was dann `-1e18` ergibt, wohingegen das „korrekte“ Ergebnis wieder `1.0` wäre.
- Der Grund ist dass zuerst `1e18 + 1 == 1e18` berechnet wird.
- Danach ergibt `1e18 + 1e36` dann `1e36`, wovon wir wiederung `1e36` abziehen und `0.0` bekommen.

```
Python 3.14.10 (main, Oct 27 2026, 09:35:01) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
# Python's float supports 15 to 16 digits of precision.
>>> 1e15 + 1          # This works and will give us 1_000_000_000_000_001
1000000000000001.0
>>> 1e16 + 1          # This gives us 1e16, since we would need 17 digits
1e+16

# While 1e16 + 1 cannot be represented exactly with a `float`, one
# may wonder what happens if the computation produces results that
# can be represented exactly ... but has intermediate steps that
# would exceed the range that can be covered by a `float`.
>>> 1e16 + 1 - 1e16  # Since 1e16 + 1 == 1e16, this does yield 0.
0.0

# The problems with the range of `float` can also occur if we have
# two very large numbers ... as long as one is much larger than the
# other.
>>> 1e18 + 1e36       # 1_000_000_000_000_000_001 * 1e18 needs 19 digits
1e+36

# We here can again observe the problem with intermediate results.
# The final result may fit well into a `float`, but if precision is
# lost in intermediate steps, the final result can be off quite a
# bit.
>>> 1e18 + 1e36 - 1e36  # Since 1e18 + 1e36 == 1e36, this gives us 0.
0.0
>>> 1e18 + 1 + 1e36 - 1e36 - 1e18  # The exact result would be 1, but...
-1e+18
```

Grenzen des Datentyps `float`

- Der Grund ist, dass das Zwischenergebnis `1e16 + 1 == 1e16` und dann `1e16 - 1e16 == 0` erfolgt.
- Ähnliches passiert, wenn wir `1e18 + 1 + 1e36 - 1e36 - 1e18` berechnen, was dann `-1e18` ergibt, wohingegen das „korrekte“ Ergebnis wieder `1.0` wäre.
- Der Grund ist dass zuerst `1e18 + 1 == 1e18` berechnet wird.
- Danach ergibt `1e18 + 1e36` dann `1e36`, wovon wir wiederung `1e36` abziehen und `0.0` bekommen.
- Die letzte Subtraktion von `1e18` liefert dann `-1e18`.

```
Python 3.14.10 (main, Oct 27 2026, 09:35:01) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
# Python's float supports 15 to 16 digits of precision.
>>> 1e15 + 1          # This works and will give us 1_000_000_000_000_001
1000000000000001.0
>>> 1e16 + 1          # This gives us 1e16, since we would need 17 digits
1e+16

# While 1e16 + 1 cannot be represented exactly with a `float`, one
# may wonder what happens if the computation produces results that
# can be represented exactly ... but has intermediate steps that
# would exceed the range that can be covered by a `float`.
>>> 1e16 + 1 - 1e16  # Since 1e16 + 1 == 1e16, this does yield 0.
0.0

# The problems with the range of `float` can also occur if we have
# two very large numbers ... as long as one is much larger than the
# other.
>>> 1e18 + 1e36       # 1_000_000_000_000_000_001 * 1e18 needs 19 digits
1e+36

# We here can again observe the problem with intermediate results.
# The final result may fit well into a `float`, but if precision is
# lost in intermediate steps, the final result can be off quite a
# bit.
>>> 1e18 + 1e36 - 1e36  # Since 1e18 + 1e36 == 1e36, this gives us 0.
0.0
>>> 1e18 + 1 + 1e36 - 1e36 - 1e18  # The exact result would be 1, but...
-1e+18
```

Grenzen des Datentyps `float`

- Ähnliches passiert, wenn wir `1e18 + 1 + 1e36 - 1e36 - 1e18` berechnen, was dann `-1e18` ergibt, wohingegen das „korrekte“ Ergebnis wieder `1.0` wäre.
- Der Grund ist dass zuerst `1e18 + 1 == 1e18` berechnet wird.
- Danach ergibt `1e18 + 1e36` dann `1e36`, wovon wir wiederung `1e36` abziehen und `0.0` bekommen.
- Die letzte Subtraktion von `1e18` liefert dann `-1e18`.
- Hätten wir unendliche Genauigkeit, dann würde der erste Rechenschritt $10^{18} + 1 = 1\,000\,000\ \,000\,000\,000\,001$ ergeben.

```
Python 3.14.10 (main, Oct 27 2026, 09:35:01) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
# Python's float supports 15 to 16 digits of precision.
>>> 1e15 + 1          # This works and will give us 1_000_000_000_000_001
1000000000000001.0
>>> 1e16 + 1          # This gives us 1e16, since we would need 17 digits
1e+16

# While 1e16 + 1 cannot be represented exactly with a `float`, one
# may wonder what happens if the computation produces results that
# can be represented exactly ... but has intermediate steps that
# would exceed the range that can be covered by a `float`.
>>> 1e16 + 1 - 1e16  # Since 1e16 + 1 == 1e16, this does yield 0.
0.0

# The problems with the range of `float` can also occur if we have
# two very large numbers ... as long as one is much larger than the
# other.
>>> 1e18 + 1e36       # 1_000_000_000_000_000_001 * 1e18 needs 19 digits
1e+36

# We here can again observe the problem with intermediate results.
# The final result may fit well into a `float`, but if precision is
# lost in intermediate steps, the final result can be off quite a
# bit.
>>> 1e18 + 1e36 - 1e36  # Since 1e18 + 1e36 == 1e36, this gives us 0.
0.0
>>> 1e18 + 1 + 1e36 - 1e36 - 1e18  # The exact result would be 1, but...
-1e+18
```

Grenzen des Datentyps `float`

- Der Grund ist dass zuerst `1e18 + 1 == 1e18` berechnet wird.
- Danach ergibt `1e18 + 1e36` dann `1e36`, wovon wir wiederung `1e36` abziehen und `0.0` bekommen.
- Die letzte Subtraktion von `1e18` liefert dann `-1e18`.
- Hätten wir unendliche Genauigkeit, dann würde der erste Rechenschritt $10^{18} + 1 = 1\,000\,000\ \ 000\,000\,000\,001$ ergeben.
- Wenn wir `1e36` zu dieser Zahl addieren, dann sollte das Ergebnis $1000\,000\,000\ \ 000\,000\,001$ $000\,000\,000\,000\ \ 000\,001$ sein.

```
Python 3.14.10 (main, Oct 27 2026, 09:35:01) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
# Python's float supports 15 to 16 digits of precision.
>>> 1e15 + 1          # This works and will give us 1_000_000_000_000_001
1000000000000001.0
>>> 1e16 + 1          # This gives us 1e16, since we would need 17 digits
1e+16

# While 1e16 + 1 cannot be represented exactly with a `float`, one
# may wonder what happens if the computation produces results that
# can be represented exactly ... but has intermediate steps that
# would exceed the range that can be covered by a `float`.
>>> 1e16 + 1 - 1e16  # Since 1e16 + 1 == 1e16, this does yield 0.
0.0

# The problems with the range of `float` can also occur if we have
# two very large numbers ... as long as one is much larger than the
# other.
>>> 1e18 + 1e36       # 1_000_000_000_000_000_001 * 1e18 needs 19 digits
1e+36

# We here can again observe the problem with intermediate results.
# The final result may fit well into a `float`, but if precision is
# lost in intermediate steps, the final result can be off quite a
# bit.
>>> 1e18 + 1e36 - 1e36  # Since 1e18 + 1e36 == 1e36, this gives us 0.
0.0
>>> 1e18 + 1 + 1e36 - 1e36 - 1e18  # The exact result would be 1, but...
-1e+18
```

Grenzen des Datentyps `float`

- Die letzte Subtraktion von `1e18` liefert dann `-1e18`.
- Hätten wir unendliche Genauigkeit, dann würde der erste Rechenschritt $10^{18} + 1 = 1\,000\,000\ \ 000\,000\,000\,001$ ergeben.
- Wenn wir `1e36` zu dieser Zahl addieren, dann sollte das Ergebnis $1000\,000\,000\ \ 000\,000\,001$ $000\,000\,000\,000\ \ 000\,001$ sein.
- Das Abziehen von `1e36` von dieser gigantischen Zahl würde uns wieder zu $10^{18} + 1$ bringen und die letzte Subtraktion von `1e18` ergäbe dann `1.0`.

```
Python 3.14.10 (main, Oct 27 2026, 09:35:01) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
# Python's float supports 15 to 16 digits of precision.
>>> 1e15 + 1          # This works and will give us 1_000_000_000_000_001
1000000000000001.0
>>> 1e16 + 1          # This gives us 1e16, since we would need 17 digits
1e+16

# While 1e16 + 1 cannot be represented exactly with a `float`, one
# may wonder what happens if the computation produces results that
# can be represented exactly ... but has intermediate steps that
# would exceed the range that can be covered by a `float`.
>>> 1e16 + 1 - 1e16  # Since 1e16 + 1 == 1e16, this does yield 0.
0.0

# The problems with the range of `float` can also occur if we have
# two very large numbers ... as long as one is much larger than the
# other.
>>> 1e18 + 1e36       # 1_000_000_000_000_000_001 * 1e18 needs 19 digits
1e+36

# We here can again observe the problem with intermediate results.
# The final result may fit well into a `float`, but if precision is
# lost in intermediate steps, the final result can be off quite a
# bit.
>>> 1e18 + 1e36 - 1e36  # Since 1e18 + 1e36 == 1e36, this gives us 0.
0.0
>>> 1e18 + 1 + 1e36 - 1e36 - 1e18  # The exact result would be 1, but...
-1e+18
```

Grenzen des Datentyps `float`

- Hätten wir unendliche Genauigkeit, dann würde der erste Rechenschritt $10^{18} + 1 = 1\,000\,000\ \ 000\,000\,000\,001$ ergeben.
- Wenn wir `1e36` zu dieser Zahl addieren, dann sollte das Ergebnis $1000\,000\,000\ \ 000\,000\,001$ $000\,000\,000\,000\ \ 000\,001$ sein.
- Das Abziehen von `1e36` von dieser gigantischen Zahl würde uns wieder zu $10^{18} + 1$ bringen und die letzte Subtraktion von `1e18` ergäbe dann `1.0`.
- Es ist leicht zu sehen, warum das nicht funktioniert.

```
Python 3.14.10 (main, Oct 27 2026, 09:35:01) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
# Python's float supports 15 to 16 digits of precision.
>>> 1e15 + 1          # This works and will give us 1_000_000_000_000_001
1000000000000001.0
>>> 1e16 + 1          # This gives us 1e16, since we would need 17 digits
1e+16

# While 1e16 + 1 cannot be represented exactly with a `float`, one
# may wonder what happens if the computation produces results that
# can be represented exactly ... but has intermediate steps that
# would exceed the range that can be covered by a `float`.
>>> 1e16 + 1 - 1e16  # Since 1e16 + 1 == 1e16, this does yield 0.
0.0

# The problems with the range of `float` can also occur if we have
# two very large numbers ... as long as one is much larger than the
# other.
>>> 1e18 + 1e36       # 1_000_000_000_000_000_001 * 1e18 needs 19 digits
1e+36

# We here can again observe the problem with intermediate results.
# The final result may fit well into a `float`, but if precision is
# lost in intermediate steps, the final result can be off quite a
# bit.
>>> 1e18 + 1e36 - 1e36  # Since 1e18 + 1e36 == 1e36, this gives us 0.
0.0
>>> 1e18 + 1 + 1e36 - 1e36 - 1e18  # The exact result would be 1, but...
-1e+18
```

Grenzen des Datentyps `float`

- Wenn wir `1e36` zu dieser Zahl addieren, dann sollte das Ergebnis 1000 000 000 000 000 001 000 000 000 000 000 001 sein.
- Das Abziehen von `1e36` von dieser gigantischen Zahl würde uns wieder zu $10^{18} + 1$ bringen und die letzte Subtraktion von `1e18` ergäbe dann `1.0`.
- Es ist leicht zu sehen, warum das nicht funktioniert.
- Die Entwickler der Fließkommaarithmetikeinheit von CPUs mussten eine bestimmte, begrenzte Anzahl von Bits für den Datentyp `float` festlegen.

```
Python 3.14.10 (main, Oct 27 2026, 09:35:01) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
# Python's float supports 15 to 16 digits of precision.
>>> 1e15 + 1          # This works and will give us 1_000_000_000_000_001
1000000000000001.0
>>> 1e16 + 1          # This gives us 1e16, since we would need 17 digits
1e+16

# While 1e16 + 1 cannot be represented exactly with a `float`, one
# may wonder what happens if the computation produces results that
# can be represented exactly ... but has intermediate steps that
# would exceed the range that can be covered by a `float`.
>>> 1e16 + 1 - 1e16  # Since 1e16 + 1 == 1e16, this does yield 0.
0.0

# The problems with the range of `float` can also occur if we have
# two very large numbers ... as long as one is much larger than the
# other.
>>> 1e18 + 1e36       # 1_000_000_000_000_000_001 * 1e18 needs 19 digits
1e+36

# We here can again observe the problem with intermediate results.
# The final result may fit well into a `float`, but if precision is
# lost in intermediate steps, the final result can be off quite a
# bit.
>>> 1e18 + 1e36 - 1e36  # Since 1e18 + 1e36 == 1e36, this gives us 0.
0.0
>>> 1e18 + 1 + 1e36 - 1e36 - 1e18  # The exact result would be 1, but...
-1e+18
```

Grenzen des Datentyps `float`

- Es ist leicht zu sehen, warum das nicht funktioniert.
- Die Entwickler der Fließkommaarithmetikeinheit von CPUs mussten eine bestimmte, begrenzte Anzahl von Bits für den Datentyp `float` festlegen.
- Sie haben sich überlegt, dass 52 Bits für den Signifikand, die uns $52/\log_2 10 \approx 15.7$ Ziffern geben, vernünftig sind und auch gut in acht Bytes Speicher passen.

```
Python 3.14.10 (main, Oct 27 2026, 09:35:01) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
# Python's float supports 15 to 16 digits of precision.
>>> 1e15 + 1          # This works and will give us 1_000_000_000_000_001
1000000000000001.0
>>> 1e16 + 1          # This gives us 1e16, since we would need 17 digits
1e+16

# While 1e16 + 1 cannot be represented exactly with a `float`, one
# may wonder what happens if the computation produces results that
# can be represented exactly ... but has intermediate steps that
# would exceed the range that can be covered by a `float`.
>>> 1e16 + 1 - 1e16  # Since 1e16 + 1 == 1e16, this does yield 0.
0.0

# The problems with the range of `float` can also occur if we have
# two very large numbers ... as long as one is much larger than the
# other.
>>> 1e18 + 1e36       # 1_000_000_000_000_000_001 * 1e18 needs 19 digits
1e+36

# We here can again observe the problem with intermediate results.
# The final result may fit well into a `float`, but if precision is
# lost in intermediate steps, the final result can be off quite a
# bit.
>>> 1e18 + 1e36 - 1e36  # Since 1e18 + 1e36 == 1e36, this gives us 0.
0.0
>>> 1e18 + 1 + 1e36 - 1e36 - 1e18  # The exact result would be 1, but...
-1e+18
```

Grenzen des Datentyps `float`

- Die Entwickler der Fließkommaarithmetikeinheit von CPUs mussten eine bestimmte, begrenzte Anzahl von Bits für den Datentyp `float` festlegen.
- Sie haben sich überlegt, dass 52 Bits für den Signifikand, die uns $52/\log_2 10 \approx 15.7$ Ziffern geben, vernünftig sind und auch gut in acht Bytes Speicher passen.
- Dagegen benötigen 36 Ziffern einen Signifikand von $36 \log_2 10 \approx 120$ Bits, was den Datentyp viel größer macht und wahrscheinlich auch nur sehr selten benötigt werden würde.

```
Python 3.14.10 (main, Oct 27 2026, 09:35:01) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
# Python's float supports 15 to 16 digits of precision.
>>> 1e15 + 1          # This works and will give us 1_000_000_000_000_001
1000000000000001.0
>>> 1e16 + 1          # This gives us 1e16, since we would need 17 digits
1e+16

# While 1e16 + 1 cannot be represented exactly with a `float`, one
# may wonder what happens if the computation produces results that
# can be represented exactly ... but has intermediate steps that
# would exceed the range that can be covered by a `float`.
>>> 1e16 + 1 - 1e16  # Since 1e16 + 1 == 1e16, this does yield 0.
0.0

# The problems with the range of `float` can also occur if we have
# two very large numbers ... as long as one is much larger than the
# other.
>>> 1e18 + 1e36       # 1_000_000_000_000_000_001 * 1e18 needs 19 digits
1e+36

# We here can again observe the problem with intermediate results.
# The final result may fit well into a `float`, but if precision is
# lost in intermediate steps, the final result can be off quite a
# bit.
>>> 1e18 + 1e36 - 1e36  # Since 1e18 + 1e36 == 1e36, this gives us 0.
0.0
>>> 1e18 + 1 + 1e36 - 1e36 - 1e18  # The exact result would be 1, but...
-1e+18
```

Grenzen des Datentyps `float`

- Sie haben sich überlegt, dass 52 Bits für den Signifikand, die uns $52/\log_2 10 \approx 15.7$ Ziffern geben, vernünftig sind und auch gut in acht Bytes Speicher passen.
- Dagegen benötigen 36 Ziffern einen Signifikand von $36 \log_2 10 \approx 120$ Bits, was den Datentyp viel größer macht und wahrscheinlich auch nur sehr selten benötigt werden würde.
- Naja, es sei denn, man addiert große und kleine Zahlen zusammen...

```
Python 3.14.10 (main, Oct 27 2026, 09:35:01) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
# Python's float supports 15 to 16 digits of precision.
>>> 1e15 + 1          # This works and will give us 1_000_000_000_000_001
1000000000000001.0
>>> 1e16 + 1          # This gives us 1e16, since we would need 17 digits
1e+16

# While 1e16 + 1 cannot be represented exactly with a `float`, one
# may wonder what happens if the computation produces results that
# can be represented exactly ... but has intermediate steps that
# would exceed the range that can be covered by a `float`.
>>> 1e16 + 1 - 1e16  # Since 1e16 + 1 == 1e16, this does yield 0.
0.0

# The problems with the range of `float` can also occur if we have
# two very large numbers ... as long as one is much larger than the
# other.
>>> 1e18 + 1e36       # 1_000_000_000_000_000_001 * 1e18 needs 19 digits
1e+36

# We here can again observe the problem with intermediate results.
# The final result may fit well into a `float`, but if precision is
# lost in intermediate steps, the final result can be off quite a
# bit.
>>> 1e18 + 1e36 - 1e36  # Since 1e18 + 1e36 == 1e36, this gives us 0.
0.0
>>> 1e18 + 1 + 1e36 - 1e36 - 1e18  # The exact result would be 1, but...
-1e+18
```

Grenzen des Datentyps `float`

- Sie haben sich überlegt, dass 52 Bits für den Signifikand, die uns $52/\log_2 10 \approx 15.7$ Ziffern geben, vernünftig sind und auch gut in acht Bytes Speicher passen.
- Dagegen benötigen 36 Ziffern einen Signifikand von $36 \log_2 10 \approx 120$ Bits, was den Datentyp viel größer macht und wahrscheinlich auch nur sehr selten benötigt werden würde.
- Naja, es sei denn, man addiert große und kleine Zahlen zusammen...
- Die interessante Frage, die wir basierend auf dem Beispiel stellen können ist...

```
Python 3.14.10 (main, Oct 27 2026, 09:35:01) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
# Python's float supports 15 to 16 digits of precision.
>>> 1e15 + 1          # This works and will give us 1_000_000_000_000_001
1000000000000001.0
>>> 1e16 + 1          # This gives us 1e16, since we would need 17 digits
1e+16

# While 1e16 + 1 cannot be represented exactly with a `float`, one
# may wonder what happens if the computation produces results that
# can be represented exactly ... but has intermediate steps that
# would exceed the range that can be covered by a `float`.
>>> 1e16 + 1 - 1e16  # Since 1e16 + 1 == 1e16, this does yield 0.
0.0

# The problems with the range of `float` can also occur if we have
# two very large numbers ... as long as one is much larger than the
# other.
>>> 1e18 + 1e36       # 1_000_000_000_000_000_001 * 1e18 needs 19 digits
1e+36

# We here can again observe the problem with intermediate results.
# The final result may fit well into a `float`, but if precision is
# lost in intermediate steps, the final result can be off quite a
# bit.
>>> 1e18 + 1e36 - 1e36  # Since 1e18 + 1e36 == 1e36, this gives us 0.
0.0
>>> 1e18 + 1 + 1e36 - 1e36 - 1e18  # The exact result would be 1, but...
-1e+18
```

Grenzen des Datentyps `float`

- Dagegen benötigen 36 Ziffern einen Signifikand von $36 \log_2 10 \approx 120$ Bits, was den Datentyp viel größer macht und wahrscheinlich auch nur sehr selten benötigt werden würde.
- Naja, es sei denn, man addiert große und kleine Zahlen zusammen...
- Die interessante Frage, die wir basierend auf dem Beispiel stellen können ist:
- *Gibt es eine Möglichkeit, große und kleine Zahlen genauer zu addieren?*

```
Python 3.14.10 (main, Oct 27 2026, 09:35:01) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
# Python's float supports 15 to 16 digits of precision.
>>> 1e15 + 1          # This works and will give us 1_000_000_000_000_001
1000000000000001.0
>>> 1e16 + 1          # This gives us 1e16, since we would need 17 digits
1e+16

# While 1e16 + 1 cannot be represented exactly with a `float`, one
# may wonder what happens if the computation produces results that
# can be represented exactly ... but has intermediate steps that
# would exceed the range that can be covered by a `float`.
>>> 1e16 + 1 - 1e16  # Since 1e16 + 1 == 1e16, this does yield 0.
0.0

# The problems with the range of `float` can also occur if we have
# two very large numbers ... as long as one is much larger than the
# other.
>>> 1e18 + 1e36       # 1_000_000_000_000_000_001 * 1e18 needs 19 digits
1e+36

# We here can again observe the problem with intermediate results.
# The final result may fit well into a `float`, but if precision is
# lost in intermediate steps, the final result can be off quite a
# bit.
>>> 1e18 + 1e36 - 1e36  # Since 1e18 + 1e36 == 1e36, this gives us 0.
0.0
>>> 1e18 + 1 + 1e36 - 1e36 - 1e18  # The exact result would be 1, but...
-1e+18
```

Grenzen des Datentyps `float`

- Dagegen benötigen 36 Ziffern einen Signifikand von $36 \log_2 10 \approx 120$ Bits, was den Datentyp viel größer macht und wahrscheinlich auch nur sehr selten benötigt werden würde.
- Naja, es sei denn, man addiert große und kleine Zahlen zusammen...
- Die interessante Frage, die wir basierend auf dem Beispiel stellen können ist:
- *Gibt es eine Möglichkeit, große und kleine Zahlen genauer zu addieren?*
- Natürlich können wir `1e16 + 1` niemlals genau mit einem `float` repräsentieren.

```
Python 3.14.10 (main, Oct 27 2026, 09:35:01) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
# Python's float supports 15 to 16 digits of precision.
>>> 1e15 + 1          # This works and will give us 1_000_000_000_000_001
1000000000000001.0
>>> 1e16 + 1          # This gives us 1e16, since we would need 17 digits
1e+16

# While 1e16 + 1 cannot be represented exactly with a `float`, one
# may wonder what happens if the computation produces results that
# can be represented exactly ... but has intermediate steps that
# would exceed the range that can be covered by a `float`.
>>> 1e16 + 1 - 1e16  # Since 1e16 + 1 == 1e16, this does yield 0.
0.0

# The problems with the range of `float` can also occur if we have
# two very large numbers ... as long as one is much larger than the
# other.
>>> 1e18 + 1e36       # 1_000_000_000_000_000_001 * 1e18 needs 19 digits
1e+36

# We here can again observe the problem with intermediate results.
# The final result may fit well into a `float`, but if precision is
# lost in intermediate steps, the final result can be off quite a
# bit.
>>> 1e18 + 1e36 - 1e36  # Since 1e18 + 1e36 == 1e36, this gives us 0.
0.0
>>> 1e18 + 1 + 1e36 - 1e36 - 1e18  # The exact result would be 1, but...
-1e+18
```

Grenzen des Datentyps `float`

- Naja, es sei denn, man addiert große und kleine Zahlen zusammen...
- Die interessante Frage, die wir basierend auf dem Beispiel stellen können ist:
- *Gibt es eine Möglichkeit, große und kleine Zahlen genauer zu addieren?*
- Natürlich können wir `1e16 + 1` niemlals genau mit einem `float` repräsentieren.
- Trotzdem wollen wir das Endergebnis eine Summe doch so genau wie möglich darstellen.

```
Python 3.14.10 (main, Oct 27 2026, 09:35:01) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
# Python's float supports 15 to 16 digits of precision.
>>> 1e15 + 1          # This works and will give us 1_000_000_000_000_001
1000000000000001.0
>>> 1e16 + 1          # This gives us 1e16, since we would need 17 digits
1e+16

# While 1e16 + 1 cannot be represented exactly with a `float`, one
# may wonder what happens if the computation produces results that
# can be represented exactly ... but has intermediate steps that
# would exceed the range that can be covered by a `float`.
>>> 1e16 + 1 - 1e16  # Since 1e16 + 1 == 1e16, this does yield 0.
0.0

# The problems with the range of `float` can also occur if we have
# two very large numbers ... as long as one is much larger than the
# other.
>>> 1e18 + 1e36       # 1_000_000_000_000_000_001 * 1e18 needs 19 digits
1e+36

# We here can again observe the problem with intermediate results.
# The final result may fit well into a `float`, but if precision is
# lost in intermediate steps, the final result can be off quite a
# bit.
>>> 1e18 + 1e36 - 1e36  # Since 1e18 + 1e36 == 1e36, this gives us 0.
0.0
>>> 1e18 + 1 + 1e36 - 1e36 - 1e18  # The exact result would be 1, but...
-1e+18
```

Grenzen des Datentyps `float`

- Die interessante Frage, die wir basierend auf dem Beispiel stellen können ist:
- *Gibt es eine Möglichkeit, große und kleine Zahlen genauer zu addieren?*
- Natürlich können wir `1e16 + 1` niemlals genau mit einem `float` repräsentieren.
- Trotzdem wollen wir das Endergebnis eine Summe doch so genau wie möglich darstellen.
- Wir wollen eine Möglichkeit, die Summe `(1e18 + 1)+ -1e18` so zu berechnen, dass am Ende `1.0` herauskommt.

```
Python 3.14.10 (main, Oct 27 2026, 09:35:01) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
# Python's float supports 15 to 16 digits of precision.
>>> 1e15 + 1          # This works and will give us 1_000_000_000_000_001
1000000000000001.0
>>> 1e16 + 1          # This gives us 1e16, since we would need 17 digits
1e+16

# While 1e16 + 1 cannot be represented exactly with a `float`, one
# may wonder what happens if the computation produces results that
# can be represented exactly ... but has intermediate steps that
# would exceed the range that can be covered by a `float`.
>>> 1e16 + 1 - 1e16  # Since 1e16 + 1 == 1e16, this does yield 0.
0.0

# The problems with the range of `float` can also occur if we have
# two very large numbers ... as long as one is much larger than the
# other.
>>> 1e18 + 1e36       # 1_000_000_000_000_000_001 * 1e18 needs 19 digits
1e+36

# We here can again observe the problem with intermediate results.
# The final result may fit well into a `float`, but if precision is
# lost in intermediate steps, the final result can be off quite a
# bit.
>>> 1e18 + 1e36 - 1e36  # Since 1e18 + 1e36 == 1e36, this gives us 0.
0.0
>>> 1e18 + 1 + 1e36 - 1e36 - 1e18  # The exact result would be 1, but...
-1e+18
```

Grenzen des Datentyps `float`

- *Gibt es eine Möglichkeit, große und kleine Zahlen genauer zu addieren?*
- Natürlich können wir `1e16 + 1` niemlals genau mit einem `float` repräsentieren.
- Trotzdem wollen wir das Endergebnis eine Summe doch so genau wie möglich darstellen.
- Wir wollen eine Möglichkeit, die Summe `(1e18 + 1)+ -1e18` so zu berechnen, dass am Ende `1.0` herauskommt.
- Geht das?

```
Python 3.14.10 (main, Oct 27 2026, 09:35:01) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
# Python's float supports 15 to 16 digits of precision.
>>> 1e15 + 1          # This works and will give us 1_000_000_000_000_001
1000000000000001.0
>>> 1e16 + 1          # This gives us 1e16, since we would need 17 digits
1e+16

# While 1e16 + 1 cannot be represented exactly with a `float`, one
# may wonder what happens if the computation produces results that
# can be represented exactly ... but has intermediate steps that
# would exceed the range that can be covered by a `float`.
>>> 1e16 + 1 - 1e16  # Since 1e16 + 1 == 1e16, this does yield 0.
0.0

# The problems with the range of `float` can also occur if we have
# two very large numbers ... as long as one is much larger than the
# other.
>>> 1e18 + 1e36       # 1_000_000_000_000_000_001 * 1e18 needs 19 digits
1e+36

# We here can again observe the problem with intermediate results.
# The final result may fit well into a `float`, but if precision is
# lost in intermediate steps, the final result can be off quite a
# bit.
>>> 1e18 + 1e36 - 1e36  # Since 1e18 + 1e36 == 1e36, this gives us 0.
0.0
>>> 1e18 + 1 + 1e36 - 1e36 - 1e18  # The exact result would be 1, but...
-1e+18
```

Kahan-Summe: Sinn und Zweck

- Zum Glück hatten Kahan[164] und Babuška[10] in den 1960ern eine Idee, wie man die Präzision für Addition erhöhen kann[125,179].

Kahan-Summe: Sinn und Zweck



- Zum Glück hatten Kahan[164] und Babuška[10] in den 1960ern eine Idee, wie man die Präzision für Addition erhöhen kann[125,179]:
- Wir addieren Zahlen in einer Variable `sum` und verwenden zusätzlich eine Variable `cs`, in der wir uns merken, wie weit das Ergebnis daneben liegt.

Kahan-Summe: Sinn und Zweck



- Zum Glück hatten Kahan[164] und Babuška[10] in den 1960ern eine Idee, wie man die Präzision für Addition erhöhen kann[125,179]:
- Wir addieren Zahlen in einer Variable `sum` und verwenden zusätzlich eine Variable `cs`, in der wir uns merken, wie weit das Ergebnis daneben liegt.
- Stellen wir uns nochmal vor, dass wir `1e18 + 1 - 1e18` berechnen wollen.

Kahan-Summe: Sinn und Zweck



- Zum Glück hatten Kahan[164] und Babuška[10] in den 1960ern eine Idee, wie man die Präzision für Addition erhöhen kann[125,179]:
- Wir addieren Zahlen in einer Variable `sum` und verwenden zusätzlich eine Variable `cs`, in der wir uns merken, wie weit das Ergebnis daneben liegt.
- Stellen wir uns nochmal vor, dass wir `1e18 + 1 - 1e18` berechnen wollen.
- Wie würden wir bei dem Ergebnis `1.0` ankommen?

Kahan-Summe: Sinn und Zweck



- Zum Glück hatten Kahan[164] und Babuška[10] in den 1960ern eine Idee, wie man die Präzision für Addition erhöhen kann[125,179]:
- Wir addieren Zahlen in einer Variable `sum` und verwenden zusätzlich eine Variable `cs`, in der wir uns merken, wie weit das Ergebnis daneben liegt.
- Stellen wir uns nochmal vor, dass wir `1e18 + 1 - 1e18` berechnen wollen.
- Wie würden wir bei dem Ergebnis `1.0` ankommen?
- In dem wir den folgenden, einfachen Algorithmus anwenden, der die Summe aller Zahlen in einer Variable `sum` bildet und zusätzlich einen Fehler-Term `cs` mitführt.

Kahan-Summe: Ausprobieren

- In dem wir den folgenden, einfachen Algorithmus anwenden:

Kahan-Summe: Ausprobieren

- In dem wir den folgenden, einfachen Algorithmus anwenden:

1. Wir beginnen mit `sum = 0` und `cs = 0`.

Kahan-Summe: Ausprobieren

- In dem wir den folgenden, einfachen Algorithmus anwenden:

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Für jede Zahl, die zur Summe dazuaddiert werden soll, machen wir die folgende Schritte.

Kahan-Summe: Ausprobieren

- In dem wir den folgenden, einfachen Algorithmus anwenden:

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Für jede Zahl, die zur Summe dazuaddiert werden soll, machen wir die folgende Schritte:
   2.1 Wir berechnen zuerst `t = sum + value`.

Kahan-Summe: Ausprobieren

- In dem wir den folgenden, einfachen Algorithmus anwenden:

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Für jede Zahl, die zur Summe dazuaddiert werden soll, machen wir die folgende Schritte:
   2.1 Wir berechnen zuerst `t = sum + value`. `t` ist damit die Summe, und zwar zu der Genauigkeit, die `float` darstellen kann.

Kahan-Summe: Ausprobieren

- In dem wir den folgenden, einfachen Algorithmus anwenden:

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Für jede Zahl, die zur Summe dazuaddiert werden soll, machen wir die folgende Schritte:
   - 2.1 Wir berechnen zuerst `t = sum + value`.
   - 2.2 Dann berechnen wir `error = (sum - t)+ value`.

Kahan-Summe: Ausprobieren

- In dem wir den folgenden, einfachen Algorithmus anwenden:

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Für jede Zahl, die zur Summe dazuaddiert werden soll, machen wir die folgende Schritte:
   - 2.1 Wir berechnen zuerst `t = sum + value`.
   - 2.2 Dann berechnen wir `error = (sum - t)+ value`. Weil `t = sum + value`, könnte man denken das `error` eigentlich immer `0.0` seien sollte, schließlich sieht das aus wie `(sum - (sum + value))+ value`.

Kahan-Summe: Ausprobieren

- In dem wir den folgenden, einfachen Algorithmus anwenden:

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Für jede Zahl, die zur Summe dazuaddiert werden soll, machen wir die folgende Schritte:
   - 2.1 Wir berechnen zuerst `t = sum + value`.
   - 2.2 Dann berechnen wir `error = (sum - t)+ value`. Weil `t = sum + value`, könnte man denken das `error` eigentlich immer `0.0` seien sollte, schließlich sieht das aus wie `(sum - (sum + value))+ value`. Aber es können ja ein paar Ziffern „verloren gehen“, wenn wir `sum + value` berechnen.

Kahan-Summe: Ausprobieren

- In dem wir den folgenden, einfachen Algorithmus anwenden:

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Für jede Zahl, die zur Summe dazuaddiert werden soll, machen wir die folgende Schritte:
   - 2.1 Wir berechnen zuerst `t = sum + value`.
   - 2.2 Dann berechnen wir `error = (sum - t)+ value`. Weil `t = sum + value`, könnte man denken das `error` eigentlich immer `0.0` seien sollte, schließlich sieht das aus wie `(sum - (sum + value))+ value`. Aber es können ja ein paar Ziffern „verloren gehen“, wenn wir `sum + value` berechnen. Diese tauchen dann wieder in `error` auf.

Kahan-Summe: Ausprobieren

- In dem wir den folgenden, einfachen Algorithmus anwenden:

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Für jede Zahl, die zur Summe dazuaddiert werden soll, machen wir die folgende Schritte:
   - 2.1 Wir berechnen zuerst `t = sum + value`.
   - 2.2 Dann berechnen wir `error = (sum - t)+ value`.
   - 2.3 Dann setzen wir `sum = t`.

Kahan-Summe: Ausprobieren

- In dem wir den folgenden, einfachen Algorithmus anwenden:

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Für jede Zahl, die zur Summe dazuaddiert werden soll, machen wir die folgende Schritte:
   - 2.1 Wir berechnen zuerst `t = sum + value`.
   - 2.2 Dann berechnen wir `error = (sum - t)+ value`.
   - 2.3 Dann setzen wir `sum = t`.
   - 2.4 Wir addieren die Fehler in `cs` auf, setzen also `cs += error`.

Kahan-Summe: Ausprobieren

- In dem wir den folgenden, einfachen Algorithmus anwenden:

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Für jede Zahl, die zur Summe dazuaddiert werden soll, machen wir die folgende Schritte:
   - 2.1 Wir berechnen zuerst `t = sum + value`.
   - 2.2 Dann berechnen wir `error = (sum - t)+ value`.
   - 2.3 Dann setzen wir `sum = t`.
   - 2.4 Wir addieren die Fehler in `cs` auf, setzen also `cs += error`.
3. Das Endergebnis ist dann `sum + cs`.

Kahan-Summe: Ausprobieren

- In dem wir den folgenden, einfachen Algorithmus anwenden:

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Für jede Zahl, die zur Summe dazuaddiert werden soll, machen wir die folgende Schritte:
   - 2.1 Wir berechnen zuerst `t = sum + value`.
   - 2.2 Dann berechnen wir `error = (sum - t)+ value`.
   - 2.3 Dann setzen wir `sum = t`.
   - 2.4 Wir addieren die Fehler in `cs` auf, setzen also `cs += error`.
3. Das Endergebnis ist dann `sum + cs`.

- Probieren wir das am Beispiel `1e18 + 1 - 1e18` aus.

Kahan-Summe: Ausprobieren

- In dem wir den folgenden, einfachen Algorithmus anwenden:

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Für jede Zahl, die zur Summe dazuaddiert werden soll, machen wir die folgende Schritte:
   - 2.1 Wir berechnen zuerst `t = sum + value`.
   - 2.2 Dann berechnen wir `error = (sum - t)+ value`.
   - 2.3 Dann setzen wir `sum = t`.
   - 2.4 Wir addieren die Fehler in `cs` auf, setzen also `cs += error`.
3. Das Endergebnis ist dann `sum + cs`.

- Probieren wir das am Beispiel `1e18 + 1 - 1e18` aus.

1. Wir beginnen mit `sum = 0` und `cs = 0`.

Kahan-Summe: Ausprobieren

- In dem wir den folgenden, einfachen Algorithmus anwenden:

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Für jede Zahl, die zur Summe dazuaddiert werden soll, machen wir die folgende Schritte:
   - 2.1 Wir berechnen zuerst `t = sum + value`.
   - 2.2 Dann berechnen wir `error = (sum - t)+ value`.
   - 2.3 Dann setzen wir `sum = t`.
   - 2.4 Wir addieren die Fehler in `cs` auf, setzen also `cs += error`.
3. Das Endergebnis ist dann `sum + cs`.

- Probieren wir das am Beispiel `1e18 + 1 - 1e18` aus.

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Als erste Zahl addieren wir `1e18` zur Summe.

Kahan-Summe: Ausprobieren

- In dem wir den folgenden, einfachen Algorithmus anwenden:

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Für jede Zahl, die zur Summe dazuaddiert werden soll, machen wir die folgende Schritte:
   - 2.1 Wir berechnen zuerst `t = sum + value`.
   - 2.2 Dann berechnen wir `error = (sum - t)+ value`.
   - 2.3 Dann setzen wir `sum = t`.
   - 2.4 Wir addieren die Fehler in `cs` auf, setzen also `cs += error`.
3. Das Endergebnis ist dann `sum + cs`.

- Probieren wir das am Beispiel `1e18 + 1 - 1e18` aus.

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Als erste Zahl addieren wir `1e18` zur Summe.
   - 2.1 In ersten Schritt berechnen wir `t = sum + value`, also `t = 0 + 1e18`, also `t = 1e18`.

Kahan-Summe: Ausprobieren

- In dem wir den folgenden, einfachen Algorithmus anwenden:

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Für jede Zahl, die zur Summe dazuaddiert werden soll, machen wir die folgende Schritte:
   - 2.1 Wir berechnen zuerst `t = sum + value`.
   - 2.2 Dann berechnen wir `error = (sum - t)+ value`.
   - 2.3 Dann setzen wir `sum = t`.
   - 2.4 Wir addieren die Fehler in `cs` auf, setzen also `cs += error`.
3. Das Endergebnis ist dann `sum + cs`.

- Probieren wir das am Beispiel `1e18 + 1 - 1e18` aus.

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Als erste Zahl addieren wir `1e18` zur Summe.
   - 2.1 In ersten Schritt berechnen wir `t = sum + value`, also `t = 0 + 1e18`, also `t = 1e18`.
   - 2.2 Dann setzen wir `error = (sum - t)+ value`, woraus dann `error = (0 - 1e18)+ 1e18` wird, was uns `error = 0.0` gibt.

Kahan-Summe: Ausprobieren

- In dem wir den folgenden, einfachen Algorithmus anwenden:

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Für jede Zahl, die zur Summe dazuaddiert werden soll, machen wir die folgende Schritte:
   - 2.1 Wir berechnen zuerst `t = sum + value`.
   - 2.2 Dann berechnen wir `error = (sum - t)+ value`.
   - 2.3 Dann setzen wir `sum = t`.
   - 2.4 Wir addieren die Fehler in `cs` auf, setzen also `cs += error`.
3. Das Endergebnis ist dann `sum + cs`.

- Probieren wir das am Beispiel `1e18 + 1 - 1e18` aus.

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Als erste Zahl addieren wir `1e18` zur Summe.
   - 2.1 In ersten Schritt berechnen wir `t = sum + value`, also `t = 0 + 1e18`, also `t = 1e18`.
   - 2.2 Dann setzen wir `error = (sum - t)+ value`, woraus dann `error = (0 - 1e18)+ 1e18` wird, was uns `error = 0.0` gibt.
   - 2.3 Wir setzen also `sum = 1e18`, also `sum = 1e18`.

Kahan-Summe: Ausprobieren

- In dem wir den folgenden, einfachen Algorithmus anwenden:

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Für jede Zahl, die zur Summe dazuaddiert werden soll, machen wir die folgende Schritte:
   - 2.1 Wir berechnen zuerst `t = sum + value`.
   - 2.2 Dann berechnen wir `error = (sum - t)+ value`.
   - 2.3 Dann setzen wir `sum = t`.
   - 2.4 Wir addieren die Fehler in `cs` auf, setzen also `cs += error`.
3. Das Endergebnis ist dann `sum + cs`.

- Probieren wir das am Beispiel `1e18 + 1 - 1e18` aus.

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Als erste Zahl addieren wir `1e18` zur Summe.
   - 2.1 In ersten Schritt berechnen wir `t = sum + value`, also `t = 0 + 1e18`, also `t = 1e18`.
   - 2.2 Dann setzen wir `error = (sum - t)+ value`, woraus dann `error = (0 - 1e18)+ 1e18` wird, was uns `error = 0.0` gibt.
   - 2.3 Wir setzen also `sum = 1e18`, also `sum = 1e18`.
   - 2.4 `cs`, was `0` war, wird `0 + 0.0`, und ist somit `cs = 0.0`.

Kahan-Summe: Ausprobieren

- In dem wir den folgenden, einfachen Algorithmus anwenden:

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Für jede Zahl, die zur Summe dazuaddiert werden soll, machen wir die folgende Schritte:
   - 2.1 Wir berechnen zuerst `t = sum + value`.
   - 2.2 Dann berechnen wir `error = (sum - t)+ value`.
   - 2.3 Dann setzen wir `sum = t`.
   - 2.4 Wir addieren die Fehler in `cs` auf, setzen also `cs += error`.
3. Das Endergebnis ist dann `sum + cs`.

- Probieren wir das am Beispiel `1e18 + 1 - 1e18` aus.

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Als erste Zahl addieren wir `1e18` zur Summe.
3. Jetzt addieren wir `1` zu der Summe.

Kahan-Summe: Ausprobieren

- In dem wir den folgenden, einfachen Algorithmus anwenden:

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Für jede Zahl, die zur Summe dazuaddiert werden soll, machen wir die folgende Schritte:
   - 2.1 Wir berechnen zuerst `t = sum + value`.
   - 2.2 Dann berechnen wir `error = (sum - t)+ value`.
   - 2.3 Dann setzen wir `sum = t`.
   - 2.4 Wir addieren die Fehler in `cs` auf, setzen also `cs += error`.
3. Das Endergebnis ist dann `sum + cs`.

- Probieren wir das am Beispiel `1e18 + 1 - 1e18` aus.

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Als erste Zahl addieren wir `1e18` zur Summe.
3. Jetzt addieren wir `1` zu der Summe.
   - 3.1 Wir berechnen `t = sum + value`, was nun `t = 1e18 + 1` ist, was uns dann `t = 1e18` gibt.

Kahan-Summe: Ausprobieren

- In dem wir den folgenden, einfachen Algorithmus anwenden:

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Für jede Zahl, die zur Summe dazuaddiert werden soll, machen wir die folgende Schritte:
   - 2.1 Wir berechnen zuerst `t = sum + value`.
   - 2.2 Dann berechnen wir `error = (sum - t)+ value`.
   - 2.3 Dann setzen wir `sum = t`.
   - 2.4 Wir addieren die Fehler in `cs` auf, setzen also `cs += error`.
3. Das Endergebnis ist dann `sum + cs`.

- Probieren wir das am Beispiel `1e18 + 1 - 1e18` aus.

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Als erste Zahl addieren wir `1e18` zur Summe.
3. Jetzt addieren wir `1` zu der Summe.
   - 3.1 Wir berechnen `t = sum + value`, was nun `t = 1e18 + 1` ist, was uns dann `t = 1e18` gibt. Die `1` geht verloren.

Kahan-Summe: Ausprobieren

- In dem wir den folgenden, einfachen Algorithmus anwenden:

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Für jede Zahl, die zur Summe dazuaddiert werden soll, machen wir die folgende Schritte:
   - 2.1 Wir berechnen zuerst `t = sum + value`.
   - 2.2 Dann berechnen wir `error = (sum - t)+ value`.
   - 2.3 Dann setzen wir `sum = t`.
   - 2.4 Wir addieren die Fehler in `cs` auf, setzen also `cs += error`.
3. Das Endergebnis ist dann `sum + cs`.

- Probieren wir das am Beispiel `1e18 + 1 - 1e18` aus.

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Als erste Zahl addieren wir `1e18` zur Summe.
3. Jetzt addieren wir `1` zu der Summe.
   - 3.1 Wir berechnen `t = sum + value`, was nun `t = 1e18 + 1` ist, was uns dann `t = 1e18` gibt. Die `1` geht verloren.
   - 3.2 Also fast zumindest.

Kahan-Summe: Ausprobieren

- In dem wir den folgenden, einfachen Algorithmus anwenden:

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Für jede Zahl, die zur Summe dazuaddiert werden soll, machen wir die folgende Schritte:
   - 2.1 Wir berechnen zuerst `t = sum + value`.
   - 2.2 Dann berechnen wir `error = (sum - t)+ value`.
   - 2.3 Dann setzen wir `sum = t`.
   - 2.4 Wir addieren die Fehler in `cs` auf, setzen also `cs += error`.
3. Das Endergebnis ist dann `sum + cs`.

- Probieren wir das am Beispiel `1e18 + 1 - 1e18` aus.

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Als erste Zahl addieren wir `1e18` zur Summe.
3. Jetzt addieren wir `1` zu der Summe.
   - 3.1 Wir berechnen `t = sum + value`, was nun `t = 1e18 + 1` ist, was uns dann `t = 1e18` gibt. Die `1` geht verloren.
   - 3.2 Also fast zumindest: `error = (sum - t)+ value` wird zu `error = (1e18 - 1e18)+ 1`, also ist `error = 1.0`.

Kahan-Summe: Ausprobieren

- In dem wir den folgenden, einfachen Algorithmus anwenden:

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Für jede Zahl, die zur Summe dazuaddiert werden soll, machen wir die folgende Schritte:
   - 2.1 Wir berechnen zuerst `t = sum + value`.
   - 2.2 Dann berechnen wir `error = (sum - t)+ value`.
   - 2.3 Dann setzen wir `sum = t`.
   - 2.4 Wir addieren die Fehler in `cs` auf, setzen also `cs += error`.
3. Das Endergebnis ist dann `sum + cs`.

- Probieren wir das am Beispiel `1e18 + 1 - 1e18` aus.

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Als erste Zahl addieren wir `1e18` zur Summe.
3. Jetzt addieren wir `1` zu der Summe.
   - 3.1 Wir berechnen `t = sum + value`, was nun `t = 1e18 + 1` ist, was uns dann `t = 1e18` gibt. Die `1` geht verloren.
   - 3.2 Also fast zumindest: `error = (sum - t)+ value` wird zu `error = (1e18 - 1e18)+ 1`, also ist `error = 1.0`.
   - 3.3 Nach diesem Schritt ist `sum = 1e18`.

Kahan-Summe: Ausprobieren

- In dem wir den folgenden, einfachen Algorithmus anwenden:

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Für jede Zahl, die zur Summe dazuaddiert werden soll, machen wir die folgende Schritte:
   - 2.1 Wir berechnen zuerst `t = sum + value`.
   - 2.2 Dann berechnen wir `error = (sum - t)+ value`.
   - 2.3 Dann setzen wir `sum = t`.
   - 2.4 Wir addieren die Fehler in `cs` auf, setzen also `cs += error`.
3. Das Endergebnis ist dann `sum + cs`.

- Probieren wir das am Beispiel `1e18 + 1 - 1e18` aus.

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Als erste Zahl addieren wir `1e18` zur Summe.
3. Jetzt addieren wir `1` zu der Summe.
   - 3.1 Wir berechnen `t = sum + value`, was nun `t = 1e18 + 1` ist, was uns dann `t = 1e18` gibt. Die `1` geht verloren.
   - 3.2 Also fast zumindest: `error = (sum - t)+ value` wird zu `error = (1e18 - 1e18)+ 1`, also ist `error = 1.0`.
   - 3.3 Nach diesem Schritt ist `sum = 1e18`.
   - 3.4 Und `cs = 0 + 1.0` wird `1.0`.

Kahan-Summe: Ausprobieren

- In dem wir den folgenden, einfachen Algorithmus anwenden:

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Für jede Zahl, die zur Summe dazuaddiert werden soll, machen wir die folgende Schritte:
   - 2.1 Wir berechnen zuerst `t = sum + value`.
   - 2.2 Dann berechnen wir `error = (sum - t)+ value`.
   - 2.3 Dann setzen wir `sum = t`.
   - 2.4 Wir addieren die Fehler in `cs` auf, setzen also `cs += error`.
3. Das Endergebnis ist dann `sum + cs`.

- Probieren wir das am Beispiel `1e18 + 1 - 1e18` aus.

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Als erste Zahl addieren wir `1e18` zur Summe.
3. Jetzt addieren wir `1` zu der Summe.
4. Wir ziehen nun `1e18` von der Summe ab, was das Gleiche wie `-1e18` dazuaddieren ist.

Kahan-Summe: Ausprobieren

- In dem wir den folgenden, einfachen Algorithmus anwenden:

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Für jede Zahl, die zur Summe dazuaddiert werden soll, machen wir die folgende Schritte:
   - 2.1 Wir berechnen zuerst `t = sum + value`.
   - 2.2 Dann berechnen wir `error = (sum - t)+ value`.
   - 2.3 Dann setzen wir `sum = t`.
   - 2.4 Wir addieren die Fehler in `cs` auf, setzen also `cs += error`.
3. Das Endergebnis ist dann `sum + cs`.

- Probieren wir das am Beispiel `1e18 + 1 - 1e18` aus.

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Als erste Zahl addieren wir `1e18` zur Summe.
3. Jetzt addieren wir `1` zu der Summe.
4. Wir ziehen nun `1e18` von der Summe ab, was das Gleiche wie `-1e18` dazuaddieren ist.
   - 4.1 Das bedeutet, dass wir zuerst `t = sum + value` berechnen, also `t = 1e18 + -1e18`, woraus dann `t = 0.0` wird.

Kahan-Summe: Ausprobieren

- In dem wir den folgenden, einfachen Algorithmus anwenden:

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Für jede Zahl, die zur Summe dazuaddiert werden soll, machen wir die folgende Schritte:
   - 2.1 Wir berechnen zuerst `t = sum + value`.
   - 2.2 Dann berechnen wir `error = (sum - t)+ value`.
   - 2.3 Dann setzen wir `sum = t`.
   - 2.4 Wir addieren die Fehler in `cs` auf, setzen also `cs += error`.
3. Das Endergebnis ist dann `sum + cs`.

- Probieren wir das am Beispiel `1e18 + 1 - 1e18` aus.

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Als erste Zahl addieren wir `1e18` zur Summe.
3. Jetzt addieren wir `1` zu der Summe.
4. Wir ziehen nun `1e18` von der Summe ab, was das Gleiche wie `-1e18` dazuaddieren ist.
   - 4.1 Das bedeutet, dass wir zuerst `t = sum + value` berechnen, also `t = 1e18 + -1e18`, woraus dann `t = 0.0` wird.
   - 4.2 Für den Fehlerterm bekommen wir `error = (1e18 - 0.0)+ -1e18`, was `error = 0.0` ergibt.

Kahan-Summe: Ausprobieren

- In dem wir den folgenden, einfachen Algorithmus anwenden:

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Für jede Zahl, die zur Summe dazuaddiert werden soll, machen wir die folgende Schritte:
   - 2.1 Wir berechnen zuerst `t = sum + value`.
   - 2.2 Dann berechnen wir `error = (sum - t)+ value`.
   - 2.3 Dann setzen wir `sum = t`.
   - 2.4 Wir addieren die Fehler in `cs` auf, setzen also `cs += error`.
3. Das Endergebnis ist dann `sum + cs`.

- Probieren wir das am Beispiel `1e18 + 1 - 1e18` aus.

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Als erste Zahl addieren wir `1e18` zur Summe.
3. Jetzt addieren wir `1` zu der Summe.
4. Wir ziehen nun `1e18` von der Summe ab, was das Gleiche wie `-1e18` dazuaddieren ist.
   - 4.1 Das bedeutet, dass wir zuerst `t = sum + value` berechnen, also `t = 1e18 + -1e18`, woraus dann `t = 0.0` wird.
   - 4.2 Für den Fehlerterm bekommen wir `error = (1e18 - 0.0)+ -1e18`, was `error = 0.0` ergibt.
   - 4.3 Wir bekommen nun `sum = 0.0`.

Kahan-Summe: Ausprobieren

- In dem wir den folgenden, einfachen Algorithmus anwenden:

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Für jede Zahl, die zur Summe dazuaddiert werden soll, machen wir die folgende Schritte:
   - 2.1 Wir berechnen zuerst `t = sum + value`.
   - 2.2 Dann berechnen wir `error = (sum - t)+ value`.
   - 2.3 Dann setzen wir `sum = t`.
   - 2.4 Wir addieren die Fehler in `cs` auf, setzen also `cs += error`.
3. Das Endergebnis ist dann `sum + cs`.

- Probieren wir das am Beispiel `1e18 + 1 - 1e18` aus.

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Als erste Zahl addieren wir `1e18` zur Summe.
3. Jetzt addieren wir `1` zu der Summe.
4. Wir ziehen nun `1e18` von der Summe ab, was das Gleiche wie `-1e18` dazuaddieren ist.
   - 4.1 Das bedeutet, dass wir zuerst `t = sum + value` berechnen, also `t = 1e18 + -1e18`, woraus dann `t = 0.0` wird.
   - 4.2 Für den Fehlerterm bekommen wir `error = (1e18 - 0.0)+ -1e18`, was `error = 0.0` ergibt.
   - 4.3 Wir bekommen nun `sum = 0.0`.
   - 4.4 Und `cs = 1.0 + 0.0`, beleibt also bei `cs = 1.0`.

Kahan-Summe: Ausprobieren

- In dem wir den folgenden, einfachen Algorithmus anwenden:

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Für jede Zahl, die zur Summe dazuaddiert werden soll, machen wir die folgende Schritte:
   - 2.1 Wir berechnen zuerst `t = sum + value`.
   - 2.2 Dann berechnen wir `error = (sum - t)+ value`.
   - 2.3 Dann setzen wir `sum = t`.
   - 2.4 Wir addieren die Fehler in `cs` auf, setzen also `cs += error`.
3. Das Endergebnis ist dann `sum + cs`.

- Probieren wir das am Beispiel `1e18 + 1 - 1e18` aus.

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Als erste Zahl addieren wir `1e18` zur Summe.
3. Jetzt addieren wir `1` zu der Summe.
4. Wir ziehen nun `1e18` von der Summe ab, was das Gleiche wie `-1e18` dazuaddieren ist.
5. Das Endergebnis ist dann `sum + cs` und das ergibt tatsächlich `0.0 + 1.0`, also `1.0`!

Kahan-Summe: Ausprobieren

- In dem wir den folgenden, einfachen Algorithmus anwenden:

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Für jede Zahl, die zur Summe dazuaddiert werden soll, machen wir die folgende Schritte:
   - 2.1 Wir berechnen zuerst `t = sum + value`.
   - 2.2 Dann berechnen wir `error = (sum - t)+ value`.
   - 2.3 Dann setzen wir `sum = t`.
   - 2.4 Wir addieren die Fehler in `cs` auf, setzen also `cs += error`.
3. Das Endergebnis ist dann `sum + cs`.

- Probieren wir das am Beispiel `1e18 + 1 - 1e18` aus.

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Als erste Zahl addieren wir `1e18` zur Summe.
3. Jetzt addieren wir `1` zu der Summe.
4. Wir ziehen nun `1e18` von der Summe ab, was das Gleiche wie `-1e18` dazuaddieren ist.
5. Das Endergebnis ist dann `sum + cs` und das ergibt tatsächlich `0.0 + 1.0`, also `1.0`!

- Mit der Kahan-Summe konnten wir also `1e18 + 1 - 1e18` berechnen und erhalten korrekt `1.0`.

Kahan-Summe: Ausprobieren

- In dem wir den folgenden, einfachen Algorithmus anwenden:

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Für jede Zahl, die zur Summe dazuaddiert werden soll, machen wir die folgende Schritte:
   - 2.1 Wir berechnen zuerst `t = sum + value`.
   - 2.2 Dann berechnen wir `error = (sum - t)+ value`.
   - 2.3 Dann setzen wir `sum = t`.
   - 2.4 Wir addieren die Fehler in `cs` auf, setzen also `cs += error`.
3. Das Endergebnis ist dann `sum + cs`.

- Probieren wir das am Beispiel `1e18 + 1 - 1e18` aus.

1. Wir beginnen mit `sum = 0` und `cs = 0`.
2. Als erste Zahl addieren wir `1e18` zur Summe.
3. Jetzt addieren wir `1` zu der Summe.
4. Wir ziehen nun `1e18` von der Summe ab, was das Gleiche wie `-1e18` dazuaddieren ist.
5. Das Endergebnis ist dann `sum + cs` und das ergibt tatsächlich `0.0 + 1.0`, also `1.0`!

- Mit der Kahan-Summe konnten wir also `1e18 + 1 - 1e18` berechnen und erhalten korrekt `1.0`.
- Der Preis ist, dass wie zusätzliche Variablen brauchen und diese bei jedem Rechnschritt updaten müssen.

Kahan-Summe: Algorithmus



- Es gibt eine Vielzahl von verschiedenen Implementierungen dieser sogenannten Kahan-Summe der Kahan-Babuška-Summe.

Kahan-Summe: Algorithmus

- Es gibt eine Vielzahl von verschiedenen Implementierungen dieser sogenannten Kahan-Summe der Kahan-Babuška-Summe.
- Neumaier, z. B., hat 1974 eine Verbesserung für den Fall, dass die laufende Summe kleiner als die zu addierende Zahl ist, beigesteuert[224].

Algorithm 1: Die Kahan-Babuška-Neumaier Summe Zweiter Ordnung [10,164,167,224] über einen Array x mit n Zahlen, nach der Spezifikation von Klein[167].

$sum \leftarrow 0; \quad cs \leftarrow 0; \quad ccs \leftarrow 0;$
for $i \in 0..n-1$ **do** ▷ *Für jede der n zu addierenden Zahlen in x.*
  $t \leftarrow sum + x[i];$ ▷ *Summe und Fehlerterm 1.Ordnung (unten)[10,164].*
  if $|sum| \geq |x[i]|$ **then** $c \leftarrow (sum - t) + x[i];$ ▷ *Neumaier's Verbesserung[224]*
  else $c \leftarrow (x[i] - t) + sum;$
  $sum \leftarrow t;$ ▷ *Die Gesamtsumme wird upgedated.*
  $t \leftarrow cs + c;$ ▷ *Fehlersumme 2. Ordnung von Klein[167] beginnt.*
  if $|cs| \geq |c|$ **then** $cc \leftarrow (cs - t) + c;$ ▷ *Neumaier's Verbesserung[224]*
  else $c \leftarrow (c - t) + cs;$
  $cs \leftarrow t; \quad ccs \leftarrow ccs + cc;$
  return $sum + cs + ccs$

Kahan-Summe: Algorithmus

- Neumaier, z. B., hat 1974 eine Verbesserung für den Fall, dass die laufende Summe kleiner als die zu addierende Zahl ist, beigesteuert[224].
- Klein[167] hat dann die Präzision weiter verbessert, in dem eine Fehlersumme für die Fehlersumme eingeführt hat, also eine Fehlersumme 2. Ordnung, und so weiter.

Algorithm 1: Die Kahan-Babuška-Neumaier Summe Zweiter Ordnung [10,164,167,224] über einen Array x mit n Zahlen, nach der Spezifikation von Klein[167].

$sum \leftarrow 0; \quad cs \leftarrow 0; \quad ccs \leftarrow 0;$
for $i \in 0..n-1$ **do** ▷ *Für jede der n zu addierenden Zahlen in x.*
  $t \leftarrow sum + x[i];$ ▷ *Summe und Fehlerterm 1.Ordnung (unten)[10,164].*
  if $|sum| \geq |x[i]|$ **then** $c \leftarrow (sum - t) + x[i];$ ▷ *Neumaier's Verbesserung[224]*
  else $c \leftarrow (x[i] - t) + sum;$
  $sum \leftarrow t;$ ▷ *Die Gesamtsumme wird upgedated.*
  $t \leftarrow cs + c;$ ▷ *Fehlersumme 2. Ordnung von Klein[167] beginnt.*
  if $|cs| \geq |c|$ **then** $cc \leftarrow (cs - t) + c;$ ▷ *Neumaier's Verbesserung[224]*
  else $c \leftarrow (c - t) + cs;$
  $cs \leftarrow t; \quad ccs \leftarrow ccs + cc;$
  return $sum + cs + ccs$

Kahan-Summe: Algorithmus

- Klein[167] hat dann die Präzision weiter verbessert, in dem eine Fehlersumme für die Fehlersumme eingeführt hat, also eine Fehlersumme 2. Ordnung, und so weiter.
- Dieser verbesserte Algorithmus ist hier dargestellt.

Algorithm 1: Die Kahan-Babuška-Neumaier Summe Zweiter Ordnung [10,164,167,224] über einen Array x mit n Zahlen, nach der Spezifikation von Klein[167].

$sum \leftarrow 0$; $cs \leftarrow 0$; $ccs \leftarrow 0$;
for $i \in 0..n-1$ **do** ▷ *Für jede der n zu addierenden Zahlen in x.*
  $t \leftarrow sum + x[i]$; ▷ *Summe und Fehlerterm 1.Ordnung (unten)[10,164].*
  if $|sum| \geq |x[i]|$ **then** $c \leftarrow (sum - t) + x[i]$; ▷ *Neumaier's Verbesserung[224]*
  else $c \leftarrow (x[i] - t) + sum$;
  $sum \leftarrow t$; ▷ *Die Gesamtsumme wird upgedated.*
  $t \leftarrow cs + c$; ▷ *Fehlersumme 2. Ordnung von Klein[167] beginnt.*
  if $|cs| \geq |c|$ **then** $cc \leftarrow (cs - t) + c$; ▷ *Neumaier's Verbesserung[224]*
  else $c \leftarrow (c - t) + cs$;
  $cs \leftarrow t$; $ccs \leftarrow ccs + cc$;
  return $sum + cs + ccs$

Haben Sie keine Angst.

- Es ist nicht so wichtig, diesen Algorithmus hier im Detail zu diskutieren.

Haben Sie keine Angst.

- Es ist nicht so wichtig, diesen Algorithmus hier im Detail zu diskutieren.
- Im Grunde ist er ja nur eine weiterentwicklent Version der einfachen Kahan-Summe, die wir schon ausprobiert haben.

Haben Sie keine Angst.

- Es ist nicht so wichtig, diesen Algorithmus hier im Detail zu diskutieren.
- Im Grunde ist er ja nur eine weiterentwicklent Version der einfachen Kahan-Summe, die wir schon ausprobiert haben.
- Ich habe mit Absicht diese weiterentwickelte, gruselig aussehende Variante des Algorithmus ausgewählt.

Haben Sie keine Angst.



- Es ist nicht so wichtig, diesen Algorithmus hier im Detail zu diskutieren.
- Im Grunde ist er ja nur eine weiterentwicklent Version der einfachen Kahan-Summe, die wir schon ausprobiert haben.
- Ich habe mit Absicht diese weiterentwickelte, gruselig aussehende Variante des Algorithmus ausgewählt.
- Ich will nämlich einen wichtigen Punkt machen.

Haben Sie keine Angst.



- Es ist nicht so wichtig, diesen Algorithmus hier im Detail zu diskutieren.
- Im Grunde ist er ja nur eine weiterentwicklent Version der einfachen Kahan-Summe, die wir schon ausprobiert haben.
- Ich habe mit Absicht diese weiterentwickelte, gruselig aussehende Variante des Algorithmus ausgewählt.
- Ich will nämlich einen wichtigen Punkt machen:
- Ja. Manchmal sehen Algorithmen gefährlich und schwierig aus.

Haben Sie keine Angst.



- Es ist nicht so wichtig, diesen Algorithmus hier im Detail zu diskutieren.
- Im Grunde ist er ja nur eine weiterentwicklent Version der einfachen Kahan-Summe, die wir schon ausprobiert haben.
- Ich habe mit Absicht diese weiterentwickelte, gruselig aussehende Variante des Algorithmus ausgewählt.
- Ich will nämlich einen wichtigen Punkt machen:
- Ja. Manchmal sehen Algorithmen gefährlich und schwierig aus.
- Aber wenn wir der Definition ordentlich folgen, dann können wir sie trotzdem implementieren.

Haben Sie keine Angst.

- Es ist nicht so wichtig, diesen Algorithmus hier im Detail zu diskutieren.
- Im Grunde ist er ja nur eine weiterentwicklent Version der einfachen Kahan-Summe, die wir schon ausprobiert haben.
- Ich habe mit Absicht diese weiterentwickelte, gruselig aussehende Variante des Algorithmus ausgewählt.
- Ich will nämlich einen wichtigen Punkt machen:
- Ja. Manchmal sehen Algorithmen gefährlich und schwierig aus.
- Aber wenn wir der Definition ordentlich folgen, dann können wir sie trotzdem implementieren.
- Selbst wenn wir am Anfang nicht alles ganz verstehen.

Haben Sie keine Angst.



- Es ist nicht so wichtig, diesen Algorithmus hier im Detail zu diskutieren.
- Im Grunde ist er ja nur eine weiterentwicklent Version der einfachen Kahan-Summe, die wir schon ausprobiert haben.
- Ich habe mit Absicht diese weiterentwickelte, gruselig aussehende Variante des Algorithmus ausgewählt.
- Ich will nämlich einen wichtigen Punkt machen:
- Ja. Manchmal sehen Algorithmen gefährlich und schwierig aus.
- Aber wenn wir der Definition ordentlich folgen, dann können wir sie trotzdem implementieren.
- Selbst wenn wir am Anfang nicht alles ganz verstehen.
- Während wir den Algorithmus implementieren, also seine Komponenten auf Elemente der Programmiersprache übertragen, können wir lernen, ihn besser zu verstehen.

Haben Sie keine Angst.

- Es ist nicht so wichtig, diesen Algorithmus hier im Detail zu diskutieren.
- Im Grunde ist er ja nur eine weiterentwicklent Version der einfachen Kahan-Summe, die wir schon ausprobiert haben.
- Ich habe mit Absicht diese weiterentwickelte, gruselig aussehende Variante des Algorithmus ausgewählt.
- Ich will nämlich einen wichtigen Punkt machen:
- Ja. Manchmal sehen Algorithmen gefährlich und schwierig aus.
- Aber wenn wir der Definition ordentlich folgen, dann können wir sie trotzdem implementieren.
- Selbst wenn wir am Anfang nicht alles ganz verstehen.
- Während wir den Algorithmus implementieren, also seine Komponenten auf Elemente der Programmiersprache übertragen, können wir lernen, ihn besser zu verstehen.
- Wir können den Kode auch Zwischenergebnisse für von uns gewählte Beispiele ausdrucken lassen.

Haben Sie keine Angst.

- Es ist nicht so wichtig, diesen Algorithmus hier im Detail zu diskutieren.
- Im Grunde ist er ja nur eine weiterentwicklent Version der einfachen Kahan-Summe, die wir schon ausprobiert haben.
- Ich habe mit Absicht diese weiterentwickelte, gruselig aussehende Variante des Algorithmus ausgewählt.
- Ich will nämlich einen wichtigen Punkt machen:
- Ja. Manchmal sehen Algorithmen gefährlich und schwierig aus.
- Aber wenn wir der Definition ordentlich folgen, dann können wir sie trotzdem implementieren.
- Selbst wenn wir am Anfang nicht alles ganz verstehen.
- Während wir den Algorithmus implementieren, also seine Komponenten auf Elemente der Programmiersprache übertragen, können wir lernen, ihn besser zu verstehen.
- Wir können den Kode auch Zwischenergebnisse für von uns gewählte Beispiele ausdrucken lassen.
- Dann können wir den Algorithmus nachverfolgen und sein Verhalten besser verstehen.

Haben Sie keine Angst.

- Im Grunde ist er ja nur eine weiterentwicklent Version der einfachen Kahan-Summe, die wir schon ausprobiert haben.
- Ich habe mit Absicht diese weiterentwickelte, gruselig aussehende Variante des Algorithmus ausgewählt.
- Ich will nämlich einen wichtigen Punkt machen:
- Ja. Manchmal sehen Algorithmen gefährlich und schwierig aus.
- Aber wenn wir der Definition ordentlich folgen, dann können wir sie trotzdem implementieren.
- Selbst wenn wir am Anfang nicht alles ganz verstehen.
- Während wir den Algorithmus implementieren, also seine Komponenten auf Elemente der Programmiersprache übertragen, können wir lernen, ihn besser zu verstehen.
- Wir können den Kode auch Zwischenergebnisse für von uns gewählte Beispiele ausdrucken lassen.
- Dann können wir den Algorithmus nachverfolgen und sein Verhalten besser verstehen.
- Wir könnten ihn auch Schritt-für-Schritt mit einem Debugger ausführen (lernen wir später).

Haben Sie keine Angst.



- Ich habe mit Absicht diese weiterentwickelte, gruselig aussehende Variante des Algorithmus ausgewählt.
- Ich will nämlich einen wichtigen Punkt machen:
- Ja. Manchmal sehen Algorithmen gefährlich und schwierig aus.
- Aber wenn wir der Definition ordentlich folgen, dann können wir sie trotzdem implementieren.
- Selbst wenn wir am Anfang nicht alles ganz verstehen.
- Während wir den Algorithmus implementieren, also seine Komponenten auf Elemente der Programmiersprache übertragen, können wir lernen, ihn besser zu verstehen.
- Wir können den Kode auch Zwischenergebnisse für von uns gewählte Beispiele ausdrucken lassen.
- Dann können wir den Algorithmus nachverfolgen und sein Verhalten besser verstehen.
- Wir könnten ihn auch Schritt-für-Schritt mit einem Debugger ausführen (lernen wir später).
- Wenn wir eine klare Algorithmusdefinition haben, dann können wir diese nach Python übersetzen.

Haben Sie keine Angst.



- Ja. Manchmal sehen Algorithmen gefährlich und schwierig aus.
- Aber wenn wir der Definition ordentlich folgen, dann können wir sie trotzdem implementieren.
- Selbst wenn wir am Anfang nicht alles ganz verstehen.
- Während wir den Algorithmus implementieren, also seine Komponenten auf Elemente der Programmiersprache übertragen, können wir lernen, ihn besser zu verstehen.
- Wir können den Kode auch Zwischenergebnisse für von uns gewählte Beispiele ausdrucken lassen.
- Dann können wir den Algorithmus nachverfolgen und sein Verhalten besser verstehen.
- Wir könnten ihn auch Schritt-für-Schritt mit einem Debugger ausführen (lernen wir später).
- Wenn wir eine klare Algorithmusdefinition haben, dann können wir diese nach Python übersetzen.
- Und wir können davon lernen.

Design der Klasse

- Jetzt implementieren wir die Kahan-Babuška-Neumaier Summe Zweiter Ordnung in Datei `kahan_sum.py`.

$sum \leftarrow 0; \quad cs \leftarrow 0; \quad ccs \leftarrow 0;$
for $i \in 0..n-1$ **do**
  $t \leftarrow sum + x[i];$
  if $|sum| \geq |x[i]|$ **then** $c \leftarrow (sum - t) + x[i];$
  else $c \leftarrow (x[i] - t) + sum;$
  $sum \leftarrow t;$
  $t \leftarrow cs + c;$
  if $|cs| \geq |c|$ **then** $cc \leftarrow (cs - t) + c;$
  else $c \leftarrow (c - t) + cs;$
  $cs \leftarrow t; \quad ccs \leftarrow ccs + cc;$
  return $sum + cs + ccs$

Design der Klasse

- Jetzt implementieren wir die Kahan-Babuška-Neumaier Summe Zweiter Ordnung in Datei `kahan_sum.py`.
- Die Frage ist nun *Wie machen wir das?*.

$sum \leftarrow 0; \quad cs \leftarrow 0; \quad ccs \leftarrow 0;$
for $i \in 0..n-1$ **do**
  $t \leftarrow sum + x[i];$
  if $|sum| \geq |x[i]|$ **then** $c \leftarrow (sum - t) + x[i];$
  else $c \leftarrow (x[i] - t) + sum;$
  $sum \leftarrow t;$
  $t \leftarrow cs + c;$
  if $|cs| \geq |c|$ **then** $cc \leftarrow (cs - t) + c;$
  else $c \leftarrow (c - t) + cs;$
  $cs \leftarrow t; \quad ccs \leftarrow ccs + cc;$
  return $sum + cs + ccs$

Design der Klasse

- Jetzt implementieren wir die Kahan-Babuška-Neumaier Summe Zweiter Ordnung in Datei `kahan_sum.py`.
- Die Frage ist nun *Wie machen wir das?*.
- Wir könnten ihn als Funktion implementieren, die ein `Iterable x` mit den zu summierenden Zahlen als Parameter erhält.

$sum \leftarrow 0; \quad cs \leftarrow 0; \quad ccs \leftarrow 0;$
for $i \in 0..n-1$ **do**
  $t \leftarrow sum + x[i];$
  if $|sum| \geq |x[i]|$ **then** $c \leftarrow (sum - t) + x[i];$
  else $c \leftarrow (x[i] - t) + sum;$
  $sum \leftarrow t;$
  $t \leftarrow cs + c;$
  if $|cs| \geq |c|$ **then** $cc \leftarrow (cs - t) + c;$
  else $c \leftarrow (c - t) + cs;$
  $cs \leftarrow t; \quad ccs \leftarrow ccs + cc;$
  return $sum + cs + ccs$

Design der Klasse

- Die Frage ist nun *Wie machen wir das?*.
- Wir könnten ihn als Funktion implementieren, die ein `Iterable x` mit den zu summierenden Zahlen als Parameter erhält.
- So ist der Algorithmus ja im Grunde definiert.

$sum \leftarrow 0$; $cs \leftarrow 0$; $ccs \leftarrow 0$;
for $i \in 0..n-1$ **do**
  $t \leftarrow sum + x[i]$;
  if $|sum| \geq |x[i]|$ **then** $c \leftarrow (sum - t) + x[i]$;
  else $c \leftarrow (x[i] - t) + sum$;
  $sum \leftarrow t$;
  $t \leftarrow cs + c$;
  if $|cs| \geq |c|$ **then** $cc \leftarrow (cs - t) + c$;
  else $c \leftarrow (c - t) + cs$;
  $cs \leftarrow t$; $ccs \leftarrow ccs + cc$;
  return $sum + cs + ccs$

Design der Klasse

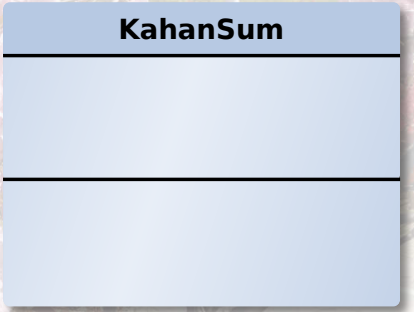
- Wir könnten ihn als Funktion implementieren, die ein `Iterable x` mit den zu summierenden Zahlen als Parameter erhält.
- So ist der Algorithmus ja im Grunde definiert.
- Wir entscheiden uns anders.

$sum \leftarrow 0; \quad cs \leftarrow 0; \quad ccs \leftarrow 0;$
for $i \in 0..n-1$ **do**
  $t \leftarrow sum + x[i];$
  if $|sum| \geq |x[i]|$ **then** $c \leftarrow (sum - t) + x[i];$
  else $c \leftarrow (x[i] - t) + sum;$
  $sum \leftarrow t;$
  $t \leftarrow cs + c;$
  if $|cs| \geq |c|$ **then** $cc \leftarrow (cs - t) + c;$
  else $c \leftarrow (c - t) + cs;$
  $cs \leftarrow t; \quad ccs \leftarrow ccs + cc;$
  return $sum + cs + ccs$

Design der Klasse

- So ist der Algorithmus ja im Grunde definiert.
- Wir entscheiden uns anders.
- Wir wollen ihn als Klasse `KahanSum` implementieren, die eine laufende Summe darstellt.

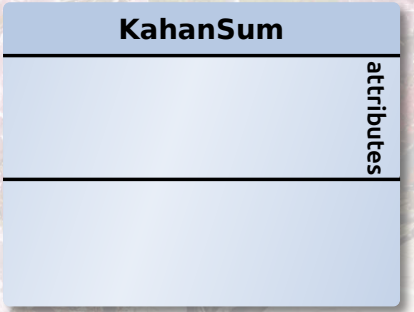
$sum \leftarrow 0; \quad cs \leftarrow 0; \quad ccs \leftarrow 0;$
for $i \in 0..n-1$ **do**
  $t \leftarrow sum + x[i];$
  if $|sum| \geq |x[i]|$ **then** $c \leftarrow (sum - t) + x[i];$
  else $c \leftarrow (x[i] - t) + sum;$
  $sum \leftarrow t;$
  $t \leftarrow cs + c;$
  if $|cs| \geq |c|$ **then** $cc \leftarrow (cs - t) + c;$
  else $c \leftarrow (c - t) + cs;$
  $cs \leftarrow t; \quad ccs \leftarrow ccs + cc;$
  return $sum + cs + ccs$



Design der Klasse

- Wir entscheiden uns anders.
- Wir wollen ihn als Klasse `KahanSum` implementieren, die eine laufende Summe darstellt.
- Eine Klasse hat Attribute.

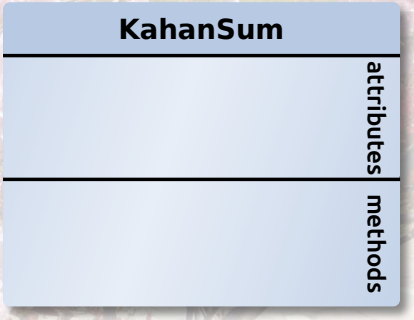
$sum \leftarrow 0; \quad cs \leftarrow 0; \quad ccs \leftarrow 0;$
for $i \in 0..n-1$ **do**
  $t \leftarrow sum + x[i];$
  if $|sum| \geq |x[i]|$ **then** $c \leftarrow (sum - t) + x[i];$
  else $c \leftarrow (x[i] - t) + sum;$
  $sum \leftarrow t;$
  $t \leftarrow cs + c;$
  if $|cs| \geq |c|$ **then** $cc \leftarrow (cs - t) + c;$
  else $c \leftarrow (c - t) + cs;$
  $cs \leftarrow t; \quad ccs \leftarrow ccs + cc;$
  return $sum + cs + ccs$



Design der Klasse

- Wir wollen ihn als Klasse `KahanSum` implementieren, die eine laufende Summe darstellt.
- Eine Klasse hat Attribute und Methoden.

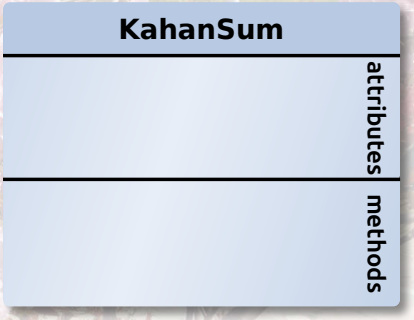
$sum \leftarrow 0; \quad cs \leftarrow 0; \quad ccs \leftarrow 0;$
for $i \in 0..n-1$ **do**
  $t \leftarrow sum + x[i];$
  if $|sum| \geq |x[i]|$ **then** $c \leftarrow (sum - t) + x[i];$
  else $c \leftarrow (x[i] - t) + sum;$
  $sum \leftarrow t;$
  $t \leftarrow cs + c;$
  if $|cs| \geq |c|$ **then** $cc \leftarrow (cs - t) + c;$
  else $c \leftarrow (c - t) + cs;$
  $cs \leftarrow t; \quad ccs \leftarrow ccs + cc;$
  return $sum + cs + ccs$



Design der Klasse

- Wir wollen ihn als Klasse `KahanSum` implementieren, die eine laufende Summe darstellt.
- Eine Klasse hat Attribute und Methoden.
- Schauen wir uns den Algorithmus genauer an.

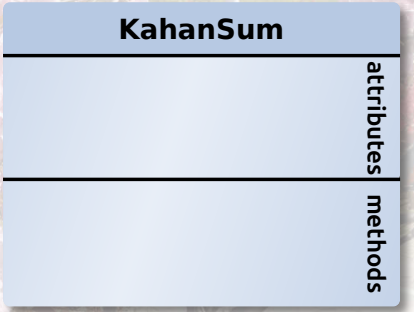
$sum \leftarrow 0; \quad cs \leftarrow 0; \quad ccs \leftarrow 0;$
for $i \in 0..n-1$ **do**
  $t \leftarrow sum + x[i];$
  if $|sum| \geq |x[i]|$ **then** $c \leftarrow (sum - t) + x[i];$
  else $c \leftarrow (x[i] - t) + sum;$
  $sum \leftarrow t;$
  $t \leftarrow cs + c;$
  if $|cs| \geq |c|$ **then** $cc \leftarrow (cs - t) + c;$
  else $c \leftarrow (c - t) + cs;$
  $cs \leftarrow t; \quad ccs \leftarrow ccs + cc;$
  return $sum + cs + ccs$



Design der Klasse

- Eine Klasse hat Attribute und Methoden.
- Schauen wir uns den Algorithmus genauer an.
- Er beginnt mit der Initialisierung der Zustandsvariablen sum, cs, und ccs.

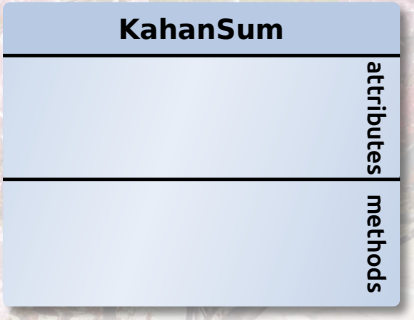
$sum \leftarrow 0; \quad cs \leftarrow 0; \quad ccs \leftarrow 0;$
for $i \in 0..n-1$ **do**
  $t \leftarrow sum + x[i];$
  if $|sum| \geq |x[i]|$ **then** $c \leftarrow (sum - t) + x[i];$
  else $c \leftarrow (x[i] - t) + sum;$
  $sum \leftarrow t;$
  $t \leftarrow cs + c;$
  if $|cs| \geq |c|$ **then** $cc \leftarrow (cs - t) + c;$
  else $c \leftarrow (c - t) + cs;$
  $cs \leftarrow t; \quad ccs \leftarrow ccs + cc;$
  return $sum + cs + ccs$



Design der Klasse

- Schauen wir uns den Algorithmus genauer an.
- Er beginnt mit der Initialisierung der Zustandsvariablen sum, cs, und ccs.
- Diese müsste man als Attribute der Instanzen der Klasse realisieren.

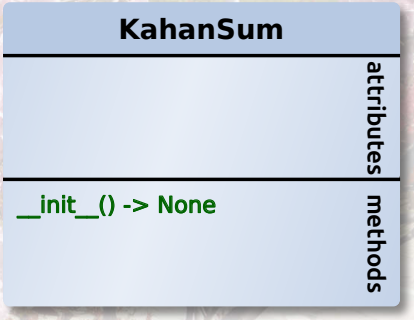
```
sum ← 0;   cs ← 0;   ccs ← 0;
for i ∈ 0..n − 1 do
    t ← sum + x[i];
    if |sum| ≥ |x[i]| then c ← (sum − t) + x[i];
    else c ← (x[i] − t) + sum;
    sum ← t;
    t ← cs + c;
    if |cs| ≥ |c| then cc ← (cs − t) + c;
    else c ← (c − t) + cs;
    cs ← t;   ccs ← ccs + cc;
    return sum + cs + ccs
```



Design der Klasse

- Er beginnt mit der Initialisierung der Zustandsvariablen sum, cs, und ccs.
- Diese müsste man als Attribute der Instanzen der Klasse realisieren.
- In einer Klasse kommt der Kode, der Attribute initialisiert, in den Initialisierer `__init__`.

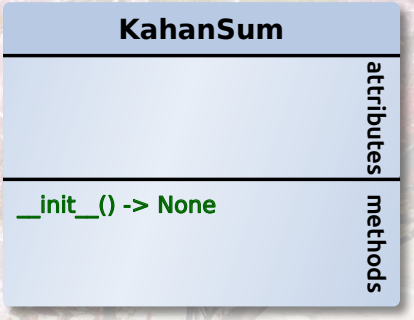
$sum \leftarrow 0;\quad cs \leftarrow 0;\quad ccs \leftarrow 0;$
for $i \in 0..n-1$ **do**
  $t \leftarrow sum + x[i];$
  if $|sum| \geq |x[i]|$ **then** $c \leftarrow (sum - t) + x[i];$
  else $c \leftarrow (x[i] - t) + sum;$
  $sum \leftarrow t;$
  $t \leftarrow cs + c;$
  if $|cs| \geq |c|$ **then** $cc \leftarrow (cs - t) + c;$
  else $c \leftarrow (c - t) + cs;$
  $cs \leftarrow t;\quad ccs \leftarrow ccs + cc;$
  return $sum + cs + ccs$



Design der Klasse

- Diese müsste man als Attribute der Instanzen der Klasse realisieren.
- In einer Klasse kommt der Kode, der Attribute initialisiert, in den Initialisierer `__init__`.
- Also tun wir diesen Teil des Algorithmus auch dort hin.

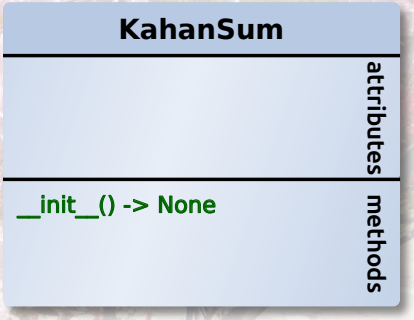
```
sum ← 0;   cs ← 0;   ccs ← 0;
for i ∈ 0..n − 1 do
    t ← sum + x[i];
    if |sum| ≥ |x[i]| then c ← (sum − t) + x[i];
    else c ← (x[i] − t) + sum;
    sum ← t;
    t ← cs + c;
    if |cs| ≥ |c| then cc ← (cs − t) + c;
    else c ← (c − t) + cs;
    cs ← t;   ccs ← ccs + cc;
    return sum + cs + ccs
```



Design der Klasse

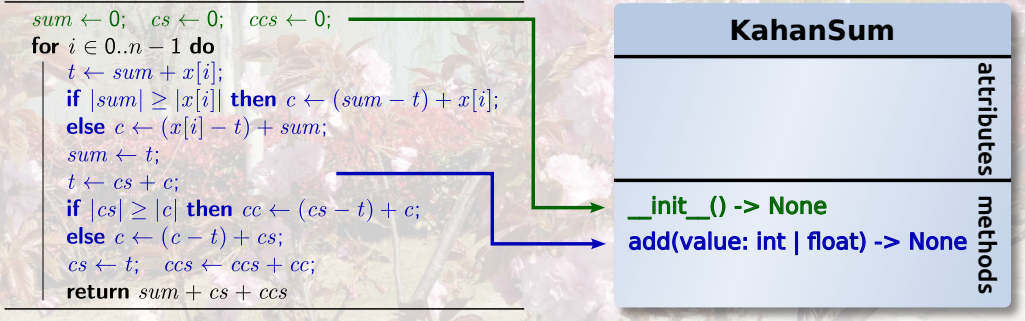
- In einer Klasse kommt der Kode, der Attribute initialisiert, in den Initialisierer `__init__`.
- Also tun wir diesen Teil des Algorithmus auch dort hin.
- In einer Schleife addiert der Algorithmus eine Zahl nach der anderen zu der Summe hinzu.

$sum \leftarrow 0; \quad cs \leftarrow 0; \quad ccs \leftarrow 0;$
for $i \in 0..n-1$ **do**
  $t \leftarrow sum + x[i];$
  if $|sum| \geq |x[i]|$ **then** $c \leftarrow (sum - t) + x[i];$
  else $c \leftarrow (x[i] - t) + sum;$
  $sum \leftarrow t;$
  $t \leftarrow cs + c;$
  if $|cs| \geq |c|$ **then** $cc \leftarrow (cs - t) + c;$
  else $c \leftarrow (c - t) + cs;$
  $cs \leftarrow t; \quad ccs \leftarrow ccs + cc;$
  return $sum + cs + ccs$



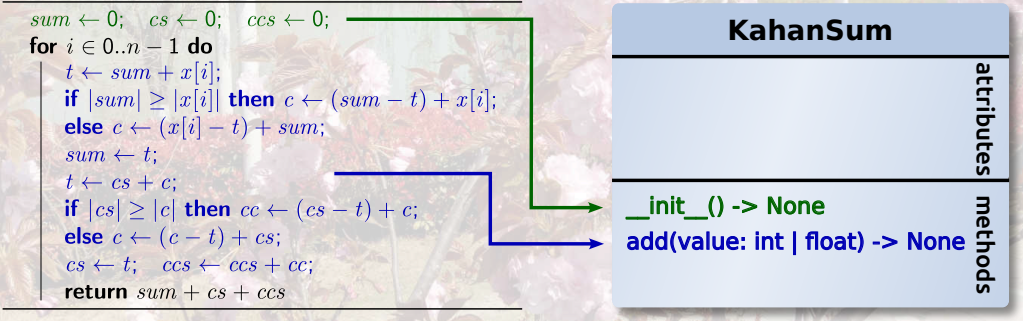
Design der Klasse

- Also tun wir diesen Teil des Algorithmus auch dort hin.
- In einer Schleife addiert der Algorithmus eine Zahl nach der anderen zu der Summe hinzu.
- Wir wollen eine Methode `add` implementieren, die jeweils genau eine Zahl zur Summe hinzuaddiert.



Design der Klasse

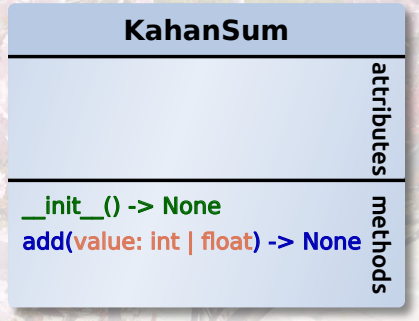
- In einer Schleife addiert der Algorithmus eine Zahl nach der anderen zu der Summe hinzu.
- Wir wollen eine Methode `add` implementieren, die jeweils genau eine Zahl zur Summe hinzuaddiert.
- Diese könnte ja dann genauso gut in einer Schleife aufgerufen werden.



Design der Klasse

- Wir wollen eine Methode `add` implementieren, die jeweils genau eine Zahl zur Summe hinzuaddiert.
- Diese könnte ja dann genauso gut in einer Schleife aufgerufen werden.
- Der Parameter `value` tritt dann an Stelle der Schleifenvariable $x[i]$.

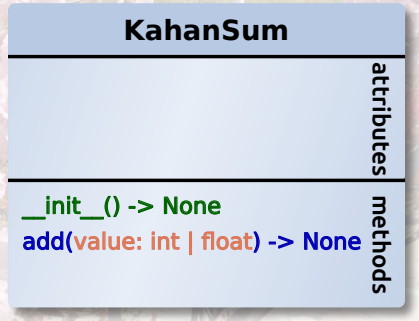
```
sum ← 0;   cs ← 0;   ccs ← 0;
for i ∈ 0..n − 1 do
    t ← sum + x[i];
    if |sum| ≥ |x[i]| then c ← (sum − t) + x[i];
    else c ← (x[i] − t) + sum;
    sum ← t;
    t ← cs + c;
    if |cs| ≥ |c| then cc ← (cs − t) + c;
    else c ← (c − t) + cs;
    cs ← t;   ccs ← ccs + cc;
    return sum + cs + ccs
```



Design der Klasse

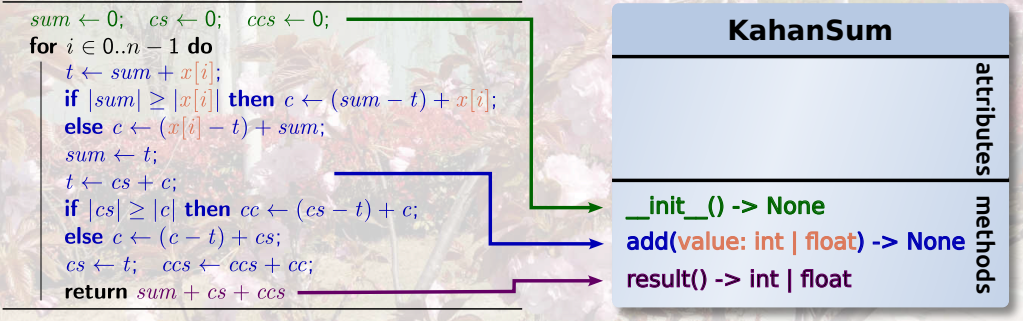
- Diese könnte ja dann genauso gut in einer Schleife aufgerufen werden.
- Der Parameter `value` tritt dann an Stelle der Schleifenvariable $x[i]$.
- Nach dem Ende der Schleife, am Ende des Algorithmus, wird die entgültige Summe berechnet.

```
sum ← 0;   cs ← 0;   ccs ← 0;
for i ∈ 0..n − 1 do
    t ← sum + x[i];
    if |sum| ≥ |x[i]| then c ← (sum − t) + x[i];
    else c ← (x[i] − t) + sum;
    sum ← t;
    t ← cs + c;
    if |cs| ≥ |c| then cc ← (cs − t) + c;
    else c ← (c − t) + cs;
    cs ← t;   ccs ← ccs + cc;
    return sum + cs + ccs
```



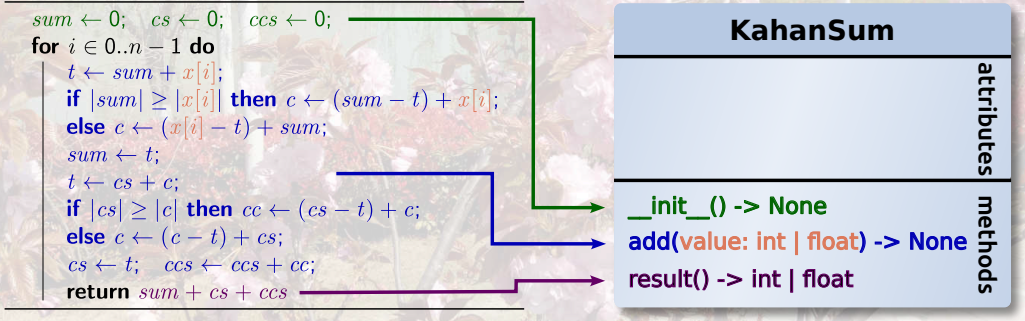
Design der Klasse

- Der Parameter `value` tritt dann an Stelle der Schleifenvariable $x[i]$.
- Nach dem Ende der Schleife, am Ende des Algorithmus, wird die entgültige Summe berechnet.
- Dazu werden die Summenvariable sum und die Fehlerterme 1. und 2. Ordnung (cs und ccs) addiert.



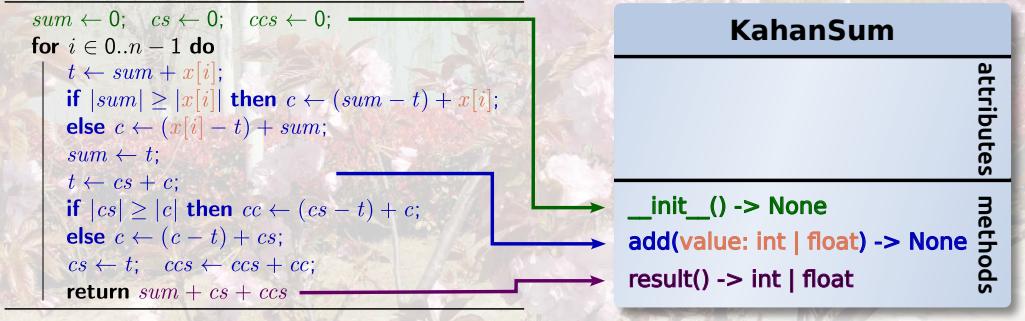
Design der Klasse

- Nach dem Ende der Schleife, am Ende des Algorithmus, wird die entgültige Summe berechnet.
- Dazu werden die Summenvariable sum und die Fehlerterme 1. und 2. Ordnung (cs und ccs) addiert.
- Wir packen das in eine Methode `result`, die das Ergebnis liefern soll.



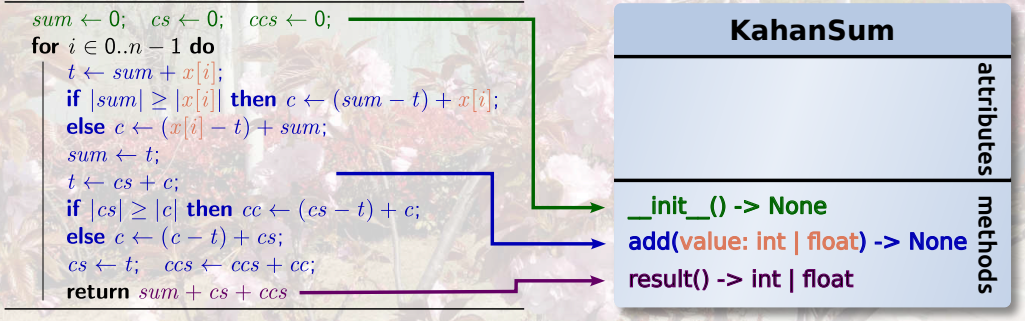
Design der Klasse

- Dazu werden die Summenvariable sum und die Fehlerterme 1. und 2. Ordnung (cs und ccs) addiert.
- Wir packen das in eine Methode `result`, die das Ergebnis liefern soll.
- Genaugenommen ist dieses „entgültige“ Ergebnis gar nicht so entgültig.



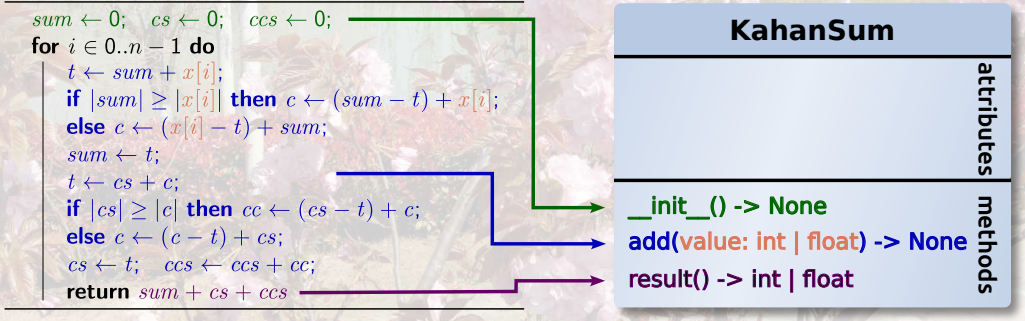
Design der Klasse

- Wir packen das in eine Methode `result`, die das Ergebnis liefern soll.
- Genaugenommen ist dieses „entgültige“ Ergebnis gar nicht so entgültig.
- Bei seiner Berechnung werden ja keine Attribute *verändert*.



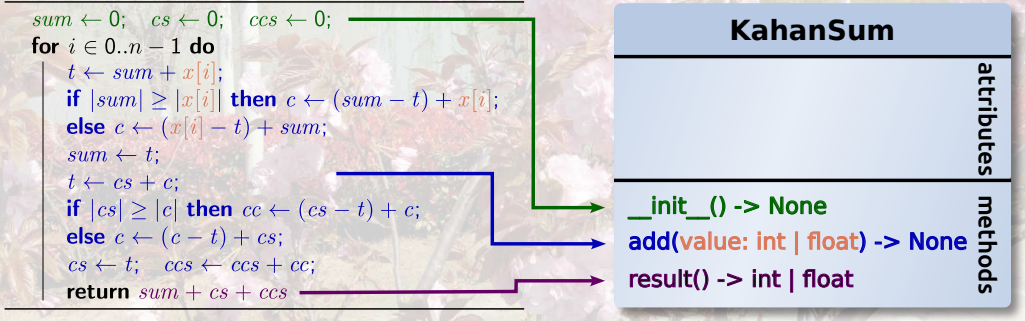
Design der Klasse

- Genaugenommen ist dieses „entgültige“ Ergebnis gar nicht so entgültig.
- Bei seiner Berechnung werden ja keine Attribute *verändert*.
- Wir könnten also einfach ein paar Zahlen mit `add` addieren, das Ergebnis mit `result` abfragen, dann noch ein paar Zahlen dazuaddieren, dann das Ergebnis nochmal abfragen.



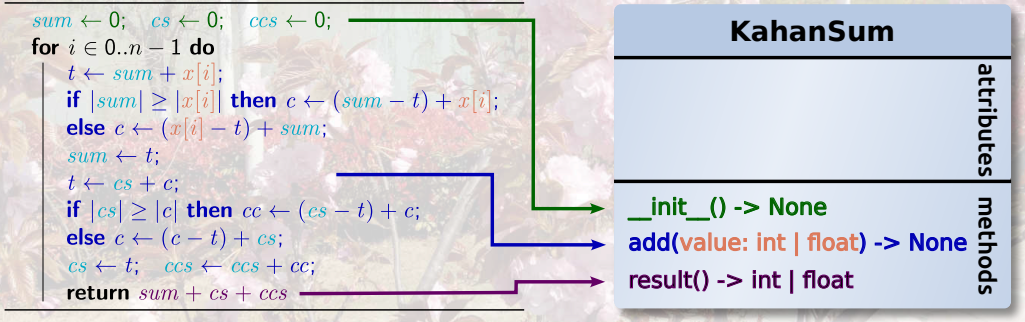
Design der Klasse

- Bei seiner Berechnung werden ja keine Attribute *verändert*.
- Wir könnten also einfach ein paar Zahlen mit `add` addieren, das Ergebnis mit `result` abfragen, dann noch ein paar Zahlen dazuaddieren, dann das Ergebnis nochmal abfragen.
- Wir haben eine wirkliche laufende Summe.



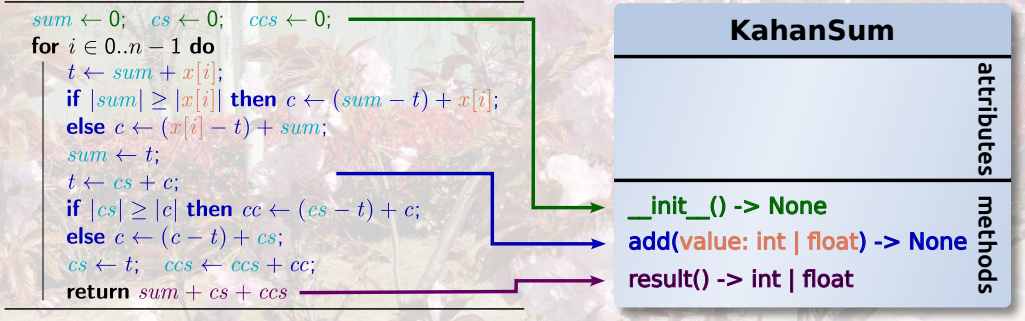
Design der Klasse

- Wir könnten also einfach ein paar Zahlen mit `add` addieren, das Ergebnis mit `result` abfragen, dann noch ein paar Zahlen dazuaddieren, dann das Ergebnis nochmal abfragen.
- Wir haben eine wirkliche laufende Summe.
- Was wir noch nicht diskutiert haben, sind die Attribute unserer Klasse.



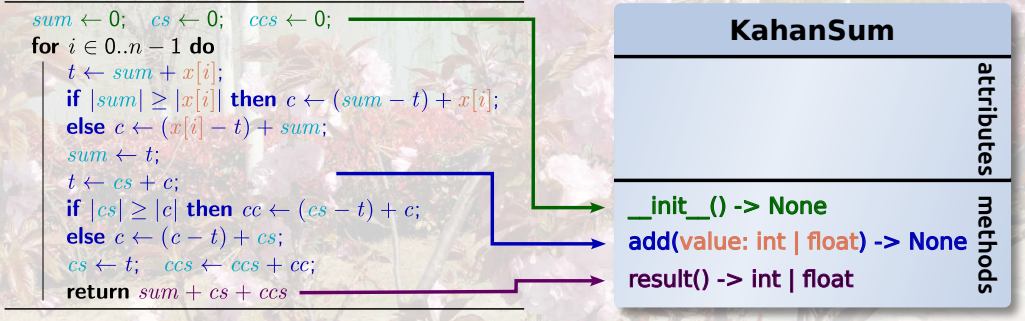
Design der Klasse

- Wir haben eine wirkliche laufende Summe.
- Was wir noch nicht diskutiert haben, sind die Attribute unserer Klasse.
- Wir brauchen Attribute für die Summe sum, den Fehlerterm erster Ordnung cs and den Fehlerterm zweiter Ordnung ccs.



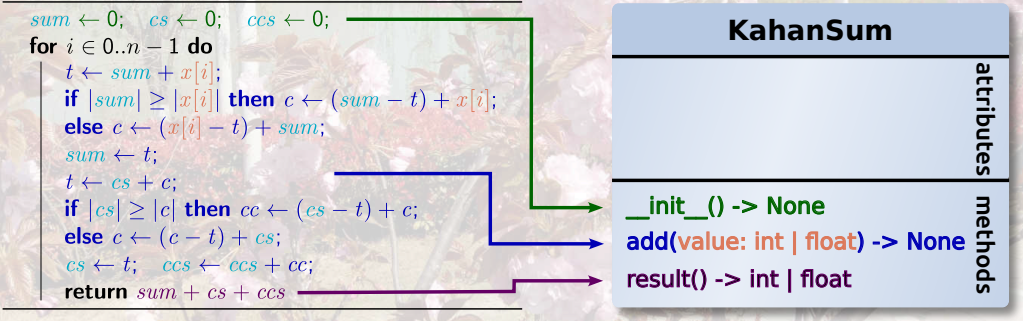
Design der Klasse

- Was wir noch nicht diskutiert haben, sind die Attribute unserer Klasse.
- Wir brauchen Attribute für die Summe sum, den Fehlerterm erster Ordnung cs and den Fehlerterm zweiter Ordnung ccs.
- Die Werte dieser Attribute ändern sich während der Summation, also sind sie nicht `Final`.



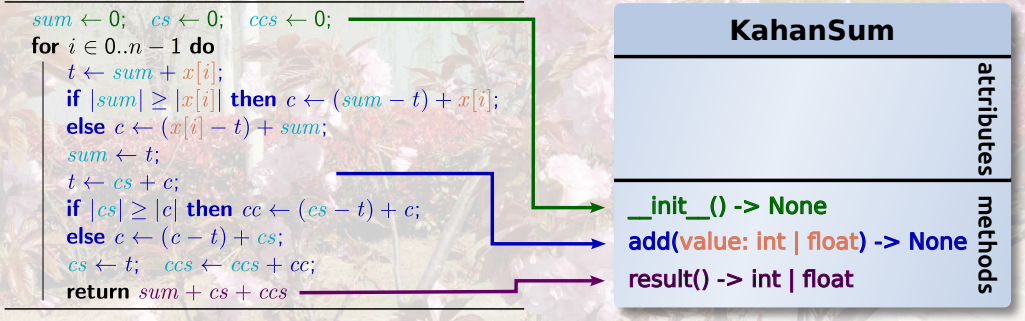
Design der Klasse

- Wir brauchen Attribute für die Summe sum, den Fehlerterm erster Ordnung cs and den Fehlerterm zweiter Ordnung ccs.
- Die Werte dieser Attribute ändern sich während der Summation, also sind sie nicht `Final`.
- Sie repräsentieren den *internen* Zustand unserer Summe.



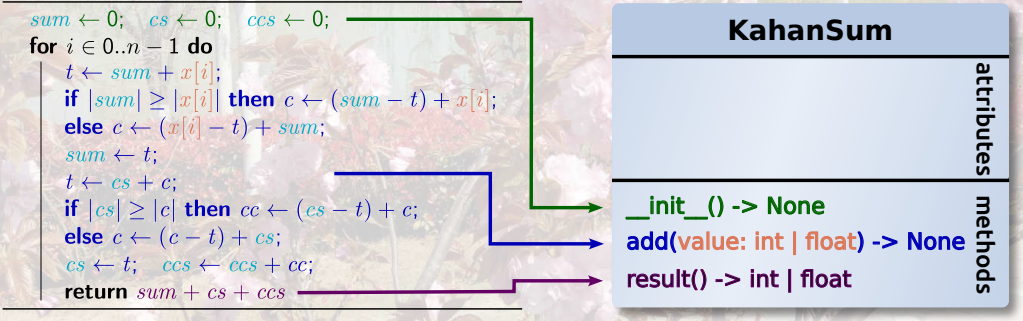
Design der Klasse

- Die Werte dieser Attribute ändern sich während der Summation, also sind sie nicht `Final`.
- Sie repräsentieren den *internen* Zustand unserer Summe.
- Sie sind bedeutungslos für jeden anderen, äußeren Kode.



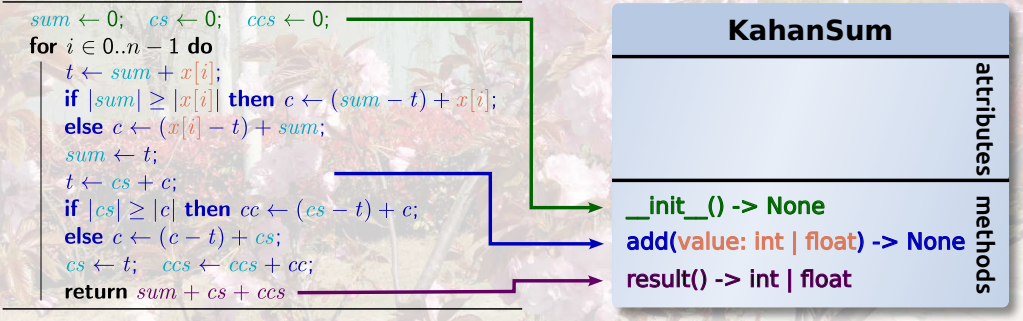
Design der Klasse

- Sie repräsentieren den *internen* Zustand unserer Summe.
- Sie sind bedeutungslos für jeden anderen, äußeren Kode.
- Wahrscheinlich kann niemand außer uns ihre Bedeutung verstehen.



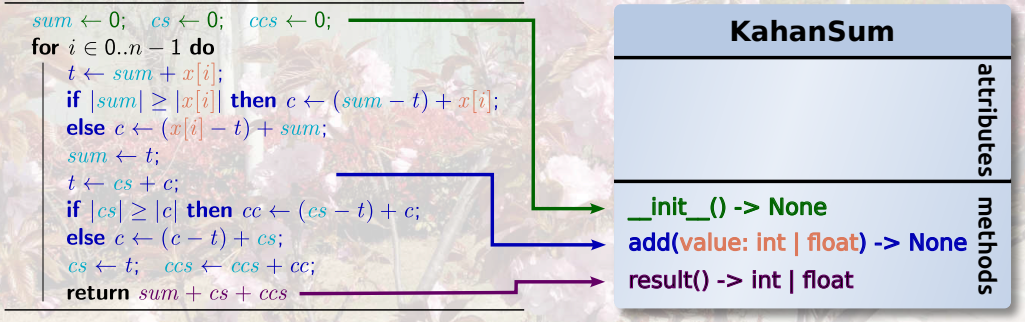
Design der Klasse

- Sie sind bedeutungslos für jeden anderen, äußeren Kode.
- Wahrscheinlich kann niemand außer uns ihre Bedeutung verstehen.
- Deshalb sollte auch niemand sie sehen oder gar verändern können.



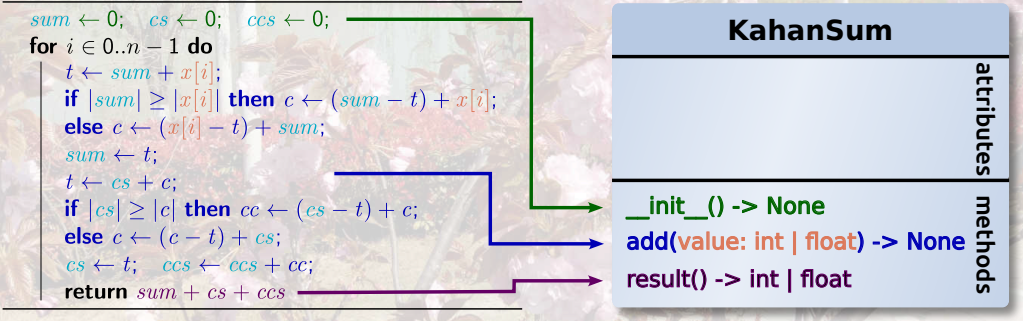
Design der Klasse

- Wahrscheinlich kann niemand außer uns ihre Bedeutung verstehen.
- Deshalb sollte auch niemand sie sehen oder gar verändern können.
- Denn dafür gibt es ja keinen Grund.



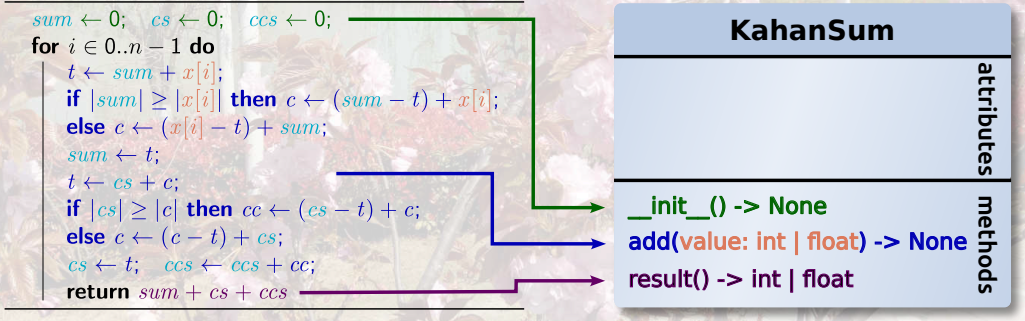
Design der Klasse

- Deshalb sollte auch niemand sie sehen oder gar verändern können.
- Denn dafür gibt es ja keinen Grund.
- Die Attribute sollten „*versteckt*“ sein.



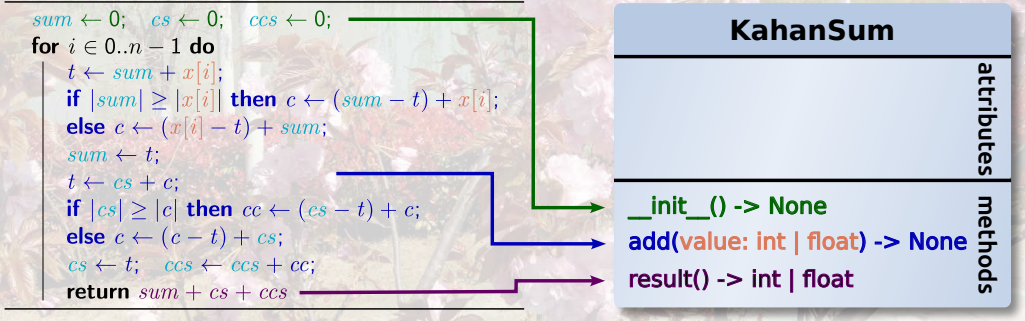
Design der Klasse

- Denn dafür gibt es ja keinen Grund.
- Die Attribute sollten „*versteckt*“ sein.
- Anders als bei den Attributen `x` und `y` unserer Klasse `Point`, hat der Benutzer (ein anderer Programmierer) keinen Grund, auf den (internen) Zustand unserer Summe zuzugreifen.



Design der Klasse

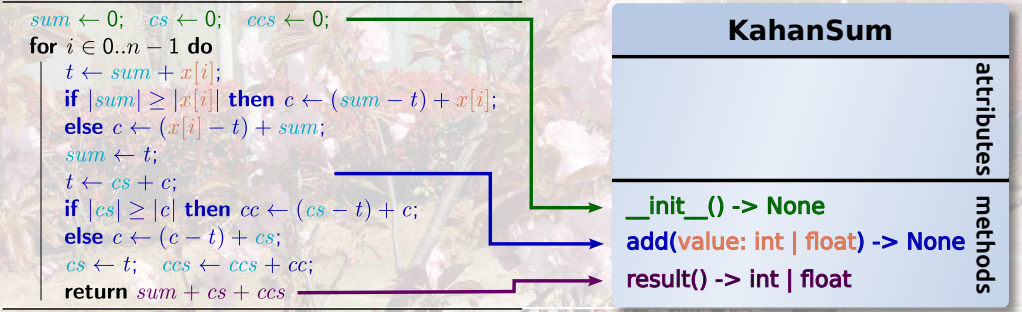
- Die Attribute sollten „*versteckt*“ sein.
- Anders als bei den Attributen `x` und `y` unserer Klasse `Point`, hat der Benutzer (ein anderer Programmierer) keinen Grund, auf den (internen) Zustand unserer Summe zuzugreifen.
- Stattdessen sollte er mit unseren Objekten nur über die Methoden `add` und `result` interagieren.



Design der Klasse

Definition: (Vollständige) Kapselung

Kapselung (EN: *encapsulation*) bedeutet, dass auf die Attribute eines Objekts nur durch seine Methoden zugegeriffen wird.

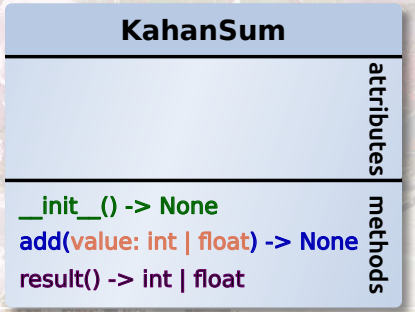


Design der Klasse

Definition: (Vollständige) Kapselung

Kapselung (EN: *encapsulation*) bedeutet, dass auf die Attribute eines Objekts nur durch seine Methoden zugegeriffen wird. Unter vollsändiger Kapselung (EN: *complete encapsulation*) können die Attribute eines Objekts nur durch seine Methoden gelesen oder verändert werden.

$sum \leftarrow 0$; $cs \leftarrow 0$; $ccs \leftarrow 0$;
for $i \in 0..n-1$ **do**
  $t \leftarrow sum + x[i]$;
  if $|sum| \geq |x[i]|$ **then** $c \leftarrow (sum - t) + x[i]$;
  else $c \leftarrow (x[i] - t) + sum$;
  $sum \leftarrow t$;
  $t \leftarrow cs + c$;
  if $|cs| \geq |c|$ **then** $cc \leftarrow (cs - t) + c$;
  else $c \leftarrow (c - t) + cs$;
  $cs \leftarrow t$; $ccs \leftarrow ccs + cc$;
  return $sum + cs + ccs$

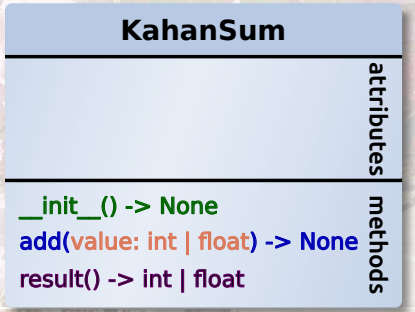


Design der Klasse

Definition: (Vollständige) Kapselung

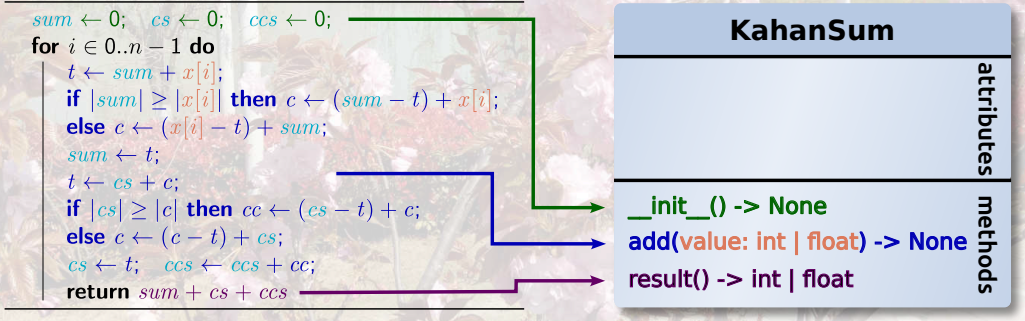
Kapselung (EN: *encapsulation*) bedeutet, dass auf die Attribute eines Objekts nur durch seine Methoden zugegeriffen wird. Unter vollsändiger Kapselung (EN: *complete encapsulation*) können die Attribute eines Objekts nur durch seine Methoden gelesen oder verändert werden. Kapselung erlaubt es daher Programmierern, sicherzustellen, dass der Zustand von Objekten nur einer konsistenten und korrekten Art und Weise verändert werden kann.

$sum \leftarrow 0$; $cs \leftarrow 0$; $ccs \leftarrow 0$;
for $i \in 0..n-1$ **do**
  $t \leftarrow sum + x[i]$;
  if $|sum| \geq |x[i]|$ **then** $c \leftarrow (sum - t) + x[i]$;
  else $c \leftarrow (x[i] - t) + sum$;
  $sum \leftarrow t$;
  $t \leftarrow cs + c$;
  if $|cs| \geq |c|$ **then** $cc \leftarrow (cs - t) + c$;
  else $c \leftarrow (c - t) + cs$;
  $cs \leftarrow t$; $ccs \leftarrow ccs + cc$;
  return $sum + cs + ccs$



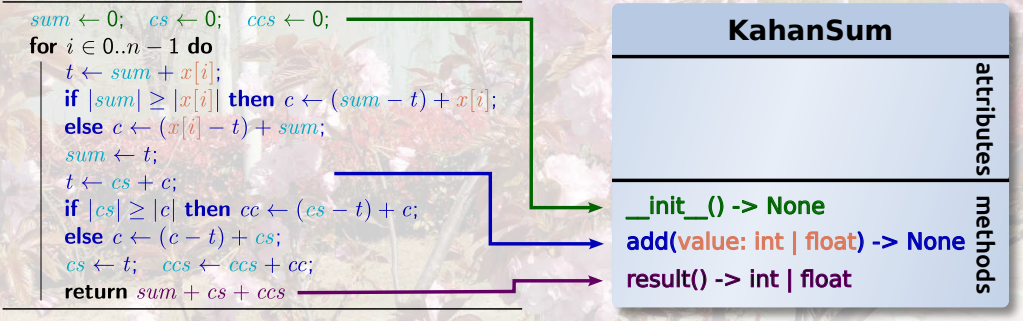
Design der Klasse

- Anders als bei den Attributen `x` und `y` unserer Klasse `Point`, hat der Benutzer (ein anderer Programmierer) keinen Grund, auf den (internen) Zustand unserer Summe zuzugreifen.
- Stattdessen sollte er mit unseren Objekten nur über die Methoden `add` und `result` interagieren.



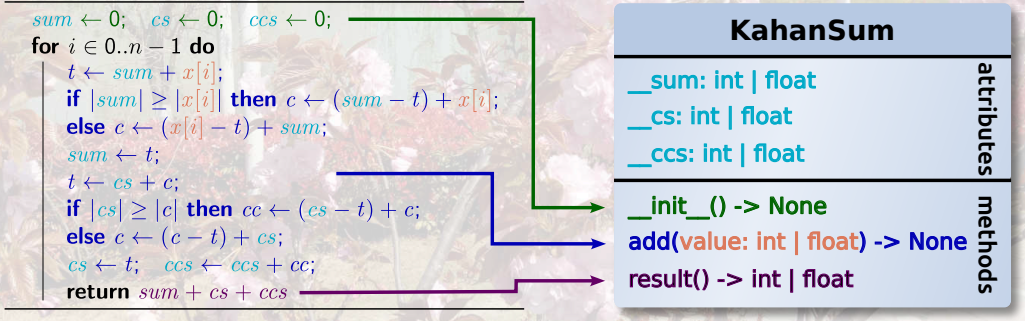
Design der Klasse

- Anders als bei den Attributen `x` und `y` unserer Klasse `Point`, hat der Benutzer (ein anderer Programmierer) keinen Grund, auf den (internen) Zustand unserer Summe zuzugreifen.
- Stattdessen sollte er mit unseren Objekten nur über die Methoden `add` und `result` interagieren.
- Wir wollen unsere Objekte vollständig kapseln.



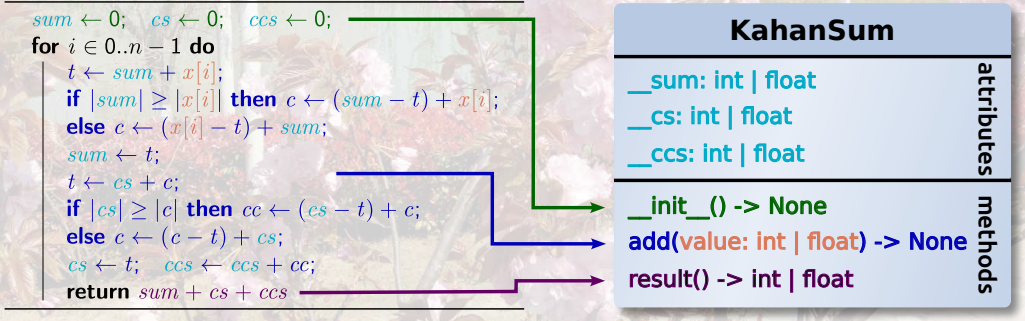
Design der Klasse

- Stattdessen sollte er mit unseren Objekten nur über die Methoden `add` und `result` interagieren.
- Wir wollen unsere Objekte vollständig kapseln.
- Wir erstellen drei Attribute `__sum`, `__cs` und `__ccs`.



Design der Klasse

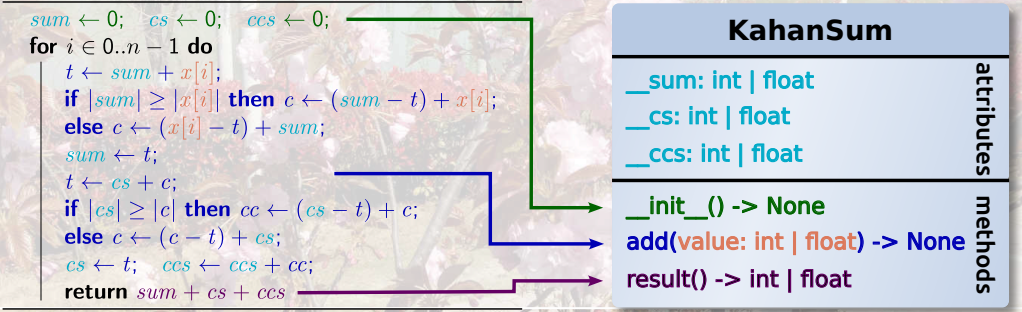
- Wir wollen unsere Objekte vollständig kapseln.
- Wir erstellen drei Attribute `__sum`, `__cs` und `__ccs`.
- Die Namen stammen aus dem Algorithmus, aber sehen Sie die beiden Unterstriche jeweils am Anfang?



Design der Klasse

Gute Praxis

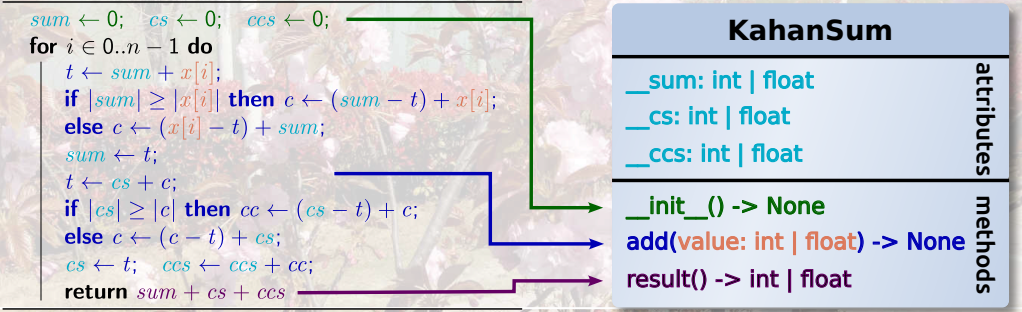
Namen von Attributen oder Methoden, die mit einem doppelten Unterstrich beginnen (__) müssen als *privat* betrachtet werden[248,249].



Design der Klasse

Gute Praxis

Namen von Attributen oder Methoden, die mit einem doppelten Unterstrich beginnen (`__`) müssen als *privat* betrachtet werden[248,249]. Auf sie sollte nicht von außerhalb der Klasse zugegriffen werden.



Design der Klasse

Gute Praxis

Namen von Attributen oder Methoden, die mit einem doppelten Unterstrich beginnen (`__`) müssen als *privat* betrachtet werden[248,249]. Auf sie sollte nicht von außerhalb der Klasse zugegriffen werden. Alle internen Attribute und Methoden einer Klasse sollte nicht von außen zugänglich sein und sollten deshalb nach dieser Konvention benannt werden, mit einem doppelten Unterstrich als Präfix.

$sum \leftarrow 0; \quad cs \leftarrow 0; \quad ccs \leftarrow 0;$
for $i \in 0..n-1$ **do**
  $t \leftarrow sum + x[i];$
  if $|sum| \geq |x[i]|$ **then** $c \leftarrow (sum - t) + x[i];$
  else $c \leftarrow (x[i] - t) + sum;$
  $sum \leftarrow t;$
  $t \leftarrow cs + c;$
  if $|cs| \geq |c|$ **then** $cc \leftarrow (cs - t) + c;$
  else $c \leftarrow (c - t) + cs;$
  $cs \leftarrow t; \quad ccs \leftarrow ccs + cc;$
  return $sum + cs + ccs$

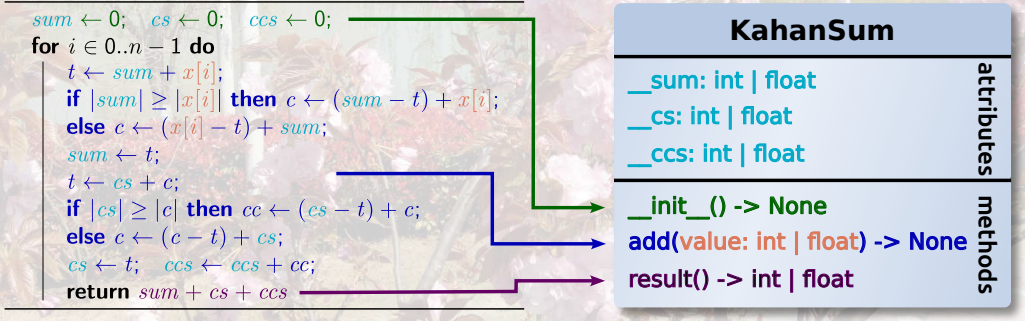
KahanSum

attributes
- `__sum: int | float`
- `__cs: int | float`
- `__ccs: int | float`

methods
- `__init__() -> None`
- `add(value: int | float) -> None`
- `result() -> int | float`

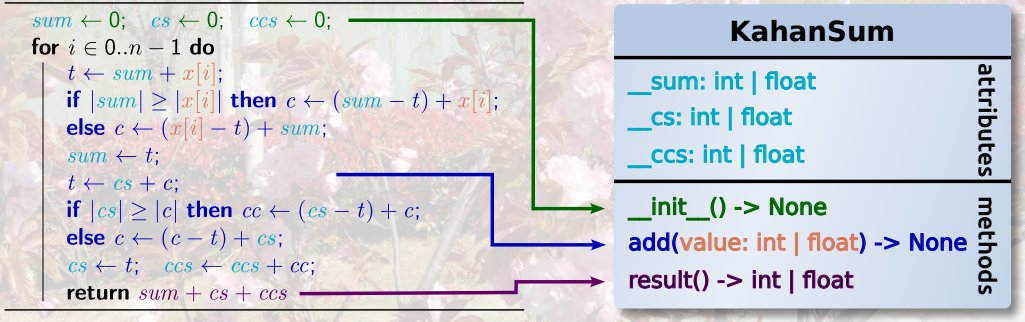
Design der Klasse

- Wir erstellen drei Attribute `__sum`, `__cs` und `__ccs`.
- Die Namen stammen aus dem Algorithmus, aber sehen Sie die beiden Unterstriche jeweils am Anfang?
- Mit anderen Worten, niemand sollte von Außen auf `__sum`, `__cs` oder `__ccs`.



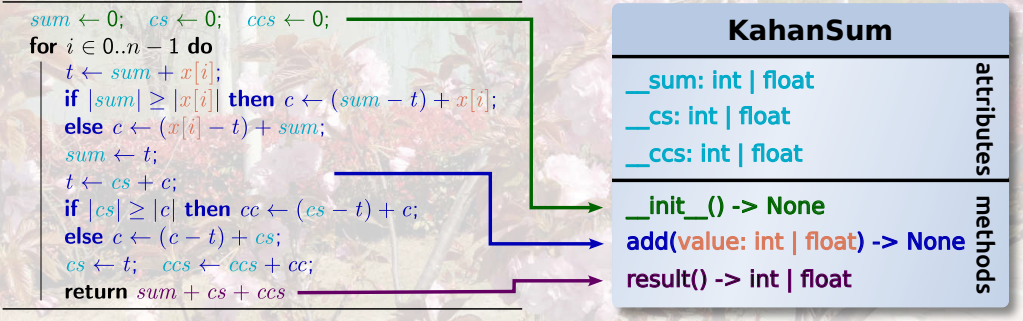
Design der Klasse

- Die Namen stammen aus dem Algorithmus, aber sehen Sie die beiden Unterstriche jeweils am Anfang?
- Mit anderen Worten, niemand sollte von Außen auf `__sum`, `__cs` oder `__ccs`.
- Wie alle solche Dinge in Python wird das natürlich nicht vom Interpreter erzwungen...



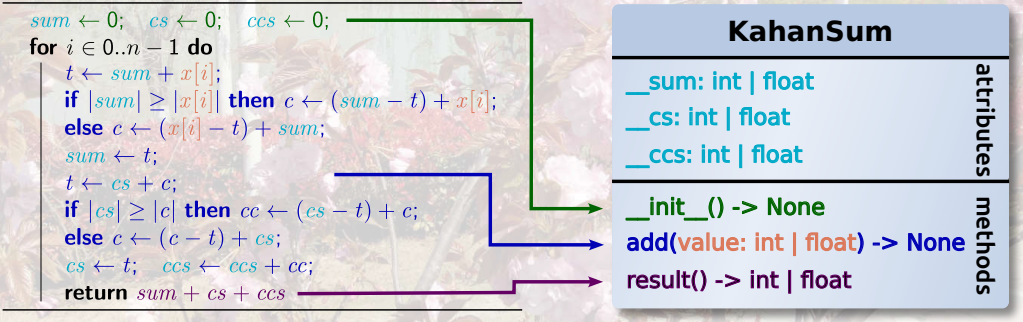
Design der Klasse

- Mit anderen Worten, niemand sollte von Außen auf `__sum`, `__cs` oder `__ccs`.
- Wie alle solche Dinge in Python wird das natürlich nicht vom Interpreter erzwungen...
- ...es ist also kein absoluter Schutz unserer Attribute.



Design der Klasse

- Wie alle solche Dinge in Python wird das natürlich nicht vom Interpreter erzwungen...
- ... es ist also kein absoluter Schutz unserer Attribute.
- Es ist nur ein sehr klarer Hinweis an andere Programmierer, dass sie da die Finger von lassen sollen.



Warum als Klasse?



- Wir hätten den Algorithmus als einfache Funktion, die auf ein `Iterable` von Zahlen angewandt wird, implementieren können.

Warum als Klasse?



- Wir hätten den Algorithmus als einfache Funktion, die auf ein `Iterable` von Zahlen angewandt wird, implementieren können.
- Das wäre wohl die „natürlichste“ Art gewesen, ihn zu implementieren.

Warum als Klasse?



- Wir hätten den Algorithmus als einfache Funktion, die auf ein `Iterable` von Zahlen angewandt wird, implementieren können.
- Das wäre wohl die „natürlichste“ Art gewesen, ihn zu implementieren.
- Wir haben aber das Klassen-basierte Design gewählt, weil es uns größere Flexibilität gibt.

Warum als Klasse?



- Wir hätten den Algorithmus als einfache Funktion, die auf ein `Iterable` von Zahlen angewandt wird, implementieren können.
- Das wäre wohl die „natürlichste“ Art gewesen, ihn zu implementieren.
- Wir haben aber das Klassen-basierte Design gewählt, weil es uns größere Flexibilität gibt.
- Wir können damit natürlich trotzdem über Sequenzen von Zahlen summieren.

Warum als Klasse?



- Wir hätten den Algorithmus als einfache Funktion, die auf ein `Iterable` von Zahlen angewandt wird, implementieren können.
- Das wäre wohl die „natürlichste“ Art gewesen, ihn zu implementieren.
- Wir haben aber das Klassen-basierte Design gewählt, weil es uns größere Flexibilität gibt.
- Wir können damit natürlich trotzdem über Sequenzen von Zahlen summieren.
- Wir können aber eben auch anders summieren.

Warum als Klasse?



- Wir hätten den Algorithmus als einfache Funktion, die auf ein `Iterable` von Zahlen angewandt wird, implementieren können.
- Das wäre wohl die „natürlichste“ Art gewesen, ihn zu implementieren.
- Wir haben aber das Klassen-basierte Design gewählt, weil es uns größere Flexibilität gibt.
- Wir können damit natürlich trotzdem über Sequenzen von Zahlen summieren.
- Wir können aber eben auch anders summieren.
- Wir können Zwischenergebnisse abfragen.

Warum als Klasse?



- Wir hätten den Algorithmus als einfache Funktion, die auf ein `Iterable` von Zahlen angewandt wird, implementieren können.
- Das wäre wohl die „natürlichste“ Art gewesen, ihn zu implementieren.
- Wir haben aber das Klassen-basierte Design gewählt, weil es uns größere Flexibilität gibt.
- Wir können damit natürlich trotzdem über Sequenzen von Zahlen summieren.
- Wir können aber eben auch anders summieren.
- Wir können Zwischenergebnisse abfragen.
- Wir können auch mehrere `KahanSum`-Objekte auf einmal verwenden.

Warum als Klasse?



- Wir hätten den Algorithmus als einfache Funktion, die auf ein `Iterable` von Zahlen angewandt wird, implementieren können.
- Das wäre wohl die „natürlichste“ Art gewesen, ihn zu implementieren.
- Wir haben aber das Klassen-basierte Design gewählt, weil es uns größere Flexibilität gibt.
- Wir können damit natürlich trotzdem über Sequenzen von Zahlen summieren.
- Wir können aber eben auch anders summieren.
- Wir können Zwischenergebnisse abfragen.
- Wir können auch mehrere `KahanSum`-Objekte auf einmal verwenden.
- Wenn wir z. B. einen Datenstrom haben, dann könnten wir zwei Instanzen von `KahanSum` verwenden, um die Zahlen (in dereinen) zu addieren und um deren Quadrate aufzusummieren (in der anderen Instanz), was durchaus nützlich seien kann, wenn wir z. B. die Sample-Varianz bestimmen wollen.

Kahan-Summe: Implementation

- Implementieren wir diesen Algorithmus nun also.

Algorithm 1: Kahan-Babuška-Neumaier Summe

$sum \leftarrow 0; \quad cs \leftarrow 0; \quad ccs \leftarrow 0;$
for $i \in 0..n-1$ **do**
  $t \leftarrow sum + x[i];$
  if $|sum| \geq |x[i]|$ **then** $c \leftarrow (sum - t) + x[i];$
  else $c \leftarrow (x[i] - t) + sum;$
  $sum \leftarrow t;$
  $t \leftarrow cs + c;$
  if $|cs| \geq |c|$ **then** $cc \leftarrow (cs - t) + c;$
  else $c \leftarrow (c - t) + cs;$
  $cs \leftarrow t; \quad ccs \leftarrow ccs + cc;$
  return $sum + cs + ccs$

Kahan-Summe: Implementation

- Eine neue Instanz von `KahanSum` beginnt mit allen ihren drei Summenattributen auf `0` gesetzt, was im Initialisierer `__init__` passiert.

Algorithm 1: Kahan-Babuška-Neumaier Summe

$sum \leftarrow 0; \quad cs \leftarrow 0; \quad ccs \leftarrow 0;$
for $i \in 0..n-1$ **do**
  $t \leftarrow sum + x[i];$
  if $|sum| \geq |x[i]|$ **then** $c \leftarrow (sum - t) + x[i];$
  else $c \leftarrow (x[i] - t) + sum;$
  $sum \leftarrow t;$
  $t \leftarrow cs + c;$
  if $|cs| \geq |c|$ **then** $cc \leftarrow (cs - t) + c;$
  else $c \leftarrow (c - t) + cs;$
  $cs \leftarrow t; \quad ccs \leftarrow ccs + cc;$
  return $sum + cs + ccs$

```python
"""
The second-order Kahan-Babuška-Neumaier-Summation by Klein.

[1] A. Klein. A Generalized Kahan-Babuška-Summation-Algorithm.
    Computing 76:279-293. 2006. doi:10.1007/s00607-005-0139-x

>>> kahan_sum = KahanSum()
>>> for xi in [1e18, 1, 1e36, -1e36, -1e18]:
...     kahan_sum.add(xi)
>>> kahan_sum.result()
1.0
"""


class KahanSum:
    """The second-order Kahan-Babuška-Neumaier sum by Klein."""

    def __init__(self) -> None:
        """Create the summation object."""
        #: the running sum, an internal variable invisible from outside
        self.__sum: float | int = 0
        #: the first correction term, another internal variable
        self.__cs: float | int = 0
        #: the second correction term, another internal variable
        self.__ccs: float | int = 0

    def add(self, value: int | float) -> None:
        """
        Add a value to the sum.

        :param value: the value to add
        """
        s: int | float = self.__sum  # Get the current running sum.
        t: int | float = s + value   # Compute the new sum value.
        c: int | float = (((s - t) + value) if abs(s) >= abs(value)
                          else ((value - t) + s))  # The Neumaier tweak.
        self.__sum = t  # Store the new sum value.
        cs: int | float = self.__cs  # the current 1st-order correction
        t = cs + c  # Compute the new first-order correction term.
        cc: int | float = ((((cs - t) + c) if abs(cs) >= abs(c)
                            else ((c - t) + cs))  # 2nd Neumaier tweak.
        self.__cs = t  # Store the updated first-order correction term.
        self.__ccs += cc  # Update the second-order correction.

    def result(self) -> int | float:
        """
        Get the current result of the summation.

        :return: the current result of the summation
        """
        return self.__sum + self.__cs + self.__ccs
```

Kahan-Summe: Implementation

- Wir implementieren zuerst die Methode `add`.

Algorithm 1: Kahan-Babuška-Neumaier Summe

$sum \leftarrow 0; \quad cs \leftarrow 0; \quad ccs \leftarrow 0;$
for $i \in 0..n-1$ **do**
  $t \leftarrow sum + x[i];$
  if $|sum| \geq |x[i]|$ **then** $c \leftarrow (sum - t) + x[i];$
  else $c \leftarrow (x[i] - t) + sum;$
  $sum \leftarrow t;$
  $t \leftarrow cs + c;$
  if $|cs| \geq |c|$ **then** $cc \leftarrow (cs - t) + c;$
  else $c \leftarrow (c - t) + cs;$
  $cs \leftarrow t; \quad ccs \leftarrow ccs + cc;$
  return $sum + cs + ccs$

```python
"""
The second-order Kahan-Babuška-Neumaier-Summation by Klein.

[1] A. Klein. A Generalized Kahan-Babuška-Summation-Algorithm.
    Computing 76:279-293. 2006. doi:10.1007/s00607-005-0139-x

>>> kahan_sum = KahanSum()
>>> for xi in [1e18, 1, 1e36, -1e36, -1e18]:
...     kahan_sum.add(xi)
>>> kahan_sum.result()
1.0
"""


class KahanSum:
    """The second-order Kahan-Babuška-Neumaier sum by Klein."""

    def __init__(self) -> None:
        """Create the summation object."""
        #: the running sum, an internal variable invisible from outside
        self.__sum: float | int = 0
        #: the first correction term, another internal variable
        self.__cs: float | int = 0
        #: the second correction term, another internal variable
        self.__ccs: float | int = 0

    def add(self, value: int | float) -> None:
        """
        Add a value to the sum.

        :param value: the value to add
        """
        s: int | float = self.__sum  # Get the current running sum.
        t: int | float = s + value   # Compute the new sum value.
        c: int | float = (((s - t) + value) if abs(s) >= abs(value)
                          else ((value - t) + s))  # The Neumaier tweak.
        self.__sum = t  # Store the new sum value.
        cs: int | float = self.__cs  # the current 1st-order correction
        t = cs + c  # Compute the new first-order correction term.
        cc: int | float = (((cs - t) + c) if abs(cs) >= abs(c)
                           else ((c - t) + cs))  # 2nd Neumaier tweak.
        self.__cs = t  # Store the updated first-order correction term.
        self.__ccs += cc  # Update the second-order correction.

    def result(self) -> int | float:
        """
        Get the current result of the summation.

        :return: the current result of the summation
        """
        return self.__sum + self.__cs + self.__ccs
```

Kahan-Summe: Implementation

- Wir implementieren zuerst die Methode `add`.
- Diese entspricht dem Köper der Schleife im Algorithmus.

Algorithm 1: Kahan-Babuška-Neumaier Summe

$sum \leftarrow 0; \quad cs \leftarrow 0; \quad ccs \leftarrow 0;$
for $i \in 0..n-1$ **do**
  $t \leftarrow sum + x[i];$
  if $|sum| \geq |x[i]|$ **then** $c \leftarrow (sum - t) + x[i];$
  else $c \leftarrow (x[i] - t) + sum;$
  $sum \leftarrow t;$
  $t \leftarrow cs + c;$
  if $|cs| \geq |c|$ **then** $cc \leftarrow (cs - t) + c;$
  else $c \leftarrow (c - t) + cs;$
  $cs \leftarrow t; \quad ccs \leftarrow ccs + cc;$
  return $sum + cs + ccs$

```python
"""
The second-order Kahan-Babuška-Neumaier-Summation by Klein.

[1] A. Klein. A Generalized Kahan-Babuška-Summation-Algorithm.
    Computing 76:279-293. 2006. doi:10.1007/s00607-005-0139-x

>>> kahan_sum = KahanSum()
>>> for xi in [1e18, 1, 1e36, -1e36, -1e18]:
...     kahan_sum.add(xi)
>>> kahan_sum.result()
1.0
"""


class KahanSum:
    """The second-order Kahan-Babuška-Neumaier sum by Klein."""

    def __init__(self) -> None:
        """Create the summation object."""
        #: the running sum, an internal variable invisible from outside
        self.__sum: float | int = 0
        #: the first correction term, another internal variable
        self.__cs: float | int = 0
        #: the second correction term, another internal variable
        self.__ccs: float | int = 0

    def add(self, value: int | float) -> None:
        """
        Add a value to the sum.

        :param value: the value to add
        """
        s: int | float = self.__sum  # Get the current running sum.
        t: int | float = s + value   # Compute the new sum value.
        c: int | float = (((s - t) + value) if abs(s) >= abs(value)
                          else ((value - t) + s))  # The Neumaier tweak.
        self.__sum = t  # Store the new sum value.
        cs: int | float = self.__cs  # the current 1st-order correction
        t = cs + c  # Compute the new first-order correction term.
        cc: int | float = ((((cs - t) + c) if abs(cs) >= abs(c)
                            else ((c - t) + cs))  # 2nd Neumaier tweak.
        self.__cs = t  # Store the updated first-order correction term.
        self.__ccs += cc  # Update the second-order correction.

    def result(self) -> int | float:
        """
        Get the current result of the summation.

        :return: the current result of the summation
        """
        return self.__sum + self.__cs + self.__ccs
```

Kahan-Summe: Implementation

- Diese entspricht dem Köper der Schleife im Algorithmus.
- Im Algorithmus wird in jeder Iteration ein Wert $x[i]$ zur Summe addiert.

Algorithm 1: Kahan-Babuška-Neumaier Summe

$sum \leftarrow 0; \quad cs \leftarrow 0; \quad ccs \leftarrow 0;$
for $i \in 0..n-1$ **do**
  $t \leftarrow sum + x[i];$
  if $|sum| \geq |x[i]|$ **then** $c \leftarrow (sum - t) + x[i];$
  else $c \leftarrow (x[i] - t) + sum;$
  $sum \leftarrow t;$
  $t \leftarrow cs + c;$
  if $|cs| \geq |c|$ **then** $cc \leftarrow (cs - t) + c;$
  else $c \leftarrow (c - t) + cs;$
  $cs \leftarrow t; \quad ccs \leftarrow ccs + cc;$
  return $sum + cs + ccs$

```python
"""
The second-order Kahan-Babuška-Neumaier-Summation by Klein.

[1] A. Klein. A Generalized Kahan-Babuška-Summation-Algorithm.
    Computing 76:279-293. 2006. doi:10.1007/s00607-005-0139-x

>>> kahan_sum = KahanSum()
>>> for xi in [1e18, 1, 1e36, -1e36, -1e18]:
...     kahan_sum.add(xi)
>>> kahan_sum.result()
1.0
"""


class KahanSum:
    """The second-order Kahan-Babuška-Neumaier sum by Klein."""

    def __init__(self) -> None:
        """Create the summation object."""
        #: the running sum, an internal variable invisible from outside
        self.__sum: float | int = 0
        #: the first correction term, another internal variable
        self.__cs: float | int = 0
        #: the second correction term, another internal variable
        self.__ccs: float | int = 0

    def add(self, value: int | float) -> None:
        """
        Add a value to the sum.

        :param value: the value to add
        """
        s: int | float = self.__sum  # Get the current running sum.
        t: int | float = s + value   # Compute the new sum value.
        c: int | float = (((s - t) + value) if abs(s) >= abs(value)
                          else ((value - t) + s))  # The Neumaier tweak.
        self.__sum = t  # Store the new sum value.
        cs: int | float = self.__cs  # the current 1st-order correction
        t = cs + c  # Compute the new first-order correction term.
        cc: int | float = ((((cs - t) + c) if abs(cs) >= abs(c)
                            else ((c - t) + cs))  # 2nd Neumaier tweak.
        self.__cs = t  # Store the updated first-order correction term.
        self.__ccs += cc  # Update the second-order correction.

    def result(self) -> int | float:
        """
        Get the current result of the summation.

        :return: the current result of the summation
        """
        return self.__sum + self.__cs + self.__ccs
```

Kahan-Summe: Implementation

- Unsere Methode `add` hat dafür den Parameter `value`, mit dem Wert den wir zur Summe addieren wollen.

Algorithm 1: Kahan-Babuška-Neumaier Summe

$sum \leftarrow 0; \quad cs \leftarrow 0; \quad ccs \leftarrow 0;$
for $i \in 0..n-1$ **do**
  $t \leftarrow sum + x[i];$
  if $|sum| \geq |x[i]|$ **then** $c \leftarrow (sum - t) + x[i];$
  else $c \leftarrow (x[i] - t) + sum;$
  $sum \leftarrow t;$
  $t \leftarrow cs + c;$
  if $|cs| \geq |c|$ **then** $cc \leftarrow (cs - t) + c;$
  else $c \leftarrow (c - t) + cs;$
  $cs \leftarrow t; \quad ccs \leftarrow ccs + cc;$
  return $sum + cs + ccs$

```python
"""
The second-order Kahan-Babuška-Neumaier-Summation by Klein.

[1] A. Klein. A Generalized Kahan-Babuška-Summation-Algorithm.
    Computing 76:279-293. 2006. doi:10.1007/s00607-005-0139-x

>>> kahan_sum = KahanSum()
>>> for xi in [1e18, 1, 1e36, -1e36, -1e18]:
...     kahan_sum.add(xi)
>>> kahan_sum.result()
1.0
"""


class KahanSum:
    """The second-order Kahan-Babuška-Neumaier sum by Klein."""

    def __init__(self) -> None:
        """Create the summation object."""
        #: the running sum, an internal variable invisible from outside
        self.__sum: float | int = 0
        #: the first correction term, another internal variable
        self.__cs: float | int = 0
        #: the second correction term, another internal variable
        self.__ccs: float | int = 0

    def add(self, value: int | float) -> None:
        """
        Add a value to the sum.

        :param value: the value to add
        """
        s: int | float = self.__sum  # Get the current running sum.
        t: int | float = s + value   # Compute the new sum value.
        c: int | float = (((s - t) + value) if abs(s) >= abs(value)
                          else ((value - t) + s))  # The Neumaier tweak.
        self.__sum = t  # Store the new sum value.
        cs: int | float = self.__cs  # the current 1st-order correction
        t = cs + c  # Compute the new first-order correction term.
        cc: int | float = ((((cs - t) + c) if abs(cs) >= abs(c)
                            else ((c - t) + cs))  # 2nd Neumaier tweak.
        self.__cs = t  # Store the updated first-order correction term.
        self.__ccs += cc  # Update the second-order correction.

    def result(self) -> int | float:
        """
        Get the current result of the summation.

        :return: the current result of the summation
        """
        return self.__sum + self.__cs + self.__ccs
```

Kahan-Summe: Implementation

- Wir müssen also nur den Schleifenkörper vom Algorithmus in die Methode `add` übertragen.

Algorithm 1: Kahan-Babuška-Neumaier Summe

$sum \leftarrow 0; \quad cs \leftarrow 0; \quad ccs \leftarrow 0;$
for $i \in 0..n-1$ **do**
  $t \leftarrow sum + x[i];$
  if $|sum| \geq |x[i]|$ **then** $c \leftarrow (sum - t) + x[i];$
  else $c \leftarrow (x[i] - t) + sum;$
  $sum \leftarrow t;$
  $t \leftarrow cs + c;$
  if $|cs| \geq |c|$ **then** $cc \leftarrow (cs - t) + c;$
  else $c \leftarrow (c - t) + cs;$
  $cs \leftarrow t; \quad ccs \leftarrow ccs + cc;$
  return $sum + cs + ccs$

```python
"""
The second-order Kahan-Babuška-Neumaier-Summation by Klein.

[1] A. Klein. A Generalized Kahan-Babuška-Summation-Algorithm.
    Computing 76:279-293. 2006. doi:10.1007/s00607-005-0139-x

>>> kahan_sum = KahanSum()
>>> for xi in [1e18, 1, 1e36, -1e36, -1e18]:
...     kahan_sum.add(xi)
>>> kahan_sum.result()
1.0
"""


class KahanSum:
    """The second-order Kahan-Babuška-Neumaier sum by Klein."""

    def __init__(self) -> None:
        """Create the summation object."""
        #: the running sum, an internal variable invisible from outside
        self.__sum: float | int = 0
        #: the first correction term, another internal variable
        self.__cs: float | int = 0
        #: the second correction term, another internal variable
        self.__ccs: float | int = 0

    def add(self, value: int | float) -> None:
        """
        Add a value to the sum.

        :param value: the value to add
        """
        s: int | float = self.__sum  # Get the current running sum.
        t: int | float = s + value   # Compute the new sum value.
        c: int | float = (((s - t) + value) if abs(s) >= abs(value)
                          else ((value - t) + s))  # The Neumaier tweak.
        self.__sum = t  # Store the new sum value.
        cs: int | float = self.__cs  # the current 1st-order correction
        t = cs + c  # Compute the new first-order correction term.
        cc: int | float = ((((cs - t) + c) if abs(cs) >= abs(c)
                            else ((c - t) + cs))  # 2nd Neumaier tweak.
        self.__cs = t  # Store the updated first-order correction term.
        self.__ccs += cc  # Update the second-order correction.

    def result(self) -> int | float:
        """
        Get the current result of the summation.

        :return: the current result of the summation
        """
        return self.__sum + self.__cs + self.__ccs
```

Kahan-Summe: Implementation

- Dabei ersetzen wir sum, cs, ccs und $x[i]$ mit `self.__sum`, `self.__cs`, `self.__ccs` und `value`.

Algorithm 1: Kahan-Babuška-Neumaier Summe

$sum \leftarrow 0; \quad cs \leftarrow 0; \quad ccs \leftarrow 0;$
for $i \in 0..n-1$ **do**
  $t \leftarrow sum + x[i];$
  if $|sum| \geq |x[i]|$ **then** $c \leftarrow (sum - t) + x[i];$
  else $c \leftarrow (x[i] - t) + sum;$
  $sum \leftarrow t;$
  $t \leftarrow cs + c;$
  if $|cs| \geq |c|$ **then** $cc \leftarrow (cs - t) + c;$
  else $c \leftarrow (c - t) + cs;$
  $cs \leftarrow t; \quad ccs \leftarrow ccs + cc;$
  return $sum + cs + ccs$

```
"""
The second-order Kahan-Babuška-Neumaier-Summation by Klein.

[1] A. Klein. A Generalized Kahan-Babuška-Summation-Algorithm.
    Computing 76:279-293. 2006. doi:10.1007/s00607-005-0139-x

>>> kahan_sum = KahanSum()
>>> for xi in [1e18, 1, 1e36, -1e36, -1e18]:
...     kahan_sum.add(xi)
>>> kahan_sum.result()
1.0
"""


class KahanSum:
    """The second-order Kahan-Babuška-Neumaier sum by Klein."""

    def __init__(self) -> None:
        """Create the summation object."""
        #: the running sum, an internal variable invisible from outside
        self.__sum: float | int = 0
        #: the first correction term, another internal variable
        self.__cs: float | int = 0
        #: the second correction term, another internal variable
        self.__ccs: float | int = 0

    def add(self, value: int | float) -> None:
        """
        Add a value to the sum.

        :param value: the value to add
        """
        s: int | float = self.__sum  # Get the current running sum.
        t: int | float = s + value   # Compute the new sum value.
        c: int | float = (((s - t) + value) if abs(s) >= abs(value)
                          else ((value - t) + s))  # The Neumaier tweak.
        self.__sum = t  # Store the new sum value.
        cs: int | float = self.__cs  # the current 1st-order correction
        t = cs + c  # Compute the new first-order correction term.
        cc: int | float = ((((cs - t) + c) if abs(cs) >= abs(c)
                            else ((c - t) + cs))  # 2nd Neumaier tweak.
        self.__cs = t  # Store the updated first-order correction term.
        self.__ccs += cc  # Update the second-order correction.

    def result(self) -> int | float:
        """
        Get the current result of the summation.

        :return: the current result of the summation
        """
        return self.__sum + self.__cs + self.__ccs
```

Kahan-Summe: Implementation

- Wir schreiben die Verbesserung von Neumaier[224] als ein inline `if...else`-Statement.

Algorithm 1: Kahan-Babuška-Neumaier Summe

$sum \leftarrow 0; \quad cs \leftarrow 0; \quad ccs \leftarrow 0;$
for $i \in 0..n-1$ **do**
  $t \leftarrow sum + x[i];$
  if $|sum| \geq |x[i]|$ **then** $c \leftarrow (sum - t) + x[i];$
  else $c \leftarrow (x[i] - t) + sum;$
  $sum \leftarrow t;$
  $t \leftarrow cs + c;$
  if $|cs| \geq |c|$ **then** $cc \leftarrow (cs - t) + c;$
  else $c \leftarrow (c - t) + cs;$
  $cs \leftarrow t; \quad ccs \leftarrow ccs + cc;$
  return $sum + cs + ccs$

```python
"""
The second-order Kahan-Babuška-Neumaier-Summation by Klein.

[1] A. Klein. A Generalized Kahan-Babuška-Summation-Algorithm.
    Computing 76:279-293. 2006. doi:10.1007/s00607-005-0139-x

>>> kahan_sum = KahanSum()
>>> for xi in [1e18, 1, 1e36, -1e36, -1e18]:
...     kahan_sum.add(xi)
>>> kahan_sum.result()
1.0
"""


class KahanSum:
    """The second-order Kahan-Babuška-Neumaier sum by Klein."""

    def __init__(self) -> None:
        """Create the summation object."""
        #: the running sum, an internal variable invisible from outside
        self.__sum: float | int = 0
        #: the first correction term, another internal variable
        self.__cs: float | int = 0
        #: the second correction term, another internal variable
        self.__ccs: float | int = 0

    def add(self, value: int | float) -> None:
        """
        Add a value to the sum.

        :param value: the value to add
        """
        s: int | float = self.__sum  # Get the current running sum.
        t: int | float = s + value   # Compute the new sum value.
        c: int | float = (((s - t) + value) if abs(s) >= abs(value)
                          else ((value - t) + s))  # The Neumaier tweak.
        self.__sum = t  # Store the new sum value.
        cs: int | float = self.__cs  # the current 1st-order correction
        t = cs + c  # Compute the new first-order correction term.
        cc: int | float = ((((cs - t) + c) if abs(cs) >= abs(c)
                            else ((c - t) + cs))  # 2nd Neumaier tweak.
        self.__cs = t  # Store the updated first-order correction term.
        self.__ccs += cc  # Update the second-order correction.

    def result(self) -> int | float:
        """
        Get the current result of the summation.

        :return: the current result of the summation
        """
        return self.__sum + self.__cs + self.__ccs
```

Kahan-Summe: Implementation

- Um den Absolutwert $|a|$ einer Zahl a zu berechnen, können wir Python's Funktion `abs` verwenden.

Algorithm 1: Kahan-Babuška-Neumaier Summe

$sum \leftarrow 0; \quad cs \leftarrow 0; \quad ccs \leftarrow 0;$
for $i \in 0..n-1$ **do**
  $t \leftarrow sum + x[i];$
  if $|sum| \geq |x[i]|$ **then** $c \leftarrow (sum - t) + x[i];$
  else $c \leftarrow (x[i] - t) + sum;$
  $sum \leftarrow t;$
  $t \leftarrow cs + c;$
  if $|cs| \geq |c|$ **then** $cc \leftarrow (cs - t) + c;$
  else $c \leftarrow (c - t) + cs;$
  $cs \leftarrow t; \quad ccs \leftarrow ccs + cc;$
  return $sum + cs + ccs$

```python
"""
The second-order Kahan-Babuška-Neumaier-Summation by Klein.

[1] A. Klein. A Generalized Kahan-Babuška-Summation-Algorithm.
    Computing 76:279-293. 2006. doi:10.1007/s00607-005-0139-x

>>> kahan_sum = KahanSum()
>>> for xi in [1e18, 1, 1e36, -1e36, -1e18]:
...     kahan_sum.add(xi)
>>> kahan_sum.result()
1.0
"""


class KahanSum:
    """The second-order Kahan-Babuška-Neumaier sum by Klein."""

    def __init__(self) -> None:
        """Create the summation object."""
        #: the running sum, an internal variable invisible from outside
        self.__sum: float | int = 0
        #: the first correction term, another internal variable
        self.__cs: float | int = 0
        #: the second correction term, another internal variable
        self.__ccs: float | int = 0

    def add(self, value: int | float) -> None:
        """
        Add a value to the sum.

        :param value: the value to add
        """
        s: int | float = self.__sum  # Get the current running sum.
        t: int | float = s + value   # Compute the new sum value.
        c: int | float = (((s - t) + value) if abs(s) >= abs(value)
                          else ((value - t) + s))  # The Neumaier tweak.
        self.__sum = t  # Store the new sum value.
        cs: int | float = self.__cs  # the current 1st-order correction
        t = cs + c  # Compute the new first-order correction term.
        cc: int | float = ((((cs - t) + c) if abs(cs) >= abs(c)
                            else ((c - t) + cs))  # 2nd Neumaier tweak.
        self.__cs = t  # Store the updated first-order correction term.
        self.__ccs += cc  # Update the second-order correction.

    def result(self) -> int | float:
        """
        Get the current result of the summation.

        :return: the current result of the summation
        """
        return self.__sum + self.__cs + self.__ccs
```

Kahan-Summe: Implementation

- Um den Absolutwert $|a|$ einer Zahl a zu berechnen, können wir Python's Funktion `abs` verwenden.
- Und das war's auch schon.

Algorithm 1: Kahan-Babuška-Neumaier Summe

$sum \leftarrow 0; \quad cs \leftarrow 0; \quad ccs \leftarrow 0;$
for $i \in 0..n-1$ **do**
  $t \leftarrow sum + x[i];$
  if $|sum| \geq |x[i]|$ **then** $c \leftarrow (sum - t) + x[i];$
  else $c \leftarrow (x[i] - t) + sum;$
  $sum \leftarrow t;$
  $t \leftarrow cs + c;$
  if $|cs| \geq |c|$ **then** $cc \leftarrow (cs - t) + c;$
  else $c \leftarrow (c - t) + cs;$
  $cs \leftarrow t; \quad ccs \leftarrow ccs + cc;$
  return $sum + cs + ccs$

```python
"""
The second-order Kahan-Babuška-Neumaier-Summation by Klein.

[1] A. Klein. A Generalized Kahan-Babuška-Summation-Algorithm.
    Computing 76:279-293. 2006. doi:10.1007/s00607-005-0139-x

>>> kahan_sum = KahanSum()
>>> for xi in [1e18, 1, 1e36, -1e36, -1e18]:
...     kahan_sum.add(xi)
>>> kahan_sum.result()
1.0
"""


class KahanSum:
    """The second-order Kahan-Babuška-Neumaier sum by Klein."""

    def __init__(self) -> None:
        """Create the summation object."""
        #: the running sum, an internal variable invisible from outside
        self.__sum: float | int = 0
        #: the first correction term, another internal variable
        self.__cs: float | int = 0
        #: the second correction term, another internal variable
        self.__ccs: float | int = 0

    def add(self, value: int | float) -> None:
        """
        Add a value to the sum.

        :param value: the value to add
        """
        s: int | float = self.__sum  # Get the current running sum.
        t: int | float = s + value   # Compute the new sum value.
        c: int | float = (((s - t) + value) if abs(s) >= abs(value)
                          else ((value - t) + s))  # The Neumaier tweak.
        self.__sum = t  # Store the new sum value.
        cs: int | float = self.__cs  # the current 1st-order correction
        t = cs + c  # Compute the new first-order correction term.
        cc: int | float = ((((cs - t) + c) if abs(cs) >= abs(c)
                            else ((c - t) + cs))  # 2nd Neumaier tweak.
        self.__cs = t  # Store the updated first-order correction term.
        self.__ccs += cc  # Update the second-order correction.

    def result(self) -> int | float:
        """
        Get the current result of the summation.

        :return: the current result of the summation
        """
        return self.__sum + self.__cs + self.__ccs
```

Kahan-Summe: Implementation

- Es gibt nichts im Schleifenkörper vom Algorithmus, das wir nicht mehr oder weniger direkt in Python-Kode übertragen können.

Algorithm 1: Kahan-Babuška-Neumaier Summe

$sum \leftarrow 0; \quad cs \leftarrow 0; \quad ccs \leftarrow 0;$
for $i \in 0..n-1$ **do**
  $t \leftarrow sum + x[i];$
  if $|sum| \geq |x[i]|$ **then** $c \leftarrow (sum - t) + x[i];$
  else $c \leftarrow (x[i] - t) + sum;$
  $sum \leftarrow t;$
  $t \leftarrow cs + c;$
  if $|cs| \geq |c|$ **then** $cc \leftarrow (cs - t) + c;$
  else $c \leftarrow (c - t) + cs;$
  $cs \leftarrow t; \quad ccs \leftarrow ccs + cc;$
  return $sum + cs + ccs$

```python
"""
The second-order Kahan-Babuška-Neumaier-Summation by Klein.

[1] A. Klein. A Generalized Kahan-Babuška-Summation-Algorithm.
    Computing 76:279-293. 2006. doi:10.1007/s00607-005-0139-x

>>> kahan_sum = KahanSum()
>>> for xi in [1e18, 1, 1e36, -1e36, -1e18]:
...     kahan_sum.add(xi)
>>> kahan_sum.result()
1.0
"""


class KahanSum:
    """The second-order Kahan-Babuška-Neumaier sum by Klein."""

    def __init__(self) -> None:
        """Create the summation object."""
        #: the running sum, an internal variable invisible from outside
        self.__sum: float | int = 0
        #: the first correction term, another internal variable
        self.__cs: float | int = 0
        #: the second correction term, another internal variable
        self.__ccs: float | int = 0

    def add(self, value: int | float) -> None:
        """
        Add a value to the sum.

        :param value: the value to add
        """
        s: int | float = self.__sum  # Get the current running sum.
        t: int | float = s + value   # Compute the new sum value.
        c: int | float = (((s - t) + value) if abs(s) >= abs(value)
                          else ((value - t) + s))  # The Neumaier tweak.
        self.__sum = t  # Store the new sum value.
        cs: int | float = self.__cs  # the current 1st-order correction
        t = cs + c  # Compute the new first-order correction term.
        cc: int | float = ((((cs - t) + c) if abs(cs) >= abs(c)
                            else ((c - t) + cs))  # 2nd Neumaier tweak.
        self.__cs = t  # Store the updated first-order correction term.
        self.__ccs += cc  # Update the second-order correction.

    def result(self) -> int | float:
        """
        Get the current result of the summation.

        :return: the current result of the summation
        """
        return self.__sum + self.__cs + self.__ccs
```

Kahan-Summe: Implementation

- Wir können nun die letzte Zeile des Algorithmus, die das Endergebnis der Summe liefert, in die neue Methode `result` packen.

Algorithm 1: Kahan-Babuška-Neumaier Summe

$sum \leftarrow 0; \quad cs \leftarrow 0; \quad ccs \leftarrow 0;$
for $i \in 0..n-1$ **do**
  $t \leftarrow sum + x[i];$
  if $|sum| \geq |x[i]|$ **then** $c \leftarrow (sum - t) + x[i];$
  else $c \leftarrow (x[i] - t) + sum;$
  $sum \leftarrow t;$
  $t \leftarrow cs + c;$
  if $|cs| \geq |c|$ **then** $cc \leftarrow (cs - t) + c;$
  else $c \leftarrow (c - t) + cs;$
  $cs \leftarrow t; \quad ccs \leftarrow ccs + cc;$
  return $sum + cs + ccs$

```python
"""
The second-order Kahan-Babuška-Neumaier-Summation by Klein.

[1] A. Klein. A Generalized Kahan-Babuška-Summation-Algorithm.
    Computing 76:279-293. 2006. doi:10.1007/s00607-005-0139-x

>>> kahan_sum = KahanSum()
>>> for xi in [1e18, 1, 1e36, -1e36, -1e18]:
...     kahan_sum.add(xi)
>>> kahan_sum.result()
1.0
"""


class KahanSum:
    """The second-order Kahan-Babuška-Neumaier sum by Klein."""

    def __init__(self) -> None:
        """Create the summation object."""
        #: the running sum, an internal variable invisible from outside
        self.__sum: float | int = 0
        #: the first correction term, another internal variable
        self.__cs: float | int = 0
        #: the second correction term, another internal variable
        self.__ccs: float | int = 0

    def add(self, value: int | float) -> None:
        """
        Add a value to the sum.

        :param value: the value to add
        """
        s: int | float = self.__sum  # Get the current running sum.
        t: int | float = s + value   # Compute the new sum value.
        c: int | float = (((s - t) + value) if abs(s) >= abs(value)
                          else ((value - t) + s))  # The Neumaier tweak.
        self.__sum = t  # Store the new sum value.
        cs: int | float = self.__cs  # the current 1st-order correction
        t = cs + c  # Compute the new first-order correction term.
        cc: int | float = ((((cs - t) + c) if abs(cs) >= abs(c)
                            else ((c - t) + cs))  # 2nd Neumaier tweak.
        self.__cs = t  # Store the updated first-order correction term.
        self.__ccs += cc  # Update the second-order correction.

    def result(self) -> int | float:
        """
        Get the current result of the summation.

        :return: the current result of the summation
        """
        return self.__sum + self.__cs + self.__ccs
```

Kahan-Summe: Implementation

- Natürlich wird diese auf die Attribute `self.__sum`, `self.__cs` und `self.__ccs` anstelle von sum, cs und ccs zugreifen.

Algorithm 1: Kahan-Babuška-Neumaier Summe

$sum \leftarrow 0; \quad cs \leftarrow 0; \quad ccs \leftarrow 0;$
for $i \in 0..n-1$ **do**
  $t \leftarrow sum + x[i];$
  if $|sum| \geq |x[i]|$ **then** $c \leftarrow (sum - t) + x[i];$
  else $c \leftarrow (x[i] - t) + sum;$
  $sum \leftarrow t;$
  $t \leftarrow cs + c;$
  if $|cs| \geq |c|$ **then** $cc \leftarrow (cs - t) + c;$
  else $c \leftarrow (c - t) + cs;$
  $cs \leftarrow t; \quad ccs \leftarrow ccs + cc;$
  return $sum + cs + ccs$

```python
"""
The second-order Kahan-Babuška-Neumaier-Summation by Klein.

[1] A. Klein. A Generalized Kahan-Babuška-Summation-Algorithm.
    Computing 76:279-293. 2006. doi:10.1007/s00607-005-0139-x

>>> kahan_sum = KahanSum()
>>> for xi in [1e18, 1, 1e36, -1e36, -1e18]:
...     kahan_sum.add(xi)
>>> kahan_sum.result()
1.0
"""


class KahanSum:
    """The second-order Kahan-Babuška-Neumaier sum by Klein."""

    def __init__(self) -> None:
        """Create the summation object."""
        #: the running sum, an internal variable invisible from outside
        self.__sum: float | int = 0
        #: the first correction term, another internal variable
        self.__cs: float | int = 0
        #: the second correction term, another internal variable
        self.__ccs: float | int = 0

    def add(self, value: int | float) -> None:
        """
        Add a value to the sum.

        :param value: the value to add
        """
        s: int | float = self.__sum  # Get the current running sum.
        t: int | float = s + value   # Compute the new sum value.
        c: int | float = (((s - t) + value) if abs(s) >= abs(value)
                          else ((value - t) + s))  # The Neumaier tweak.
        self.__sum = t  # Store the new sum value.
        cs: int | float = self.__cs  # the current 1st-order correction
        t = cs + c  # Compute the new first-order correction term.
        cc: int | float = ((((cs - t) + c) if abs(cs) >= abs(c)
                            else ((c - t) + cs))  # 2nd Neumaier tweak.
        self.__cs = t  # Store the updated first-order correction term.
        self.__ccs += cc  # Update the second-order correction.

    def result(self) -> int | float:
        """
        Get the current result of the summation.

        :return: the current result of the summation
        """
        return self.__sum + self.__cs + self.__ccs
```

Kahan-Summe: Implementation

- Und damit ist das fertig.

Algorithm 1: Kahan-Babuška-Neumaier Summe

$sum \leftarrow 0; \quad cs \leftarrow 0; \quad ccs \leftarrow 0;$
for $i \in 0..n-1$ **do**
  $t \leftarrow sum + x[i];$
  if $|sum| \geq |x[i]|$ **then** $c \leftarrow (sum - t) + x[i];$
  else $c \leftarrow (x[i] - t) + sum;$
  $sum \leftarrow t;$
  $t \leftarrow cs + c;$
  if $|cs| \geq |c|$ **then** $cc \leftarrow (cs - t) + c;$
  else $c \leftarrow (c - t) + cs;$
  $cs \leftarrow t; \quad ccs \leftarrow ccs + cc;$
  return $sum + cs + ccs$

```python
"""
The second-order Kahan-Babuška-Neumaier-Summation by Klein.

[1] A. Klein. A Generalized Kahan-Babuška-Summation-Algorithm.
    Computing 76:279-293. 2006. doi:10.1007/s00607-005-0139-x

>>> kahan_sum = KahanSum()
>>> for xi in [1e18, 1, 1e36, -1e36, -1e18]:
...     kahan_sum.add(xi)
>>> kahan_sum.result()
1.0
"""


class KahanSum:
    """The second-order Kahan-Babuška-Neumaier sum by Klein."""

    def __init__(self) -> None:
        """Create the summation object."""
        #: the running sum, an internal variable invisible from outside
        self.__sum: float | int = 0
        #: the first correction term, another internal variable
        self.__cs: float | int = 0
        #: the second correction term, another internal variable
        self.__ccs: float | int = 0

    def add(self, value: int | float) -> None:
        """
        Add a value to the sum.

        :param value: the value to add
        """
        s: int | float = self.__sum  # Get the current running sum.
        t: int | float = s + value   # Compute the new sum value.
        c: int | float = (((s - t) + value) if abs(s) >= abs(value)
                          else ((value - t) + s))  # The Neumaier tweak.
        self.__sum = t  # Store the new sum value.
        cs: int | float = self.__cs  # the current 1st-order correction
        t = cs + c  # Compute the new first-order correction term.
        cc: int | float = ((((cs - t) + c) if abs(cs) >= abs(c)
                            else ((c - t) + cs))  # 2nd Neumaier tweak.
        self.__cs = t  # Store the updated first-order correction term.
        self.__ccs += cc  # Update the second-order correction.

    def result(self) -> int | float:
        """
        Get the current result of the summation.

        :return: the current result of the summation
        """
        return self.__sum + self.__cs + self.__ccs
```

Kahan-Summe: Implementation

- Und damit ist das fertig.
- Wir haben einen relativ komplexen mathematischen Algorithmus in ein Stück Python-Kode übersetzt.

Algorithm 1: Kahan-Babuška-Neumaier Summe

$sum \leftarrow 0; \quad cs \leftarrow 0; \quad ccs \leftarrow 0;$
for $i \in 0..n-1$ **do**
  $t \leftarrow sum + x[i];$
  if $|sum| \geq |x[i]|$ **then** $c \leftarrow (sum - t) + x[i];$
  else $c \leftarrow (x[i] - t) + sum;$
  $sum \leftarrow t;$
  $t \leftarrow cs + c;$
  if $|cs| \geq |c|$ **then** $cc \leftarrow (cs - t) + c;$
  else $c \leftarrow (c - t) + cs;$
  $cs \leftarrow t; \quad ccs \leftarrow ccs + cc;$
  return $sum + cs + ccs$

```python
"""
The second-order Kahan-Babuška-Neumaier-Summation by Klein.

[1] A. Klein. A Generalized Kahan-Babuška-Summation-Algorithm.
    Computing 76:279-293. 2006. doi:10.1007/s00607-005-0139-x

>>> kahan_sum = KahanSum()
>>> for xi in [1e18, 1, 1e36, -1e36, -1e18]:
...     kahan_sum.add(xi)
>>> kahan_sum.result()
1.0
"""


class KahanSum:
    """The second-order Kahan-Babuška-Neumaier sum by Klein."""

    def __init__(self) -> None:
        """Create the summation object."""
        #: the running sum, an internal variable invisible from outside
        self.__sum: float | int = 0
        #: the first correction term, another internal variable
        self.__cs: float | int = 0
        #: the second correction term, another internal variable
        self.__ccs: float | int = 0

    def add(self, value: int | float) -> None:
        """
        Add a value to the sum.

        :param value: the value to add
        """
        s: int | float = self.__sum  # Get the current running sum.
        t: int | float = s + value   # Compute the new sum value.
        c: int | float = (((s - t) + value) if abs(s) >= abs(value)
                          else ((value - t) + s))  # The Neumaier tweak.
        self.__sum = t  # Store the new sum value.
        cs: int | float = self.__cs  # the current 1st-order correction
        t = cs + c  # Compute the new first-order correction term.
        cc: int | float = ((((cs - t) + c) if abs(cs) >= abs(c)
                            else ((c - t) + cs))  # 2nd Neumaier tweak.
        self.__cs = t  # Store the updated first-order correction term.
        self.__ccs += cc  # Update the second-order correction.

    def result(self) -> int | float:
        """
        Get the current result of the summation.

        :return: the current result of the summation
        """
        return self.__sum + self.__cs + self.__ccs
```

Kahan-Summe: Implementation

- Wir haben einen monolithischen Algorithmus in eine API verpackt, die iterativ aufgerufen werden kann.

Algorithm 1: Kahan-Babuška-Neumaier Summe

$sum \leftarrow 0; \quad cs \leftarrow 0; \quad ccs \leftarrow 0;$
for $i \in 0..n-1$ **do**
  $t \leftarrow sum + x[i];$
  if $|sum| \geq |x[i]|$ **then** $c \leftarrow (sum - t) + x[i];$
  else $c \leftarrow (x[i] - t) + sum;$
  $sum \leftarrow t;$
  $t \leftarrow cs + c;$
  if $|cs| \geq |c|$ **then** $cc \leftarrow (cs - t) + c;$
  else $c \leftarrow (c - t) + cs;$
  $cs \leftarrow t; \quad ccs \leftarrow ccs + cc;$
  return $sum + cs + ccs$

```python
"""
The second-order Kahan-Babuška-Neumaier-Summation by Klein.

[1] A. Klein. A Generalized Kahan-Babuška-Summation-Algorithm.
    Computing 76:279-293. 2006. doi:10.1007/s00607-005-0139-x

>>> kahan_sum = KahanSum()
>>> for xi in [1e18, 1, 1e36, -1e36, -1e18]:
...     kahan_sum.add(xi)
>>> kahan_sum.result()
1.0
"""


class KahanSum:
    """The second-order Kahan-Babuška-Neumaier sum by Klein."""

    def __init__(self) -> None:
        """Create the summation object."""
        #: the running sum, an internal variable invisible from outside
        self.__sum: float | int = 0
        #: the first correction term, another internal variable
        self.__cs: float | int = 0
        #: the second correction term, another internal variable
        self.__ccs: float | int = 0

    def add(self, value: int | float) -> None:
        """
        Add a value to the sum.

        :param value: the value to add
        """
        s: int | float = self.__sum  # Get the current running sum.
        t: int | float = s + value   # Compute the new sum value.
        c: int | float = (((s - t) + value) if abs(s) >= abs(value)
                          else ((value - t) + s))  # The Neumaier tweak.
        self.__sum = t  # Store the new sum value.
        cs: int | float = self.__cs  # the current 1st-order correction
        t = cs + c  # Compute the new first-order correction term.
        cc: int | float = ((((cs - t) + c) if abs(cs) >= abs(c)
                            else ((c - t) + cs))  # 2nd Neumaier tweak.
        self.__cs = t  # Store the updated first-order correction term.
        self.__ccs += cc  # Update the second-order correction.

    def result(self) -> int | float:
        """
        Get the current result of the summation.

        :return: the current result of the summation
        """
        return self.__sum + self.__cs + self.__ccs
```

Kahan-Summe: Implementation

- Wir haben einen monolithischen Algorithmus in eine API verpackt, die iterativ aufgerufen werden kann.
- Aber funktioniert er auch?

Algorithm 1: Kahan-Babuška-Neumaier Summe

$sum \leftarrow 0; \quad cs \leftarrow 0; \quad ccs \leftarrow 0;$
for $i \in 0..n-1$ **do**
  $t \leftarrow sum + x[i];$
  if $|sum| \geq |x[i]|$ **then** $c \leftarrow (sum - t) + x[i];$
  else $c \leftarrow (x[i] - t) + sum;$
  $sum \leftarrow t;$
  $t \leftarrow cs + c;$
  if $|cs| \geq |c|$ **then** $cc \leftarrow (cs - t) + c;$
  else $c \leftarrow (c - t) + cs;$
  $cs \leftarrow t; \quad ccs \leftarrow ccs + cc;$
  return $sum + cs + ccs$

```python
"""
The second-order Kahan-Babuška-Neumaier-Summation by Klein.

[1] A. Klein. A Generalized Kahan-Babuška-Summation-Algorithm.
    Computing 76:279-293. 2006. doi:10.1007/s00607-005-0139-x

>>> kahan_sum = KahanSum()
>>> for xi in [1e18, 1, 1e36, -1e36, -1e18]:
...     kahan_sum.add(xi)
>>> kahan_sum.result()
1.0
"""


class KahanSum:
    """The second-order Kahan-Babuška-Neumaier sum by Klein."""

    def __init__(self) -> None:
        """Create the summation object."""
        #: the running sum, an internal variable invisible from outside
        self.__sum: float | int = 0
        #: the first correction term, another internal variable
        self.__cs: float | int = 0
        #: the second correction term, another internal variable
        self.__ccs: float | int = 0

    def add(self, value: int | float) -> None:
        """
        Add a value to the sum.

        :param value: the value to add
        """
        s: int | float = self.__sum  # Get the current running sum.
        t: int | float = s + value   # Compute the new sum value.
        c: int | float = (((s - t) + value) if abs(s) >= abs(value)
                          else ((value - t) + s))  # The Neumaier tweak.
        self.__sum = t  # Store the new sum value.
        cs: int | float = self.__cs  # the current 1st-order correction
        t = cs + c  # Compute the new first-order correction term.
        cc: int | float = ((((cs - t) + c) if abs(cs) >= abs(c)
                            else ((c - t) + cs))  # 2nd Neumaier tweak.
        self.__cs = t  # Store the updated first-order correction term.
        self.__ccs += cc  # Update the second-order correction.

    def result(self) -> int | float:
        """
        Get the current result of the summation.

        :return: the current result of the summation
        """
        return self.__sum + self.__cs + self.__ccs
```

Kahan-Summe: Implementation

- Aber funktioniert er auch?
- Wir platzieren erstmal einen Doctest in den Docstring des Moduls.

Algorithm 1: Kahan-Babuška-Neumaier Summe

$sum \leftarrow 0; \quad cs \leftarrow 0; \quad ccs \leftarrow 0;$
for $i \in 0..n-1$ **do**
  $t \leftarrow sum + x[i];$
  if $|sum| \geq |x[i]|$ **then** $c \leftarrow (sum - t) + x[i];$
  else $c \leftarrow (x[i] - t) + sum;$
  $sum \leftarrow t;$
  $t \leftarrow cs + c;$
  if $|cs| \geq |c|$ **then** $cc \leftarrow (cs - t) + c;$
  else $c \leftarrow (c - t) + cs;$
  $cs \leftarrow t; \quad ccs \leftarrow ccs + cc;$
  return $sum + cs + ccs$

```
"""
The second-order Kahan-Babuška-Neumaier-Summation by Klein.

[1] A. Klein. A Generalized Kahan-Babuška-Summation-Algorithm.
    Computing 76:279-293. 2006. doi:10.1007/s00607-005-0139-x

>>> kahan_sum = KahanSum()
>>> for xi in [1e18, 1, 1e36, -1e36, -1e18]:
...     kahan_sum.add(xi)
>>> kahan_sum.result()
1.0
"""



class KahanSum:
    """The second-order Kahan-Babuška-Neumaier sum by Klein."""

    def __init__(self) -> None:
        """Create the summation object."""
        #: the running sum, an internal variable invisible from outside
        self.__sum: float | int = 0
        #: the first correction term, another internal variable
        self.__cs: float | int = 0
        #: the second correction term, another internal variable
        self.__ccs: float | int = 0

    def add(self, value: int | float) -> None:
        """
        Add a value to the sum.

        :param value: the value to add
        """
        s: int | float = self.__sum  # Get the current running sum.
        t: int | float = s + value   # Compute the new sum value.
        c: int | float = (((s - t) + value) if abs(s) >= abs(value)
                          else ((value - t) + s))  # The Neumaier tweak.
        self.__sum = t  # Store the new sum value.
        cs: int | float = self.__cs  # the current 1st-order correction
        t = cs + c  # Compute the new first-order correction term.
        cc: int | float = ((((cs - t) + c) if abs(cs) >= abs(c)
                            else ((c - t) + cs))  # 2nd Neumaier tweak.
        self.__cs = t  # Store the updated first-order correction term.
        self.__ccs += cc  # Update the second-order correction.

    def result(self) -> int | float:
        """
        Get the current result of the summation.

        :return: the current result of the summation
        """
        return self.__sum + self.__cs + self.__ccs
```

Kahan-Summe: Implementation

- Dieser Doctest berechnet die Summe `1e18 + 1 + 1e36 - 1e36 - 1e18`, die unser Beispiel am Anfang der Einheit war.

Algorithm 1: Kahan-Babuška-Neumaier Summe

$sum \leftarrow 0; \quad cs \leftarrow 0; \quad ccs \leftarrow 0;$
for $i \in 0..n-1$ **do**
  $t \leftarrow sum + x[i];$
  if $|sum| \geq |x[i]|$ **then** $c \leftarrow (sum - t) + x[i];$
  else $c \leftarrow (x[i] - t) + sum;$
  $sum \leftarrow t;$
  $t \leftarrow cs + c;$
  if $|cs| \geq |c|$ **then** $cc \leftarrow (cs - t) + c;$
  else $c \leftarrow (c - t) + cs;$
  $cs \leftarrow t; \quad ccs \leftarrow ccs + cc;$
  return $sum + cs + ccs$

```python
"""
The second-order Kahan-Babuška-Neumaier-Summation by Klein.

[1] A. Klein. A Generalized Kahan-Babuška-Summation-Algorithm.
    Computing 76:279-293. 2006. doi:10.1007/s00607-005-0139-x

>>> kahan_sum = KahanSum()
>>> for xi in [1e18, 1, 1e36, -1e36, -1e18]:
...     kahan_sum.add(xi)
>>> kahan_sum.result()
1.0
"""


class KahanSum:
    """The second-order Kahan-Babuška-Neumaier sum by Klein."""

    def __init__(self) -> None:
        """Create the summation object."""
        #: the running sum, an internal variable invisible from outside
        self.__sum: float | int = 0
        #: the first correction term, another internal variable
        self.__cs: float | int = 0
        #: the second correction term, another internal variable
        self.__ccs: float | int = 0

    def add(self, value: int | float) -> None:
        """
        Add a value to the sum.

        :param value: the value to add
        """
        s: int | float = self.__sum  # Get the current running sum.
        t: int | float = s + value   # Compute the new sum value.
        c: int | float = (((s - t) + value) if abs(s) >= abs(value)
                          else ((value - t) + s))  # The Neumaier tweak.
        self.__sum = t  # Store the new sum value.
        cs: int | float = self.__cs  # the current 1st-order correction
        t = cs + c  # Compute the new first-order correction term.
        cc: int | float = ((((cs - t) + c) if abs(cs) >= abs(c)
                            else ((c - t) + cs))  # 2nd Neumaier tweak.
        self.__cs = t  # Store the updated first-order correction term.
        self.__ccs += cc  # Update the second-order correction.

    def result(self) -> int | float:
        """
        Get the current result of the summation.

        :return: the current result of the summation
        """
        return self.__sum + self.__cs + self.__ccs
```

Kahan-Summe: Implementation

- Dort haben wir gelernt, dass wenn wir die Summe direkt berechnen, einfach `0.0` rauskommt.

Algorithm 1: Kahan-Babuška-Neumaier Summe

$sum \leftarrow 0; \quad cs \leftarrow 0; \quad ccs \leftarrow 0;$
for $i \in 0..n-1$ **do**
  $t \leftarrow sum + x[i];$
  if $|sum| \geq |x[i]|$ **then** $c \leftarrow (sum - t) + x[i];$
  else $c \leftarrow (x[i] - t) + sum;$
  $sum \leftarrow t;$
  $t \leftarrow cs + c;$
  if $|cs| \geq |c|$ **then** $cc \leftarrow (cs - t) + c;$
  else $c \leftarrow (c - t) + cs;$
  $cs \leftarrow t; \quad ccs \leftarrow ccs + cc;$
  return $sum + cs + ccs$

```python
"""
The second-order Kahan-Babuška-Neumaier-Summation by Klein.

[1] A. Klein. A Generalized Kahan-Babuška-Summation-Algorithm.
    Computing 76:279-293. 2006. doi:10.1007/s00607-005-0139-x

>>> kahan_sum = KahanSum()
>>> for xi in [1e18, 1, 1e36, -1e36, -1e18]:
...     kahan_sum.add(xi)
>>> kahan_sum.result()
1.0
"""


class KahanSum:
    """The second-order Kahan-Babuška-Neumaier sum by Klein."""

    def __init__(self) -> None:
        """Create the summation object."""
        #: the running sum, an internal variable invisible from outside
        self.__sum: float | int = 0
        #: the first correction term, another internal variable
        self.__cs: float | int = 0
        #: the second correction term, another internal variable
        self.__ccs: float | int = 0

    def add(self, value: int | float) -> None:
        """
        Add a value to the sum.

        :param value: the value to add
        """
        s: int | float = self.__sum  # Get the current running sum.
        t: int | float = s + value   # Compute the new sum value.
        c: int | float = (((s - t) + value) if abs(s) >= abs(value)
                          else ((value - t) + s))  # The Neumaier tweak.
        self.__sum = t  # Store the new sum value.
        cs: int | float = self.__cs  # the current 1st-order correction
        t = cs + c  # Compute the new first-order correction term.
        cc: int | float = ((((cs - t) + c) if abs(cs) >= abs(c)
                            else ((c - t) + cs))  # 2nd Neumaier tweak.
        self.__cs = t  # Store the updated first-order correction term.
        self.__ccs += cc  # Update the second-order correction.

    def result(self) -> int | float:
        """
        Get the current result of the summation.

        :return: the current result of the summation
        """
        return self.__sum + self.__cs + self.__ccs
```

Kahan-Summe: Implementation

- Wir wissen aber, dass das richtige Ergebnis `1.0` ist.

Algorithm 1: Kahan-Babuška-Neumaier Summe

$sum \leftarrow 0; \quad cs \leftarrow 0; \quad ccs \leftarrow 0;$
for $i \in 0..n-1$ **do**
  $t \leftarrow sum + x[i];$
  if $|sum| \geq |x[i]|$ **then** $c \leftarrow (sum - t) + x[i];$
  else $c \leftarrow (x[i] - t) + sum;$
  $sum \leftarrow t;$
  $t \leftarrow cs + c;$
  if $|cs| \geq |c|$ **then** $cc \leftarrow (cs - t) + c;$
  else $c \leftarrow (c - t) + cs;$
  $cs \leftarrow t; \quad ccs \leftarrow ccs + cc;$
  return $sum + cs + ccs$

```python
"""
The second-order Kahan-Babuška-Neumaier-Summation by Klein.

[1] A. Klein. A Generalized Kahan-Babuška-Summation-Algorithm.
    Computing 76:279-293. 2006. doi:10.1007/s00607-005-0139-x

>>> kahan_sum = KahanSum()
>>> for xi in [1e18, 1, 1e36, -1e36, -1e18]:
...     kahan_sum.add(xi)
>>> kahan_sum.result()
1.0
"""


class KahanSum:
    """The second-order Kahan-Babuška-Neumaier sum by Klein."""

    def __init__(self) -> None:
        """Create the summation object."""
        #: the running sum, an internal variable invisible from outside
        self.__sum: float | int = 0
        #: the first correction term, another internal variable
        self.__cs: float | int = 0
        #: the second correction term, another internal variable
        self.__ccs: float | int = 0

    def add(self, value: int | float) -> None:
        """
        Add a value to the sum.

        :param value: the value to add
        """
        s: int | float = self.__sum  # Get the current running sum.
        t: int | float = s + value   # Compute the new sum value.
        c: int | float = (((s - t) + value) if abs(s) >= abs(value)
                          else ((value - t) + s))  # The Neumaier tweak.
        self.__sum = t  # Store the new sum value.
        cs: int | float = self.__cs  # the current 1st-order correction
        t = cs + c  # Compute the new first-order correction term.
        cc: int | float = (((cs - t) + c) if abs(cs) >= abs(c)
                           else ((c - t) + cs))  # 2nd Neumaier tweak.
        self.__cs = t  # Store the updated first-order correction term.
        self.__ccs += cc  # Update the second-order correction.

    def result(self) -> int | float:
        """
        Get the current result of the summation.

        :return: the current result of the summation
        """
        return self.__sum + self.__cs + self.__ccs
```

Kahan-Summe: Implementation

- Wenn wir mit unserer Klasse `KahanSum` die Zahlen `[1e18, 1, 1e36, -1e36, -1e18]` aufaddieren, dann *sollte* dieses richtige Ergebnis herauskommen.

Algorithm 1: Kahan-Babuška-Neumaier Summe

$sum \leftarrow 0; \quad cs \leftarrow 0; \quad ccs \leftarrow 0;$
for $i \in 0..n-1$ **do**
  $t \leftarrow sum + x[i];$
  if $|sum| \geq |x[i]|$ **then** $c \leftarrow (sum - t) + x[i];$
  else $c \leftarrow (x[i] - t) + sum;$
  $sum \leftarrow t;$
  $t \leftarrow cs + c;$
  if $|cs| \geq |c|$ **then** $cc \leftarrow (cs - t) + c;$
  else $c \leftarrow (c - t) + cs;$
  $cs \leftarrow t; \quad ccs \leftarrow ccs + cc;$
  return $sum + cs + ccs$

```python
"""
The second-order Kahan-Babuška-Neumaier-Summation by Klein.

[1] A. Klein. A Generalized Kahan-Babuška-Summation-Algorithm.
    Computing 76:279-293. 2006. doi:10.1007/s00607-005-0139-x

>>> kahan_sum = KahanSum()
>>> for xi in [1e18, 1, 1e36, -1e36, -1e18]:
...     kahan_sum.add(xi)
>>> kahan_sum.result()
1.0
"""


class KahanSum:
    """The second-order Kahan-Babuška-Neumaier sum by Klein."""

    def __init__(self) -> None:
        """Create the summation object."""
        #: the running sum, an internal variable invisible from outside
        self.__sum: float | int = 0
        #: the first correction term, another internal variable
        self.__cs: float | int = 0
        #: the second correction term, another internal variable
        self.__ccs: float | int = 0

    def add(self, value: int | float) -> None:
        """
        Add a value to the sum.

        :param value: the value to add
        """
        s: int | float = self.__sum  # Get the current running sum.
        t: int | float = s + value   # Compute the new sum value.
        c: int | float = (((s - t) + value) if abs(s) >= abs(value)
                          else ((value - t) + s))  # The Neumaier tweak.
        self.__sum = t  # Store the new sum value.
        cs: int | float = self.__cs  # the current 1st-order correction
        t = cs + c  # Compute the new first-order correction term.
        cc: int | float = ((((cs - t) + c) if abs(cs) >= abs(c)
                            else ((c - t) + cs))  # 2nd Neumaier tweak.
        self.__cs = t  # Store the updated first-order correction term.
        self.__ccs += cc  # Update the second-order correction.

    def result(self) -> int | float:
        """
        Get the current result of the summation.

        :return: the current result of the summation
        """
        return self.__sum + self.__cs + self.__ccs
```

Kahan-Summe: Implementation

- Wenn wir mit unserer Klasse `KahanSum` die Zahlen `[1e18, 1, 1e36, -1e36, -1e18]` aufaddieren, dann *sollte* dieses richtige Ergebnis herauskommen.
- Wir wir sehen, tut es das auch.

```
$ pytest --timeout=10 --no-header --tb=short --doctest-modules kahan_sum
    ↪ .py
========================= test session starts
    ↪ ==========================
collected 1 item
kahan_sum.py .                                                      [100%]
========================== 1 passed in 0.01s
    ↪ ===========================
# pytest 9.2.0 with pytest-timeout 2.4.0 succeeded with exit code 0.
```

```
"""
The second-order Kahan-Babuška-Neumaier-Summation by Klein.

[1] A. Klein. A Generalized Kahan-Babuška-Summation-Algorithm.
    Computing 76:279-293. 2006. doi:10.1007/s00607-005-0139-x

>>> kahan_sum = KahanSum()
>>> for xi in [1e18, 1, 1e36, -1e36, -1e18]:
...     kahan_sum.add(xi)
>>> kahan_sum.result()
1.0
"""


class KahanSum:
    """The second-order Kahan-Babuška-Neumaier sum by Klein."""

    def __init__(self) -> None:
        """Create the summation object."""
        #: the running sum, an internal variable invisible from outside
        self.__sum: float | int = 0
        #: the first correction term, another internal variable
        self.__cs: float | int = 0
        #: the second correction term, another internal variable
        self.__ccs: float | int = 0

    def add(self, value: int | float) -> None:
        """
        Add a value to the sum.

        :param value: the value to add
        """
        s: int | float = self.__sum  # Get the current running sum.
        t: int | float = s + value   # Compute the new sum value.
        c: int | float = (((s - t) + value) if abs(s) >= abs(value)
                          else ((value - t) + s))  # The Neumaier tweak.
        self.__sum = t  # Store the new sum value.
        cs: int | float = self.__cs  # the current 1st-order correction
        t = cs + c  # Compute the new first-order correction term.
        cc: int | float = ((((cs - t) + c) if abs(cs) >= abs(c)
                            else ((c - t) + cs))  # 2nd Neumaier tweak.
        self.__cs = t  # Store the updated first-order correction term.
        self.__ccs += cc  # Update the second-order correction.

    def result(self) -> int | float:
        """
        Get the current result of the summation.

        :return: the current result of the summation
        """
        return self.__sum + self.__cs + self.__ccs
```

Kahan Summe: Vergleich

- Im Programm `kahan_user.py` benutzen wir nun unsere neue `KahanSum` Klasse.

```python
"""Examples for using our class :class:`KahanSum`."""

from math import fsum  # An (even more) precise summation algorithm.

from kahan_sum import KahanSum  # Import our new own class.

# Iterate over four example arrays.
for numbers in [[1e-15, 1e-14, 1e-13, 1e-16, 1e-12], [1e18, 1, -1e18],
                [ 1e36, 1e18, 1, -1e36, -1e18],
                [1e36, 1e72, 1e18, -1e36, -1e72, 1, -1e18],
                [1, -1e-16, 1e-16, 1e-16]]:
    print(f"======= numbers = [{', '.join(map(str, numbers))}] =======")
    k: KahanSum = KahanSum()  # Create our Kahan summation object.
    for n in numbers:         # Iterate over the numbers...
        k.add(n)              # ...and let our object add them up.
    print(f"sum(numbers)  = {sum(numbers)}")    # the normal sum
    print(f"Kahan sum     = {k.result()}")      # our better result
    print(f"fsum(numbers) = {fsum(numbers)}")   # the exact result
```

↓ python3 kahan_user.py ↓

```
======= numbers = [1e-15, 1e-14, 1e-13, 1e-16, 1e-12] =======
sum(numbers)  = 1.1111e-12
Kahan sum     = 1.1111e-12
fsum(numbers) = 1.1111e-12
======= numbers = [1e+18, 1, -1e+18] =======
sum(numbers)  = 1.0
Kahan sum     = 1.0
fsum(numbers) = 1.0
======= numbers = [1e+36, 1e+18, 1, -1e+36, -1e+18] =======
sum(numbers)  = 0.0
Kahan sum     = 1.0
fsum(numbers) = 1.0
======= numbers = [1e+36, 1e+72, 1e+18, -1e+36, -1e+72, 1, -1e+18]
    ↪ =======
sum(numbers)  = -1e+18
Kahan sum     = 0.0
fsum(numbers) = 1.0
======= numbers = [1, -1e-16, 1e-16, 1e-16] =======
sum(numbers)  = 1.0
Kahan sum     = 1.0
fsum(numbers) = 1.0
```

Kahan Summe: Vergleich

- Im Programm `kahan_user.py` benutzen wir nun unsere neue `KahanSum` Klasse.
- Wir vergleichen ihre Ergebnisse mit der built-in Funktion `sum` und der genauen Summenfunktion `fsum` aus dem `math` Modul.

```python
"""Examples for using our class :class:`KahanSum`."""

from math import fsum  # An (even more) precise summation algorithm.

from kahan_sum import KahanSum  # Import our new own class.

# Iterate over four example arrays.
for numbers in [[1e-15, 1e-14, 1e-13, 1e-16, 1e-12], [1e18, 1, -1e18],
                [ 1e36, 1e18, 1, -1e36, -1e18],
                [1e36, 1e72, 1e18, -1e36, -1e72, 1, -1e18],
                [1, -1e-16, 1e-16, 1e-16]]:
    print(f"======= numbers = [{', '.join(map(str, numbers))}] =======")
    k: KahanSum = KahanSum()  # Create our Kahan summation object.
    for n in numbers:         # Iterate over the numbers...
        k.add(n)              # ...and let our object add them up.
    print(f"sum(numbers)  = {sum(numbers)}")   # the normal sum
    print(f"Kahan sum     = {k.result()}")     # our better result
    print(f"fsum(numbers) = {fsum(numbers)}")  # the exact result
```

↓ `python3 kahan_user.py` ↓

```
======= numbers = [1e-15, 1e-14, 1e-13, 1e-16, 1e-12] =======
sum(numbers)  = 1.1111e-12
Kahan sum     = 1.1111e-12
fsum(numbers) = 1.1111e-12
======= numbers = [1e+18, 1, -1e+18] =======
sum(numbers)  = 1.0
Kahan sum     = 1.0
fsum(numbers) = 1.0
======= numbers = [1e+36, 1e+18, 1, -1e+36, -1e+18] =======
sum(numbers)  = 0.0
Kahan sum     = 1.0
fsum(numbers) = 1.0
======= numbers = [1e+36, 1e+72, 1e+18, -1e+36, -1e+72, 1, -1e+18]
    ↪ =======
sum(numbers)  = -1e+18
Kahan sum     = 0.0
fsum(numbers) = 1.0
======= numbers = [1, -1e-16, 1e-16, 1e-16] =======
sum(numbers)  = 1.0
Kahan sum     = 1.0
fsum(numbers) = 1.0
```

Kahan Summe: Vergleich

- Im Programm `kahan_user.py` benutzen wir nun unsere neue `KahanSum` Klasse.
- Wir vergleichen ihre Ergebnisse mit der built-in Funktion `sum` und der genauen Summenfunktion `fsum` aus dem `math` Modul.
- Alle drei können `[1e-15, 1e-14, 1e-13, 1e-16, 1e-12]` exakt zu `1.1111e-12` aufaddieren.

```python
"""Examples for using our class :class:`KahanSum`."""

from math import fsum  # An (even more) precise summation algorithm.

from kahan_sum import KahanSum  # Import our new own class.

# Iterate over four example arrays.
for numbers in [[1e-15, 1e-14, 1e-13, 1e-16, 1e-12], [1e18, 1, -1e18],
                [ 1e36, 1e18, 1, -1e36, -1e18],
                [1e36, 1e72, 1e18, -1e36, -1e72, 1, -1e18],
                [1, -1e-16, 1e-16, 1e-16]]:
    print(f"======= numbers = [{', '.join(map(str, numbers))}] =======")
    k: KahanSum = KahanSum()  # Create our Kahan summation object.
    for n in numbers:         # Iterate over the numbers...
        k.add(n)              # ...and let our object add them up.
    print(f"sum(numbers)  = {sum(numbers)}")   # the normal sum
    print(f"Kahan sum     = {k.result()}")     # our better result
    print(f"fsum(numbers) = {fsum(numbers)}")  # the exact result
```

↓ `python3 kahan_user.py` ↓

```
======= numbers = [1e-15, 1e-14, 1e-13, 1e-16, 1e-12] =======
sum(numbers)  = 1.1111e-12
Kahan sum     = 1.1111e-12
fsum(numbers) = 1.1111e-12
======= numbers = [1e+18, 1, -1e+18] =======
sum(numbers)  = 1.0
Kahan sum     = 1.0
fsum(numbers) = 1.0
======= numbers = [1e+36, 1e+18, 1, -1e+36, -1e+18] =======
sum(numbers)  = 0.0
Kahan sum     = 1.0
fsum(numbers) = 1.0
======= numbers = [1e+36, 1e+72, 1e+18, -1e+36, -1e+72, 1, -1e+18]
    ↪ =======
sum(numbers)  = -1e+18
Kahan sum     = 0.0
fsum(numbers) = 1.0
======= numbers = [1, -1e-16, 1e-16, 1e-16] =======
sum(numbers)  = 1.0
Kahan sum     = 1.0
fsum(numbers) = 1.0
```

Kahan Summe: Vergleich

- Im Programm `kahan_user.py` benutzen wir nun unsere neue `KahanSum` Klasse.
- Wir vergleichen ihre Ergebnisse mit der built-in Funktion `sum` und der genauen Summenfunktion `fsum` aus dem `math` Modul.
- Alle drei können `[1e-15, 1e-14, 1e-13, 1e-16, 1e-12]` exakt zu `1.1111e-12` aufaddieren.
- `sum` liefert `0.0` für die Summe von `[1e+18, 1, -1e+18]` und auch für `[1e+36, 1e+18, 1, -1e+36, -1e+18]`.

```python
"""Examples for using our class :class:`KahanSum`."""

from math import fsum  # An (even more) precise summation algorithm.

from kahan_sum import KahanSum  # Import our new own class.

# Iterate over four example arrays.
for numbers in [[1e-15, 1e-14, 1e-13, 1e-16, 1e-12], [1e18, 1, -1e18],
                [ 1e36, 1e18, 1, -1e36, -1e18],
                [1e36, 1e72, 1e18, -1e36, -1e72, 1, -1e18],
                [1, -1e-16, 1e-16, 1e-16]]:
    print(f"======= numbers = [{', '.join(map(str, numbers))}] =======")
    k: KahanSum = KahanSum()  # Create our Kahan summation object.
    for n in numbers:         # Iterate over the numbers...
        k.add(n)              # ...and let our object add them up.
    print(f"sum(numbers)  = {sum(numbers)}")    # the normal sum
    print(f"Kahan sum     = {k.result()}")      # our better result
    print(f"fsum(numbers) = {fsum(numbers)}")   # the exact result
```

↓ python3 kahan_user.py ↓

```
======= numbers = [1e-15, 1e-14, 1e-13, 1e-16, 1e-12] =======
sum(numbers)  = 1.1111e-12
Kahan sum     = 1.1111e-12
fsum(numbers) = 1.1111e-12
======= numbers = [1e+18, 1, -1e+18] =======
sum(numbers)  = 1.0
Kahan sum     = 1.0
fsum(numbers) = 1.0
======= numbers = [1e+36, 1e+18, 1, -1e+36, -1e+18] =======
sum(numbers)  = 0.0
Kahan sum     = 1.0
fsum(numbers) = 1.0
======= numbers = [1e+36, 1e+72, 1e+18, -1e+36, -1e+72, 1, -1e+18]
    ↪ =======
sum(numbers)  = -1e+18
Kahan sum     = 0.0
fsum(numbers) = 1.0
======= numbers = [1, -1e-16, 1e-16, 1e-16] =======
sum(numbers)  = 1.0
Kahan sum     = 1.0
fsum(numbers) = 1.0
```

Kahan Summe: Vergleich

- Wir vergleichen ihre Ergebnisse mit der built-in Funktion `sum` und der genauen Summenfunktion `fsum` aus dem `math` Modul.
- Alle drei können `[1e-15, 1e-14, 1e-13, 1e-16, 1e-12]` exakt zu `1.1111e-12` aufaddieren.
- `sum` liefert `0.0` für die Summe von `[1e+18, 1, -1e+18]` und auch für `[1e+36, 1e+18, 1, -1e+36, -1e+18]`.
- `KahanSum` und `fsum` können für beide Fälle korrekt `1.0` ausrechnen.

```python
"""Examples for using our class :class:`KahanSum`."""

from math import fsum  # An (even more) precise summation algorithm.

from kahan_sum import KahanSum  # Import our new own class.

# Iterate over four example arrays.
for numbers in [[1e-15, 1e-14, 1e-13, 1e-16, 1e-12], [1e18, 1, -1e18],
                [ 1e36, 1e18, 1, -1e36, -1e18],
                [1e36, 1e72, 1e18, -1e36, -1e72, 1, -1e18],
                [1, -1e-16, 1e-16, 1e-16]]:
    print(f"======= numbers = [{', '.join(map(str, numbers))}] =======")
    k: KahanSum = KahanSum()  # Create our Kahan summation object.
    for n in numbers:         # Iterate over the numbers...
        k.add(n)              # ...and let our object add them up.
    print(f"sum(numbers)  = {sum(numbers)}")    # the normal sum
    print(f"Kahan sum     = {k.result()}")      # our better result
    print(f"fsum(numbers) = {fsum(numbers)}")   # the exact result
```

↓ python3 kahan_user.py ↓

```
======= numbers = [1e-15, 1e-14, 1e-13, 1e-16, 1e-12] =======
sum(numbers)  = 1.1111e-12
Kahan sum     = 1.1111e-12
fsum(numbers) = 1.1111e-12
======= numbers = [1e+18, 1, -1e+18] =======
sum(numbers)  = 1.0
Kahan sum     = 1.0
fsum(numbers) = 1.0
======= numbers = [1e+36, 1e+18, 1, -1e+36, -1e+18] =======
sum(numbers)  = 0.0
Kahan sum     = 1.0
fsum(numbers) = 1.0
======= numbers = [1e+36, 1e+72, 1e+18, -1e+36, -1e+72, 1, -1e+18]
    ↪ =======
sum(numbers)  = -1e+18
Kahan sum     = 0.0
fsum(numbers) = 1.0
======= numbers = [1, -1e-16, 1e-16, 1e-16] =======
sum(numbers)  = 1.0
Kahan sum     = 1.0
fsum(numbers) = 1.0
```

Kahan Summe: Vergleich

- Alle drei können `[1e-15, 1e-14, 1e-13, 1e-16, 1e-12]` exakt zu `1.1111e-12` aufaddieren.
- `sum` liefert `0.0` für die Summe von `[1e+18, 1, -1e+18]` und auch für `[1e+36, 1e+18, 1, -1e+36, -1e+18]`.
- `KahanSum` und `fsum` können für beide Fälle korrekt `1.0` ausrechnen.
- Wenn wir aber noch größere Zahlen mit einbeziehen und z. B. die Summe über `[1e+36, 1e+72, 1e+18, -1e+36, -1e+72, 1, -1e+18]` berechnen, dann liefert unsere `KahanSum` `0.0` anstatt des korrekten Ergebnisses `1.0`.

```python
"""Examples for using our class :class:`KahanSum`."""

from math import fsum  # An (even more) precise summation algorithm.

from kahan_sum import KahanSum  # Import our new own class.

# Iterate over four example arrays.
for numbers in [[1e-15, 1e-14, 1e-13, 1e-16, 1e-12], [1e18, 1, -1e18],
                [ 1e36, 1e18, 1, -1e36, -1e18],
                [1e36, 1e72, 1e18, -1e36, -1e72, 1, -1e18],
                [1, -1e-16, 1e-16, 1e-16]]:
    print(f"======= numbers = [{', '.join(map(str, numbers))}] =======")
    k: KahanSum = KahanSum()  # Create our Kahan summation object.
    for n in numbers:         # Iterate over the numbers...
        k.add(n)              # ...and let our object add them up.
    print(f"sum(numbers)  = {sum(numbers)}")    # the normal sum
    print(f"Kahan sum     = {k.result()}")      # our better result
    print(f"fsum(numbers) = {fsum(numbers)}")   # the exact result
```

↓ python3 kahan_user.py ↓

```
======= numbers = [1e-15, 1e-14, 1e-13, 1e-16, 1e-12] =======
sum(numbers)  = 1.1111e-12
Kahan sum     = 1.1111e-12
fsum(numbers) = 1.1111e-12
======= numbers = [1e+18, 1, -1e+18] =======
sum(numbers)  = 1.0
Kahan sum     = 1.0
fsum(numbers) = 1.0
======= numbers = [1e+36, 1e+18, 1, -1e+36, -1e+18] =======
sum(numbers)  = 0.0
Kahan sum     = 1.0
fsum(numbers) = 1.0
======= numbers = [1e+36, 1e+72, 1e+18, -1e+36, -1e+72, 1, -1e+18]
    ↪ =======
sum(numbers)  = -1e+18
Kahan sum     = 0.0
fsum(numbers) = 1.0
======= numbers = [1, -1e-16, 1e-16, 1e-16] =======
sum(numbers)  = 1.0
Kahan sum     = 1.0
fsum(numbers) = 1.0
```

Kahan Summe: Vergleich

- `sum` liefert `0.0` für die Summe von `[1e+18, 1, -1e+18]` und auch für `[1e+36, 1e+18, 1, -1e+36, -1e+18]`.
- `KahanSum` und `fsum` können für beide Fälle korrekt `1.0` ausrechnen.
- Wenn wir aber noch größere Zahlen mit einbeziehen und z. B. die Summe über `[1e+36, 1e+72, 1e+18, -1e+36, -1e+72, 1, -1e+18]` berechnen, dann liefert unsere `KahanSum` `0.0` anstatt des korrekten Ergebnisses `1.0`.
- `fsum` finded das richtige Ergebnis, wohingegen `sum` mit `-1e18` weit daneben liegt.

```python
"""Examples for using our class :class:`KahanSum`."""

from math import fsum  # An (even more) precise summation algorithm.

from kahan_sum import KahanSum  # Import our new own class.

# Iterate over four example arrays.
for numbers in [[1e-15, 1e-14, 1e-13, 1e-16, 1e-12], [1e18, 1, -1e18],
                [ 1e36, 1e18, 1, -1e36, -1e18],
                [1e36, 1e72, 1e18, -1e36, -1e72, 1, -1e18],
                [1, -1e-16, 1e-16, 1e-16]]:
    print(f"======= numbers = [{', '.join(map(str, numbers))}] =======")
    k: KahanSum = KahanSum()  # Create our Kahan summation object.
    for n in numbers:         # Iterate over the numbers...
        k.add(n)              # ...and let our object add them up.
    print(f"sum(numbers)  = {sum(numbers)}")    # the normal sum
    print(f"Kahan sum     = {k.result()}")      # our better result
    print(f"fsum(numbers) = {fsum(numbers)}")   # the exact result
```

↓ python3 kahan_user.py ↓

```
======= numbers = [1e-15, 1e-14, 1e-13, 1e-16, 1e-12] =======
sum(numbers)  = 1.1111e-12
Kahan sum     = 1.1111e-12
fsum(numbers) = 1.1111e-12
======= numbers = [1e+18, 1, -1e+18] =======
sum(numbers)  = 1.0
Kahan sum     = 1.0
fsum(numbers) = 1.0
======= numbers = [1e+36, 1e+18, 1, -1e+36, -1e+18] =======
sum(numbers)  = 0.0
Kahan sum     = 1.0
fsum(numbers) = 1.0
======= numbers = [1e+36, 1e+72, 1e+18, -1e+36, -1e+72, 1, -1e+18]
    ↪ =======
sum(numbers)  = -1e+18
Kahan sum     = 0.0
fsum(numbers) = 1.0
======= numbers = [1, -1e-16, 1e-16, 1e-16] =======
sum(numbers)  = 1.0
Kahan sum     = 1.0
fsum(numbers) = 1.0
```

Kahan Summe: Vergleich

- `KahanSum` und `fsum` können für beide Fälle korrekt `1.0` ausrechnen.
- Wenn wir aber noch größere Zahlen mit einbeziehen und z. B. die Summe über `[1e+36, 1e+72, 1e+18, -1e+36, -1e+72, 1, -1e+18]` berechnen, dann liefert unsere `KahanSum` `0.0` anstatt des korrekten Ergebnisses `1.0`.
- `fsum` finded das richtige Ergebnis, wohingegen `sum` mit `-1e18` weit daneben liegt.
- Zuletzt addieren wir noch `[1, -1e-16, 1e-16, 1e-16]`.

```python
"""Examples for using our class :class:`KahanSum`."""

from math import fsum  # An (even more) precise summation algorithm.

from kahan_sum import KahanSum  # Import our new own class.

# Iterate over four example arrays.
for numbers in [[1e-15, 1e-14, 1e-13, 1e-16, 1e-12], [1e18, 1, -1e18],
                [ 1e36, 1e18, 1, -1e36, -1e18],
                [1e36, 1e72, 1e18, -1e36, -1e72, 1, -1e18],
                [1, -1e-16, 1e-16, 1e-16]]:
    print(f"======= numbers = [{', '.join(map(str, numbers))}] =======")
    k: KahanSum = KahanSum()  # Create our Kahan summation object.
    for n in numbers:         # Iterate over the numbers...
        k.add(n)              # ...and let our object add them up.
    print(f"sum(numbers)  = {sum(numbers)}")   # the normal sum
    print(f"Kahan sum     = {k.result()}")     # our better result
    print(f"fsum(numbers) = {fsum(numbers)}")  # the exact result
```

↓ python3 kahan_user.py ↓

```
======= numbers = [1e-15, 1e-14, 1e-13, 1e-16, 1e-12] =======
sum(numbers)  = 1.1111e-12
Kahan sum     = 1.1111e-12
fsum(numbers) = 1.1111e-12
======= numbers = [1e+18, 1, -1e+18] =======
sum(numbers)  = 1.0
Kahan sum     = 1.0
fsum(numbers) = 1.0
======= numbers = [1e+36, 1e+18, 1, -1e+36, -1e+18] =======
sum(numbers)  = 0.0
Kahan sum     = 1.0
fsum(numbers) = 1.0
======= numbers = [1e+36, 1e+72, 1e+18, -1e+36, -1e+72, 1, -1e+18]
    ↪ =======
sum(numbers)  = -1e+18
Kahan sum     = 0.0
fsum(numbers) = 1.0
======= numbers = [1, -1e-16, 1e-16, 1e-16] =======
sum(numbers)  = 1.0
Kahan sum     = 1.0
fsum(numbers) = 1.0
```

Kahan Summe: Vergleich

- Wenn wir aber noch größere Zahlen mit einbeziehen und z. B. die Summe über `[1e+36, 1e+72, 1e+18, -1e+36, -1e+72, 1, -1e+18]` berechnen, dann liefert unsere `KahanSum` `0.0` anstatt des korrekten Ergebnisses `1.0`.
- `fsum` finded das richtige Ergebnis, wohingegen `sum` mit `-1e18` weit daneben liegt.
- Zuletzt addieren wir noch `[1, -1e-16, 1e-16, 1e-16]`.
- Alle drei Methoden liefern `1.0`, wohingegen das genaue Ergebnis $1 + 10^{-16}$ wäre.

```python
"""Examples for using our class :class:`KahanSum`."""

from math import fsum  # An (even more) precise summation algorithm.

from kahan_sum import KahanSum  # Import our new own class.

# Iterate over four example arrays.
for numbers in [[1e-15, 1e-14, 1e-13, 1e-16, 1e-12], [1e18, 1, -1e18],
                [ 1e36, 1e18, 1, -1e36, -1e18],
                [1e36, 1e72, 1e18, -1e36, -1e72, 1, -1e18],
                [1, -1e-16, 1e-16, 1e-16]]:
    print(f"======= numbers = [{', '.join(map(str, numbers))}] =======")
    k: KahanSum = KahanSum()  # Create our Kahan summation object.
    for n in numbers:         # Iterate over the numbers...
        k.add(n)              # ...and let our object add them up.
    print(f"sum(numbers)  = {sum(numbers)}")    # the normal sum
    print(f"Kahan sum     = {k.result()}")      # our better result
    print(f"fsum(numbers) = {fsum(numbers)}")   # the exact result
```

↓ python3 kahan_user.py ↓

```
======= numbers = [1e-15, 1e-14, 1e-13, 1e-16, 1e-12] =======
sum(numbers)  = 1.1111e-12
Kahan sum     = 1.1111e-12
fsum(numbers) = 1.1111e-12
======= numbers = [1e+18, 1, -1e+18] =======
sum(numbers)  = 1.0
Kahan sum     = 1.0
fsum(numbers) = 1.0
======= numbers = [1e+36, 1e+18, 1, -1e+36, -1e+18] =======
sum(numbers)  = 0.0
Kahan sum     = 1.0
fsum(numbers) = 1.0
======= numbers = [1e+36, 1e+72, 1e+18, -1e+36, -1e+72, 1, -1e+18]
    ↪ =======
sum(numbers)  = -1e+18
Kahan sum     = 0.0
fsum(numbers) = 1.0
======= numbers = [1, -1e-16, 1e-16, 1e-16] =======
sum(numbers)  = 1.0
Kahan sum     = 1.0
fsum(numbers) = 1.0
```

Kahan Summe: Vergleich

- `fsum` finded das richtige Ergebnis, wohingegen `sum` mit `-1e18` weit daneben liegt.
- Zuletzt addieren wir noch `[1, -1e-16, 1e-16, 1e-16]`.
- Alle drei Methoden liefern `1.0`, wohingegen das genaue Ergebnis $1 + 10^{-16}$ wäre.
- Dies kann allerdings nicht mit dem Datentyp `float` dargestellt werden, also sind die Ergebnisse schon richtig.

```python
"""Examples for using our class :class:`KahanSum`."""

from math import fsum  # An (even more) precise summation algorithm.

from kahan_sum import KahanSum  # Import our new own class.

# Iterate over four example arrays.
for numbers in [[1e-15, 1e-14, 1e-13, 1e-16, 1e-12], [1e18, 1, -1e18],
                [ 1e36, 1e18, 1, -1e36, -1e18],
                [1e36, 1e72, 1e18, -1e36, -1e72, 1, -1e18],
                [1, -1e-16, 1e-16, 1e-16]]:
    print(f"======= numbers = [{', '.join(map(str, numbers))}] =======")
    k: KahanSum = KahanSum()  # Create our Kahan summation object.
    for n in numbers:         # Iterate over the numbers...
        k.add(n)              # ...and let our object add them up.
    print(f"sum(numbers)  = {sum(numbers)}")   # the normal sum
    print(f"Kahan sum     = {k.result()}")     # our better result
    print(f"fsum(numbers) = {fsum(numbers)}")  # the exact result
```

↓ python3 kahan_user.py ↓

```
======= numbers = [1e-15, 1e-14, 1e-13, 1e-16, 1e-12] =======
sum(numbers)  = 1.1111e-12
Kahan sum     = 1.1111e-12
fsum(numbers) = 1.1111e-12
======= numbers = [1e+18, 1, -1e+18] =======
sum(numbers)  = 1.0
Kahan sum     = 1.0
fsum(numbers) = 1.0
======= numbers = [1e+36, 1e+18, 1, -1e+36, -1e+18] =======
sum(numbers)  = 0.0
Kahan sum     = 1.0
fsum(numbers) = 1.0
======= numbers = [1e+36, 1e+72, 1e+18, -1e+36, -1e+72, 1, -1e+18]
    ↪ =======
sum(numbers)  = -1e+18
Kahan sum     = 0.0
fsum(numbers) = 1.0
======= numbers = [1, -1e-16, 1e-16, 1e-16] =======
sum(numbers)  = 1.0
Kahan sum     = 1.0
fsum(numbers) = 1.0
```

Ergebnis des Vergleichs

- Mit unserem Experiment haben wir herausgefunden, dass `fsum` uns das präziseste Ergebnis gibt.

Ergebnis des Vergleichs

- Mit unserem Experiment haben wir herausgefunden, dass `fsum` uns das präziseste Ergebnis gibt.
- Unsere `KahanSum` ist aber definitiv besser als die built-in Funktion `sum`.

Ergebnis des Vergleichs

- Mit unserem Experiment haben wir herausgefunden, dass `fsum` uns das präziseste Ergebnis gibt.
- Unsere `KahanSum` ist aber definitiv besser als die built-in Funktion `sum`.
- Das bringt uns zu zwei Fragen.

Ergebnis des Vergleichs

- Mit unserem Experiment haben wir herausgefunden, dass `fsum` uns das präziseste Ergebnis gibt.
- Unsere `KahanSum` ist aber definitiv besser als die built-in Funktion `sum`.
- Das bringt uns zu zwei Fragen:
  1. Warum kann unsere `KahanSum` uns nicht immer das genaue Ergebnis liefern?

Ergebnis des Vergleichs

- Mit unserem Experiment haben wir herausgefunden, dass `fsum` uns das präziseste Ergebnis gibt.
- Unsere `KahanSum` ist aber definitiv besser als die built-in Funktion `sum`.
- Das bringt uns zu zwei Fragen:
  1. Warum kann unsere `KahanSum` uns nicht immer das genaue Ergebnis liefern?
  2. Wie und warum ist `fsum` besser?

Ergebnis des Vergleichs

- Mit unserem Experiment haben wir herausgefunden, dass `fsum` uns das präziseste Ergebnis gibt.
- Unsere `KahanSum` ist aber definitiv besser als die built-in Funktion `sum`.
- Das bringt uns zu zwei Fragen:
  1. Warum kann unsere `KahanSum` uns nicht immer das genaue Ergebnis liefern?
  2. Wie und warum ist `fsum` besser?
- Die Antwort auf die erste Frage ist sehr einfach.

Ergebnis des Vergleichs

- Mit unserem Experiment haben wir herausgefunden, dass `fsum` uns das präziseste Ergebnis gibt.
- Unsere `KahanSum` ist aber definitiv besser als die built-in Funktion `sum`.
- Das bringt uns zu zwei Fragen:
  1. Warum kann unsere `KahanSum` uns nicht immer das genaue Ergebnis liefern?
  2. Wie und warum ist `fsum` besser?
- Die Antwort auf die erste Frage ist sehr einfach:
- Wenn wir genau eine Summationsvariable verwenden, so wie `sum` das tut, dann können präzise bis zu 15 oder 16 Ziffern sein und verliegen alle Ziffern darüber hinaus.

Ergebnis des Vergleichs

- Mit unserem Experiment haben wir herausgefunden, dass `fsum` uns das präziseste Ergebnis gibt.
- Unsere `KahanSum` ist aber definitiv besser als die built-in Funktion `sum`.
- Das bringt uns zu zwei Fragen:
  1. Warum kann unsere `KahanSum` uns nicht immer das genaue Ergebnis liefern?
  2. Wie und warum ist `fsum` besser?
- Die Antwort auf die erste Frage ist sehr einfach:
- Wenn wir genau eine Summationsvariable verwenden, so wie `sum` das tut, dann können präzise bis zu 15 oder 16 Ziffern sein und verliegen alle Ziffern darüber hinaus.
- Wenn wir eine Summationsvariable und einen Fehlerterm `cs` erster Ordnung verwenden, dann bekommen wir im Grunde 15 bis 16 Ziffern dazu, um Zwischenergebnisse genauer darzustellen.

Ergebnis des Vergleichs

- Mit unserem Experiment haben wir herausgefunden, dass `fsum` uns das präziseste Ergebnis gibt.
- Unsere `KahanSum` ist aber definitiv besser als die built-in Funktion `sum`.
- Das bringt uns zu zwei Fragen:
  1. Warum kann unsere `KahanSum` uns nicht immer das genaue Ergebnis liefern?
  2. Wie und warum ist `fsum` besser?
- Die Antwort auf die erste Frage ist sehr einfach:
- Wenn wir genau eine Summationsvariable verwenden, so wie `sum` das tut, dann können präzise bis zu 15 oder 16 Ziffern sein und verliegen alle Ziffern darüber hinaus.
- Wenn wir eine Summationsvariable und einen Fehlerterm `cs` erster Ordnung verwenden, dann bekommen wir im Grunde 15 bis 16 Ziffern dazu, um Zwischenergebnisse genauer darzustellen.
- Mit dem Fehlerterm zweiter Ordnung `ccs` können wir dann Zwischenergebnisse mit 45 bis 48 Ziffern genau darstellen.

Ergebnis des Vergleichs

- Mit unserem Experiment haben wir herausgefunden, dass `fsum` uns das präziseste Ergebnis gibt.
- Unsere `KahanSum` ist aber definitiv besser als die built-in Funktion `sum`.
- Das bringt uns zu zwei Fragen:
  1. Warum kann unsere `KahanSum` uns nicht immer das genaue Ergebnis liefern?
  2. Wie und warum ist `fsum` besser?
- Die Antwort auf die erste Frage ist sehr einfach:
- Wenn wir genau eine Summationsvariable verwenden, so wie `sum` das tut, dann können präzise bis zu 15 oder 16 Ziffern sein und verliegen alle Ziffern darüber hinaus.
- Wenn wir eine Summationsvariable und einen Fehlerterm `cs` erster Ordnung verwenden, dann bekommen wir im Grunde 15 bis 16 Ziffern dazu, um Zwischenergebnisse genauer darzustellen.
- Mit dem Fehlerterm zweiter Ordnung `ccs` können wir dann Zwischenergebnisse mit 45 bis 48 Ziffern genau darstellen.
- Damit können wir `1` zu `1e36` addieren und dann wieder `1e36` abziehen und bekommen `1`.

Ergebnis des Vergleichs

- Unsere `KahanSum` ist aber definitiv besser als die built-in Funktion `sum`.
- Das bringt uns zu zwei Fragen:
  1. Warum kann unsere `KahanSum` uns nicht immer das genaue Ergebnis liefern?
  2. Wie und warum ist `fsum` besser?
- Die Antwort auf die erste Frage ist sehr einfach:
- Wenn wir genau eine Summationsvariable verwenden, so wie `sum` das tut, dann können präzise bis zu 15 oder 16 Ziffern sein und verliegen alle Ziffern darüber hinaus.
- Wenn wir eine Summationsvariable und einen Fehlerterm `cs` erster Ordnung verwenden, dann bekommen wir im Grunde 15 bis 16 Ziffern dazu, um Zwischenergebnisse genauer darzustellen.
- Mit dem Fehlerterm zweiter Ordnung `ccs` können wir dann Zwischenergebnisse mit 45 bis 48 Ziffern genau darstellen.
- Damit können wir `1` zu `1e36` addieren und dann wieder `1e36` abziehen und bekommen `1`.
- Aber wenn wir noch `1e72` dazuaddieren, dann überschreiten wir das Fenster von 48 Ziffern und die `1` geht verloren.

Ergebnis des Vergleichs

- Das bringt uns zu zwei Fragen:
  1. Warum kann unsere `KahanSum` uns nicht immer das genaue Ergebnis liefern?
  2. Wie und warum ist `fsum` besser?
- Die Antwort auf die erste Frage ist sehr einfach:
- Wenn wir genau eine Summationsvariable verwenden, so wie `sum` das tut, dann können präzise bis zu 15 oder 16 Ziffern sein und verliegen alle Ziffern darüber hinaus.
- Wenn wir eine Summationsvariable und einen Fehlerterm `cs` erster Ordnung verwenden, dann bekommen wir im Grunde 15 bis 16 Ziffern dazu, um Zwischenergebnisse genauer darzustellen.
- Mit dem Fehlerterm zweiter Ordnung `ccs` können wir dann Zwischenergebnisse mit 45 bis 48 Ziffern genau darstellen.
- Damit können wir `1` zu `1e36` addieren und dann wieder `1e36` abziehen und bekommen `1`.
- Aber wenn wir noch `1e72` dazuaddieren, dann überschreiten wir das Fenster von 48 Ziffern und die `1` geht verloren.
- Mit unserer `KahanSum` mit zwei Fehlertermen können wir also Zwischenergebnisse von bis zu 48 Ziffern darstellen.

Ergebnis des Vergleichs

- Die Antwort auf die erste Frage ist sehr einfach:
- Wenn wir genau eine Summationsvariable verwenden, so wie `sum` das tut, dann können präzise bis zu 15 oder 16 Ziffern sein und verliegen alle Ziffern darüber hinaus.
- Wenn wir eine Summationsvariable und einen Fehlerterm `cs` erster Ordnung verwenden, dann bekommen wir im Grunde 15 bis 16 Ziffern dazu, um Zwischenergebnisse genauer darzustellen.
- Mit dem Fehlerterm zweiter Ordnung `ccs` können wir dann Zwischenergebnisse mit 45 bis 48 Ziffern genau darstellen.
- Damit können wir `1` zu `1e36` addieren und dann wieder `1e36` abziehen und bekommen `1`.
- Aber wenn wir noch `1e72` dazuaddieren, dann überschreiten wir das Fenster von 48 Ziffern und die `1` geht verloren.
- Mit unserer `KahanSum` mit zwei Fehlertermen können wir also Zwischenergebnisse von bis zu 48 Ziffern darstellen.
- Die zweite Frage war *Was macht `fsum` anders?*

Ergebnis des Vergleichs

- Die Antwort auf die erste Frage ist sehr einfach:
- Wenn wir genau eine Summationsvariable verwenden, so wie `sum` das tut, dann können präzise bis zu 15 oder 16 Ziffern sein und verliegen alle Ziffern darüber hinaus.
- Wenn wir eine Summationsvariable und einen Fehlerterm `cs` erster Ordnung verwenden, dann bekommen wir im Grunde 15 bis 16 Ziffern dazu, um Zwischenergebnisse genauer darzustellen.
- Mit dem Fehlerterm zweiter Ordnung `ccs` können wir dann Zwischenergebnisse mit 45 bis 48 Ziffern genau darstellen.
- Damit können wir `1` zu `1e36` addieren und dann wieder `1e36` abziehen und bekommen `1`.
- Aber wenn wir noch `1e72` dazuaddieren, dann überschreiten wir das Fenster von 48 Ziffern und die `1` geht verloren.
- Mit unserer `KahanSum` mit zwei Fehlertermen können wir also Zwischenergebnisse von bis zu 48 Ziffern darstellen.
- Die zweite Frage war *Was macht `fsum` anders?*
- Eigentlich nicht viel.

Ergebnis des Vergleichs

- Wenn wir eine Summationsvariable und einen Fehlerterm `cs` erster Ordnung verwenden, dann bekommen wir im Grunde 15 bis 16 Ziffern dazu, um Zwischenergebnisse genauer darzustellen.
- Mit dem Fehlerterm zweiter Ordnung `ccs` können wir dann Zwischenergebnisse mit 45 bis 48 Ziffern genau darstellen.
- Damit können wir `1` zu `1e36` addieren und dann wieder `1e36` abziehen und bekommen `1`.
- Aber wenn wir noch `1e72` dazuaddieren, dann überschreiten wir das Fenster von 48 Ziffern und die `1` geht verloren.
- Mit unserer `KahanSum` mit zwei Fehlertermen können wir also Zwischenergebnisse von bis zu 48 Ziffern darstellen.
- Die zweite Frage war *Was macht `fsum` anders?*
- Eigentlich nicht viel.
- Es basiert auf dem Algorithmus von Shewchuk[144,267], welcher dynamisch eine Liste von Hilfsvariablen managed um eine beweisbar genaue Summe zu berechnen (mit Ergebnis `float`).

Ergebnis des Vergleichs

- Mit dem Fehlerterm zweiter Ordnung `ccs` können wir dann Zwischenergebnisse mit 45 bis 48 Ziffern genau darstellen.
- Damit können wir `1` zu `1e36` addieren und dann wieder `1e36` abziehen und bekommen `1`.
- Aber wenn wir noch `1e72` dazuaddieren, dann überschreiten wir das Fenster von 48 Ziffern und die `1` geht verloren.
- Mit unserer `KahanSum` mit zwei Fehlertermen können wir also Zwischenergebnisse von bis zu 48 Ziffern darstellen.
- Die zweite Frage war *Was macht `fsum` anders?*
- Eigentlich nicht viel.
- Es basiert auf dem Algorithmus von Shewchuk[144,267], welcher dynamisch eine Liste von Hilfsvariablen managed um eine beweisbar genaue Summe zu berechnen (mit Ergebnis `float`).
- Im Grunde ist es eine dynamische Version der Kahan-Summe, welche bei Bedarf mehr Fehlerterme benutzt und diese auch wieder freigibt, wenn sie nicht mehr benötigt werden.

Ergebnis des Vergleichs

- Mit dem Fehlerterm zweiter Ordnung `ccs` können wir dann Zwischenergebnisse mit 45 bis 48 Ziffern genau darstellen.
- Damit können wir `1` zu `1e36` addieren und dann wieder `1e36` abziehen und bekommen `1`.
- Aber wenn wir noch `1e72` dazuaddieren, dann überschreiten wir das Fenster von 48 Ziffern und die `1` geht verloren.
- Mit unserer `KahanSum` mit zwei Fehlertermen können wir also Zwischenergebnisse von bis zu 48 Ziffern darstellen.
- Die zweite Frage war *Was macht `fsum` anders?*
- Eigentlich nicht viel.
- Es basiert auf dem Algorithmus von Shewchuk[144,267], welcher dynamisch eine Liste von Hilfsvariablen managed um eine beweisbar genaue Summe zu berechnen (mit Ergebnis `float`).
- Im Grunde ist es eine dynamische Version der Kahan-Summe, welche bei Bedarf mehr Fehlerterme benutzt und diese auch wieder freigibt, wenn sie nicht mehr benötigt werden.
- Das Prinzip ist das gleiche, nur das eine Liste die Variablen `cs` und `ccs` ersetzt.

Ergebnis des Vergleichs

- Damit können wir `1` zu `1e36` addieren und dann wieder `1e36` abziehen und bekommen `1`.
- Aber wenn wir noch `1e72` dazuaddieren, dann überschreiten wir das Fenster von 48 Ziffern und die `1` geht verloren.
- Mit unserer `KahanSum` mit zwei Fehlertermen können wir also Zwischenergebnisse von bis zu 48 Ziffern darstellen.
- Die zweite Frage war *Was macht `fsum` anders?*
- Eigentlich nicht viel.
- Es basiert auf dem Algorithmus von Shewchuk[144,267], welcher dynamisch eine Liste von Hilfsvariablen managed um eine beweisbar genaue Summe zu berechnen (mit Ergebnis `float`).
- Im Grunde ist es eine dynamische Version der Kahan-Summe, welche bei Bedarf mehr Fehlerterme benutzt und diese auch wieder freigibt, wenn sie nicht mehr benötigt werden.
- Das Prinzip ist das gleiche, nur das eine Liste die Variablen `cs` und `ccs` ersetzt.
- Unser `KahanSum` benutzt genau drei Variablen für die ganze Summation.

Ergebnis des Vergleichs

- Aber wenn wir noch `1e72` dazuaddieren, dann überschreiten wir das Fenster von 48 Ziffern und die `1` geht verloren.
- Mit unserer `KahanSum` mit zwei Fehlertermen können wir also Zwischenergebnisse von bis zu 48 Ziffern darstellen.
- Die zweite Frage war *Was macht `fsum` anders?*
- Eigentlich nicht viel.
- Es basiert auf dem Algorithmus von Shewchuk[144,267], welcher dynamisch eine Liste von Hilfsvariablen managed um eine beweisbar genaue Summe zu berechnen (mit Ergebnis `float`).
- Im Grunde ist es eine dynamische Version der Kahan-Summe, welche bei Bedarf mehr Fehlerterme benutzt und diese auch wieder freigibt, wenn sie nicht mehr benötigt werden.
- Das Prinzip ist das gleiche, nur das eine Liste die Variablen `cs` und `ccs` ersetzt.
- Unser `KahanSum` benutzt genau drei Variablen für die ganze Summation.
- Es allokiert nicht dynamisch mehr Speicher.

Ergebnis des Vergleichs

- Mit unserer `KahanSum` mit zwei Fehlertermen können wir also Zwischenergebnisse von bis zu 48 Ziffern darstellen.
- Die zweite Frage war *Was macht `fsum` anders?*
- Eigentlich nicht viel.
- Es basiert auf dem Algorithmus von Shewchuk[144,267], welcher dynamisch eine Liste von Hilfsvariablen managed um eine beweisbar genaue Summe zu berechnen (mit Ergebnis `float`).
- Im Grunde ist es eine dynamische Version der Kahan-Summe, welche bei Bedarf mehr Fehlerterme benutzt und diese auch wieder freigibt, wenn sie nicht mehr benötigt werden.
- Das Prinzip ist das gleiche, nur das eine Liste die Variablen `cs` und `ccs` ersetzt.
- Unser `KahanSum` benutzt genau drei Variablen für die ganze Summation.
- Es allokiert nicht dynamisch mehr Speicher.
- Deshalb ist die Anzahl der Rechenschritte, die es für jede hinzuaddierte Zahl macht, auch konstant.

Ergebnis des Vergleichs

- Die zweite Frage war *Was macht `fsum` anders?*
- Eigentlich nicht viel.
- Es basiert auf dem Algorithmus von Shewchuk[144,267], welcher dynamisch eine Liste von Hilfsvariablen managed um eine beweisbar genaue Summe zu berechnen (mit Ergebnis `float`).
- Im Grunde ist es eine dynamische Version der Kahan-Summe, welche bei Bedarf mehr Fehlerterme benutzt und diese auch wieder freigibt, wenn sie nicht mehr benötigt werden.
- Das Prinzip ist das gleiche, nur das eine Liste die Variablen `cs` und `ccs` ersetzt.
- Unser `KahanSum` benutzt genau drei Variablen für die ganze Summation.
- Es allokiert nicht dynamisch mehr Speicher.
- Deshalb ist die Anzahl der Rechenschritte, die es für jede hinzuaddierte Zahl macht, auch konstant.
- `fsum` macht eine variable Zahl von Rechenschritten, deren Anzahl auf der Länge der internen Datensturktur baisert.

Ergebnis des Vergleichs

- Eigentlich nicht viel.
- Es basiert auf dem Algorithmus von Shewchuk[144,267], welcher dynamisch eine Liste von Hilfsvariablen managed um eine beweisbar genaue Summe zu berechnen (mit Ergebnis `float`).
- Im Grunde ist es eine dynamische Version der Kahan-Summe, welche bei Bedarf mehr Fehlerterme benutzt und diese auch wieder freigibt, wenn sie nicht mehr benötigt werden.
- Das Prinzip ist das gleiche, nur das eine Liste die Variablen `cs` und `ccs` ersetzt.
- Unser `KahanSum` benutzt genau drei Variablen für die ganze Summation.
- Es allokiert nicht dynamisch mehr Speicher.
- Deshalb ist die Anzahl der Rechenschritte, die es für jede hinzuaddierte Zahl macht, auch konstant.
- `fsum` macht eine variable Zahl von Rechenschritten, deren Anzahl auf der Länge der internen Datensturktur baisert.
- Damit ist `KahanSum` tatsächlich ein sehr schöne Kompromisslösung, die eine höhere Präzision als die normale Addition bietet und aber immer noch die selbe konstante Speicher- und Zeitkomplexität hat.

Ergebnis des Vergleichs

- Es basiert auf dem Algorithmus von Shewchuk[144,267], welcher dynamisch eine Liste von Hilfsvariablen managed um eine beweisbar genaue Summe zu berechnen (mit Ergebnis `float`).
- Im Grunde ist es eine dynamische Version der Kahan-Summe, welche bei Bedarf mehr Fehlerterme benutzt und diese auch wieder freigibt, wenn sie nicht mehr benötigt werden.
- Das Prinzip ist das gleiche, nur das eine Liste die Variablen `cs` und `ccs` ersetzt.
- Unser `KahanSum` benutzt genau drei Variablen für die ganze Summation.
- Es allokiert nicht dynamisch mehr Speicher.
- Deshalb ist die Anzahl der Rechenschritte, die es für jede hinzuaddierte Zahl macht, auch konstant.
- `fsum` macht eine variable Zahl von Rechenschritten, deren Anzahl auf der Länge der internen Datensturktur baisert.
- Damit ist `KahanSum` tatsächlich ein sehr schöne Kompromisslösung, die eine höhere Präzision als die normale Addition bietet und aber immer noch die selbe konstante Speicher- und Zeitkomplexität hat.
- Es ist auch vielseitiger als `fsum`.

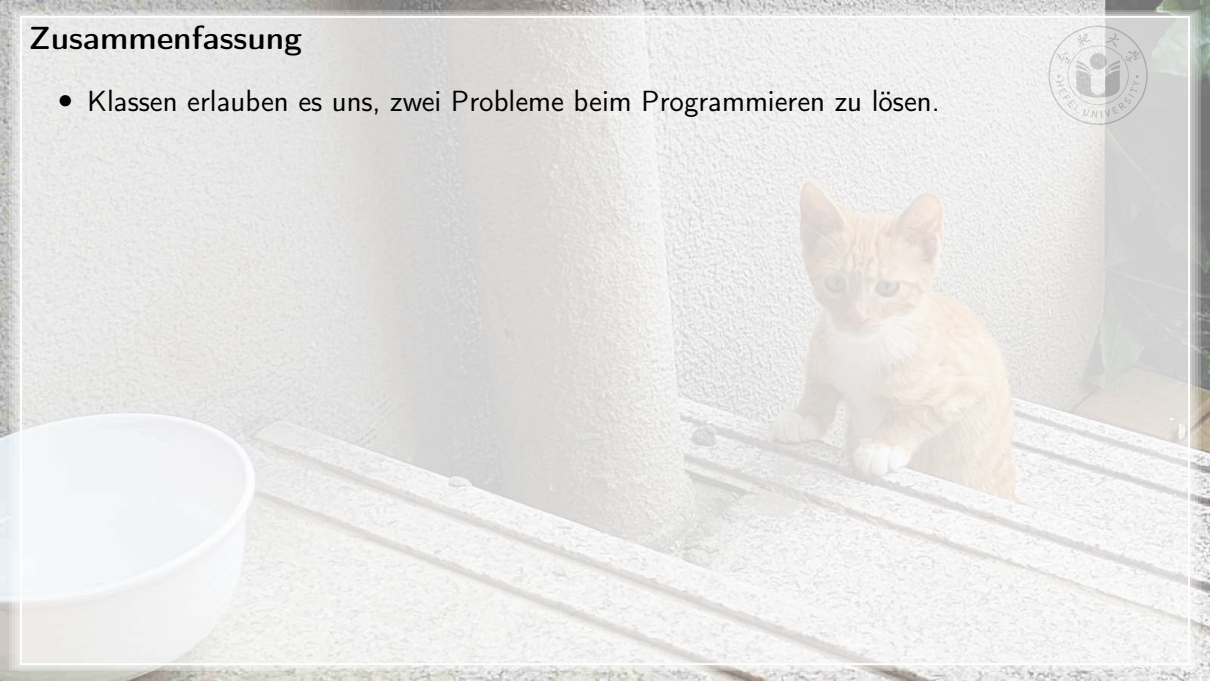
Ergebnis des Vergleichs

- Im Grunde ist es eine dynamische Version der Kahan-Summe, welche bei Bedarf mehr Fehlerterme benutzt und diese auch wieder freigibt, wenn sie nicht mehr benötigt werden.
- Das Prinzip ist das gleiche, nur das eine Liste die Variablen `cs` und `ccs` ersetzt.
- Unser `KahanSum` benutzt genau drei Variablen für die ganze Summation.
- Es allokiert nicht dynamisch mehr Speicher.
- Deshalb ist die Anzahl der Rechenschritte, die es für jede hinzuaddierte Zahl macht, auch konstant.
- `fsum` macht eine variable Zahl von Rechenschritten, deren Anzahl auf der Länge der internen Datensturktur baisert.
- Damit ist `KahanSum` tatsächlich ein sehr schöne Kompromisslösung, die eine höhere Präzision als die normale Addition bietet und aber immer noch die selbe konstante Speicher- und Zeitkomplexität hat.
- Es ist auch vielseitiger als `fsum`.
- Wir müssen die aufzuaddierenden Werte nicht in einem `Iterable` haben.

Zusammenfassung

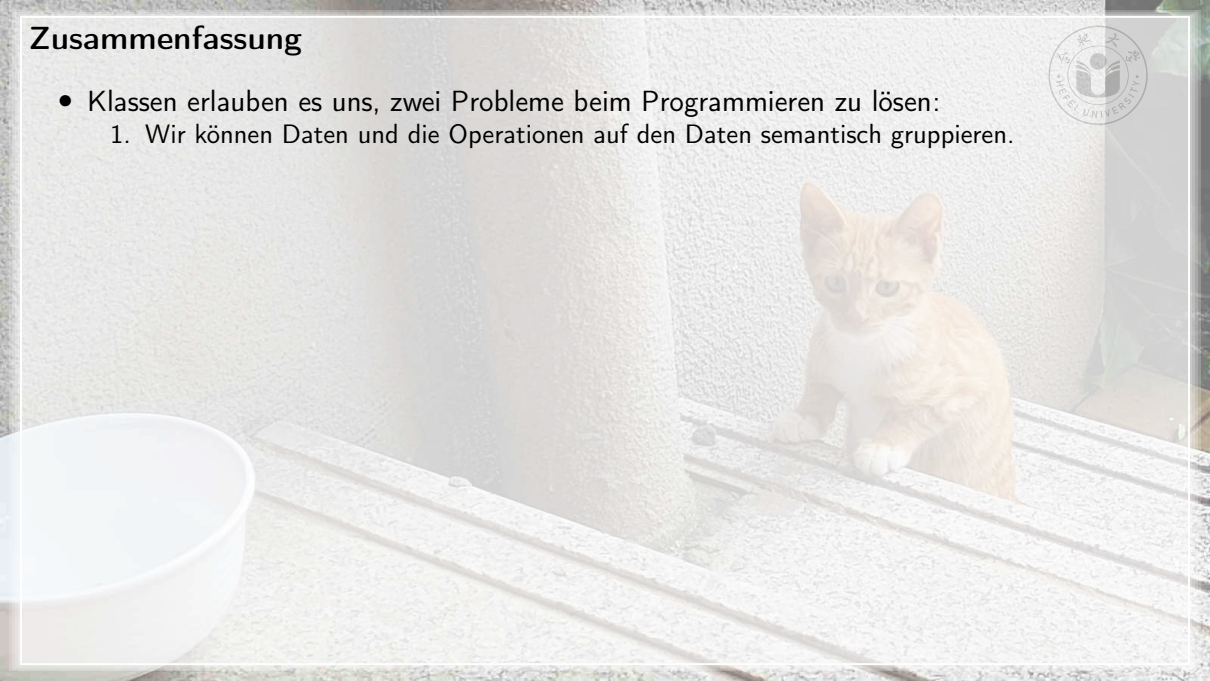
Zusammenfassung

- Klassen erlauben es uns, zwei Probleme beim Programmieren zu lösen.



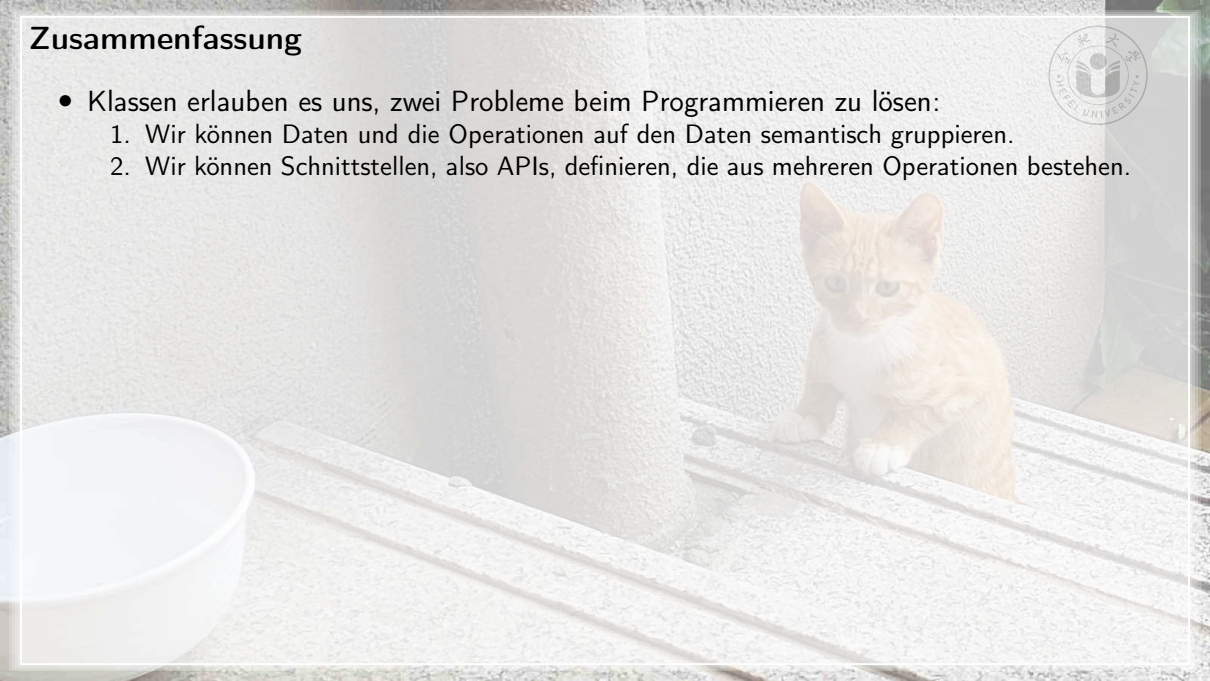
Zusammenfassung

- Klassen erlauben es uns, zwei Probleme beim Programmieren zu lösen:
  1. Wir können Daten und die Operationen auf den Daten semantisch gruppieren.



Zusammenfassung

- Klassen erlauben es uns, zwei Probleme beim Programmieren zu lösen:
  1. Wir können Daten und die Operationen auf den Daten semantisch gruppieren.
  2. Wir können Schnittstellen, also APIs, definieren, die aus mehreren Operationen bestehen.

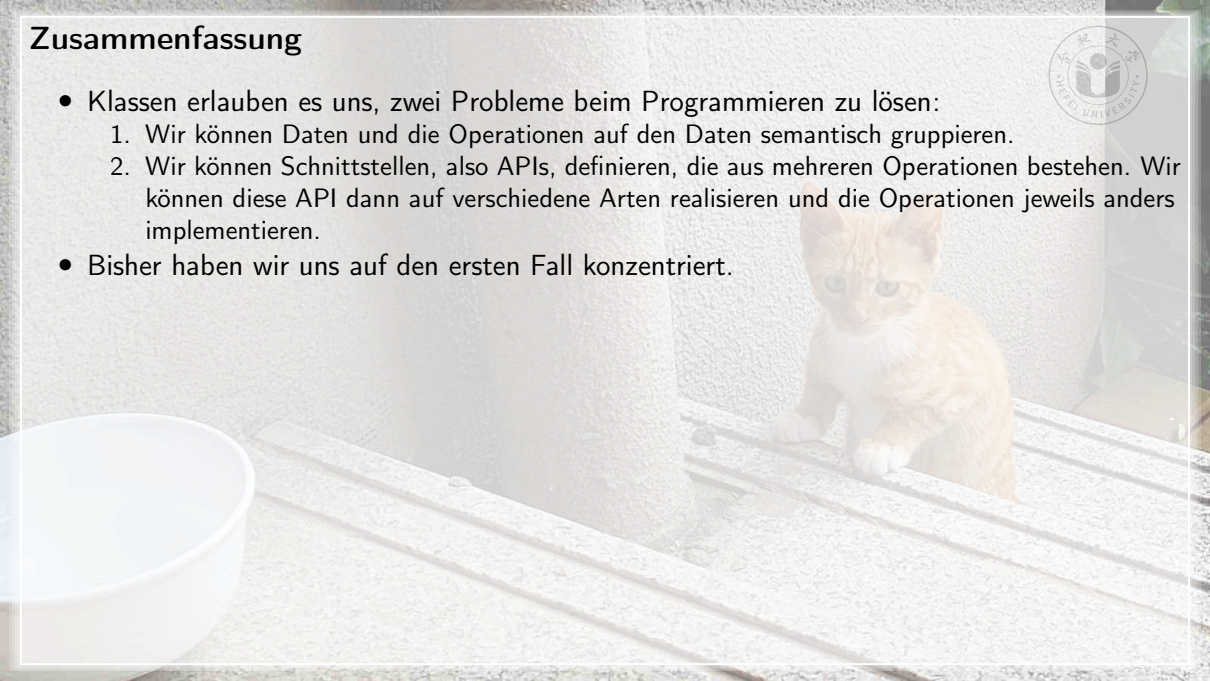


Zusammenfassung

- Klassen erlauben es uns, zwei Probleme beim Programmieren zu lösen:
  1. Wir können Daten und die Operationen auf den Daten semantisch gruppieren.
  2. Wir können Schnittstellen, also APIs, definieren, die aus mehreren Operationen bestehen. Wir können diese API dann auf verschiedene Arten realisieren und die Operationen jeweils anders implementieren.

Zusammenfassung

- Klassen erlauben es uns, zwei Probleme beim Programmieren zu lösen:
  1. Wir können Daten und die Operationen auf den Daten semantisch gruppieren.
  2. Wir können Schnittstellen, also APIs, definieren, die aus mehreren Operationen bestehen. Wir können diese API dann auf verschiedene Arten realisieren und die Operationen jeweils anders implementieren.
- Bisher haben wir uns auf den ersten Fall konzentriert.



Zusammenfassung

- Klassen erlauben es uns, zwei Probleme beim Programmieren zu lösen:
  1. Wir können Daten und die Operationen auf den Daten semantisch gruppieren.
  2. Wir können Schnittstellen, also APIs, definieren, die aus mehreren Operationen bestehen. Wir können diese API dann auf verschiedene Arten realisieren und die Operationen jeweils anders implementieren.
- Bisher haben wir uns auf den ersten Fall konzentriert.
- Wir haben zwei Beispiele dafür angeschaut.

Zusammenfassung

- Klassen erlauben es uns, zwei Probleme beim Programmieren zu lösen:
  1. Wir können Daten und die Operationen auf den Daten semantisch gruppieren.
  2. Wir können Schnittstellen, also APIs, definieren, die aus mehreren Operationen bestehen. Wir können diese API dann auf verschiedene Arten realisieren und die Operationen jeweils anders implementieren.
- Bisher haben wir uns auf den ersten Fall konzentriert.
- Wir haben zwei Beispiele dafür angeschaut.
- Wir haben auch zwei Prinzipien kennengelernt, wie wir Klassen entwerfen können.

Zusammenfassung

- Klassen erlauben es uns, zwei Probleme beim Programmieren zu lösen:
  1. Wir können Daten und die Operationen auf den Daten semantisch gruppieren.
  2. Wir können Schnittstellen, also APIs, definieren, die aus mehreren Operationen bestehen. Wir können diese API dann auf verschiedene Arten realisieren und die Operationen jeweils anders implementieren.
- Bisher haben wir uns auf den ersten Fall konzentriert.
- Wir haben zwei Beispiele dafür angeschaut.
- Wir haben auch zwei Prinzipien kennengelernt, wie wir Klassen entwerfen können
  1. Wir können unveränderliche Klassen als Kontainer konstanter Information erstellen, deren Attribute sich nicht ändern.

Zusammenfassung



- Klassen erlauben es uns, zwei Probleme beim Programmieren zu lösen:
  1. Wir können Daten und die Operationen auf den Daten semantisch gruppieren.
  2. Wir können Schnittstellen, also APIs, definieren, die aus mehreren Operationen bestehen. Wir können diese API dann auf verschiedene Arten realisieren und die Operationen jeweils anders implementieren.
- Bisher haben wir uns auf den ersten Fall konzentriert.
- Wir haben zwei Beispiele dafür angeschaut.
- Wir haben auch zwei Prinzipien kennengelernt, wie wir Klassen entwerfen können
  1. Wir können unveränderliche Klassen als Kontainer konstanter Information erstellen, deren Attribute sich nicht ändern. Unsere Klasse `Point` ist ein Beispiel dafür.

Zusammenfassung

- Klassen erlauben es uns, zwei Probleme beim Programmieren zu lösen:
  1. Wir können Daten und die Operationen auf den Daten semantisch gruppieren.
  2. Wir können Schnittstellen, also APIs, definieren, die aus mehreren Operationen bestehen. Wir können diese API dann auf verschiedene Arten realisieren und die Operationen jeweils anders implementieren.
- Bisher haben wir uns auf den ersten Fall konzentriert.
- Wir haben zwei Beispiele dafür angeschaut.
- Wir haben auch zwei Prinzipien kennengelernt, wie wir Klassen entwerfen können
  1. Wir können unveränderliche Klassen als Kontainer konstanter Information erstellen, deren Attribute sich nicht ändern. Unsere Klasse `Point` ist ein Beispiel dafür.
  2. Wir können auch gekapselte Klassen erstellen, wobei der Zugriff auf Attribute nur durch die Methoden der Klasse möglich ist.

Zusammenfassung



- Bisher haben wir uns auf den ersten Fall konzentriert.
- Wir haben zwei Beispiele dafür angeschaut.
- Wir haben auch zwei Prinzipien kennengelernt, wie wir Klassen entwerfen können
  1. Wir können unveränderliche Klassen als Kontainer konstanter Information erstellen, deren Attribute sich nicht ändern. Unsere Klasse `Point` ist ein Beispiel dafür.
  2. Wir können auch gekapselte Klassen erstellen, wobei der Zugriff auf Attribute nur durch die Methoden der Klasse möglich ist. Dadurch wird sichergestellt, dass jede Zustandsänderung konsistent und korrekt ist.

Zusammenfassung



- Bisher haben wir uns auf den ersten Fall konzentriert.
- Wir haben zwei Beispiele dafür angeschaut.
- Wir haben auch zwei Prinzipien kennengelernt, wie wir Klassen entwerfen können
  1. Wir können unveränderliche Klassen als Kontainer konstanter Information erstellen, deren Attribute sich nicht ändern. Unsere Klasse `Point` ist ein Beispiel dafür.
  2. Wir können auch gekapselte Klassen erstellen, wobei der Zugriff auf Attribute nur durch die Methoden der Klasse möglich ist. Dadurch wird sichergestellt, dass jede Zustandsänderung konsistent und korrekt ist. Unsere Klasse `KahanSum` ist ein Beispiel dafür.

Zusammenfassung



- Bisher haben wir uns auf den ersten Fall konzentriert.
- Wir haben zwei Beispiele dafür angeschaut.
- Wir haben auch zwei Prinzipien kennengelernt, wie wir Klassen entwerfen können
  1. Wir können unveränderliche Klassen als Kontainer konstanter Information erstellen, deren Attribute sich nicht ändern. Unsere Klasse `Point` ist ein Beispiel dafür.
  2. Wir können auch gekapselte Klassen erstellen, wobei der Zugriff auf Attribute nur durch die Methoden der Klasse möglich ist. Dadurch wird sichergestellt, dass jede Zustandsänderung konsistent und korrekt ist. Unsere Klasse `KahanSum` ist ein Beispiel dafür.
- Die meisten Klassen, die entweder Daten speichern oder irgendein Verhalten realisieren, können auf eine dieser beiden Arten implementiert werden.

Zusammenfassung



- Bisher haben wir uns auf den ersten Fall konzentriert.
- Wir haben zwei Beispiele dafür angeschaut.
- Wir haben auch zwei Prinzipien kennengelernt, wie wir Klassen entwerfen können
  1. Wir können unveränderliche Klassen als Kontainer konstanter Information erstellen, deren Attribute sich nicht ändern. Unsere Klasse `Point` ist ein Beispiel dafür.
  2. Wir können auch gekapselte Klassen erstellen, wobei der Zugriff auf Attribute nur durch die Methoden der Klasse möglich ist. Dadurch wird sichergestellt, dass jede Zustandsänderung konsistent und korrekt ist. Unsere Klasse `KahanSum` ist ein Beispiel dafür.
- Die meisten Klassen, die entweder Daten speichern oder irgendein Verhalten realisieren, können auf eine dieser beiden Arten implementiert werden.
- Es kann natürlich auch Designs geben, die dazwischen liegen, wo einige Attribute `Final` und von außen zugänglich sind und andere Attribute wieder nur durch Methoden geändert werden können.

Zusammenfassung: Unveränderliche Objekte



- Instanzen unserer Klasse `Point` speichern ein Paar Koordinaten in der zwei-dimensionalen Euklidischen Ebene.

Zusammenfassung: Unveränderliche Objekte



- Instanzen unserer Klasse `Point` speichern ein Paar Koordinaten in der zwei-dimensionalen Euklidischen Ebene.
- Die Operation `distance` ist untrennbar mit dieser Datenstruktur verbunden.

Zusammenfassung: Unveränderliche Objekte



- Instanzen unserer Klasse `Point` speichern ein Paar Koordinaten in der zwei-dimensionalen Euklidischen Ebene.
- Die Operation `distance` ist untrennbar mit dieser Datenstruktur verbunden.
- Wir haben gelernt, dass es oft eine gute Idee ist, Objekte unveränderlich zu machen.

Zusammenfassung: Unveränderliche Objekte



- Instanzen unserer Klasse `Point` speichern ein Paar Koordinaten in der zwei-dimensionalen Euklidischen Ebene.
- Die Operation `distance` ist untrennbar mit dieser Datenstruktur verbunden.
- Wir haben gelernt, dass es oft eine gute Idee ist, Objekte unveränderlich zu machen.
- Dann werden die Werte der Attribute eines Objektes nur während seiner Initialisierung (durch die `__init__` Methode) gesetzt und ändern sich danach nicht.

Zusammenfassung: Unveränderliche Objekte



- Instanzen unserer Klasse `Point` speichern ein Paar Koordinaten in der zwei-dimensionalen Euklidischen Ebene.
- Die Operation `distance` ist untrennbar mit dieser Datenstruktur verbunden.
- Wir haben gelernt, dass es oft eine gute Idee ist, Objekte unveränderlich zu machen.
- Dann werden die Werte der Attribute eines Objektes nur während seiner Initialisierung (durch die `__init__` Methode) gesetzt und ändern sich danach nicht.
- Dann kann es niemals Verwirrung über die Attributwerte geben.

Zusammenfassung: Unveränderliche Objekte



- Instanzen unserer Klasse `Point` speichern ein Paar Koordinaten in der zwei-dimensionalen Euklidischen Ebene.
- Die Operation `distance` ist untrennbar mit dieser Datenstruktur verbunden.
- Wir haben gelernt, dass es oft eine gute Idee ist, Objekte unveränderlich zu machen.
- Dann werden die Werte der Attribute eines Objektes nur während seiner Initialisierung (durch die `__init__` Methode) gesetzt und ändern sich danach nicht.
- Dann kann es niemals Verwirrung über die Attributwerte geben.
- Es kann nie passieren, dass ein Teil unseres Prozesses eine Referenz auf eine `Point`-Variable hat und „denkt“ das diese die Koordinaten $(0, 1)$ hat, aber das anderer Kode die Werte auf etwas anderes geändert hat.

Zusammenfassung: Unveränderliche Objekte



- Instanzen unserer Klasse `Point` speichern ein Paar Koordinaten in der zwei-dimensionalen Euklidischen Ebene.
- Die Operation `distance` ist untrennbar mit dieser Datenstruktur verbunden.
- Wir haben gelernt, dass es oft eine gute Idee ist, Objekte unveränderlich zu machen.
- Dann werden die Werte der Attribute eines Objektes nur während seiner Initialisierung (durch die `__init__` Methode) gesetzt und ändern sich danach nicht.
- Dann kann es niemals Verwirrung über die Attributwerte geben.
- Es kann nie passieren, dass ein Teil unseres Prozesses eine Referenz auf eine `Point`-Variable hat und „denkt“ das diese die Koordinaten $(0, 1)$ hat, aber das anderer Kode die Werte auf etwas anderes geändert hat.
- Das kann nicht passieren, weil sich die Koordinatenwerte einer Instanz von `Point` niemals ändern.

Zusammenfassung: Gekapselte Klasse



- Wenn sich Attribute ändern müssen, dann ist es oft eine gute Idee, sie zu kapseln *encapsulate*.

Zusammenfassung: Gekapselte Klasse



- Wenn sich Attribute ändern müssen, dann ist es oft eine gute Idee, sie zu kapseln *encapsulate*.
- Kapselung bedeutet, dass die Attribute eines Objekts nur durch die Methoden des Objekts gelesen oder geändert werden können.

Zusammenfassung: Gekapselte Klasse



- Wenn sich Attribute ändern müssen, dann ist es oft eine gute Idee, sie zu kapseln *encapsulate*.
- Kapselung bedeutet, dass die Attribute eines Objekts nur durch die Methoden des Objekts gelesen oder geändert werden können.
- Unsere Klasse `KahanSum` ist ein Beispiel dafür.

Zusammenfassung: Gekapselte Klasse



- Wenn sich Attribute ändern müssen, dann ist es oft eine gute Idee, sie zu kapseln *encapsulate*.
- Kapselung bedeutet, dass die Attribute eines Objekts nur durch die Methoden des Objekts gelesen oder geändert werden können.
- Unsere Klasse `KahanSum` ist ein Beispiel dafür.
- Die Klasse erlaubt es uns, Zahlen präziser zu addieren, in dem sie intern Fehlerterme zusätzlich zur normalen Summe unterhält.

Zusammenfassung: Gekapselte Klasse



- Wenn sich Attribute ändern müssen, dann ist es oft eine gute Idee, sie zu kapseln *encapsulate*.
- Kapselung bedeutet, dass die Attribute eines Objekts nur durch die Methoden des Objekts gelesen oder geändert werden können.
- Unsere Klasse `KahanSum` ist ein Beispiel dafür.
- Die Klasse erlaubt es uns, Zahlen präziser zu addieren, in dem sie intern Fehlerterme zusätzlich zur normalen Summe unterhält.
- Der Benutzer bekommt diese internen Attribute niemals zu sehen und kann sie auch nicht direkt verändern.

Zusammenfassung: Gekapselte Klasse



- Wenn sich Attribute ändern müssen, dann ist es oft eine gute Idee, sie zu kapseln *encapsulate*.
- Kapselung bedeutet, dass die Attribute eines Objekts nur durch die Methoden des Objekts gelesen oder geändert werden können.
- Unsere Klasse `KahanSum` ist ein Beispiel dafür.
- Die Klasse erlaubt es uns, Zahlen präziser zu addieren, in dem sie intern Fehlerterme zusätzlich zur normalen Summe unterhält.
- Der Benutzer bekommt diese internen Attribute niemals zu sehen und kann sie auch nicht direkt verändern.
- Stattdessen können sie verändert werden, in dem wir neue Zahlen an die Methode `add` liefern.

Zusammenfassung: Gekapselte Klasse



- Wenn sich Attribute ändern müssen, dann ist es oft eine gute Idee, sie zu kapseln *encapsulate*.
- Kapselung bedeutet, dass die Attribute eines Objekts nur durch die Methoden des Objekts gelesen oder geändert werden können.
- Unsere Klasse `KahanSum` ist ein Beispiel dafür.
- Die Klasse erlaubt es uns, Zahlen präziser zu addieren, in dem sie intern Fehlerterme zusätzlich zur normalen Summe unterhält.
- Der Benutzer bekommt diese internen Attribute niemals zu sehen und kann sie auch nicht direkt verändern.
- Stattdessen können sie verändert werden, in dem wir neue Zahlen an die Methode `add` liefern.
- Über die Methode `result` kann der Nutzer eine konsistente Sicht auf den Zustand der Summe bekommen, ohne durch den internen Zustand verwirrt zu werden.

Zusammenfassung: Python is Nachsichtig

- Python ist eine sehr nachsichtige Sprache.



Zusammenfassung: Python is Nachsichtig

- Python ist eine sehr nachsichtige Sprache.
- Es ist sehr nachsichtig darin, was erlaubt ist und was nicht.



Zusammenfassung: Python is Nachsichtig

- Python ist eine sehr nachsichtige Sprache.
- Es ist sehr nachsichtig darin, was erlaubt ist und was nicht.
- Sie erinnern sich daran, dass Type Hints nur Hinweise für Werkzeuge und Programmierer sind.

Zusammenfassung: Python is Nachsichtig

- Python ist eine sehr nachsichtige Sprache.
- Es ist sehr nachsichtig darin, was erlaubt ist und was nicht.
- Sie erinnern sich daran, dass Type Hints nur Hinweise für Werkzeuge und Programmierer sind.
- Der Python-Interpreter setzt sie nicht durch.

Zusammenfassung: Python is Nachsichtig

- Python ist eine sehr nachsichtige Sprache.
- Es ist sehr nachsichtig darin, was erlaubt ist und was nicht.
- Sie erinnern sich daran, dass Type Hints nur Hinweise für Werkzeuge und Programmierer sind.
- Der Python-Interpreter setzt sie nicht durch.
- Es ist problemlos möglich, so etwas wir `a: str = 5` auszuführen.

Zusammenfassung: Python is Nachsichtig

- Python ist eine sehr nachsichtige Sprache.
- Es ist sehr nachsichtig darin, was erlaubt ist und was nicht.
- Sie erinnern sich daran, dass Type Hints nur Hinweise für Werkzeuge und Programmierer sind.
- Der Python-Interpreter setzt sie nicht durch.
- Es ist problemlos möglich, so etwas wir `a: str = 5` auszuführen.
- Diese Nachsichtigkeit betrifft auch die beiden Designprinzipien von oben.

Zusammenfassung: Python is Nachsichtig

- Python ist eine sehr nachsichtige Sprache.
- Es ist sehr nachsichtig darin, was erlaubt ist und was nicht.
- Sie erinnern sich daran, dass Type Hints nur Hinweise für Werkzeuge und Programmierer sind.
- Der Python-Interpreter setzt sie nicht durch.
- Es ist problemlos möglich, so etwas wir `a: str = 5` auszuführen.
- Diese Nachsichtigkeit betrifft auch die beiden Designprinzipien von oben.
- Attribute und Variablen können „unveränderlich“ gemacht werden, in dem wir sie mit dem Type Hint `Final` annotieren.

Zusammenfassung: Python is Nachsichtig

- Python ist eine sehr nachsichtige Sprache.
- Es ist sehr nachsichtig darin, was erlaubt ist und was nicht.
- Sie erinnern sich daran, dass Type Hints nur Hinweise für Werkzeuge und Programmierer sind.
- Der Python-Interpreter setzt sie nicht durch.
- Es ist problemlos möglich, so etwas wir `a: str = 5` auszuführen.
- Diese Nachsichtigkeit betrifft auch die beiden Designprinzipien von oben.
- Attribute und Variablen können „unveränderlich“ gemacht werden, in dem wir sie mit dem Type Hint `Final` annotieren.
- Das ist natürlich wieder nur ein Type Hint, nur eine Information für Programmierer und Werkzeuge.

Zusammenfassung: Python is Nachsichtig



- Python ist eine sehr nachsichtige Sprache.
- Es ist sehr nachsichtig darin, was erlaubt ist und was nicht.
- Sie erinnern sich daran, dass Type Hints nur Hinweise für Werkzeuge und Programmierer sind.
- Der Python-Interpreter setzt sie nicht durch.
- Es ist problemlos möglich, so etwas wir `a: str = 5` auszuführen.
- Diese Nachsichtigkeit betrifft auch die beiden Designprinzipien von oben.
- Attribute und Variablen können „unveränderlich“ gemacht werden, in dem wir sie mit dem Type Hint `Final` annotieren.
- Das ist natürlich wieder nur ein Type Hint, nur eine Information für Programmierer und Werkzeuge.
- Der Python-Interpreter ignoriert das.

Zusammenfassung: Python is Nachsichtig



- Python ist eine sehr nachsichtige Sprache.
- Es ist sehr nachsichtig darin, was erlaubt ist und was nicht.
- Sie erinnern sich daran, dass Type Hints nur Hinweise für Werkzeuge und Programmierer sind.
- Der Python-Interpreter setzt sie nicht durch.
- Es ist problemlos möglich, so etwas wir `a: str = 5` auszuführen.
- Diese Nachsichtigkeit betrifft auch die beiden Designprinzipien von oben.
- Attribute und Variablen können „unveränderlich“ gemacht werden, in dem wir sie mit dem Type Hint `Final` annotieren.
- Das ist natürlich wieder nur ein Type Hint, nur eine Information für Programmierer und Werkzeuge.
- Der Python-Interpreter ignoriert das.
- Wie können erst `a: Final[int] = 5` und dann `a = 6` machen, ohne bestraft zu werden.

Zusammenfassung: Python is Nachsichtig

- Es ist sehr nachsichtig darin, was erlaubt ist und was nicht.
- Sie erinnern sich daran, dass Type Hints nur Hinweise für Werkzeuge und Programmierer sind.
- Der Python-Interpreter setzt sie nicht durch.
- Es ist problemlos möglich, so etwas wir `a: str = 5` auszuführen.
- Diese Nachsichtigkeit betrifft auch die beiden Designprinzipien von oben.
- Attribute und Variablen können „unveränderlich“ gemacht werden, in dem wir sie mit dem Type Hint `Final` annotieren.
- Das ist natürlich wieder nur ein Type Hint, nur eine Information für Programmierer und Werkzeuge.
- Der Python-Interpreter ignoriert das.
- Wie können erst `a: Final[int] = 5` und dann `a = 6` machen, ohne bestraft zu werden.
- Ähnliches gilt wenn wir Attribute durch Namen die mit einem doppelten Unterstrich (`__`) anfangen als privat markieren.

Zusammenfassung: Python is Nachsichtig

- Sie erinnern sich daran, dass Type Hints nur Hinweise für Werkzeuge und Programmierer sind.
- Der Python-Interpreter setzt sie nicht durch.
- Es ist problemlos möglich, so etwas wir `a: str = 5` auszuführen.
- Diese Nachsichtigkeit betrifft auch die beiden Designprinzipien von oben.
- Attribute und Variablen können „unveränderlich“ gemacht werden, in dem wir sie mit dem Type Hint `Final` annotieren.
- Das ist natürlich wieder nur ein Type Hint, nur eine Information für Programmierer und Werkzeuge.
- Der Python-Interpreter ignoriert das.
- Wie können erst `a: Final[int] = 5` und dann `a = 6` machen, ohne bestraft zu werden.
- Ähnliches gilt wenn wir Attribute durch Namen die mit einem doppelten Unterstrich (`__`) anfangen als privat markieren.
- Sie werden dadurch nicht privat.

Zusammenfassung: Python is Nachsichtig



- Der Python-Interpreter setzt sie nicht durch.
- Es ist problemlos möglich, so etwas wir `a: str = 5` auszuführen.
- Diese Nachsichtigkeit betrifft auch die beiden Designprinzipien von oben.
- Attribute und Variablen können „unveränderlich“ gemacht werden, in dem wir sie mit dem Type Hint `Final` annotieren.
- Das ist natürlich wieder nur ein Type Hint, nur eine Information für Programmierer und Werkzeuge.
- Der Python-Interpreter ignoriert das.
- Wie können erst `a: Final[int] = 5` und dann `a = 6` machen, ohne bestraft zu werden.
- Ähnliches gilt wenn wir Attribute durch Namen die mit einem doppelten Unterstrich (`__`) anfangen als privat markieren.
- Sie werden dadurch nicht privat.
- Der Python-Interpreter verändert zwar ihre Namen intern[306], aber ein cleverer Programmierer kann sie trotzdem auslesen und ändern.

Zusammenfassung: Python is Nachsichtig



- Es ist problemlos möglich, so etwas wir `a: str = 5` auszuführen.
- Diese Nachsichtigkeit betrifft auch die beiden Designprinzipien von oben.
- Attribute und Variablen können „unveränderlich“ gemacht werden, in dem wir sie mit dem Type Hint `Final` annotieren.
- Das ist natürlich wieder nur ein Type Hint, nur eine Information für Programmierer und Werkzeuge.
- Der Python-Interpreter ignoriert das.
- Wie können erst `a: Final[int] = 5` und dann `a = 6` machen, ohne bestraft zu werden.
- Ähnliches gilt wenn wir Attribute durch Namen die mit einem doppelten Unterstrich (`__`) anfangen als privat markieren.
- Sie werden dadurch nicht privat.
- Der Python-Interpreter verändert zwar ihre Namen intern[306], aber ein cleverer Programmierer kann sie trotzdem auslesen und ändern.
- Werkzeuge wie Mypy können solches Fehlverhalten finden.

Zusammenfassung: Python is Nachsichtig

- Diese Nachsichtigkeit betrifft auch die beiden Designprinzipien von oben.
- Attribute und Variablen können „unveränderlich“ gemacht werden, in dem wir sie mit dem Type Hint `Final` annotieren.
- Das ist natürlich wieder nur ein Type Hint, nur eine Information für Programmierer und Werkzeuge.
- Der Python-Interpreter ignoriert das.
- Wie können erst `a: Final[int] = 5` und dann `a = 6` machen, ohne bestraft zu werden.
- Ähnliches gilt wenn wir Attribute durch Namen die mit einem doppelten Unterstrich (`__`) anfangen als privat markieren.
- Sie werden dadurch nicht privat.
- Der Python-Interpreter verändert zwar ihre Namen intern[306], aber ein cleverer Programmierer kann sie trotzdem auslesen und ändern.
- Werkzeuge wie Mypy können solches Fehlverhalten finden.
- Python gibt die Verantwortung in unsere Hände.

Zusammenfassung: Python is Nachsichtig

- Attribute und Variablen können „unveränderlich“ gemacht werden, in dem wir sie mit dem Type Hint `Final` annotieren.
- Das ist natürlich wieder nur ein Type Hint, nur eine Information für Programmierer und Werkzeuge.
- Der Python-Interpreter ignoriert das.
- Wie können erst `a: Final[int] = 5` und dann `a = 6` machen, ohne bestraft zu werden.
- Ähnliches gilt wenn wir Attribute durch Namen die mit einem doppelten Unterstrich (`__`) anfangen als privat markieren.
- Sie werden dadurch nicht privat.
- Der Python-Interpreter verändert zwar ihre Namen intern[306], aber ein cleverer Programmierer kann sie trotzdem auslesen und ändern.
- Werkzeuge wie Mypy können solches Fehlverhalten finden.
- Python gibt die Verantwortung in unsere Hände.
- Wir müssen uns an Standards und Programmierregeln halten.

Zusammenfassung: Python is Nachsichtig



- Das ist natürlich wieder nur ein Type Hint, nur eine Information für Programmierer und Werkzeuge.
- Der Python-Interpreter ignoriert das.
- Wie können erst `a: Final[int] = 5` und dann `a = 6` machen, ohne bestraft zu werden.
- Ähnliches gilt wenn wir Attribute durch Namen die mit einem doppelten Unterstrich (`__`) anfangen als privat markieren.
- Sie werden dadurch nicht privat.
- Der Python-Interpreter verändert zwar ihre Namen intern[306], aber ein cleverer Programmierer kann sie trotzdem auslesen und ändern.
- Werkzeuge wie Mypy können solches Fehlverhalten finden.
- Python gibt die Verantwortung in unsere Hände.
- Wir müssen uns an Standards und Programmierregeln halten.
- Aber Python erzwingt das nicht.

谢谢您们！

Thank you!

Vielen Dank!



References I

[1] Joseph Adams, Egon Balas und Daniel Zawack. "The Shifting Bottleneck Procedure for Job Shop Scheduling". *Management Science* 34(3):391–401, März 1988. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 0025-1909. doi:10.1287/mnsc.34.3.391 (siehe S. 336).

[2] David J. Agans. *Debugging*. New York, NY, USA: AMACOM, Sep. 2002. ISBN: 978-0-8144-2678-4 (siehe S. 333).

[3] Miguel F. Anjos. *The QAPLIB Challenge*. Edinburgh, Scotland, UK: Miguel F. Anjos @ University of Edinburgh, 19. Nov. 2023. URL: https://www.miguelanjos.com/projects/2999 (besucht am 2026-09-28). The document lists the 28 instances of QAPLIB[43,45] that had not yet been solved to optimality by the time of its writing. (Siehe S. 341).

[4] Denis Antipov und Benjamin Doerr. "Precise Runtime Analysis for Plateaus". In: *15th International Conference on Parallel Problem Solving from Nature (PPSN XV)*. Bd. 2. 8.–12. Sep. 2018, Coimbra, Portugal. Hrsg. von Anne Auger, Carlos M. Fonseca, Nuno Lourenço, Penousal Machado, Luís Paquete und L. Darrell Whitley. Bd. 11102 der Reihe Lecture Notes in Computer Science (LNCS). Cham, Switzerland: Springer, 2018, S. 117–128. ISSN: 0302-9743. ISBN: 978-3-319-99258-7. doi:10.1007/978-3-319-99259-4_10 (siehe S. 340).

[5] David Lee Applegate, Robert E. Bixby, Vašek Chvátal und William John Cook. *The Traveling Salesman Problem: A Computational Study*. 2. Aufl. Bd. 17 der Reihe Princeton Series in Applied Mathematics. Princeton, NJ, USA: Princeton University Press, 2007. ISBN: 978-0-691-12993-8. This book also discusses the Concorde Traveling Salesperson Problem (TSP) solver. (Siehe S. 332, 344).

[6] David Lee Applegate und William John Cook. "A Computational Study of the Job-Shop Scheduling Problem". *ORSA Journal on Computing* 3(2):149–156, Mai 1991. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 0899-1499. doi:10.1287/ijoc.3.2.149. The Job Shop Scheduling Problem (JSSP) instances used were generated in Bonn in 1986. (Siehe S. 336).

[7] Adam Aspin und Karine Aspin. *Query Answers with MariaDB – Volume I: Introduction to SQL Queries*. Tetras Publishing, Okt. 2018. ISBN: 978-1-9996172-4-0. See also[8] (siehe S. 283, 338).

[8] Adam Aspin und Karine Aspin. *Query Answers with MariaDB – Volume II: In-Depth Querying*. Tetras Publishing, Okt. 2018. ISBN: 978-1-9996172-5-7. See also[7] (siehe S. 283, 338).

References II

[9] Anne Auger, Carlos M. Fonseca, Nuno Lourenço, Penousal Machado, Luís Paquete und L. Darrell Whitley, Hrsg. *15th International Conference on Parallel Problem Solving from Nature (PPSN XV)*. Bd. 2. 8.–12. Sep. 2018, Coimbra, Portugal. Bd. 11102 der Reihe Lecture Notes in Computer Science (LNCS). Cham, Switzerland: Springer, 2018. ISSN: 0302-9743. ISBN: 978-3-319-99258-7. doi:10.1007/978-3-319-99259-4.

[10] Ivo Babuška. "Numerical Stability in Mathematical Analysis". In: *World Congress on Information Processing (IFIP'1968)*. Bd. 1: Mathematics, Software. 5. Aug. 1966–10. Aug. 1968, Edinburgh, Scotland, UK. Hrsg. von A.J.H. Morrell. Laxenburg, Austria: International Federation for Information Processing (IFIP). Amsterdam, The Netherlands: North-Holland Publishing Co., 1969, S. 11–23. ISBN: 978-0-7204-2032-6 (siehe S. 65–69, 104–107, 121–123, 178–200, 304).

[11] Paul Gustav Heinrich Bachmann. *Die Analytische Zahlentheorie / Dargestellt von Paul Bachmann*. Bd. Zweiter Theil der Reihe Zahlentheorie: Versuch einer Gesamtdarstellung dieser Wissenschaft in ihren Haupttheilen. Leipzig, Sachsen, Germany: B. G. Teubner, 1894. ISBN: 978-1-4181-6963-3. URL: http://gallica.bnf.fr/ark:/12148/bpt6k994750 (besucht am 2023-12-13) (siehe S. 350).

[12] Thomas Bäck, David B. Fogel und Zbigniew „Zbyszek" Michalewicz, Hrsg. *Handbook of Evolutionary Computation*. Bristol, England, UK: IOP Publishing Ltd und Oxford, Oxfordshire, England, UK: Oxford University Press, 1997. ISBN: 978-0-7503-0392-7 (siehe S. 333, 334, 342).

[13] Daniel J. Barrett. *Efficient Linux at the Command Line*. Sebastopol, CA, USA: O'Reilly Media, Inc., Feb. 2022. ISBN: 978-1-0981-1340-7 (siehe S. 337, 343).

[14] Daniel Bartholomew. *Learning the MariaDB Ecosystem: Enterprise-level Features for Scalability and Availability*. New York, NY, USA: Apress Media, LLC, Okt. 2019. ISBN: 978-1-4842-5514-8 (siehe S. 338).

[15] Roberto Battiti und Giampietro Tecchiolli. "Local Search with Memory: Benchmarking RTS". *OR Spectrum: Quantitative Approaches in Management. Formerly Operations-Research-Spektrum* 17:67–86, Juni 1995. London, England, UK: Springer Nature Limited. ISSN: 0171-6468. doi:10.1007/BF01719249 (siehe S. 342).

[16] Roberto Battiti und Giampietro Tecchiolli. "Simulated Annealing and Tabu Search in the Long Run: A Comparison on QAP Tasks". *Computers & Mathematics with Applications* 28(6):1–8, Sep. 1994. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0898-1221. doi:10.1016/0898-1221(94)00147-2 (siehe S. 342, 344).

References III

[17] Roberto Battiti und Giampietro Tecchiolli. "The Reactive Tabu Search". 6(2):126–140, Mai 1994. doi:10.1287/ijoc.6.2.126. Applications of Reactive Tabu Search (RTS) to the 0-1 Knapsack and Quadratic Assignment Problem (QAP). (Siehe S. 342, 344).

[18] Roberto Battiti und Giampietro Tecchiolli. "Training Neural Nets with the Reactive Tabu Search". *IEEE Transactions on Neural Networks* 6(5):1185–1200, Sep. 1995. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE). ISSN: 1045-9227. doi:10.1109/72.410361. See also "Training Neural Nets with the Reactive Tabu Search, Technical Report UTM 421, Trento, Alto Adige/Südtirol, Italy: University of Trento, 11 1993" (siehe S. 342).

[19] James C. Bean. "Genetic Algorithms and Random Keys for Sequencing and Optimization". *ORSA Journal on Computing* 6(2):107–220, Frühling 1994. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 0899-1499. doi:10.1287/ijoc.6.2.154. Applies the Random Keys encoding to scheduling and the QAP. See [20]. (Siehe S. 285, 332, 341).

[20] James C. Bean. *Genetics and Random Keys for Sequencing and Optimization*. Techn. Ber. 92-43. Ann Arbor, MI, USA: University of Michigan, Department of Industrial & Operations Engineering, 30. Juni 1992. URL: https://www.math.wm.edu/~rrkinc/homework/RdmKeyGA.pdf (besucht am 2026-10-01). See [19]. (Siehe S. 285, 332, 341).

[21] John Edward Beasley. "OR-Library: Distributing Test Problems by Electronic Mail". *The Journal of the Operational Research Society (JORS)* 41:1069–1072, Nov. 1990. London, England, UK: Taylor and Francis Ltd. and London, England, UK: Springer Nature Limited. ISSN: 0160-5682. doi:10.1057/jors.1990.166. See the website[22] (siehe S. 285, 336, 340).

[22] John Edward Beasley. *Welcome to OR-Library*. Uxbridge, Middlesex, England, UK: Brunel University of London, Juni 1990–Feb. 2018. URL: https://people.brunel.ac.uk/~mastjjb/jeb/info.html (besucht am 2026-09-30). See the article[21]. Contains instances of the JSSP, 1D- and 2D-Packing, Knapsack, periodic TSPs, amongst others. (Siehe S. 285, 336, 340).

[23] Kent L. Beck. *JUnit Pocket Guide*. Sebastopol, CA, USA: O'Reilly Media, Inc., Sep. 2004. ISBN: 978-0-596-00743-0 (siehe S. 346).

References IV

[24] Andreas Beham, Michael Affenzeller und Erik Pitzer. "Metaheuristic Algorithms for the Quadratic Assignment Problem: Performance and Comparison". In: *Innovative Technologies in Management and Science*. Hrsg. von Ryszard Klempous und Jan Nikodem. Bd. 10 der Reihe Topics in Intelligent Engineering and Informatics (TIEI). Cham, Switzerland: Springer, 2015, S. 171–190. ISSN: 2193-9411. ISBN: 978-3-319-12651-7. doi:10.1007/978-3-319-12652-4_10. Algorithms like stochastic local search (SLS)/iterated local search (ILS), Simulated Annealing (SA), Tabu Search (TS), Variable Neighborhood Search (VNS), evolutionary algorithms (EAs), Memetic Algorithms (MAs) are compared and, with the exception of SA and EA, all perform best on some instance groups of QAPLIB. (Siehe S. 341).

[25] Richard Beigel. "Irrationality without Number Theory". *The American Mathematical Monthly* 98(4):332–335, Apr. 1991. London, England, UK: Taylor and Francis Ltd. ISSN: 1930-0972. doi:10.1080/00029890.1991.12000762. URL: https://cis.temple.edu/~beigel/papers/irrationality-amm.PS.gz (besucht am 2026-11-04) (siehe S. 335).

[26] Richard Ernest Bellman. *Dynamic Programming*. Princeton Landmarks in Mathematics and Physics. Princeton, NJ, USA: Princeton University Press, 1957–2003. ISBN: 978-0-691-14668-3 (siehe S. 349).

[27] Bengt-Erik Bengtsson. "Packing Rectangular Pieces— A Heuristic Approach". *The Computer Journal* 25(3):353–357, Aug. 1982. Oxford, Oxfordshire, England, UK: Oxford University Press. ISSN: 0010-4620. doi:10.1093/comjnl/25.3.353 (siehe S. 330).

[28] Tim Berners-Lee. *Re: Qualifiers on Hypertext links...* Geneva, Switzerland: World Wide Web project, European Organization for Nuclear Research (CERN) und Newsgroups: alt.hypertext, 6. Aug. 1991. URL: https://www.w3.org/People/Berners-Lee/1991/08/art-6484.txt (besucht am 2025-02-05) (siehe S. 347).

[29] Tim Berners-Lee, Larry Masinter und Mark P. McCahill. *Uniform Resource Locators (URL)*. Request for Comments (RFC) 1738. Wilmington, DE, USA: Internet Engineering Task Force (IETF), Dez. 1994. URL: https://www.ietf.org/rfc/rfc1738.txt (besucht am 2025-02-05) (siehe S. 346).

[30] Alex Berson. *Client/Server Architecture*. 2. Aufl. Computer Communications Series. New York, NY, USA: McGraw-Hill, 29. März 1996. ISBN: 978-0-07-005664-0 (siehe S. 332).

[31] Muhammed Beyaz, Tansel Dökeroğlu und Ahmet Cosar. "Robust Hyper-Heuristic Algorithms for the Offline Oriented/Non-Oriented 2D Bin Packing Problems". *Applied Soft Computing (ASOC)* 36:236–245, Nov. 2015. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 1568-4946. doi:10.1016/j.asoc.2015.06.063 (siehe S. 330).

References V

[32] Christian Bierwirth. "A Generalized Permutation Approach to Job Shop Scheduling with Genetic Algorithms". *OR Spectrum: Quantitative Approaches in Management. Formerly Operations-Research-Spektrum* 17(2–3):87–92, Juni 1995. London, England, UK: Springer Nature Limited. ISSN: 0171-6468. doi:10.1007/BF01719250. URL: https://www.researchgate.net/publication/240263036 (besucht am 2026-09-30). Proposes a permutations-with-repetitions based encoding for the JSSP, see also[33]. (Siehe S. 287, 299, 336, 348).

[33] Christian Bierwirth, Dirk C. Mattfeld und Herbert Kopfer. "On Permutation Representations for Scheduling Problems". In: *4th International Conference on Parallel Problem Solving from Nature (PPSN IV)*. 22.–24. Sep. 1996, Berlin, Germany. Hrsg. von Hans-Michael Voigt, Werner Ebeling, Ingo Rechenberg und Hans-Paul Schwefel. Bd. 1141 der Reihe Lecture Notes in Computer Science (LNCS). Berlin/Heidelberg, Germany: Springer-Verlag GmbH Germany, 1996, S. 310–318. ISSN: 0302-9743. ISBN: 978-3-540-61723-5. doi:10.1007/3-540-61723-X_995. URL: https://www.researchgate.net/publication/2753293 (besucht am 2026-09-30). Proposes a permutations-with-repetitions based encoding for the JSSP, see also[32]. (Siehe S. 287, 299, 336, 348).

[34] Jacek Błażewicz, Wolfgang Domschke und Erwin Pesch. "The Job Shop Scheduling Problem: Conventional and New Solution Techniques". *European Journal of Operational Research* 93(1):1–33, Aug. 1996. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/0377-2217(95)00362-2 (siehe S. 336).

[35] Joshua Bloch. *Effective Java*. Reading, MA, USA: Addison-Wesley Professional, Mai 2008. ISBN: 978-0-321-35668-0 (siehe S. 336).

[36] Christian Blum und Verena Schmid. "Solving the 2D Bin Packing Problem by Means of a Hybrid Evolutionary Algorithm". In: *International Conference on Computational Science (ICCS'2013)*. 5.–7. Juni 2013, Barcelona, Spain. Hrsg. von Vassil Alexandrov, Michael Lees, Valeria V. Krzhizhanovskaya, Jack J. Dongarra und Peter M. A. Sloot. Bd. 18 der Reihe Procedia Computer Science. Amsterdam, The Netherlands: Elsevier B.V., 2013, S. 899–908. ISSN: 1877-0509. doi:10.1016/j.procs.2013.05.255. An EA for the oriented two-dimensional Bin Packing Problem with free cuts (2BP|O|F) using the heuristic by Wong und Lee [332]. (Siehe S. 330).

[37] Borko Bošković und Janez Brest. "Computational Search of Long Skew-Symmetric Binary Sequences with High Merit Factors". *MENDEL – Soft Computing Journal* 28(2):17–24, Dez. 2022. Brno, Czech Republic: Brno University of Technology (Vysoké učení technické v Brně). ISSN: 1803-3814. doi:10.13164/mendel.2022.2.017 (siehe S. 337).

References VI

[38] Borko Bošković, Franc Brglez und Janez Brest. "Low-Autocorrelation Binary Sequences: On Improved Merit Factors and Runtime Predictions to Achieve Them". *Applied Soft Computing (ASOC)* 56:262–285, Juli 2017. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 1568-4946. doi:10.1016/J.ASOC.2017.02.024 (siehe S. 337).

[39] Silvia Botros und Jeremy Tinley. *High Performance MySQL*. 4. Aufl. Sebastopol, CA, USA: O'Reilly Media, Inc., Nov. 2021. ISBN: 978-1-4920-8051-0 (siehe S. 339).

[40] Ed Bott. *Windows 11 Inside Out*. Hoboken, NJ, USA: Microsoft Press, Pearson Education, Inc., Feb. 2023. ISBN: 978-0-13-769132-6 (siehe S. 339).

[41] Ron Brash und Ganesh Naik. *Bash Cookbook*. Birmingham, England, UK: Packt Publishing Ltd, Juli 2018. ISBN: 978-1-78862-936-2 (siehe S. 331).

[42] Janez Brest und Borko Bošković. "A Heuristic Algorithm for a Low Autocorrelation Binary Sequence Problem with Odd Length and High Merit Factor". *IEEE Access* 6:4127–4134, Jan. 2018. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE). ISSN: 2169-3536. doi:10.1109/ACCESS.2018.2789916 (siehe S. 337).

[43] Rainer E. Burkard, Eranda Çela, Stefan E. Karisch, Franz Rendl, Miguel F. Anjos und Peter M. Hahn. *QAPLIB – A Quadratic Assignment Problem Library – Problem Instances and Solutions*. Edinburgh, Scotland, UK: University of Edinburgh, 20. Apr. 2012–31. März 2022. doi:10.7488/ds/3428. This is the 2012 status of the QAPLIB dataset from[45] (siehe S. 283, 288, 341).

[44] Rainer E. Burkard, Eranda Çela, Panos Miltiades Pardalos und Leonidas S. Pitsoulis. "The Quadratic Assignment Problem". In: *Handbook of Combinatorial Optimization*. Hrsg. von Panos Miltiades Pardalos, Ding-Zhu Du und Ronald Lewis Graham. 1. Aufl. Boston, MA, USA: Springer, 1998, S. 1713–1809. ISBN: 978-1-4613-7987-4. doi:10.1007/978-1-4613-0303-9_27 (siehe S. 341).

[45] Rainer E. Burkard, Stefan E. Karisch und Franz Rendl. "QAPLIB – A Quadratic Assignment Problem Library". *Journal of Global Optimization* 10(4):391–403, Juni 1997. London, England, UK: Springer Nature Limited. ISSN: 0925-5001. doi:10.1023/A:1008293323270. This article introduces QAPLIB, a set of benchmark problems for the QAP, now hosted at [43]. (Siehe S. 283, 288, 341).

References VII

[46] Rainer E. Burkard und Josef Offermann. "Entwurf von Schreibmaschinentastaturen mittels quadratischer Zuordnungsprobleme". *Zeitschrift für Operations Research (ZOR). now Mathematical Methods of Operations Research (MMOR)* 21:B121–B132, Aug. 1977. Heidelberg, Baden-Württemberg, Germany: Physica-Verlag; now London, England, UK: Springer Nature Limited. ISSN: 0340-9422. doi:10.1007/BF01918175. Keyboard layout modeled as QAPs. (Siehe S. 341).

[47] Jason Cannon. *High Availability for the LAMP Stack*. Shelter Island, NY, USA: Manning Publications, Juni 2022 (siehe S. 337, 342).

[48] Jacques Carlier und Éric Pinson. "An Algorithm for Solving the Job-Shop Problem". *Management Science* 35(2):164–176, 1989. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 0025-1909. doi:10.1287/mnsc.35.2.164. A branch and bound (B&B) method for the JSSP. (Siehe S. 331).

[49] Stephan C. Carlson und The Editors of Encyclopaedia Britannica. *Golden Ratio*. Hrsg. von The Editors of Encyclopaedia Britannica. Chicago, IL, USA: Encyclopædia Britannica, Inc., 21. Okt. 2024. URL: https://www.britannica.com/science/golden-ratio (besucht am 2024-12-14) (siehe S. 347).

[50] Eduardo Carvalho Pinto und Carola Doerr. "A Simple Proof for the Usefulness of Crossover in Black-Box Optimization". In: *15th International Conference on Parallel Problem Solving from Nature (PPSN XV)*. Bd. 2. 8.–12. Sep. 2018, Coimbra, Portugal. Hrsg. von Anne Auger, Carlos M. Fonseca, Nuno Lourenço, Penousal Machado, Luís Paquete und L. Darrell Whitley. Bd. 11102 der Reihe Lecture Notes in Computer Science (LNCS). Cham, Switzerland: Springer, 2018, S. 29–41. ISSN: 0302-9743. ISBN: 978-3-319-99258-7. doi:10.1007/978-3-319-99259-4_3 (siehe S. 334).

[51] Eduardo Carvalho Pinto und Carola Doerr. *Towards a More Practice-Aware Runtime Analysis of Evolutionary Algorithms*. arXiv.org: Computing Research Repository (CoRR) abs/1812.00493. Ithaca, NY, USA: Cornell Universiy Library, 3. Dez. 2018. doi:10.48550/arXiv.1812.00493. URL: https://arxiv.org/abs/1812.00493 (besucht am 2025-08-08). arXiv:1812.00493v1 [cs.NE] 3 Dec 2018 (siehe S. 329, 334, 342).

[52] Antonio Cavacini. "Is the CE/BCE notation becoming a standard in scholarly literature?" *Scientometrics* 102(2):1661–1668, Juli 2015. London, England, UK: Springer Nature Limited. ISSN: 0138-9130. doi:10.1007/s11192-014-1352-1 (siehe S. 331, 332).

[53] Vladimír Černý. "Thermodynamical Approach to the Traveling Salesman Problem: An Efficient Simulation Algorithm". *Journal of Optimization Theory and Applications* 45(1):41–51, Jan. 1985. New York, NY, USA: Springer Science+Business Media, LLC. ISSN: 0022-3239. doi:10.1007/BF00940812 (siehe S. 342).

References VIII

[54] Noureddine Chabini und Rachid Beguenane. "FPGA-Based Designs of the Factorial Function". In: *IEEE Canadian Conference on Electrical and Computer Engineering (CCECE'2022)*. 18.–20. Sep. 2022, Halifax, NS, Canada. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE), 2022, S. 16–20. ISBN: 978-1-6654-8432-9. doi:10.1109/CCECE49351.2022.9918302 (siehe S. 347).

[55] Donald D. Chamberlin. "50 Years of Queries". *Communications of the ACM (CACM)* 67(8):110–121, Aug. 2024. New York, NY, USA: Association for Computing Machinery (ACM). ISSN: 0001-0782. doi:10.1145/3649887. URL: https://cacm.acm.org/research/50-years-of-queries (besucht am 2025-01-09) (siehe S. 343).

[56] Tony F. Chan, Gene H. Golub und Randall J. LeVeque. "Algorithms for Computing the Sample Variance: Analysis and Recommendations". *The American Statistician* 37(3):242–247, Aug. 1983. London, England, UK: Taylor and Francis Ltd. ISSN: 0003-1305. doi:10.2307/2683386. URL: https://engineering.yale.edu/application/files/3817/3714/7395/tr222.pdf (besucht am 2026-10-30) (siehe S. 325, 351).

[57] Bo Chen, Chris N. Potts und Gerhard J. Woeginger. "A Review of Machine Scheduling: Complexity, Algorithms and Approximability". In: *Handbook of Combinatorial Optimization*. Hrsg. von Panos Miltiades Pardalos, Ding-Zhu Du und Ronald Lewis Graham. 1. Aufl. Boston, MA, USA: Springer, 1998, S. 1493–1641. ISBN: 978-1-4613-7987-4. doi:10.1007/978-1-4613-0303-9_25. See also pages 21–169 in volume 3/3 by Norwell, MA, USA: Kluwer Academic Publishers. Discusses, amongst others, that the JSSP is $\mathcal{NP}$-hard. (Siehe S. 336, 350).

[58] Stephen Chen, James Montgomery und Antonio Bolufé-Röhler. "Measuring the Curse of Dimensionality and its Effects on Particle Swarm Optimization and Differential Evolution". *Applied Intelligence – The International Journal of Research on Intelligent Systems for Real Life Complex Problems* 42:514–526, Apr. 2015. London, England, UK: Springer Nature Limited. ISSN: 0924-669X. doi:10.1007/s10489-014-0613-2 (siehe S. 349).

[59] Jiayang Chen (陈嘉阳), Zhize Wu (吴志泽), Sarah Louise Thomson und Thomas Weise (汤卫思). "Frequency Fitness Assignment: Optimization Without Bias for Good Solution Outperforms Randomized Local Search on the Quadratic Assignment Problem". In: *16th International Joint Conference on Computational Intelligence (IJCCI'24)*. 20.–22. Nov. 2024, Porto, Portugal. Hrsg. von Francesco Marcelloni, Kurosh Madani, Niki van Stein und Joaquim Filipe. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2024, S. 27–37. ISSN: 2184-3236. ISBN: 978-989-758-721-4. doi:10.5220/0012888600003837 (siehe S. 333, 339, 341).

References IX

[60] T. C. Edwin Cheng, Bo Peng und Zhipeng Lü. "A Hybrid Evolutionary Algorithm to Solve the Job Shop Scheduling Problem". *Annals of Operations Research* 242(2):223–237, Juli 2016. London, England, UK: Springer Nature Limited. ISSN: 0254-5330. doi:10.1007/s10479-013-1332-5. A MA utlizing TS (siehe S. 338).

[61] Philippe Chrétienne, Edward G. Coffman, Jan Karel Lenstra und Zhen Liu, Hrsg. *Scheduling Theory and Its Applications*. Chichester, West Sussex, England, UK: John Wiley and Sons Ltd., Sep. 1995. ISBN: 978-0-471-94059-3 (siehe S. 336).

[62] Nestor M. Cid-Garcia und Yasmin A. Rios-Solis. "Positions and Covering: A Two-Stage Methodology to Obtain Optimal Solutions for the 2D-Bin Packing Problem". *PLoS ONE* 15(4):e0229358, Apr. 2020. San Francisco, CA, USA: Public Library of Science (PLOS). ISSN: 1932-6203. doi:10.1371/journal.pone.0229358. An exact method for the two-dimensional Bin Packing Problem with free cuts and rotation (2BP|R|F) and the 2BP|O|F. (Siehe S. 330).

[63] David Clinton und Christopher Negus. *Ubuntu Linux Bible*. 10. Aufl. Bible Series. Chichester, West Sussex, England, UK: John Wiley and Sons Ltd., 10. Nov. 2020. ISBN: 978-1-119-72233-5 (siehe S. 343, 345).

[64] Edgar Frank „Ted" Codd. "A Relational Model of Data for Large Shared Data Banks". *Communications of the ACM (CACM)* 13(6):377–387, Juni 1970. New York, NY, USA: Association for Computing Machinery (ACM). ISSN: 0001-0782. doi:10.1145/362384.362685. URL: https://www.seas.upenn.edu/~zives/03f/cis550/codd.pdf (besucht am 2025-01-05) (siehe S. 341).

[65] Stephen Arthur Cook. "The Complexity of Theorem-Proving Procedures". In: *Third Annual ACM Symposium on Theory of Computing (STOC'1971)*. 3.–5. Mai 1971, Shaker Heights, OH, USA. Hrsg. von Michael A. Harrison, Ranan B. Banerji und Jeffrey D. Ullman. New York, NY, USA: Association for Computing Machinery (ACM), 1971, S. 151–158. ISBN: 978-1-4503-7464-4. doi:10.1145/800157.805047 (siehe S. 338, 350).

[66] William John Cook. *Concorde TSP Solver*. Waterloo, ON, Canada: University of Waterloo, 1997. URL: https://math.uwaterloo.ca/tsp/concorde (besucht am 2026-09-26) (siehe S. 332).

[67] William John Cook. *In Pursuit of the Traveling Salesman: Mathematics at the Limits of Computation*. Princeton, NJ, USA: Princeton University Press, 2012–9. Nov. 2014. ISBN: 978-0-691-16352-9 (siehe S. 332, 338).

[68] William John Cook. *World TSP*. Waterloo, ON, Canada: University of Waterloo, 2013–25. Okt. 2025. URL: http://www.math.uwaterloo.ca/tsp/world (besucht am 2026-01-03) (siehe S. 338, 344).

References X

[69] Thomas H. Cormen, Charles E. Leiserson, Ronald Linn Rivest und Clifford Stein. "Asymptotic Notation". In: *Introduction to Algorithms*. 3. Aufl. Cambridge, MA, USA: MIT Press, 2009. Kap. 3.1, S. 43–53. ISBN: 978-0-262-03384-8. Discusses Θ, $\mathcal{O}$, Ω, o, and ω (siehe S. 350, 351).

[70] Cemre Çubukçuoğlu, Pirouz Nourian, M. Fatih Tasgetiren, I. Sevil Sariyildiz und Shervin Azadi. "Hospital Layout Design Renovation as a Quadratic Assignment Problem with Geodesic Distances". *Journal of Building Engineering* 44:102952, Dez. 2021. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 2352-7102. doi:10.1016/j.jobe.2021.102952 (siehe S. 341).

[71] Christopher Cullen. "Learning from Liu Hui? A Different Way to Do Mathematics". *Notices of the American Mathematical Society* 49(7):783–790, Aug. 2002. Providence, RI, USA: American Mathematical Society (AMS). ISSN: 1088-9477. URL: https://www.ams.org/notices/200207/comm-cullen.pdf (besucht am 2024-08-09) (siehe S. 347).

[72] *Database Language SQL*. Techn. Ber. ANSI X3.135-1986. Washington, D.C., USA: American National Standards Institute (ANSI), 1986 (siehe S. 343).

[73] Joseph W. Dauben. "Archimedes and Liu Hui on Circles and Spheres". *Ontology Studies (Cuadernos de Ontología)* 10:21–38, 2010. Leioa, Bizkaia, Spain: Universidad del País Vasco / Euskal Herriko Unibertsitatea. ISSN: 1576-2270. URL: https://ddd.uab.cat/pub/ontstu/ontstu_a2010n10/ontstu_a2010n10p21.pdf (besucht am 2026-11-03). Also discusses LIU Hui's (刘徽) method of approximating π (siehe S. 347).

[74] Matt David und Blake Barnhill. *How to Teach People SQL*. San Francisco, CA, USA: The Data School, Chart.io, Inc., 10. Dez. 2019–10. Apr. 2023. URL: https://dataschool.com/how-to-teach-people-sql (besucht am 2025-02-27) (siehe S. 343).

[75] *Database Language SQL*. International Standard ISO 9075-1987. Geneva, Switzerland: International Organization for Standardization (ISO), 1987 (siehe S. 343).

[76] Kenneth Alan De Jong. *Evolutionary Computation: A Unified Approach*. Bd. 4 der Reihe Complex Adaptive Systems. Cambridge, MA, USA: MIT Press, 2006. ISBN: 978-0-262-04194-2. URL: https://www.researchgate.net/publication/220740669 (besucht am 2025-08-08) (siehe S. 334, 342).

[77] *Definition of Operations Research*. University of Western Ontario, London, ON, Canada: International Federation of Operational Research Societies (IFORS), 2020. URL: https://www.ifors.org/what-is-or (besucht am 2026-01-01) (siehe S. 340).

References XI

[78] Paul Deitel, Harvey Deitel und Abbey Deitel. *Internet & World Wide WebW[: How to Program*. 5. Aufl. Hoboken, NJ, USA: Pearson Education, Inc., Nov. 2011. ISBN: 978-0-13-299045-5 (siehe S. 347).

[79] Maxence Delorme, Manuel Iori und Silvano Martello. "Bin Packing and Cutting Stock Problems: Mathematical Models and Exact Algorithms". *European Journal of Operational Research* 255(1):1–20, Nov. 2016. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/j.ejor.2016.04.030 (siehe S. 332).

[80] Ebru Demirkol, Sanjay V. Mehta und Reha Uzsoy. "Benchmarks for Shop Scheduling Problems". *European Journal of Operational Research* 109(1):137–141, Aug. 1998. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/S0377-2217(97)00019-2 (siehe S. 336).

[81] Alfredo Deza und Noah Gift. *Testing In Python*. San Francisco, CA, USA: Pragmatic AI Labs, Feb. 2020. ISBN: 979-8-6169-6064-1 (siehe S. 340).

[82] Miroslav Dimitrov. *New Classes of Binary Sequences with High Merit Factor*. arXiv.org: Computing Research Repository (CoRR). Ithaca, NY, USA: Cornell Universiy Library, 24. Juni 2022. doi:10.48550/arXiv.2206.12070. URL: https://arxiv.org/pdf/2206.12070 (besucht am 2026-09-21). arXiv:2206.12070v1 [cs.IT] 24 Jun 2022 (siehe S. 337).

[83] Miroslav Dimitrov, Tsonka Baitcheva und Nikolay Nikolov. "Efficient Generation of Low Autocorrelation Binary Sequences". *IEEE Signal Processing Letters* 27:341–345, 2020. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE). ISSN: 1558-2361. doi:10.1109/LSP.2020.2972127 (siehe S. 337).

[84] Slobodan Dmitrović. *Modern C for Absolute Beginners: A Friendly Introduction to the C Programming Language*. New York, NY, USA: Apress Media, LLC, März 2024. ISBN: 979-8-8688-0224-9 (siehe S. 332).

[85] "Doctest – Test Interactive Python Examples". In: *Python 3 Documentation. The Python Standard Library*. Beaverton, OR, USA: Python Software Foundation (PSF), 2001–2025. URL: https://docs.python.org/3/library/doctest.html (besucht am 2024-11-07) (siehe S. 333).

[86] Benjamin Doerr und Carola Doerr. "Optimal Static and Self-Adjusting Parameter Choices for the $(1 + (\lambda, \lambda))$ Genetic Algorithm". *Algorithmica* 80(5):1658–1709, Mai 2018. London, England, UK: Springer Nature Limited. ISSN: 0178-4617. doi:10.1007/s00453-017-0354-9 (siehe S. 342).

References XII

[87] Benjamin Doerr, Carola Doerr und Franziska Ebel. "From Black-Box Complexity to Designing New Genetic Algorithms". *Theoretical Computer Science* 567:87–104, Feb. 2015. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0304-3975. doi:10.1016/J.TCS.2014.11.028 (siehe S. 340).

[88] Carola Doerr, Furong Ye, Naama Horesh, Hao Wang und Ofer M. ShirThomas Bäck. "Benchmarking Discrete Optimization Heuristics with IOHprofiler". In: *21st Genetic and Evolutionary Computation Conference Companion (GECCO'2019)*. 13.–17. Juli 2019, Prague, Czech Republic. Hrsg. von Manuel López-Ibáñez, Anne Auger und Thomas Stützle. New York, NY, USA: Association for Computing Machinery (ACM), 2019, S. 1798–1806. ISBN: 978-1-4503-6748-6. doi:10.1145/3319619.3326810. See also[89,90] (siehe S. 294).

[89] Carola Doerr, Furong Ye, Naama Horesh, Hao Wang und Ofer M. ShirThomas Bäck. "Benchmarking Discrete Optimization Heuristics with IOHprofiler". *Applied Soft Computing (ASOC)* 88:106027, März 2020. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 1568-4946. doi:10.1016/j.asoc.2019.106027. See also[88,90] (siehe S. 294, 329, 335, 336, 339).

[90] Carola Doerr, Furong Ye, Naama Horesh, Hao Wang und Ofer M. ShirThomas Bäck. "Benchmarking Discrete Optimization Heuristics with IOHprofiler". (abs/1912.09237), 31. Jan. 2021. doi:10.48550/arXiv.1912.09237. URL: https://arxiv.org/abs/1912.09237 (besucht am 2024-06-27). arXiv:1912.09237v2 [cs.NE] 31 Jan 2021, see also[88,89] (siehe S. 294).

[91] Marco Dorigo, Mauro Birattari und Thomas Stützle. "Ant Colony Optimization". *IEEE Computational Intelligence Magazine (CIM)* 1(4):28–39, Nov. 2006. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE). ISSN: 1556-603X. doi:10.1109/MCI.2006.329691. URL: https://www.researchgate.net/publication/308953674 (besucht am 2025-07-25) (siehe S. 330).

[92] Marco Dorigo, Vittorio Maniezzo und Alberto Colorni. "Ant System: Optimization by a Colony of Cooperating Agents". *IEEE Transactions on Systems, Man, and Cybernetics – Part B: Cybernetics* 26(1):29–41, Feb. 1996. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE). ISSN: 1083-4419. doi:10.1109/3477.484436. URL: https://www.researchgate.net/publication/5589170 (besucht am 2025-07-26) (siehe S. 330).

[93] Marco Dorigo und Thomas Stützle. *Ant Colony Optimization*. Cambridge, MA, USA: MIT Press, 2004. ISBN: 978-0-262-04219-2 (siehe S. 330).

[94] Johann Dréo, Alain Pétrowski, Patrick Siarry und Éric D. Taillard. *Metaheuristics for Hard Optimization – Methods and Case Studies*. Berlin/Heidelberg, Germany: Springer-Verlag GmbH Germany, 2006. ISBN: 978-3-540-23022-9. doi:10.1007/3-540-30966-7. Discusses SA, TS, EAs, and Ant Colony Optimization (ACO). (Siehe S. 341).

References XIII

[95] Stefan Droste, Thomas Jansen und Ingo Wegener. "On the Analysis of the $(1 + 1)$ Evolutionary Algorithm". *Theoretical Computer Science* 276(1-2):51–81, Apr. 2002. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0304-3975. doi:10.1016/S0304-3975(01)00182-7 (siehe S. 329, 337, 343).

[96] Stefan Droste, Thomas Jansen und Ingo Wegener. "Upper and Lower Bounds for Randomized Search Heuristics in Black-Box Optimization". *Theoretical Computer Science* 39(4):525–544, Juli 2006. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0304-3975. doi:10.1007/S00224-004-1177-Z (siehe S. 340).

[97] Jacques Dutka. "The Early History of the Factorial Function". *Archive for History of Exact Sciences* 43(3):225–249, Sep. 1991. Berlin/Heidelberg, Germany: Springer-Verlag GmbH Germany. ISSN: 0003-9519. doi:10.1007/BF00389433. Communicated by Umberto Bottazzini (siehe S. 347).

[98] Harald Dyckhoff und Ute Finke. *Cutting and Packing in Production and Distribution: A Typology and Bibliography*. Contributions to Management Science. Heidelberg, Baden-Württemberg, Germany: Physica-Verlag, 1992. ISSN: 1431-1941. ISBN: 978-3-642-63487-1. doi:10.1007/978-3-642-58165-6 (siehe S. 332).

[99] Russell J.T. Dyer. *Learning MySQL and MariaDB*. Sebastopol, CA, USA: O'Reilly Media, Inc., März 2015. ISBN: 978-1-4493-6290-4 (siehe S. 338, 339).

[100] Kelly Easton, George L. Nemhauser und Michael A. Trick. "The Traveling Tournament Problem Description and Benchmarks". In: *7th International Conference on Principles and Practice of Constraint Programming (CP'01)*. 26. Nov.–1. Dez. 2001, Paphos, Cyprus. Hrsg. von Toby Walsh. Bd. 2239 der Reihe Lecture Notes in Computer Science (LNCS). Berlin/Heidelberg, Germany: Springer-Verlag GmbH Germany, 2001, S. 580–584. ISSN: 0302-9743. ISBN: 978-3-540-42863-3. doi:10.1007/3-540-45578-7_43 (siehe S. 345).

[101] Alwalid N. Elshafei. "Hospital Layout as a Quadratic Assignment Problem". *The Journal of the Operational Research Society (JORS)* 28(1):167–179, Apr. 1977. London, England, UK: Taylor and Francis Ltd. and London, England, UK: Springer Nature Limited. ISSN: 0160-5682. doi:10.1057/jors.1977.29 (siehe S. 341).

[102] Bernhard Eschermann und Hans-Joachim Wunderlich. "Optimized Synthesis of Self-Testable Finite State Machines". In: *20th International Symposium Fault-Tolerant Computing (FTCS'1990): Digest of Papers*. 26.–28. Juni 1990, Newcastle Upon Tyne, England, UK. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE), 1990. ISBN: 978-0-8186-2051-5. doi:10.1109/FTCS.1990.89393 (siehe S. 341).

References XIV

[103] Euclid of Alexandria (*Εὐκλείδης*). *Euclid's Elements of Geometry (Στοιχεῖα). The Greek Text of J.L. Heiberg (1883-1885) from Euclidis Elementa, Edidit et Latine Interpretatus est I.L. Heiberg in Aedibus B. G. Teubneri, 1883-1885. Edited, and provided with a modern English translation, by Richard Fitzpatrick*. Hrsg. von Richard Fitzpatrick. Übers. von Johan Ludvig Heiberg. revised and corrected. Austin, TX, USA: The University of Texas at Austin, 2008. ISBN: 978-0-615-17984-1. URL: https://farside.ph.utexas.edu/Books/Euclid/Elements.pdf (besucht am 2024-09-30) (siehe S. 347).

[104] Leonhard Euler. "An Essay on Continued Fractions". Übers. von Myra F. Wyman und Bostwick F. Wyman. *Mathematical Systems Theory* 18(1):295–328, Dez. 1985. New York, NY, USA: Springer Science+Business Media, LLC. ISSN: 1432-4350. doi:10.1007/BF01699475. URL: https://www.researchgate.net/publication/301720080 (besucht am 2024-09-24). Translation of [105]. (Siehe S. 296).

[105] Leonhard Euler. "De Fractionibus Continuis Dissertation". *Commentarii Academiae Scientiarum Petropolitanae* 9:98–137, 1737–1744. Petropolis (St. Petersburg), Russia: Typis Academiae. URL: https://scholarlycommons.pacific.edu/cgi/viewcontent.cgi?article=1070 (besucht am 2024-09-24). See [104] for a translation. (Siehe S. 296, 348).

[106] Thomas A. Feo und Mauricio G. C. Resende. "Greedy Randomized Adaptive Search Procedures". *Journal of Global Optimization* 6(2):109–133, März 1995. London, England, UK: Springer Nature Limited. ISSN: 0925-5001. doi:10.1007/BF01096763. Introduces Greedy Randomized Adaptive Search Procedures (GRASPs) and applies them to the set covering problem, among others, and lists many problem domains. (Siehe S. 334).

[107] Luca Ferrari und Enrico Pirozzi. *Learn PostgreSQL*. 2. Aufl. Birmingham, England, UK: Packt Publishing Ltd, Okt. 2023. ISBN: 978-1-83763-564-1 (siehe S. 340).

[108] Paola Festa und Mauricio G. C. Resende. "An Annotated Bibliography of GRASP—Part I: Algorithms". *International Transactions on Operational Research (ITOR)* 16(1):1–24, Jan. 2009. Chichester, West Sussex, England, UK: John Wiley and Sons Ltd. ISSN: 0969-6016. doi:10.1111/j.1475-3995.2009.00663.x. See also[109] (siehe S. 296, 334).

[109] Paola Festa und Mauricio G. C. Resende. "An Annotated Bibliography of GRASP—Part II: Applications". *International Transactions on Operational Research (ITOR)* 16(2):131–172, März 2009. Chichester, West Sussex, England, UK: John Wiley and Sons Ltd. ISSN: 0969-6016. doi:10.1111/j.1475-3995.2009.00664.x. See also[108] (siehe S. 296, 334).

References XV

[110] Michael Filaseta. "The Transcendence of e and π". In: *Math 785: Transcendental Number Theory*. Columbia, SC, USA: University of South Carolina, Frühling 2011. Kap. 6. URL: https://people.math.sc.edu/filaseta/gradcourses/Math785/Math785Notes6.pdf (besucht am 2024-07-05) (siehe S. 347, 348).

[111] Simon Fischer. "A Polynomial Upper Bound for a Mutation-Based Algorithm on the Two-Dimensional Ising Model". In: *6th Genetic and Evolutionary Computation Conference (GECCO'2004), Part I*. 26.–30. Juni 2004, Seattle, WA, USA. Hrsg. von Kalyanmoy Deb, Riccardo Poli, Wolfgang Banzhaf, Hans-Georg Beyer, Edmund K. Burke, Paul J. Darwen, Dipankar Dasgupta, Dario Floreano, James A. Foster, Mark Harman, Owen Holland, Pier Luca Lanzi, Lee Spector, Andrea Tettamanzi, Dirk Thierens und Andrew M. Tyrrell. Bd. 3102 der Reihe Lecture Notes in Computer Science (LNCS). Berlin/Heidelberg, Germany: Springer-Verlag GmbH Germany, 2004, S. 1100–1112. ISSN: 0302-9743. ISBN: 978-3-540-22344-3. doi:10.1007/978-3-540-24854-5_108 (siehe S. 335, 336).

[112] Simon Fischer und Ingo Wegener. "The One-Dimensional Ising Model: Mutation versus Recombination". *Theoretical Computer Science* 344(2-3):208–225, Nov. 2005. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0304-3975. doi:10.1016/J.TCS.2005.04.002 (siehe S. 335, 336).

[113] Henry Fisher und Gerald L. Thompson. "Probabilistic Learning Combinations of Local Job-Shop Scheduling Rules". In: *Industrial Scheduling*. Hrsg. von John F. Muth und Gerald L. Thompson. Englewood Cliffs, NJ, USA: Prentice-Hall, 1963. Kap. 3.2, S. 225–251 (siehe S. 336).

[114] Charles Fleurent und Jacques A. Ferland. "Genetic Hybrids for the Quadratic Assignment Problem". In: *Quadratic Assignment and Related Problems, Proceedings of a DIMACS Workshop*. 20.–21. Mai 1993, New Brunswick, NJ, USA: Rutgers University. Hrsg. von Panos Miltiades Pardalos und Henry Wolkowicz. Providence, RI, USA: American Mathematical Society (AMS), 1994, S. 173–187. ISBN: 978-0-8218-6607-8. A MA using the TS from[288] as local search performs well. (Siehe S. 338, 341).

[115] "Floating-Point Arithmetic: Issues and Limitations". In: *Python 3 Documentation. The Python Tutorial*. Beaverton, OR, USA: Python Software Foundation (PSF), 2001–2025. Kap. 15. URL: https://docs.python.org/3/tutorial/floatingpoint.html (besucht am 2024-12-08) (siehe S. 30–41).

[116] Tobias Friedrich, Pietro Simone Oliveto, Dirk Sudholt und Carsten Witt. "Analysis of Diversity-Preserving Mechanisms for Global Exploration". *Evolutionary Computation* 17(4):455–476, Winter 2009. Cambridge, MA, USA: MIT Press. ISSN: 1063-6560. doi:10.1162/evco.2009.17.4.17401 (siehe S. 345).

References XVI

[117] Tobias Friedrich, Francesco Quinzan und Markus Wagner. "Escaping Large Deceptive Basins of Attraction with Heavy-Tailed Mutation Operators". In: *20th Genetic and Evolutionary Computation Conference (GECCO'2018)*. 15.–19. Juli 2018, Kyōto, Kyōto-fu, Japan. Hrsg. von Hernán E. Aguirre und Keiki Takadama. New York, NY, USA: Association for Computing Machinery (ACM), 2018, S. 293–300. ISBN: 978-1-4503-5618-3. doi:10.1145/3205455.3205515 (siehe S. 337, 345).

[118] José E. Gallardo, Carlos Cotta und Antonio J. Fernández. "A Memetic Algorithm for the Low Autocorrelation Binary Sequence Problem". In: *9th Genetic and Evolutionary Computation Conference (GECCO'2007)*. 7.–11. Juli 2007, London, England, UK. Hrsg. von Hod Lipson. New York, NY, USA: Association for Computing Machinery (ACM), 2007, S. 1226–1233. ISBN: 978-1-59593-697-4. doi:10.1145/1276958.1277195 (siehe S. 337).

[119] Michael R. Garey und David Stifler Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. New York, NY, USA: W. H. Freeman and Company, 1979. ISBN: 978-0-7167-1045-5 (siehe S. 344).

[120] Mitsuo Gen, Yasuhiro Tsujimura und Erika Kubota. "Solving Job-Shop Scheduling Problems by Genetic Algorithm". In: *Humans, Information and Technology: Proceedings of the 1994 IEEE International Conference on Systems, Man and Cybernetics (SMC'1994)*. Bd. 2. 2.–5. Okt. 1994, San Antonio, TX, USA. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE), 1994, S. 1577–1582. ISBN: 978-0-7803-2129-8. doi:10.1109/ICSMC.1994.400072. Proposes a permutations-with-repetitions based encoding for the JSSP. (Siehe S. 299, 336, 348).

[121] Fred Glover. "Tabu Search – Part I". *ORSA Journal on Computing* 1(3):190–206, Sommer 1989. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 0899-1499. doi:10.1287/ijoc.1.3.190 (siehe S. 344).

[122] Fred Glover. "Tabu Search – Part II". *ORSA Journal on Computing* 2(1):4–32, Winter 1990. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 0899-1499. doi:10.1287/ijoc.2.1.4 (siehe S. 344).

[123] Fred Glover und Gary A. Kochenberger, Hrsg. *Handbook of Metaheuristics*. Bd. 57 der Reihe International Series in Operations Research & Management Science (ISOR). Cham, Switzerland: Springer, 2003. ISSN: 0884-8289. ISBN: 978-0-306-48056-0. doi:10.1007/b101874 (siehe S. 338).

[124] Fred Glover, Éric D. Taillard und Dominique de Werra. "A User's Guide to Tabu Search". *Annals of Operations Research* 41(1):3–28, März 1993. London, England, UK: Springer Nature Limited. ISSN: 0254-5330. doi:10.1007/BF02078647. Special issue on TS. (Siehe S. 344).

References XVII

[125] David Goldberg. "What Every Computer Scientist Should Know About Floating-Point Arithmetic". *ACM Computing Surveys (CSUR)* 23(1):5–48, März 1991. New York, NY, USA: Association for Computing Machinery (ACM). ISSN: 0360-0300. doi:10.1145/103162.103163. URL: https://pages.cs.wisc.edu/~david/courses/cs552/S12/handouts/goldberg-floating-point.pdf (besucht am 2025-09-03) (siehe S. 65–69, 306).

[126] David Edward Goldberg. *Genetic Algorithms in Search, Optimization, and Machine Learning*. Redwood City, CA, USA: Addison Wesley Longman Publishing Co., Inc., 1989. ISBN: 978-0-201-15767-3 (siehe S. 334, 342).

[127] José Fernando Gonçalves und Mauricio G. C. Resende. "A Biased Random Key Genetic Algorithm for two-dimensional Bin Packing Problem (2BP) and 3D Bin Packing Problems". *International Journal of Production Economics (IJPE)* 145(2):500–510, Okt. 2013. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0925-5273. doi:10.1016/j.ijpe.2013.04.019. Applies the Biased Random Key Genetic Algorithm (BRKGA) to the 2BP|R|F and 2BP|O|F as well as 3D packing. Continuous vectors are translated to permutations and orientation data which is then the input of a placement heuristic. (Siehe S. 330, 332).

[128] José Fernando Gonçalves und Mauricio G. C. Resende. "An Extended Akers Graphical Method with a Biased Random-Key Genetic Algorithm for Job-Shop Scheduling". *International Transactions on Operational Research (ITOR)* 21(2):215–246, März 2014. Chichester, West Sussex, England, UK: John Wiley and Sons Ltd. ISSN: 0969-6016. doi:10.1111/itor.12044. URL: http://mauricio.resende.info/doc/brkga-jss2011.pdf (besucht am 2026-09-30). The BRKGA basically is a MA using a continuous encoding of permutations that are decoded to schedules using the method from[32,33,120,268] and then refined via a TS. (Siehe S. 332, 336).

[129] José Fernando Gonçalves und Mauricio G. C. Resende. "Biased Random-Key Genetic Algorithms for Combinatorial Optimization". *Journal of Heuristics* 17:487–525, Okt. 2010. London, England, UK: Springer Nature Limited. ISSN: 1381-1231. doi:10.1007/s10732-010-9143-1 (siehe S. 332).

[130] David Goodger und Guido van Rossum. *Docstring Conventions*. Python Enhancement Proposal (PEP) 257. Beaverton, OR, USA: Python Software Foundation (PSF), 29. Mai–13. Juni 2001. URL: https://peps.python.org/pep-0257 (besucht am 2024-07-27) (siehe S. 333).

[131] Michael T. Goodrich. *A Gentle Introduction to NP-Completeness*. Irvine, CA, USA: University of California, Irvine, Apr. 2022. URL: https://ics.uci.edu/~goodrich/teach/cs165/notes/NPComplete.pdf (besucht am 2025-08-01) (siehe S. 349, 350).

References XVIII

[132] Michael Goodwin. *What is an API?* Armonk, NY, USA: International Business Machines Corporation (IBM), 9. Apr. 2024. URL: https://www.ibm.com/topics/api (besucht am 2024-12-12) (siehe S. 331).

[133] Ronald Lewis Graham, Eugene Leighton Lawler, Jan Karel Lenstra und Alexander Hendrik George Rinnooy Kan. "Optimization and Approximation in Deterministic Sequencing and Scheduling: A Survey". Annals of Discrete Mathematics (5):287–326. Hrsg. von Peter Ladislaw Hammer, Ellis Lane Johnson und Bernhard H. Korte, 1979. ISSN: 0167-5060. doi:10.1016/S0167-5060(08)70356-X. An early review on JSSPs and FSSPs. (Siehe S. 333, 336).

[134] Martin Grötschel, Michael Jünger und Gerhard Reinelt. "Optimal Control of Plotting and Drilling Machines: A Case Study". *Zeitschrift für Operations Research (ZOR). now Mathematical Methods of Operations Research (MMOR)* 35(1):61–84, Jan. 1991. Heidelberg, Baden-Württemberg, Germany: Physica-Verlag; now London, England, UK: Springer Nature Limited. ISSN: 0340-9422. doi:10.1007/BF01415960. This paper tackles circuit board plotting and drilling problems by modelling (parts of) them as symmetric Traveling Salesperson Problems (STSPs). (Siehe S. 344).

[135] Gregory Z. Gutin und Abraham P. Punnen, Hrsg. *The Traveling Salesman Problem and its Variations*. Bd. 12 der Reihe Combinatorial Optimization (COOP). New York, NY, USA: Springer New York, Mai 2002. ISSN: 1388-3011. doi:10.1007/b101971 (siehe S. 344).

[136] Terry Halpin und Tony Morgan. *Information Modeling and Relational Databases*. 3. Aufl. Burlington, MA, USA/San Mateo, CA, USA: Morgan Kaufmann Publishers, Juli 2024. ISBN: 978-0-443-23791-1 (siehe S. 341).

[137] Pierre Hansen, Nenad Mladenović und José Andrés Moreno Pérez. "Variable Neighbourhood Search: Methods and Applications. Invited Survey". *4OR* 6(4):319–360, Dez. 2008. ISSN: 1619-4500. doi:10.1007/s10288-008-0089-1. URL: https://www.researchgate.net/publication/225155945 (besucht am 2026-10-02) (siehe S. 347).

[138] Jan L. Harrington. *Relational Database Design and Implementation*. 4. Aufl. Burlington, MA, USA/San Mateo, CA, USA: Morgan Kaufmann Publishers, Apr. 2016. ISBN: 978-0-12-849902-3 (siehe S. 341).

[139] Michael Hausenblas. *Learning Modern Linux*. Sebastopol, CA, USA: O'Reilly Media, Inc., Apr. 2022. ISBN: 978-1-0981-0894-6 (siehe S. 337).

[140] Matthew Helmke. *Ubuntu Linux Unleashed 2021 Edition*. 14. Aufl. Reading, MA, USA: Addison-Wesley Professional, Aug. 2020. ISBN: 978-0-13-668539-5 (siehe S. 337, 345).

References XIX

[141] Keld Helsgaun. "General k-opt Submoves for the Lin–Kernighan TSP Heuristic". *Mathematical Programming Computation* 1(2-3):119–163, Juli 2009. London, England, UK: Springer Nature Limited. ISSN: 1867-2949. doi:10.1007/s12532-009-0004-6. URL: https://www2.seas.gwu.edu/~simhaweb/champalg/tsp/papers/Helsgaun2.pdf (besucht am 2026-09-26) (siehe S. 338).

[142] Keld Helsgaun. "An Effective Implementation of the Lin-Kernighan Traveling Salesman Heuristic". 126(1):106–130, Okt. 2000. doi:10.1016/S0377-2217(99)00284-2. URL: http://akira.ruc.dk/~keld/research/LKH/LKH-2.0/DOC/LKH_REPORT.pdf (besucht am 2026-09-26) (siehe S. 338).

[143] André Henning. "Praktische Job-Shop Scheduling-Probleme". Diss. Jena, Thüringen, Germany: Friedrich-Schiller-Universität Jena, Aug. 2002. URL: http://www.db-thueringen.de/servlets/DocumentServlet?id=873 (besucht am 2026-09-30). Alternate url: https://nbn-resolving.org/urn:nbn:de:gbv:27-20060809-115700-4. Applies MA-like algorithms to the JSSP. (Siehe S. 336, 338).

[144] Raymond Hettinger. *Binary Floating Point Summation Accurate to Full Precision (Python Recipe)*. Vancouver, BC, Canada: ActiveState Software Inc., 28. März 2005. URL: http://code.activestate.com/recipes/393090 (besucht am 2024-11-19) (siehe S. 212–234).

[145] Steve Hollasch. "IEEE Standard 754 Floating Point Numbers". In: *CSE401: Introduction to Compiler Construction*. Seattle, WA, USA: University of Washington, 8. Jan. 1997. URL: https://courses.cs.washington.edu/courses/cse401/01au/details/fp.html (besucht am 2024-07-05) (siehe S. 30–41, 342).

[146] Holger H. Hoos und Thomas Stützle. *Stochastic Local Search: Foundations & Applications*. The Morgan Kaufmann Series in Artificial Intelligence. Amsterdam, The Netherlands: Elsevier B.V., 2005. ISBN: 978-1-55860-872-6. doi:10.1016/B978-1-55860-872-6.X5016-1 (siehe S. 338, 343).

[147] Holger H. Hoos und Thomas Stützle. *SATLIB – The Satisfiability Library*. 1.4.4. Vancouver, BC, Canada: The University of British Columbia (UBC), 11. Mai 2001. URL: https://www.cs.ubc.ca/~hoos/SATLIB/index-ubc.html (besucht am 2026-09-20). See paper[148] (siehe S. 301, 342).

[148] Holger H. Hoos und Thomas Stützle. "SATLIB: An Online Resource for Research on SAT". In: *SAT2000 – Highlights of Satisfiability Research in the Year 2000*. Hrsg. von Ian Philip Gent, Toby Walsh und Hans van Maaren. Bd. 63 der Reihe Frontiers in Artificial Intelligence and Applications. Amsterdam, The Netherlands: IOS Press BV, Jan. 2000, S. 283–292. ISSN: 0922-6389. ISBN: 978-1-58603-061-2. See the website[147] (siehe S. 301, 342).

References XX

[149] John Hunt. *A Beginners Guide to Python 3 Programming*. 2. Aufl. Undergraduate Topics in Computer Science (UTICS). Cham, Switzerland: Springer, 2023. ISSN: 1863-7310. ISBN: 978-3-031-35121-1. doi:10.1007/978-3-031-35122-8 (siehe S. 340).

[150] *IEEE Standard for Floating-Point Arithmetic*. IEEE Std 754™-2019 (Revision of IEEE Std 754-2008). New York, NY, USA: Institute of Electrical and Electronics Engineers (IEEE), 13. Juni 2019 (siehe S. 30–41, 342).

[151] *Information Technology – Database Languages – SQL – Part 1: Framework (SQL/Framework), Part 1*. International Standard ISO/IEC 9075-1:2023(E), Sixth Edition, (ANSI X3.135). Geneva, Switzerland: International Organization for Standardization (ISO) und International Electrotechnical Commission (IEC), Juni 2023. URL: https://standards.iso.org/ittf/PubliclyAvailableStandards/ISO_IEC_9075-1_2023_ed_6_-_id_76583_Publication_PDF_(en).zip (besucht am 2025-01-08). Consists of several parts, see https://modern-sql.com/standard for information where to obtain them. (Siehe S. 343).

[152] *Information Technology – Universal Coded Character Set (UCS)*. International Standard ISO/IEC 10646:2020. Geneva, Switzerland: International Organization for Standardization (ISO) und International Electrotechnical Commission (IEC), Dez. 2020 (siehe S. 326, 346).

[153] Manuel Iori, Vinícius Loti de Lima, Silvano Martello, Flávio K. Miyazaw und Michele Monaci. "Exact Solution Techniques for Two-Dimensional Cutting and Packing". *European Journal of Operational Research* 289(2):399–415, März 2021. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/j.ejor.2020.06.050. See also arXiv:2004.12619v1 [math.OC] 27 Apr 2020 at doi:10.1016/j.ejor.2020.06.050. (Siehe S. 330).

[154] Christoph Jabs, Dieter Vandesande, Jeremias Berg, Johannes Fichte, George Katsirelos, Ruben Martins, André Schidler und Dominik Schreiber. *MaxSAT Evaluation: Complete Benchmark Collection*. 2026. Aufl. Geneva, Switzerland: zenodo (hosted by European Organization for Nuclear Research (CERN)), 9. Juli 2026. doi:10.5281/zenodo.21283148. All versions of this dataset can be reached directly via doi:10.5281/zenodo.21283147. See also https://github.com/maxsat-evaluations/data and https://maxsat-evaluations.github.io. (Siehe S. 339).

References XXI

[155] Dean Jacobs, Jan Prins, Peter Siegel und Kenneth Wilson. "Monte Carlo Techniques in Code Optimization". In: *15th Annual Workshop on Microprogramming (MICRO 15)*. 5.–7. Okt. 1982. Hrsg. von Joseph Allen Fisher, William J. Tracz und Bill Hopkins. Palo Alto, CA, USA: Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE) und New York, NY, USA: Association for Computing Machinery (ACM), Okt. 1982, S. 143–148. doi:10.5555/800036.800944. See [156] (siehe S. 303).

[156] Dean Jacobs, Jan Prins, Peter Siegel und Kenneth Wilson. "Monte Carlo Techniques in Code Optimization". *ACM SIGMICRO Newsletter* 13(4):143–148, Dez. 1982. New York, NY, USA: Association for Computing Machinery (ACM). ISSN: 1050-916X. doi:10.1145/1014194.800944. See [155] (siehe S. 303, 342).

[157] Stefan Jakobs. "On Genetic Algorithms for the Packing of Polygons". *European Journal of Operational Research* 88(1):165–181, Jan. 1996. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/0377-2217(94)00166-9 (siehe S. 335).

[158] Wassim Jaziri, Hrsg. *Tabu Search*. London, England, UK: IntechOpen Limited, 1. Aug. 2008. ISBN: 978-3-902613-34-9. doi:10.5772/65. URL: https://www.intechopen.com/books/6114 (besucht am 2026-09-30) (siehe S. 344).

[159] David Stifler Johnson, Gregory Z. Gutin, Lyle Andrew McGeoch, Anders Yeo, Weixiong Zhang und Alexei Zverovitch. *Experimental Analysis of Heuristics for the ATSP*. Hrsg. von Gregory Z. Gutin und Abraham P. Punnen. New York, NY, USA, Mai 2002. doi:10.1007/0-306-48213-4_10. URL: https://archive.dimacs.rutgers.edu/Challenges/TSP/papers/atspchap.pdf (besucht am 2026-09-25) (siehe S. 344).

[160] David Stifler Johnson und Lyle Andrew McGeoch. *Experimental Analysis of Heuristics for the STSP*. Hrsg. von Gregory Z. Gutin und Abraham P. Punnen. New York, NY, USA, Mai 2002. doi:10.1007/0-306-48213-4_9. URL: https://archive.dimacs.rutgers.edu/Challenges/TSP/papers/stspchap.pdf (besucht am 2026-09-25) (siehe S. 344).

[161] David Stifler Johnson, Lyle Andrew McGeoch, Fred Glover und Cesar Rego. *8th DIMACS Implementation Challenge: The Traveling Salesman Problem*. 1. Juni 2000–19. Nov. 2013. URL: https://archive.dimacs.rutgers.edu/Challenges/TSP (besucht am 2026-09-25) (siehe S. 344).

[162] Selmer Martin Johnson. "Optimal Two- and Three-Stage Production Schedules with Setup Times Included". *Naval Research Logistics Quarterly* I(1):61–68, März 1954. Chichester, West Sussex, England, UK: John Wiley and Sons Ltd. ISSN: 0894-069X. doi:10.1002/nav.3800010110. URL: https://www.rand.org/content/dam/rand/pubs/papers/2008/P402.pdf (besucht am 2023-12-06) (siehe S. 333).

References XXII

[163] Arthur Jones, Kenneth R. Pearson und Sidney A. Morris. "Transcendence of e and π". In: *Abstract Algebra and Famous Impossibilities*. Universitext (UTX). New York, NY, USA: Springer New York, 1991. Kap. 9, S. 115–161. ISSN: 0172-5939. ISBN: 978-1-4419-8552-1. doi:10.1007/978-1-4419-8552-1_8 (siehe S. 347, 348).

[164] William Kahan. "Pracniques: Further Remarks on Reducing Truncation Errors". *Communications of the ACM (CACM)* 8(1):40, Jan. 1965. New York, NY, USA: Association for Computing Machinery (ACM). ISSN: 0001-0782. doi:10.1145/363707.363723. URL: https://www.convexoptimization.com/TOOLS/Kahan.pdf (besucht am 2024-11-18) (siehe S. 65–69, 83–118, 121–123, 178–200, 212–235, 304).

[165] Shen Kangshen, John Newsome Crossley und Anthony W.-C. Lun. *The Nine Chapters on the Mathematical Art: Companion and Commentary*. Oxford, Oxfordshire, England, UK: Oxford University Press, 7. Okt. 1999. ISBN: 978-0-19-853936-0. doi:10.1093/oso/9780198539360.001.0001. Translation of the historical Chinese mathematics book *Jiu Zhang Suanshu* (九章算术), including the commentaries of LIU Hui (刘徽). (Siehe S. 347).

[166] Scott Kirkpatrick, C. Daniel Gelatt, Jr. und Mario P. Vecchi. "Optimization by Simulated Annealing". *Science Magazine* 220(4598):671–680, 13. Mai 1983. Washington, D.C., USA: American Association for the Advancement of Science (AAAS). ISSN: 0036-8075. doi:10.1126/science.220.4598.671. URL: https://www.researchgate.net/publication/6026283 (besucht am 2025-08-08) (siehe S. 342).

[167] Andreas Klein. "A Generalized Kahan-Babuška-Summation-Algorithm". *Computing* 76(3-4):279–293, Jan. 2006. Berlin/Heidelberg, Germany: Springer-Verlag GmbH Germany. ISSN: 0010-485X. doi:10.1007/s00607-005-0139-x. Based on[10,164,224] (siehe S. 104–107).

[168] Robert Klein. *Scheduling of Resource-Constrained Projects*. Bd. 10 der Reihe Operations Research/Computer Science Interfaces Series (ORCS). New York, NY, USA: Springer New York, 2000. ISSN: 1387-666X. ISBN: 978-0-7923-8637-7. doi:10.1007/978-1-4615-4629-0 (siehe S. 334).

[169] Donald Ervin Knuth. "Big Omicron and Big Omega and Big Theta". *ACM SIGACT News* 8(2):18–24, Apr.–Juni 1976. New York, NY, USA: Association for Computing Machinery (ACM). ISSN: 0163-5700. doi:10.1145/1008328.1008329 (siehe S. 350).

[170] Donald Ervin Knuth. *Fundamental Algorithms*. 3. Aufl. Bd. 1 der Reihe The Art of Computer Programming. Reading, MA, USA: Addison-Wesley Professional, 1997. ISBN: 978-0-201-89683-1 (siehe S. 350).

References XXIII

[171] Donald Ervin Knuth. *Seminumerical Algorithms*. Bd. 2 der Reihe The Art of Computer Programming. Reading, MA, USA: Addison-Wesley Professional, Nov. 1997. ISBN: 978-0-201-89684-8 (siehe S. 325, 351).

[172] Donald Ervin Knuth. *Sorting and Searching*. Bd. 3 der Reihe The Art of Computer Programming. Reading, MA, USA: Addison-Wesley Professional, 1998. ISBN: 978-0-201-89685-5 (siehe S. 349).

[173] Tjalling Charles Koopmans und Martin Beckmann. "Assignment Problems and the Location of Economic Activities". *Econometrica* 25(1):53–76, 1957. New Haven, CT, USA: The Econometric Society and Chichester, West Sussex, England, UK: John Wiley and Sons Ltd. ISSN: 0012-9682. doi:10.2307/1907742 (siehe S. 341).

[174] Jakob Krarup und Peter Mark Pruzan. "Computer-aided Layout Design". In: *Mathematical Programming in Use*. Hrsg. von Michel Louis Balinski und Claude Lemaréchal. Bd. 9 der Reihe. Berlin/Heidelberg, Germany: Springer-Verlag GmbH Germany, 1978. Kap. 6, S. 75–94. ISSN: 0303-3929. ISBN: 978-3-642-00796-5. doi:10.1007/BFb0120827. Uses QAPs as model for designing architectural layouts. (Siehe S. 341).

[175] Holger Krekel und pytest-Dev Team. "How to Run Doctests". In: *pytest Documentation*. Release 8.4. Freiburg, Baden-Württemberg, Germany: merlinux GmbH. Kap. 2.8, S. 65–69. URL: https://docs.pytest.org/en/stable/how-to/doctest.html (besucht am 2024-11-07) (siehe S. 333).

[176] Holger Krekel und pytest-Dev Team. *pytest Documentation*. Release 8.4. Freiburg, Baden-Württemberg, Germany: merlinux GmbH. URL: https://readthedocs.org/projects/pytest/downloads/pdf/latest (besucht am 2024-11-07) (siehe S. 340).

[177] Mark W. Krentel. "The Complexity of Optimization Problems". *Journal of Computer and System Sciences* 36(3):490–509, Juni 1988. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0022-0000. Explores the hardness (optimization) problems such TSP, Clique, and Coloring, where computing the optimal value is $\mathcal{NP}$-complete. (Siehe S. 344).

[178] Jay LaCroix. *Mastering Ubuntu Server*. 4. Aufl. Birmingham, England, UK: Packt Publishing Ltd, Sep. 2022. ISBN: 978-1-80323-424-3 (siehe S. 342).

References XXIV

[179] Vincent Lafage. *Revisiting "What Every Computer Scientist Should Know About Floating-Point Arithmetic"*. arXiv.org: Computing Research Repository (CoRR) abs/2012.02492. Ithaca, NY, USA: Cornell Universiy Library, 4. Dez. 2020. doi:10.48550/arXiv.2012.02492. URL: https://arxiv.org/abs/2012.02492 (besucht am 2025-09-03). arXiv:2012.02492v1 [math.NA] 4 Dec 2020, see also[125] (siehe S. 65–69).

[180] Edmund Landau. *Handbuch der Lehre von der Verteilung der Primzahlen*. Leipzig, Sachsen, Germany: B. G. Teubner, 1909. ISBN: 978-0-8218-2650-8 (siehe S. 350).

[181] Łukasz Langa. *Literature Overview for Type Hints*. Python Enhancement Proposal (PEP) 482. Beaverton, OR, USA: Python Software Foundation (PSF), 8. Jan. 2015. URL: https://peps.python.org/pep-0482 (besucht am 2024-10-09) (siehe S. 345).

[182] Eugene Leighton Lawler, Jan Karel Lenstra und Alexander Hendrik George Rinnooy Kan. "Recent Developments in Deterministic Sequencing and Scheduling: A Survey". In: *Deterministic and Stochastic Scheduling: Proceedings of the NATO Advanced Study and Research Institute on Theoretical Approaches to Scheduling Problems*. 6.–17. Juni 1981, Durham, England, UK. Hrsg. von Michael Anthony Howe Dempster, Jan Karel Lenstra und Alexander Hendrik George Rinnooy Kan. Bd. 84 der Reihe Nato Science Series C: Mathematical and Physical Science (ASIC). London, England, UK: Springer Nature Limited, Apr. 1982, S. 35–73. ISBN: 978-90-277-1397-1. doi:10.1007/978-94-009-7801-0_3. The JSSP is $\mathcal{NP}$-hard. (Siehe S. 336).

[183] Eugene Leighton Lawler, Jan Karel Lenstra, Alexander Hendrik George Rinnooy Kan und David B. Shmoys. "Sequencing and Scheduling: Algorithms and Complexity". In: *Production Planning and Inventory*. Hrsg. von Stephen C. Graves, Alexander Hendrik George Rinnooy Kan und Paul H. Zipkin. Bd. IV der Reihe Handbooks of Operations Research and Management Science. Amsterdam, The Netherlands: Elsevier B.V., 1993. Kap. 9, S. 445–522. ISSN: 0927-0507. ISBN: 978-0-444-87472-6. doi:10.1016/S0927-0507(05)80189-6. URL: http://alexandria.tue.nl/repository/books/339776.pdf (besucht am 2023-12-06). Discusses, amongst others, that the JSSP is $\mathcal{NP}$-hard. (Siehe S. 336, 350).

[184] Eugene Leighton Lawler, Jan Karel Lenstra, Alexander Hendrik George Rinnooy Kan und David B. Shmoys. *The Traveling Salesman Problem: A Guided Tour of Combinatorial Optimization*. Estimation, Simulation, and Control – Wiley-Interscience Series in Discrete Mathematics and Optimization. Chichester, West Sussex, England, UK: Wiley Interscience, Sep. 1985. ISSN: 0277-2698. ISBN: 978-0-471-90413-7 (siehe S. 344).

References XXV

[185] Stephen R. Lawrence. "Resource Constrained Project Scheduling: An Experimental Investigation of Heuristic Scheduling Techniques". Diss. Pittsburgh, PA, USA: Graduate School of Industrial Administration (GSIA), Carnegie-Mellon University, 1984 (siehe S. 336).

[186] Kent D. Lee und Steve Hubbard. *Data Structures and Algorithms with Python*. Undergraduate Topics in Computer Science (UTICS). Cham, Switzerland: Springer, 2015. ISSN: 1863-7310. ISBN: 978-3-319-13071-2. doi:10.1007/978-3-319-13072-9 (siehe S. 340).

[187] Jukka Lehtosalo, Ivan Levkivskyi, Jared Hance, Ethan Smith, Guido van Rossum, Jelle „JelleZijlstra" Zijlstra, Michael J. Sullivan, Shantanu Jain, Xuanda Yang, Jingchen Ye, Nikita Sobolev und Mypy Contributors. *Mypy – Static Typing for Python*. San Francisco, CA, USA: GitHub Inc, 2024. URL: https://github.com/python/mypy (besucht am 2024-08-17) (siehe S. 339).

[188] Xueping Li und Kaike Zhang. "Single Batch Processing Machine Scheduling with Two-Dimensional Bin Packing Constraints". *International Journal of Production Economics (IJPE)* 196:113–121, Feb. 2018. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0925-5273. doi:10.1016/j.ijpe.2017.11.015. Mentions some scheduling tasks of Additive Manufacturing jobs can be viewed as instance of the 2BP|R|F. (Siehe S. 330).

[189] Tianyu Liang (梁天宇), Zhize Wu (吴志泽), Jörg Lässig, Daan van den Berg, Sarah Louise Thomson und Thomas Weise (汤卫思). "Addressing the Traveling Salesperson Problem with Frequency Fitness Assignment and Hybrid Algorithms". *Soft Computing* 28(17-18):9495–9508, Juli 2024. London, England, UK: Springer Nature Limited. ISSN: 1432-7643. doi:10.1007/S00500-024-09718-8 (siehe S. 333, 339, 344).

[190] Tianyu Liang (梁天宇), Zhize Wu (吴志泽), Jörg Lässig, Daan van den Berg und Thomas Weise (汤卫思). "Solving the Traveling Salesperson Problem using Frequency Fitness Assignment". In: *IEEE Symposium Series on Computational Intelligence (SSCI'2022)*. 4.–7. Dez. 2022, Singapore. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE), 2022. ISBN: 978-1-6654-8769-6. doi:10.1109/SSCI51031.2022.10022296 (siehe S. 333, 339).

[191] Tianyu Liang (梁天宇), Zhize Wu (吴志泽), Matthias Thürer, Markus Wagner und Thomas Weise (汤卫思). "Generating Small Instances with Interesting Features for the Traveling Salesperson Problem". In: *16th International Joint Conference on Computational Intelligence (IJCCI'24)*. 20.–22. Nov. 2024, Porto, Portugal. Hrsg. von Francesco Marcelloni, Kurosh Madani, Niki van Stein und Joaquim Filipe. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2024, S. 173–180. ISSN: 2184-3236. ISBN: 978-989-758-721-4. doi:10.5220/0012888800003837 (siehe S. 339).

References XXVI

[192] Shen Lin und Brian W. Kernighan. "An Effective Heuristic Algorithm for the Traveling-Salesman Problem". *Operations Research* 21(2):498–516, März–Apr. 1973. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 0030-364X. doi:10.1287/opre.21.2.498. URL: https://www.cs.princeton.edu/~bwk/btl.mirror/tsp.pdf (besucht am 2026-09-26) (siehe S. 338).

[193] John D. C. Little, Katta G. Murty, Dura W. Sweeny und Caroline Karel. *An Algorithm for the Traveling Salesman Problem*. Sloan Working Papers 07-63. Massachusetts Institute of Technology (MIT): Massachusetts Institute of Technology (MIT), Sloan School of Management, März 1963. URL: http://dspace.mit.edu/bitstream/handle/1721.1/46828/algorithmfortrav00litt.pdf (besucht am 2026-01-03) (siehe S. 331).

[194] Dequan Liu (刘德全) und Hongfei Teng (滕弘飞). "An Improved BL-Algorithm for EAGenetic Algorithm of the Orthogonal Packing of Rectangles". *European Journal of Operational Research* 112(2):413–420, Jan. 1999. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/S0377-2217(97)00437-2 (siehe S. 335).

[195] Andrea Lodi, Silvano Martello und Michele Monaci. "Two-Dimensional Packing Problems: A Survey". *European Journal of Operational Research* 141(2):241–252, Sep. 2002. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/S0377-2217(02)00123-6 (siehe S. 330, 332).

[196] Andrea Lodi, Silvano Martello und Daniele Vigo. "Heuristic and Metaheuristic Approaches for a Class of Two-Dimensional Bin Packing Problems". *INFORMS Journal on Computing* 11(4):345–357, Herbst 1999. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 1091-9856. doi:10.1287/ijoc.11.4.345 (siehe S. 330).

[197] Andrea Lodi, Silvano Martello und Daniele Vigo. "Recent Advances on Two-Dimensional Bin Packing Problems". *Discrete Applied Mathematics* 123(1–3):379–396, Nov. 2002. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0166-218X. doi:10.1016/S0166-218X(01)00347-X (siehe S. 330, 332).

[198] Eliane Maria Loiola, Nair Maria Maia de Abreu, Paulo Oswaldo Boaventura-Netto, Peter M. Hahn und Tania Querido. "A Survey for the Quadratic Assignment Problem". *European Journal of Operational Research* 176(2):657–690, 2007. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/j.ejor.2005.09.032 (siehe S. 341).

References XXVII

[199] Mariana A. Londe, Luciana S. Pessoa, Carlos E. Andrade und Mauricio G. C. Resende. *Biased Random-Key Genetic Algorithms: A Review*. arXiv.org: Computing Research Repository (CoRR) abs/2312.00961v2. Ithaca, NY, USA: Cornell Universiy Library, 6. Dez. 2023. doi:10.48550/arXiv.2312.00961. URL: https://arxiv.org/abs/2312.00961v2 (besucht am 2024-06-27). arXiv:2312.00961v2 [cs.NE] 06 Dec 2023; See [200] (siehe S. 309).

[200] Mariana A. Londe, Luciana S. Pessoa, Carlos E. Andrade und Mauricio G. C. Resende. "Biased Random-Key Genetic Algorithms: A Review". *European Journal of Operational Research* 321(1):1–22, Feb. 2025. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/j.ejor.2024.03.030. See [199] (siehe S. 309, 332).

[201] Gloria Lotha, Aakanksha Gaur, Erik Gregersen, Swati Chopra und William L. Hosch. "Client-Server Architecture". In: *Encyclopaedia Britannica*. Hrsg. von The Editors of Encyclopaedia Britannica. Chicago, IL, USA: Encyclopædia Britannica, Inc., 3. Jan. 2025. URL: https://www.britannica.com/technology/client-server-architecture (besucht am 2025-01-20) (siehe S. 332).

[202] Marc Loy, Patrick Niemeyer und Daniel Leuck. *Learning Java*. 5. Aufl. Sebastopol, CA, USA: O'Reilly Media, Inc., März 2020. ISBN: 978-1-4920-5627-0 (siehe S. 336).

[203] Peter Luschny. *A New Kind of Factorial Function*. Highland Park, NJ, USA: The OEIS Foundation Inc., 4. Okt. 2015. URL: https://oeis.org/A000142/a000142.pdf (besucht am 2024-09-29) (siehe S. 347).

[204] Mark Lutz. *Learning Python*. 6. Aufl. Sebastopol, CA, USA: O'Reilly Media, Inc., März 2025. ISBN: 978-1-0981-7130-8 (siehe S. 340).

[205] Rita Macedo, Cláudio Alves und J. M. Valério de Carvalho. "Arc-Flow Model for the Two-Dimensional Guillotine Cutting Stock Problem". *Computers & Operations Research* 37(6):991–1001, Juni 2010. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0305-0548. doi:10.1016/j.cor.2009.08.005 (siehe S. 332).

[206] Reinhard Männer und Bernard Manderick, Hrsg. *Parallel Problem Solving from Nature 2 (PPSN-II)*. 28.–30. Sep. 1992, Brussels, Belgium. Amsterdam, The Netherlands: Elsevier B.V., 1992.

[207] Francesco Marcelloni, Kurosh Madani, Niki van Stein und Joaquim Filipe, Hrsg. *16th International Joint Conference on Computational Intelligence (IJCCI'24)*. 20.–22. Nov. 2024, Porto, Portugal. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2024. ISSN: 2184-3236. ISBN: 978-989-758-721-4. doi:10.5220/0000195000003837.

References XXVIII

[208] *MariaDB Server Documentation*. Milpitas, CA, USA: MariaDB, 2025. URL: https://mariadb.com/kb/en/documentation (besucht am 2025-04-24) (siehe S. 338).

[209] Silvano Martello und Daniele Vigo. "Exact Solution of the Two-Dimensional Finite Bin Packing Problem". 44(3):388–399, März 1998. doi:10.1287/mnsc.44.3.388 (siehe S. 330, 331).

[210] "Mathematical Functions and Operators". In: *PostgreSQL Documentation*. 17.4. The PostgreSQL Global Development Group (PGDG), 20. Feb. 2025. Kap. 9.3. URL: https://www.postgresql.org/docs/17/functions-math.html (besucht am 2025-02-27) (siehe S. 347, 348).

[211] Jim Melton und Alan R. Simon. *SQL: 1999 – Understanding Relational Language Components*. The Morgan Kaufmann Series in Data Management Systems. Burlington, MA, USA/San Mateo, CA, USA: Morgan Kaufmann Publishers, Juni 2001. ISSN: 1046-1698. ISBN: 978-1-55860-456-8 (siehe S. 343).

[212] Stephan Mertens. "Exhaustive Search for Low-Autocorrelation Binary Sequences". *Journal of Physics A: Mathematical and General* 29(18):L473, Sep. 1996. Bristol, England, UK: IOP Publishing Ltd. ISSN: 0305-4470. doi:10.1088/0305-4470/29/18/005 (siehe S. 337).

[213] Peter Merz und Bernd Freisleben. "A Comparison of MAMemetic Algorithms, TSTabu Search, and ACOAnt Colonies for the QAPQuadratic Assignment Problem". In: *Congress on Evolutionary Computation (CEC'1999)*. 6.–9. Juli 1999, Washington, D.C., USA. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE), 1999, S. 2063–2070. doi:10.1109/CEC.1999.785529 (siehe S. 341).

[214] Zbigniew „Zbyszek" Michalewicz. *Genetic Algorithms + Data Structures = Evolution Programs*. Berlin/Heidelberg, Germany: Springer-Verlag GmbH Germany, 1996. ISBN: 978-3-540-58090-4. doi:10.1007/978-3-662-03315-9 (siehe S. 334, 342).

[215] Burkhard Militzer, Michele Zamparelli und Dieter Beule. "Evolutionary Search for Low Autocorrelated Binary Sequences". *IEEE Transactions on Evolutionary Computation* 2(1):34–39, Apr. 1998. Los Alamitos, CA, USA: IEEE Computer Society. ISSN: 1089-778X. doi:10.1109/4235.728212 (siehe S. 337).

[216] Alfonsas Misevičius. "A Tabu Search Algorithm for the Quadratic Assignment Problem". *Computational Optimization and Applications* 30:95–111, Jan. 2005. London, England, UK: Springer Nature Limited. ISSN: 0926-6003. doi:10.1007/s10589-005-4562-x (siehe S. 344).

References XXIX

[217] Alfonsas Misevičius. "A Modified Simulated Annealing Algorithm for the Quadratic Assignment Problem". *Informatica* 14(4):497–514, Jan. 2003. Vilnius, Lithuania: Vilnius University, Institute of Data Science and Digital Technologies (DMSTI), Lithuanian Academy of Sciences. ISSN: 0868-4952. doi:10.15388/Informatica.2003.037 (siehe S. 342).

[218] Alfonsas Misevičius. *An Implementation of the Iterated Tabu Search Algorithm for the Quadratic Assignment Problem*. Working Paper. Kaunas, Lithuania: Kaunas University of Technology, 2008. See also[219] (siehe S. 311).

[219] Alfonsas Misevičius. "An Implementation of the Iterated Tabu Search Algorithm for the Quadratic Assignment Problem". *OR Spectrum: Quantitative Approaches in Management. Formerly Operations-Research-Spektrum* 34(3):665–690, Juli 2012. London, England, UK: Springer Nature Limited. ISSN: 0171-6468. doi:10.1007/s00291-011-0274-z. See also[218] (siehe S. 311, 341, 344).

[220] Melanie Mitchell. *An Introduction to Genetic Algorithms*. Complex Adaptive Systems. Cambridge, MA, USA: MIT Press, Feb. 1998. ISBN: 978-0-262-13316-6. URL: http://boente.eti.br/fuzzy/ebook-fuzzy-mitchell.pdf (besucht am 2025-08-08) (siehe S. 334, 342).

[221] Nenad Mladenović und Pierre Hansen. "Variable Neighborhood Search". *Computers & Operations Research* 24(11):1097–1100, Nov. 1997. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0305-0548. doi:10.1016/S0305-0548(97)00031-2 (siehe S. 347).

[222] Wai Ho Mow, Ke-Lin Du und Wei Hsiang Wu. "New Evolutionary Search for Long Low Autocorrelation Binary Sequences". *IEEE Transactions on Aerospace and Electronic Systems* 51(1):290–303, Jan. 2015. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE). ISSN: 0018-9251. doi:10.1109/TAES.2014.130518 (siehe S. 337).

[223] Heinz Mühlenbein. "How Genetic Algorithms Really Work: Mutation and Hillclimbing". In: *Parallel Problem Solving from Nature 2 (PPSN-II)*. 28.–30. Sep. 1992, Brussels, Belgium. Hrsg. von Reinhard Männer und Bernard Manderick. Amsterdam, The Netherlands: Elsevier B.V., 1992, S. 15–26. URL: https://www.researchgate.net/publication/284805798 (besucht am 2025-08-15) (siehe S. 340).

[224] Arnold Neumaier. "Rundungsfehleranalyse einiger Verfahren zur Summation endlicher Summen". *ZAMM – Journal of Applied Mathematics and Mechanics / Zeitschrift für Angewandte Mathematik und Mechanik* 54(1):39–51, 1974. Weinheim, Baden-Württemberg, Germany: Wiley-VCH GmbH. ISSN: 0044-2267. doi:10.1002/zamm.19740540106. URL: https://arnold-neumaier.at/scan/01.pdf (besucht am 2024-11-18) (siehe S. 104–107, 121–123, 178–200, 304).

[225] Cameron Newham und Bill Rosenblatt. *Learning the Bash Shell – Unix Shell Programming: Covers Bash 3.0*. 3. Aufl. Sebastopol, CA, USA: O'Reilly Media, Inc., 2005. ISBN: 978-0-596-00965-6 (siehe S. 331).

References XXX

[226] Siegfried Nijssen und Thomas Bäck. "An Analysis of the Behavior of Simplified Evolutionary Algorithms on Trap Functions". *IEEE Transactions on Evolutionary Computation* 7(1):11–22, Feb. 2003. Los Alamitos, CA, USA: IEEE Computer Society. ISSN: 1089-778X. doi:10.1109/TEVC.2002.806169 (siehe S. 343).

[227] Ivan Niven. "The Transcendence of π". *The American Mathematical Monthly* 46(8):469–471, Okt. 1939. London, England, UK: Taylor and Francis Ltd. ISSN: 1930-0972. doi:10.2307/2302515 (siehe S. 347).

[228] Eugeniusz Nowicki und Czesław Smutnicki. "A Fast Taboo Search Algorithm for the Job Shop Problem". *Management Science* 42(6):783–938, Juni 1996. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 0025-1909. doi:10.1287/mnsc.42.6.797 (siehe S. 344).

[229] John J. O'Connor und Edmund F. Robertson. *Liu Hui*. St Andrews, Scotland, UK: University of St Andrews, School of Mathematics and Statistics, Dez. 2003. URL: https://mathshistory.st-andrews.ac.uk/Biographies/Liu_Hui (besucht am 2024-08-10). Discusses LIU Hui's (刘徽) method of approximating π (siehe S. 347).

[230] Regina O. Obe und Leo S. Hsu. *PostgreSQL: Up and Running*. 3. Aufl. Sebastopol, CA, USA: O'Reilly Media, Inc., Okt. 2017. ISBN: 978-1-4919-6336-4 (siehe S. 340).

[231] A. Jefferson Offutt. "Unit Testing Versus Integration Testing". In: *Test: Faster, Better, Sooner – IEEE International Test Conference (ITC'1991)*. 26.–30. Okt. 1991, Nashville, TN, USA. Los Alamitos, CA, USA: IEEE Computer Society, 1991. Kap. Paper P2.3, S. 1108–1109. ISSN: 1089-3539. ISBN: 978-0-8186-9156-0. doi:10.1109/TEST.1991.519784 (siehe S. 346).

[232] Brian Okken. *Python Testing with pytest*. Flower Mound, TX, USA: Pragmatic Bookshelf by The Pragmatic Programmers, L.L.C., Feb. 2022. ISBN: 978-1-68050-860-4 (siehe S. 340).

[233] Michael Olan. "Unit Testing: Test Early, Test Often". *Journal of Computing Sciences in Colleges (JCSC)* 19(2):319–328, Dez. 2003. New York, NY, USA: Association for Computing Machinery (ACM). ISSN: 1937-4771. doi:10.5555/948785.948830. URL: https://www.researchgate.net/publication/255673967 (besucht am 2025-09-05) (siehe S. 346).

[234] Robert Orfali, Dan Harkey und Jeri Edwards. *Client/Server Survival Guide*. 3. Aufl. Chichester, West Sussex, England, UK: John Wiley and Sons Ltd., 25. Jan. 1999. ISBN: 978-0-471-31615-2 (siehe S. 332).

References XXXI

[235] Tom Packebusch und Stephan Mertens. "Low Autocorrelation Binary Sequences". *Journal of Physics A: Mathematical and Theoretical* 49(16):165001, März 2016. Bristol, England, UK: IOP Publishing Ltd. ISSN: 1751-8121. doi:10.1088/1751-8113/49/16/165001 (siehe S. 337).

[236] Ashwin Pajankar. *Python Unit Test Automation: Automate, Organize, and Execute Unit Tests in Python*. New York, NY, USA: Apress Media, LLC, Dez. 2021. ISBN: 978-1-4842-7854-3 (siehe S. 340, 346).

[237] Panos Miltiades Pardalos, Ding-Zhu Du und Ronald Lewis Graham, Hrsg. *Handbook of Combinatorial Optimization*. 1. Aufl. Boston, MA, USA: Springer, 1998. ISBN: 978-1-4613-7987-4.

[238] Panos Miltiades Pardalos, Oleg V. Shylo und Alkis Vazacopoulos. "Solving Job Shop Scheduling Problems Utilizing the Properties of Backbone and "Big Valley"". *Computational Optimization and Applications* 47(1):61–76, Sep. 2010. London, England, UK: Springer Nature Limited. ISSN: 0926-6003. doi:10.1007/s10589-008-9206-5. An algorithm similar to SA. (Siehe S. 336, 342).

[239] Francisco Parreño, Ramón Alvarez-Valdés, José Fernando Oliveira und José Manuel Tamarit. "A Hybrid GRASP/VND Algorithm for Two- and Three-Dimensional Bin Packing". *Annals of Operations Research* 179(1):203–220, Sep. 2010. London, England, UK: Springer Nature Limited. ISSN: 0254-5330. doi:10.1007/s10479-008-0449-4 (siehe S. 330, 334, 346).

[240] Bo Peng, Zhipeng Lü und T. C. Edwin Cheng. "A Tabu Search/Path Relinking Algorithm to Solve the Job Shop Scheduling Problem". *Computers & Operations Research* 53:154–164, Jan. 2015. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0305-0548. doi:10.1016/j.cor.2014.08.006. URL: http://arxiv.org/abs/1402.5613 (besucht am 2026-09-30) (siehe S. 336, 344).

[241] Yasset Pérez-Riverol, Laurent Gatto, Rui Wang, Timo Sachsenberg, Julian Uszkoreit, Felipe da Veiga Leprevost, Christian Fufezan, Tobias Ternent, Stephen J. Eglen, Daniel S. Katz, Tom J. Pollard, Alexander Konovalov, Robert M. Flight, Kai Blin und Juan Antonio Vizcaíno. "Ten Simple Rules for Taking Advantage of Git and GitHub". *PLOS Computational Biology* 12(7), 14. Juli 2016. San Francisco, CA, USA: Public Library of Science (PLOS). ISSN: 1553-7358. doi:10.1371/JOURNAL.PCBI.1004947 (siehe S. 334).

[242] Ferdinando Pezzella und Emanuela Merelli. "A Tabu Search Method Guided by Shifting Bottleneck for the Job Shop Scheduling Problem". *European Journal of Operational Research* 120(2):297–310, Jan. 2000. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/S0377-2217(99)00158-7. URL: https://www2.cs.sfu.ca/CourseCentral/827/havens/papers/topic%5C%2310(JobShop)/Tabu%5C%20With%5C%20Shifting.pdf (besucht am 2026-09-30) (siehe S. 344).

References XXXII

[243] Martin Pincus. "Letter to the Editor – A Monte Carlo Method for the Approximate Solution of Certain Types of Constrained Optimization Problems". *Operations Research* 18(6):1225–1228, Nov.–Dez. 1970. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 0030-364X. doi:10.1287/opre.18.6.1225 (siehe S. 342).

[244] Marcelo Pinto, Cristóvão Silva, Matthias Thürer und Samuel Moniz. "Survey in Operations Research and Management Science. Nesting and Scheduling Optimization of Additive Manufacturing Systems: Mapping the Territory". *Computers & Operations Research* 165(106592), Feb. 2024. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0305-0548. doi:10.1016/j.cor.2024.106592. Mentions some scheduling tasks of Additive Manufacturing jobs can be viewed as instance of the 2BP|R|F. (Siehe S. 330).

[245] *PostgreSQL Documentation*. 17.4. The PostgreSQL Global Development Group (PGDG), Feb. 2025. URL: https://www.postgresql.org/docs/17/index.html (besucht am 2025-02-25).

[246] *PostgreSQL Essentials: Leveling Up Your Data Work*. Sebastopol, CA, USA: O'Reilly Media, Inc., März 2024 (siehe S. 340).

[247] S. D. Prestwich. *Improved Branch-and-Bound for Low Autocorrelation Binary Sequences*. arXiv.org: Computing Research Repository (CoRR) abs/1305.6187. Ithaca, NY, USA: Cornell Universiy Library, 3. Apr. 2024. doi:10.48550/arXiv.1305.6187. URL: https://arxiv.org/abs/1305.6187 (besucht am 2013-07-23). arXiv:1305.6187v2 [cs.AI] 23 Jul 2013 (siehe S. 337).

[248] "Private Name Mangling". In: *Python 3 Documentation. The Python Language Reference*. Beaverton, OR, USA: Python Software Foundation (PSF), 2001–2025. Kap. 6.2.1.1. URL: https://docs.python.org/3/reference/expressions.html#private-name-mangling (besucht am 2025-09-23) (siehe S. 163–165).

[249] "Private Variables". In: *Python 3 Documentation. The Python Tutorial*. Beaverton, OR, USA: Python Software Foundation (PSF), 2001–2025. Kap. 9.6. URL: https://docs.python.org/3/tutorial/classes.html#private-variables (besucht am 2025-09-23) (siehe S. 163–165).

[250] *Programming Languages – C, Working Document of SC22/WG14*. International Standard ISO/IEC9899:2017 C17 Ballot N2176. Geneva, Switzerland: International Organization for Standardization (ISO) und International Electrotechnical Commission (IEC), Nov. 2017. URL: https://files.lhmouse.com/standards/ISO%20C%20N2176.pdf (besucht am 2024-06-29) (siehe S. 332).

References XXXIII

[251] Sanatan Rai und George Vairaktarakis. "NP-Complete Problems and Proof Methodology". In: *Encyclopedia of Optimization*. Hrsg. von Christodoulos A. Floudas und Panos Miltiades Pardalos. 2. Aufl. Boston, MA, USA: Springer, Sep. 2008, S. 2675–2682. ISBN: 978-0-387-74758-3. doi:10.1007/978-0-387-74759-0_462 (siehe S. 350).

[252] Soraya B. Rana und L. Darrell Whitley. "Genetic Algorithm Behavior in the MAXSAT Domain". In: *5th International Conference on Parallel Problem Solving from Nature (PPSN V)*. 27.–30. Sep. 1998, Amsterdam, The Netherlands. Hrsg. von Ágoston Endre Eiben, Thomas Bäck, Marc Schoenauer und Hans-Paul Schwefel. Lecture Notes in Computer Science (LNCS). 1998, S. 785–794. ISSN: 0302-9743. ISBN: 978-3-540-65078-2. doi:10.1007/BFb0056920 (siehe S. 338).

[253] Abhishek Ratan, Eric Chou, Pradeeban Kathiravelu und Dr. M.O. Faruque Sarker. *Python Network Programming*. Birmingham, England, UK: Packt Publishing Ltd, Jan. 2019. ISBN: 978-1-78883-546-6 (siehe S. 332).

[254] Federico Razzoli. *Mastering MariaDB*. Birmingham, England, UK: Packt Publishing Ltd, Sep. 2014. ISBN: 978-1-78398-154-0 (siehe S. 338).

[255] Mike Reichardt, Michael Gundall und Hans D. Schotten. "Benchmarking the Operation Times of NoSQL and MySQL Databases for Python Clients". In: *47th Annual Conference of the IEEE Industrial Electronics Society (IECON'2021*. 13.–15. Okt. 2021, Toronto, ON, Canada. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE), 2021, S. 1–8. ISSN: 2577-1647. ISBN: 978-1-6654-3554-3. doi:10.1109/IECON48115.2021.9589382 (siehe S. 339).

[256] Gerhard Reinelt. "TSPLIB – A Traveling Salesman Problem Library". *ORSA Journal on Computing* 3(4):376–384, Herbst 1991. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 0899-1499. doi:10.1287/ijoc.3.4.376 (siehe S. 344).

[257] Gerhard Reinelt. *TSPLIB95*. Techn. Ber. Heidelberg, Baden-Württemberg, Germany: Universität Heidelberg, Institut für Angewandte Mathematik, 1995–28. Apr. 2016. URL: https://github.com/shredderzwj/TSPLIB (besucht am 2026-09-25). The original Uniform Resource Locator (URL) of the repository has become defunct in 2026. Therefore, we provide the repository listed as main URL which contains the data. Additionally, you may also check https://github.com/thomasWeise/tspSuite of our own project *TSPSuite*, which contains all the TSPLib instances as well as Java code to load them. (Siehe S. 344).

[258] Mark Richards und Neal Ford. *Fundamentals of Software Architecture: An Engineering Approach*. Sebastopol, CA, USA: O'Reilly Media, Inc., Jan. 2020. ISBN: 978-1-4920-4345-4 (siehe S. 332).

References XXXIV

[259] Kristian Rother. *Pro Python Best Practices: Debugging, Testing and Maintenance*. New York, NY, USA: Apress Media, LLC, März 2017. ISBN: 978-1-4842-2241-6 (siehe S. 333).

[260] Günter Rudolph. *Convergence Properties of Evolutionary Algorithms*. Bd. 35 der Reihe Forschungsergebnisse zur Informatik. Hamburg, Germany: Verlag Dr. Kovač, 1997. ISSN: 1435-6260. ISBN: 978-3-86064-554-3 (siehe S. 337).

[261] Per Runeson. "A Survey of Unit Testing Practices". *IEEE Software* 23(4):22–29, Juli–Aug. 2006. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE). ISSN: 0740-7459. doi:10.1109/MS.2006.91 (siehe S. 346).

[262] Stuart J. Russell und Peter Norvig. *Artificial Intelligence: A Modern Approach (AIMA)*. 4. Aufl. Hoboken, NJ, USA: Pearson Education, Inc. ISBN: 978-1-292-40113-3. URL: https://aima.cs.berkeley.edu (besucht am 2024-06-27) (siehe S. 330).

[263] Sartaj Sahni und Teofilo Francisco Gonzalez Arce. "P-complete Approximation Problems". *Journal of the ACM (JACM)* 23(3):555–565, Juli 1976. New York, NY, USA: Association for Computing Machinery (ACM). ISSN: 0004-5411. doi:10.1145/321958.321975. URL: https://sites.cs.ucsb.edu/~teo/papers/JACM-PCom.pdf (besucht am 2026-02-21) (siehe S. 341).

[264] Guntram Scheithauer. *Introduction to Cutting and Packing Optimization: Problems, Modeling Approaches, Solution Methods*. Bd. 263 der Reihe International Series in Operations Research & Management Science (ISOR). Cham, Switzerland: Springer, 2018. ISSN: 0884-8289. ISBN: 978-3-319-64402-8. doi:10.1007/978-3-319-64403-5 (siehe S. 332).

[265] Syamal K. Sen und Ravi P. Agarwal. "Existence of year zero in astronomical counting is advantageous and preserves compatibility with significance of AD, BC, CE, and BCE". In: *Zero – A Landmark Discovery, the Dreadful Void, and the Ultimate Mind*. Amsterdam, The Netherlands: Elsevier B.V., 2016. Kap. 5.5, S. 94–95. ISBN: 978-0-08-100774-7. doi:10.1016/C2015-0-02299-7 (siehe S. 331, 332).

[266] Shai Shalev-Shwartz und Shai Ben-David. *Understanding Machine Learning: From Theory to Algorithms*. Cambridge, England, UK: Cambridge University Press (CUP), Juli 2014. ISBN: 978-1-107-05713-5. URL: http://www.cs.huji.ac.il/~shais/UnderstandingMachineLearning (besucht am 2024-06-27) (siehe S. 339).

References XXXV

[267] Jonathan Richard Shewchuk. "Adaptive Precision Floating-Point Arithmetic and Fast Robust Geometric Predicates". *Discrete & Computational Geometry* 18(3):305–363, Okt. 1997. London, England, UK: Springer Nature Limited. ISSN: 0179-5376. doi:10.1007/PL00009321. URL: https://people.eecs.berkeley.edu/~jrs/papers/robustr.pdf (besucht am 2024-11-19). Discusses LIU Hui's (刘徽) comments on the *Jiu Zhang Suanshu* (九章算术), including an algorithm for computing the square root and one for approximating π (siehe S. 212–234).

[268] Guoyong Shi, Hitoshi Iima und Nobuo Sannomiya. "New Encoding Scheme for Solving Job Shop Problems by Genetic Algorithm". In: *35th IEEE Conference on Decision and Control (CDC'96)*. Bd. 4. 11.–13. Dez. 1996. Kōbe, Hyōgo-ken, Japan: Institute of Electrical and Electronics Engineers (IEEE), 1997, S. 4395–4400. ISBN: 978-0-7803-3590-5. doi:10.1109/CDC.1996.577484. URL: https://www.researchgate.net/publication/224238934 (besucht am 2026-09-30). Proposes a permutations-with-repetitions based encoding for the JSSP. (Siehe S. 299, 336, 348).

[269] Oleg V. Shylo. *Job Shop Scheduling (Personal Homepage)*. Knoxville, TN, USA: Oleg V. Shylo, Aug. 2019. URL: http://optimizizer.com/jobshop.php (besucht am 2026-09-30). Contains many JSSP benchmark instances. (Siehe S. 317, 336).

[270] Oleg V. Shylo und Hesam Shams. *Boosting Binary Optimization via Binary Classification: A Case Study of Job Shop Scheduling*. arXiv.org: Computing Research Repository (CoRR) abs/1808.10813. Proposes a TS using regression to adapt the tabu tenures for each variable of the solution vector of a JSSP separately. Many results are available in the GitHub repository https://github.com/quasiquasar/gta-jobshop-data; other results are published Shylo's website http://optimizizer.com/jobshop.php. Ithaca, NY, USA: Cornell Universiy Library, 31. Aug. 2018. doi:10.48550/arXiv.1808.10813. URL: https://arxiv.org/abs/1808.10813 (besucht am 2026-09-30) (siehe S. 336, 344).

[271] Ellen Siever, Stephen Figgins, Robert Love und Arnold Robbins. *Linux in a Nutshell*. 6. Aufl. Sebastopol, CA, USA: O'Reilly Media, Inc., Sep. 2009. ISBN: 978-0-596-15448-6 (siehe S. 337).

[272] Anna Skoulikari. *Learning Git*. Sebastopol, CA, USA: O'Reilly Media, Inc., Mai 2023. ISBN: 978-1-0981-3391-7 (siehe S. 334).

[273] Neil James Alexander Sloane. *Decimal Expansion of Golden Ratio phi (or tau) = (1 + sqrt(5))/2*. Hrsg. von John Horton Conway. Bd. A001622 der Reihe The On-Line Encyclopedia of Integer Sequences. Highland Park, NJ, USA: The OEIS Foundation Inc., 13. Dez. 2024. URL: https://oeis.org/A001622 (besucht am 2024-12-14) (siehe S. 347).

References XXXVI

[274] John Miles Smith und Philip Yen-Tang Chang. "Optimizing the Performance of a Relational Algebra Database Interface". *Communications of the ACM (CACM)* 18(10):568–579, Okt. 1975. New York, NY, USA: Association for Computing Machinery (ACM). ISSN: 0001-0782. doi:10.1145/361020.361025 (siehe S. 341).

[275] Hossein M. Soroush. "Scheduling in Stochastic Bicriteria Single Machine Systems with Set-up Times". *International Journal of Planning and Scheduling* 1(1–2):109–145, Jan. 2011. Geneva, Switzerland: Inderscience Publishers. ISSN: 2044-494X. doi:10.1504/IJPS.2011.044605. A scheduling problem is phrased as QAP. (Siehe S. 341).

[276] "SQL Commands". In: *PostgreSQL Documentation*. 17.4. The PostgreSQL Global Development Group (PGDG), 20. Feb. 2025. Kap. Part VI. Reference. URL: https://www.postgresql.org/docs/17/sql-commands.html (besucht am 2025-02-25) (siehe S. 343).

[277] Leon Steinberg. "The Backboard Wiring Problem: A Placement Algorithm". *SIAM Review* 3(1):37–50, Jan. 1961. Philadelphia, PA, USA: Society for Industrial and Applied Mathematics Publications (SIAM). ISSN: 0036-1445. doi:10.1137/1003003. The problem is modeled as a QAP (siehe S. 341).

[278] Ryan K. Stephens und Ronald R. Plew. *Sams Teach Yourself SQL in 21 Days*. 4. Aufl. Sams Teach Yourself. Indianapolis, IN, USA: SAMS Technical Publishing und Hoboken, NJ, USA: Pearson Education, Inc., Okt. 2002. ISBN: 978-0-672-32451-2 (siehe S. 318, 343).

[279] Ryan K. Stephens, Ronald R. Plew, Bryan Morgan und Jeff Perkins. *SQL in 21 Tagen. Die Datenbank-Abfragesprache SQL vollständig erklärt (in 14/21 Tagen)*. 6. Aufl. Burgthann, Bayern, Germany: Markt+Technik Verlag GmbH, Feb. 1998. ISBN: 978-3-8272-2020-2. Translation of [278] (siehe S. 343).

[280] Robert H. Storer, S. David Wu und Renzo Vaccari. "New Search Spaces for Sequencing Problems with Application to Job Shop Scheduling". *Management Science* 38(10):1495–1509, Okt. 1992. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 0025-1909. doi:10.1287/mnsc.38.10.1495 (siehe S. 336).

[281] Philip D. Straffin Jr. "Liu Hui and the First Golden Age of Chinese Mathematics". *Mathematics Magazine* 71(3):163–181, Juni 1998. London, England, UK: Taylor and Francis Ltd. ISSN: 0025-570X. doi:10.2307/2691200. URL: https://www.researchgate.net/publication/237334342 (besucht am 2024-08-10) (siehe S. 347).

References XXXVII

[282] Thomas Strecker und Leonhard Hennig. "Automatic Layouting of Personalized Newspaper Pages". In: *Operations Research Proceedings 2008: Selected Papers of the Annual International Conference of the German Operations Research Society (GOR)*. 3.–5. Sep. 2008, Augsburg, Bayern, Germany: University of Augsburg. Hrsg. von Bernhard Fleischmann, Karl-Heinz Borgwardt, Robert Klein und Axel Tuma. Bd. 2008 der Reihe Operations Research Proceedings (ORP), Gesellschaft für Operations Research e.V., (GOR). Berlin/Heidelberg, Germany: Springer-Verlag GmbH Germany, 2009, S. 469–474. ISSN: 0721-5924. doi:10.1007/978-3-642-00142-0_76 (siehe S. 330).

[283] Thomas Stützle. "Iterated Local Search for the Quadratic Assignment Problem". *European Journal of Operational Research* 174(3):1519–1539, Nov. 2006. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/j.ejor.2005.01.066 (siehe S. 335, 341).

[284] Dirk Sudholt. "Crossover Speeds up Building-Block Assembly". In: *14th Genetic and Evolutionary Computation Conference (GECCO'2012)*. 7.–11. Juli 2012, Philadelphia, PA, USA. Hrsg. von Terence Soule und Jason Hall Moore. New York, NY, USA: Association for Computing Machinery (ACM), 2012, S. 689–702. ISBN: 978-1-4503-1177-9. doi:10.1145/2330163.2330260. This paper is on the Greedy $(2+1)$ Genetic Algorithm (Greedy $(2+1)$ GA), see also[285] (siehe S. 319, 334).

[285] Dirk Sudholt. "How Crossover Speeds up Building Block Assembly in Genetic Algorithms". *Evolutionary Computation* 25(2):237–274, 2017. Cambridge, MA, USA: MIT Press. ISSN: 1063-6560. doi:10.1162/EVCO_a_00171. This paper is on the Greedy $(2+1)$ GA, see also[284] (siehe S. 319, 334).

[286] Éric D. Taillard. "Benchmarks for Basic Scheduling Problems". *European Journal of Operational Research* 64(2):278–285, Jan. 1993. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/0377-2217(93)90182-M. URL: https://arodes.hes-so.ch/record/8224/files/Preprint.pdf (besucht am 2026-02-20) (siehe S. 336).

[287] Éric D. Taillard. "Comparison of Iterative Searches for the Quadratic Assignment Problem". *Location Science. Now incorporated into Computers & Operations Research* 3(2):87–105, Aug. 1995. Amsterdam, The Netherlands: Elsevier B.V. doi:10.1016/0966-8349(95)00008-6 (siehe S. 344).

[288] Éric D. Taillard. "Robust Taboo Search for the Quadratic Assignment Problem". *Parallel Computing* 17(4–5):443–455, Juli 1991. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0167-8191. doi:10.1016/S0167-8191(05)80147-4 (siehe S. 297, 320, 341, 344).

References XXXVIII

[289] Éric D. Taillard und Luca M. Gambardella. *Adaptive Memories for the Quadratic Assignment Problem*. Techn. Ber. IDSIA-87-97. Lugano, Switzerland: Istituto Dalle Molle Di Studi Sull Intelligenza Artificiale (IDSIA), 10. Sep. 1997. URL: https://mistic.iict-heig-vd.ch/taillard/articles.dir/TaillardGambardella1997.pdf (besucht am 2026-09-28). A MA using TS from [288] as well as a ACO-based approach refining its solutions with a very simple SLS are presented. (Siehe S. 341).

[290] Allen Taylor. *Introducing SQL and Relational Databases*. New York, NY, USA: Apress Media, LLC, Sep. 2018. ISBN: 978-1-4842-3841-7 (siehe S. 341, 343).

[291] Alkin Tezuysal und Ibrar Ahmed. *Database Design and Modeling with PostgreSQL and MySQL*. Birmingham, England, UK: Packt Publishing Ltd, Juli 2024. ISBN: 978-1-80323-347-5 (siehe S. 339, 340).

[292] The Editors of Encyclopaedia Britannica, Hrsg. *Encyclopaedia Britannica*. Chicago, IL, USA: Encyclopædia Britannica, Inc.

[293] *Python 3 Documentation. The Python Tutorial*. Beaverton, OR, USA: Python Software Foundation (PSF), 2001–2025. URL: https://docs.python.org/3/tutorial (besucht am 2025-04-26).

[294] *The Unicode Standard, Version 15.1: Archived Code Charts*. South San Francisco, CA, USA: The Unicode Consortium, 25. Aug. 2023. URL: https://www.unicode.org/Public/15.1.0/charts/CodeCharts.pdf (besucht am 2024-07-26) (siehe S. 346).

[295] Dirk Thierens, David Edward Goldberg und Angela Guimarães Pereira. "Domino Convergence, Drift, and the Temporal-Salience Structure of Problems". In: S. 535–540. doi:10.1109/ICEC.1998.700085 (siehe S. 329).

[296] George K. Thiruvathukal, Konstantin Läufer und Benjamin Gonzalez. "Unit Testing Considered Useful". *Computing in Science & Engineering* 8(6):76–87, Nov.–Dez. 2006. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE). ISSN: 1521-9615. doi:10.1109/MCSE.2006.124. URL: https://www.researchgate.net/publication/220094077 (besucht am 2024-10-01) (siehe S. 346).

References XXXIX

[297] Sarah Louise Thomson, Gabriela Ochoa, Daan van den Berg, Tianyu Liang (梁天宇) und Thomas Weise (汤卫思). "Entropy, Search Trajectories, and Explainability for Frequency Fitness Assignment". In: *Parallel Problem Solving from Nature (PPSN XVIII)*. Bd. 1. 14.–18. Sep. 2024, Hagenberg, Mühlkreis, Austria. Hrsg. von Michael Affenzeller, Stephan M. Winkler, Anna V. Kononova, Heike Trautmann, Tea Tušar, Penousal Machado und Thomas Bäck. Bd. 15148 der Reihe Lecture Notes in Computer Science (LNCS). Cham, Switzerland: Springer. ISSN: 0302-9743. ISBN: 978-3-031-70054-5. doi:10.1007/978-3-031-70055-2_23. Explores randomized local search (RLS), RLS with FFA (FRLS), and SA on the QAP. (Siehe S. 333, 339).

[298] Ulrich Wilhelm Thonemann und Andreas Bölte. *An Improved Simulated Annealing Algorithm for the Quadratic Assignment Problem*. Working Paper. Paderborn, Nordrhein-Westfalen, Germany: School of Business, Department of Production and Operations Research, University of Paderborn, 1994 (siehe S. 342).

[299] Marco Tomassini. "Complex Networks Analysis of the Energy Landscape of the Low Autocorrelation Binary Sequences Problem". *Physica A: Statistical Mechanics and its Applications* 577:126089, 1. Sep. 2021. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0378-4371. doi:10.1016/j.physa.2021.126089. Lists the LABS as $\mathcal{NP}$-hard. (Siehe S. 337).

[300] Linus Torvalds. "The Linux Edge". *Communications of the ACM (CACM)* 42(4):38–39, Apr. 1999. New York, NY, USA: Association for Computing Machinery (ACM). ISSN: 0001-0782. doi:10.1145/299157.299165 (siehe S. 337).

[301] Mariot Tsitoara. *Beginning Git and GitHub: Version Control, Project Management and Teamwork for the New Developer*. New York, NY, USA: Apress Media, LLC, März 2024. ISBN: 979-8-8688-0215-7 (siehe S. 334, 346).

[302] *Unicode® 15.1.0*. South San Francisco, CA, USA: The Unicode Consortium, 12. Sep. 2023. ISBN: 978-1-936213-33-7. URL: https://www.unicode.org/versions/Unicode15.1.0 (besucht am 2024-07-26) (siehe S. 346).

[303] Daan van den Berg, Florian Braam, Mark Moes, Emiel Suilen und Sandjai Bhulai. "Almost Squares in Almost Squares: Solving the Final Instance". In: *5th International Conference on Data Analytics (DATA ANALYTICS'2016)*. 9.–13. Okt. 2016, Venice, Venezia, Veneto, Italy. Hrsg. von Sandjai Bhulai und Ivana Semanjski. Wilmington, DE, USA: International Academy, Research, and Industry Association (IARIA), 2016, S. 69–74. ISSN: 2308-4464. ISBN: 978-1-61208-510-4. URL: https://math.vu.nl/~sbhulai/publications/data_analytics2016b.pdf (besucht am 2026-10-01). This poses a very interesting 2BP|R|F problem, where „almost squars," i.e., rectangles with sizes $(i, i+1)$, must be placed in an almost square. (Siehe S. 330, 331).

References XL

[304] Clarissa Van Hoyweghen, David Edward Goldberg und Bart Naudts. "From Twomax To The Ising Model: Easy And Hard Symmetrical Problems". In: *4th Genetic and Evolutionary Computation Conference (GECCO'2002)*. 9.–13. Juli 2002, New York, NY, USA. Hrsg. von William B. Langdon, Erick Cantú-Paz, Keith E. Mathias, Rajkumar Roy, David Davis, Riccardo Poli, Karthik Balakrishnan, Vasant G. Honavar, Günter Rudolph, Joachim Wegener, Larry Bull, Mitchell A. Potter, Alan C. Schultz, Julian F. Miller, Edmund K. Burke und Natasa Jonoska. Burlington, MA, USA/San Mateo, CA, USA: Morgan Kaufmann Publishers, 2002, S. 626–633. ISBN: 978-1-55860-878-8 (siehe S. 345).

[305] Guido van Rossum und Łukasz Langa. *Type Hints*. Python Enhancement Proposal (PEP) 484. Beaverton, OR, USA: Python Software Foundation (PSF), 29. Sep. 2014. URL: https://peps.python.org/pep-0484 (besucht am 2024-08-22) (siehe S. 345).

[306] Guido van Rossum, Barry Warsaw und Alyssa Coghlan. *Style Guide for Python Code*. Python Enhancement Proposal (PEP) 8. Beaverton, OR, USA: Python Software Foundation (PSF), 5. Juli 2001. URL: https://peps.python.org/pep-0008 (besucht am 2024-07-27) (siehe S. 265–281, 333).

[307] Sander van Vugt. *Linux Fundamentals*. 2. Aufl. Hoboken, NJ, USA: Pearson IT Certification, Juni 2022. ISBN: 978-0-13-792931-3 (siehe S. 337).

[308] Dieter Vandesande, Christoph Jabs, Jeremias Berg, Johannes Fichte, George Katsirelos, Ruben Martins, André Schidler und Dominik Schreiber. *MaxSAT Evaluation 2026: Solvers, Benchmarks & Results*. v1. Geneva, Switzerland: zenodo (hosted by European Organization for Nuclear Research (CERN)), 14. Juli 2026. doi:10.5281/zenodo.21358014. See also https://maxsat-evaluations.github.io. (Siehe S. 339).

[309] Kristian Verduin, Sarah Louise Thomson und Daan van den Berg. "Too Constrained for Genetic Algorithms. Too Hard for Evolutionary Computing. The Traveling Tournament Problem". In: *15th International Joint Conference on Computational Intelligence (IJCCI'23)*. 13.–15. Nov. 2023, Rome, Italy. Hrsg. von Niki van Stein, Francesco Marcelloni, H. K. Lam, Marie Cottrell und Joaquim Filipe. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2023, S. 246–257. ISSN: 2184-3236. ISBN: 978-989-758-674-3. doi:10.5220/0012192100003595 (siehe S. 345).

References XLI

[310] Petr Vilím, Philippe Laborie und Paul Shaw. "Failure-Directed Search for Constraint-Based Scheduling". In: *12th International Conference on the Integration of AI and OR Techniques in Constraint Programming (CPAIOR'2015)*. Hrsg. von Laurent Michel. Bd. 9075 der Reihe Theoretical Computer Science and General Issues (LNTCS), sub-series of Lecture Notes in Computer Science (LNCS). Barcelona, Spain: Springer, 2015, S. 437–453. ISSN: 2512-2010. ISBN: 978-3-319-18007-6. doi:10.1007/978-3-319-18008-3_30. URL: https://www.researchgate.net/publication/280223626 (besucht am 2026-09-30) (siehe S. 336).

[311] Thomas Weise (汤卫思). *Databases*. Hefei, Anhui, China (中国安徽省合肥市): Hefei University (合肥大学), School of Artificial Intelligence and Big Data (人工智能与大数据学院), 2025. URL: https://thomasweise.github.io/databases (besucht am 2025-01-05) (siehe S. 333, 341).

[312] Thomas Weise (汤卫思). *Global Optimization Algorithms – Theory and Application*. self-published, 2009. URL: https://www.researchgate.net/publication/200622167 (besucht am 2025-07-25) (siehe S. 333, 334, 338, 342).

[313] Thomas Weise (汤卫思). *jsspInstancesAndResults: Results, Data, and Instances of the Job Shop Scheduling Problem*. San Francisco, CA, USA: GitHub Inc, 2019–2020. URL: https://github.com/thomasWeise/jsspInstancesAndResults (besucht am 2025-08-20) (siehe S. 336).

[314] Thomas Weise (汤卫思). *Programming with C*. Hefei, Anhui, China (中国安徽省合肥市): Hefei University (合肥大学), School of Artificial Intelligence and Big Data (人工智能与大数据学院), 2026. URL: https://thomasweise.github.io/programmingWithC (besucht am 2026-05-10) (siehe S. 332).

[315] Thomas Weise (汤卫思). *Programming with Python*. Hefei, Anhui, China (中国安徽省合肥市): Hefei University (合肥大学), School of Artificial Intelligence and Big Data (人工智能与大数据学院), 2024–2026. URL: https://thomasweise.github.io/programmingWithPython (besucht am 2025-01-05) (siehe S. 333, 339, 340).

[316] Thomas Weise (汤卫思), Raymond Chiong, Jörg Lässig, Ke Tang (唐珂), Shigeyoshi Tsutsui, Wenxiang Chen (陈文祥), Zbigniew „Zbyszek" Michalewicz und Xin Yao (姚新). "Benchmarking Optimization Algorithms: An Open Source Framework for the Traveling Salesman Problem". *IEEE Computational Intelligence Magazine (CIM)* 9(3):40–52, Aug. 2014. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE). ISSN: 1556-603X. doi:10.1109/MCI.2014.2326101. Applies a multitude of algorithms including SLSes, EAs, MAs, ACO, and EDAs to the TSPLib instances. (Siehe S. 330, 344, 347).

References XLII

[317] Thomas Weise (汤卫思), Xinlu Li (李新路), Yan Chen (陈岩) und Zhize Wu (吴志泽). "Solving Job Shop Scheduling Problems without using a Bias for Good Solutions". In: *23rd Genetic and Evolutionary Computation Conference (GECCO'2021), Companion Volume*. 10.–14. Juli 2021, Lille, France. Hrsg. von Krzysztof Krawiec. New York, NY, USA: Association for Computing Machinery (ACM), 2021, S. 1459–1466. ISBN: 978-1-4503-8351-6. doi:10.1145/3449726.3463124. Applies Frequency Fitness Assignment (FFA) to the JSSP. (Siehe S. 348).

[318] Thomas Weise (汤卫思), Mingxu Wan (万明绪), Pu Wang (王璞), Ke Tang (唐珂), Alexandre Devert und Xin Yao (姚新). "Frequency Fitness Assignment". *IEEE Transactions on Evolutionary Computation* 18(2):226–243, Apr. 2014. Los Alamitos, CA, USA: IEEE Computer Society. ISSN: 1089-778X. doi:10.1109/TEVC.2013.2251885. The article introducing FFA. (Siehe S. 333).

[319] Thomas Weise (汤卫思) und Zhize Wu (吴志泽). "Replicable Self-Documenting Experiments with Arbitrary Search Spaces and Algorithms". In: *25th Conference on Genetic and Evolutionary Computation (GECCO'2023), Companion Volume*. 15.–19. Juli 2023, Lisbon, Portugal. Hrsg. von Sara Silva und Luís Paquete. New York, NY, USA: Association for Computing Machinery (ACM), 2023, S. 1891–1899. ISBN: 979-8-4007-0120-7. doi:10.1145/3583133.3596306. Introduces the experiment execution and data collection facilities of moptipy. (Siehe S. 339).

[320] Thomas Weise (汤卫思), Zhize Wu (吴志泽), Xinlu Li (李新路) und Yan Chen (陈岩). "Frequency Fitness Assignment: Making Optimization Algorithms Invariant under Bijective Transformations of the Objective Function Value". *IEEE Transactions on Evolutionary Computation* 25(2):307–319, Apr. 2021. Los Alamitos, CA, USA: IEEE Computer Society. ISSN: 1089-778X. doi:10.1109/TEVC.2020.3032090 (siehe S. 329, 333, 336, 337, 340, 343, 345).

[321] Thomas Weise (汤卫思), Zhize Wu (吴志泽), Xinlu Li (李新路), Yan Chen (陈岩) und Jörg Lässig. "Frequency Fitness Assignment: Optimization without Bias for Good Solutions can be Efficient". *IEEE Transactions on Evolutionary Computation* 27(4):980–992, 2023. Los Alamitos, CA, USA: IEEE Computer Society. ISSN: 1089-778X. doi:10.1109/TEVC.2022.3191698. Applies many FFA-based algorithm variants to problems such as OneMax, LeadingOnes, Linear Harmonic Function (LinHarm), TwoMax, Trap, Jump, Plateau, Ising Model, and N-Queens. (Siehe S. 329, 333, 337, 343, 345).

[322] Thomas Weise (汤卫思), Yuezhong Wu (吴越钟), Raymond Chiong, Ke Tang (唐珂) und Jörg Lässig. "Global versus Local Search: The Impact of Population Sizes on Evolutionary Algorithm Performance". *Journal of Global Optimization* 66(3):511–534, Feb. 2016. London, England, UK: Springer Nature Limited. ISSN: 0925-5001. doi:10.1007/s10898-016-0417-5 (siehe S. 330, 344).

References XLIII

[323] Eric Wolfgang Weisstein. "Irrational Number". In: *MathWorld – A Wolfram Web Resource*. Champaign, IL, USA: Wolfram Research, Inc., 22. Aug. 2024. URL: https://mathworld.wolfram.com/IrrationalNumber.html (besucht am 2026-11-04) (siehe S. 335).

[324] Eric Wolfgang Weisstein. *MathWorld – A Wolfram Web Resource*. Champaign, IL, USA: Wolfram Research, Inc., 22. Aug. 2024. URL: https://mathworld.wolfram.com (besucht am 2024-09-24).

[325] Eric Wolfgang Weisstein. "Transcendental Number". In: *MathWorld – A Wolfram Web Resource*. Champaign, IL, USA: Wolfram Research, Inc., 22. Aug. 2024. URL: https://mathworld.wolfram.com/TranscendentalNumber.html (besucht am 2026-11-04) (siehe S. 343).

[326] B. P. Welford. "Note on a Method for Calculating Corrected Sums of Squares and Products". *Technometrics* 4(3):419–420, Aug. 1962. London, England, UK: Taylor and Francis Ltd. ISSN: 0040-1706. doi:10.2307/1266577. This algorithm is often used to compute the sample variance[56,171] (siehe S. 351).

[327] *What is a Relational Database?* Armonk, NY, USA: International Business Machines Corporation (IBM), 20. Okt. 2021–12. Dez. 2024. URL: https://www.ibm.com/think/topics/relational-databases (besucht am 2025-01-05) (siehe S. 341).

[328] L. Darrell Whitley. "The GENITOR Algorithm and Selection Pressure: Why Rank-Based Allocation of Reproductive Trials is Best". In: *3rd International Conference on Genetic Algorithms (ICGA'1989)*. Juni 1989, Fairfax, VA, USA: George Mason University. Hrsg. von J. David Schaffer. Burlington, MA, USA/San Mateo, CA, USA: Morgan Kaufmann Publishers, 1989, S. 116–123. ISBN: 978-1-55860-066-9. URL: https://www.researchgate.net/publication/2527551 (besucht am 2025-08-08) (siehe S. 337, 342).

[329] Ulf Michael „Monty" Widenius, David Axmark und Uppsala, Sweden: MySQL AB. *MySQL Reference Manual – Documentation from the Source*. Sebastopol, CA, USA: O'Reilly Media, Inc., 9. Juli 2002. ISBN: 978-0-596-00265-7 (siehe S. 339).

[330] James M. Wilson. "Gantt Charts: A Centenary Appreciation". *European Journal of Operational Research* 149(2):430–437, Sep. 2003. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/S0377-2217(02)00769-5 (siehe S. 334).

[331] Kevin Wilson. *Python Made Easy*. Birmingham, England, UK: Packt Publishing Ltd, Aug. 2024. ISBN: 978-1-83664-615-0 (siehe S. 333, 340).

References XLIV

[332] L. Wong und Lai-Soon Lee. "Heuristic Placement Routines for Two-Dimensional Bin Packing Problem". 5(4):334–341, Dez. 2009. doi:/10.3844/jmssp.2009.334.341. URL: https://www.thescipub.com/pdf/jmssp.2009.334.341 (besucht am 2026-10-02). A heuristic for the 2BP|R|F and the 2BP|O|F (siehe S. 287).

[333] CAO Xiang (曹翔), Zhize Wu (吴志泽), Daan van den Berg und Thomas Weise (汤卫恩). "Randomized Local Search vs. NSGA-II vs. Frequency Fitness Assignment on The Traveling Tournament Problem". In: *16th International Joint Conference on Computational Intelligence (IJCCI'24)*. 20.–22. Nov. 2024, Porto, Portugal. Hrsg. von Francesco Marcelloni, Kurosh Madani, Niki van Stein und Joaquim Filipe. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2024, S. 38–49. ISSN: 2184-3236. ISBN: 978-989-758-721-4. doi:10.5220/0012891500003837 (siehe S. 339, 345, 348).

[334] Zhou Xu und Chung-Yee Lee. "New Lower Bound and Exact Method for the Continuous Berth Allocation Problem". *Operations Research* 66(3):ii–iv, 597–892, Mai–Juni 2018. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 0030-364X. doi:10.1287/opre.2017.1687. URL: https://ira.lib.polyu.edu.hk/bitstream/10397/78451/1/Xu_New_Lower_Bound.pdf (besucht am 2026-10-01) (siehe S. 331).

[335] Takeshi Yamada und Ryohei Nakano. "A Genetic Algorithm Applicable to Large-Scale Job-Shop Instances". In: *Parallel Problem Solving from Nature 2 (PPSN-II)*. 28.–30. Sep. 1992, Brussels, Belgium. Hrsg. von Reinhard Männer und Bernard Manderick. Amsterdam, The Netherlands: Elsevier B.V., 1992, S. 281–290. URL: https://www.researchgate.net/publication/220701684 (besucht am 2026-09-30) (siehe S. 336).

[336] Kinza Yasar und Craig S. Mullins. *Definition: Database Management System (DBMS)*. Newton, MA, USA: TechTarget, Inc., Juni 2024. URL: https://www.techtarget.com/searchdatamanagement/definition/database-management-system (besucht am 2025-01-11) (siehe S. 333).

[337] François Yergeau. *UTF-8, A Transformation Format of ISO 10646*. Request for Comments (RFC) 3629. Wilmington, DE, USA: Internet Engineering Task Force (IETF), Nov. 2003. URL: https://www.ietf.org/rfc/rfc3629.txt (besucht am 2025-02-05). See Unicode and[152] (siehe S. 346).

References XLV

[338] Wenqi Ying (应雯棋), Hrsg. *Commemoration of Ancient Chinese Mathematical Master Liu Hui for his Timeless Influence on Mathematics and Civilizational Exchange*. Bd. 48 (Special Issue) der Reihe CAST Newsletter. China, Beijing (中国北京市): 中国科学技术协会 (China Association for Science and Technology, CAST), Nov. 2024. URL: https://english.cast.org.cn/cms_files/filemanager/1941250207/attach/202412/8f23655a82364d19ad7874eb37b23035.pdf (besucht am 2025-08-24). Proofreader: Yumeng Wei (魏雨萌), Designer: Shan Zhang (张珊) (siehe S. 347).

[339] Giorgio Zarrelli. *Mastering Bash*. Birmingham, England, UK: Packt Publishing Ltd, Juni 2017. ISBN: 978-1-78439-687-9 (siehe S. 331).

[340] Jiazheng Zeng (曾嘉政), Thomas Weise (汤卫思), Zhize Wu (吴志泽) und Markus Wagner. "When Frequency Fitness Assignment Fails: Trapped States in Frequency-Guided Local Search". In: *28th Genetic and Evolutionary Computation Conference (GECCO'2026)*. 13.–17. Juli 2026, San José, Costa Rica. Hrsg. von Leonardo Trujillo und Ting Hu. New York, NY, USA: Association for Computing Machinery (ACM), 2026, S. 233–242. ISBN: 979-8-4007-2487-9. doi:10.1145/3795095.3805098. Explores $(1+1)$ evolutionary algorithm ($(1+1)$ EA), $(1+1)$ EA with FFA ($(1+1)$ FEA), RLS, and FRLS as well as hybrid algorithms on a variety of discrete benchmarks such as OneMax, LeadingOnes, LinHarm, Binary Integer (BinInt), TwoMax, Trap, Jump, Plateau, Ising Model, N-Queens and also applies them to the search for Low-Autocorrelation Binary Sequences (LABS). (Siehe S. 329, 333, 335–337, 339, 340, 342, 343, 345).

[341] Chaoyong Zhang, Xinyu Shao, Yunqing Rao und Haobo Qiu. "Some New Results on Tabu Search Algorithm Applied to the Job-Shop Scheduling Problem". In: *Tabu Search*. Hrsg. von Wassim Jaziri. London, England, UK: IntechOpen Limited, 1. Aug. 2008. Kap. 8. ISBN: 978-3-902613-34-9. doi:10.5772/5593. URL: https://www.intechopen.com/chapters/4596 (besucht am 2026-09-30) (siehe S. 336, 344).

[342] Rui Zhao (赵睿), Tianyu Liang (梁天宇), Zhize Wu (吴志泽), Daan van den Berg, Matthias Thürer und Thomas Weise (汤卫思). "Randomized Local Search on the 2D Rectangular Bin Packing Problem with Item Rotation". In: *26th Genetic and Evolutionary Computation Conference (GECCO'2024)*. 14.–18. Juli 2024, Melbourne, VIC, Australia. Hrsg. von Xiaodong Li und Julia Handl. New York, NY, USA: Association for Computing Machinery (ACM), 2024, S. 235–238. ISBN: 979-8-4007-0494-9. doi:10.1145/3638530.3654139 (siehe S. 330, 335, 339).

References XLVI

[343] Rui Zhao (赵睿), Zhize Wu (吴志泽), Daan van den Berg, Matthias Thürer, Tianyu Liang (梁天宇), Ming Tan (檀明) und Thomas Weise (汤卫思). "Randomized Local Search for Two-Dimensional Bin Packing and a Negative Result for Frequency Fitness Assignment". In: *16th International Joint Conference on Computational Intelligence (IJCCI'24)*. 20.–22. Nov. 2024, Porto, Portugal. Hrsg. von Francesco Marcelloni, Kurosh Madani, Niki van Stein und Joaquim Filipe. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2024, S. 15–26. ISSN: 2184-3236. ISBN: 978-989-758-721-4. doi:10.5220/0012888500003837 (siehe S. 330, 332, 335, 339).

[344] Nicola Abdo Ziadeh, Michael B. Rowton, A. Geoffrey Woodhead, Wolfgang Helck, Jean L.A. Filliozat, Hiroyuki Momo, Eric Thompson, E.J. Wiesenberg und Shih-ch'ang Wu. "Chronology – Christian History, Dates, Events". In: *Encyclopaedia Britannica*. Hrsg. von The Editors of Encyclopaedia Britannica. Chicago, IL, USA: Encyclopædia Britannica, Inc., 26. Juli 1999–20. März 2024. URL: https://www.britannica.com/topic/chronology/Christian (besucht am 2025-08-27) (siehe S. 331, 332).

Glossary (in English) I

(1 + 1) EA The $(1+1)$ EA is a local search algorithm that retains the best solution x_c discovered so far during the search[51,95]. In each step, it applies a unary search operator to this best-so-far solution x_c and derives a new solution x_n. If the new solution x_n is *better or equally good* when compared with x_c, i.e., not worse, then it replaces it, i.e., is stored as the new x_c. If the search space are bit strings of length n, then the $(1+1)$ EA uses a unary search operator that flips each bit independently with probability m/n, where usually $m = 1$. This operator is the main difference to randomized local search (RLS). The $(1+1)$ EA is a special case of the $(\mu+\lambda)$ evolutionary algorithm ($(\mu+\lambda)$ EA) where $\mu = \lambda = 1$.

(1 + 1) FEA The $(1+1)$ EA with FFA plugged in [320, 321, 340]. On the discrete bit-string domains, this algorithm has a very specific structure: Its unary search operator flips each bit with probability $1/n$, where n is the dimension of the search space, distinguishing it from the unary operator in FRLS which flips exactly one bit. On other domains, such as permutations, the term is used synonymously with FRLS.

(μ + λ) EA The $(\mu+\lambda)$ EA is an evolutionary algorithm (EA) where, in each generation, λ offspring solutions are generated from the current population of μ parent solutions. The offspring and parent populations are merged, yielding $\mu+\lambda$ solutions, from which then the best μ solutions are ratained to form the parent population of the next generation. If the search space is the bit strings of length n, then this algorithm usually applies a mutation operator flipping each bit independently with probability $1/n$.

BinInt The *Binary Integer Problem* is a benchmark problem for bit-string based black-box search[295,340]. It is very similar to OneMax: It tries to maximize the number of 1 bits in the bit string of a chosen dimension n. Different from OneMax, where each bit has the same weight in the objective function, the bits of BinInt have a exponentially increasing weight: The bit string is treated as a binary-encoded integer whose value should be maximized. The objective function $f(x)$ of the minimization variant of this problem can be defined as $f(x) = 2^n - 1 - \sum_{i=1}^{n} 2^{i-1} \cdot x_i$. The problem is unimodal, i.e., has only a single global optimum and no local optimum. It is fully separable, i.e., we can optimize each variable separately.

LinHarm The Linear Harmonic Function is a benchmark problem for bit-string based black-box search[89,95,340]. It is very similar to OneMax: It tries to maximize the number of 1 bits in the bit string of a chosen dimension n. Different from OneMax, where each bit has the same weight in the objective function, the bits of LinHarm have a linear increasing weight. The objective function $f(x)$ of the minimization variant of this problem can be defined as $f(x) = \frac{1}{2}n(n+1) - \sum_{i=1}^{n} i \cdot x_i$. The problem is unimodal, i.e., has only a single global optimum and no local optimum. It is fully separable, i.e., we can optimize each variable separately. The $(1+1)$ EA can solve this problem with an expected runtime in $\Theta(n \ln n)$[89,95].

Glossary (in English) II

TSP Suite is a open source benchmarking framework for the TSP written in Java[316]. It uses the instances of the TSPLib and offers both automated experiment execution as well as several metrics to measure runtime. It is available at https://thomasweise.github.io/tspSuite and has been used in many research works, such as[322]. It can be viewed as a predecessor of moptipy.

2BP In the two-dimensional Bin Packing Problem (BP), the goal is to pack rectangles into rectangular bins without overlap and such that their edges are parallel to the edges of the bins[153,195,197]. The 2BP is strongly $\mathcal{NP}$-hard[197]. The variants of this problem include the 2BP|O|F, 2BP|R|F, 2BP|O|G, and 2BP|R|G[196,197]. Different objective function for the 2BP|O|F and 2BP|R|F are presented in [342]. 2BPs have many practical applications, including cutting (wood, glas)[197], packing (transportation, warehouse)[197], and newspaper page layout[282].

2BP|O|F In the oriented 2BP with free cuts, the rectangles to pack have fixed orientations, i.e., cannot be rotated, but can otherwise be arranged freely[196,197,209,343]. Permutation-based search spaces can be used for this problem by using the Improved Bottom Left (IBL) encoding. The state-of-the-art (SOTA) for this problem comprises[31,36,62,127,239,343].

2BP|O|G In the oriented 2BP with guillotine cuts, the rectangles to pack have fixed orientations, i.e., cannot be rotated, and must be arranged such that they can be extracted by edge-to-edge cuts[196,197].

2BP|R|F In this variant of the 2BP with free cuts, the rectangles to pack can be rotated by 90° if need be and can freely be arranged[27,196,197,342,343]. Besides packing and cutting, this problem also occurs in scheduling and build formation in additive manufacturing[188,244]. The almost-squares-in-almost-squares (Asqas) problem[303] is also a 2BP|R|F. Search spaces based on signed permutations can be used for this problem by using the IBL encoding. The SOTA for this problem comprises[31,62,127,342,343].

2BP|R|G In this variant of the 2BP with guillotine cuts, the rectangles to pack can be rotated by 90° if need be, but must be arranged such that they can be extracted by edge-to-edge cuts[196,197].

ACO *Ant Colony Optimization* is a nature-inspired optimization method for combinatorial problems where solutions are generated by „ants“ that move from node to node in a graph choosing edges based on (1) the simulated pheromone on the edges and (2) a per-edge heuristic value[91–93]. If an ant produced a good solution, „pheromone“ is distributed over the edges it visited, making it more likely to be re-visited by other ants.

AI Artificial Intelligence, see, e.g., [262]

Glossary (in English) III



API An *Application Programming Interface* is a set of rules or protocols that enables one software application or component to use or communicate with another[132].

Asqas An „almost-square" be a rectangle with size $i \times (i+1)$ with $i \in \mathbb{N}_\mathbf{1}$. The *almost-squares-in-almost-squares* problem[303] is a special case of the 2BP|R|F. Here, almost-square rectangles of sizes 1×2, 2×3, $\ldots$, $n \times (n+1)$ must be placed into an almost-square bin without waste. This requires that there exists a natural number $p \in \mathbb{N}_\mathbf{1}$ such that $p(p+1) = \sum_{i=\mathbf{1}}^{n} i(i+1)$[303]. There are exacty five Asqas instances[303], namely with bin sizes 1×2 for $n = 1$, $p = 1$, 5×4 for $n = 3$, $p = 4$, 15×16 for $n = 8$, $p = 15$, 55×56 for $n = 20$, $p = 55$, and 119×120 for $n = 34$, $p = 119$.

ATSP The general case for TSPs are *asymmetric Traveling Salesperson Problems*, where the $n \times n$ distance matrix A does not need to be symmetric. In other words, it is possible that $\exists a, b \in 1..n$ with $A[a, b] \neq A[b, a]$. Notice that while such index tuple *may* exist, this is not always or necessarily the case. In this case, one may consider the STSP as a special case of the ATSP, while ATSPs may be considered as synonymous to TSPs.

B&B *Branch and bound* is a family of optimization methods that combines systematic exploration of the search space with both and objective function f subject to minimization and a lower bound function. The idea is that we divide the search space into two sets. We then pick the set that seems more interesting and divide it again. We do this recursively until we have arrived at a set of size 1, which is our first solution y. We can compute its objective value $z = f(y)$. Now we continue with the sets that we have created and not yet explored. However, we can discard all sets for which the lower bound of the solution quality is $\geq z$, because they can never give us a better solution than y. If we have divided another set down to size 1 and it yields a better solution than y, of cause we update these values. Eventually, we have either discarded or fully expanded all sets – and then the solution we got must be optimal. B&B-algorithms have been applied to the TSP[193], the JSSP[48], the 2BP|O|F[209], and maritime logistics[334], for instance.

Bash is a shell used under Ubuntu Linux, i.e., the program that „runs" in the terminal and interprets your commands, allowing you to start and interact with other programs[41,225,339]. Learn more at `https://www.gnu.org/software/bash`.

BCE The time notation *before Common Era* is a non-religious but chronological equivalent alternative to the traditional *Before Christ (BC)* notation, which refers to the years *before* the birth of Jesus Christ[52]. The years BCE are counted down, i.e., the larger the year, the farther in the past. The year 1 BCE comes directly before the year 1 Common Era (CE)[265,344].

Glossary (in English) IV

BP The *Bin Packing Problem*[98,264] is one of the classical problems from Operations Research (OR). The goal is to pack a set of objects into containers, using as few containers as possible. Different flavors of BPs can be distinguished, e.g., by the number of dimensions[79,195] that the objects and containers have (see, e.g., the 2BP), and/or whether the objects can be rotated[343], as well as other constraints and limitations. Both the one- and two-dimensional BP are strongly $\mathcal{NP}$-hard[197].

BRKGA The *Biased Random Key Genetic Algorithm* is a EA with a continuous search space $\mathbb{X} \subset [0,1]^n$ and a decoding function γ translating this space to something else and biased, uniform, and elitist mating strategies[129,200].The *bias* comes from the fact that one of the parents used for the binary operation is biased to be of higher fitness than the other parent[129]. Often, γ is the Random Keys encoding[19,20] procedure translating $\mathbb{X}$ to permutations of the first n natural numbers[128]. Sometimes, it is actually a MA further refining the translation result with a local search (LS)[128]. Sometimes the permutations are then further translated to more complex data structures, such as Gantt charts for the JSSP[128], or packing plans for 2BP 3D packing[127].

C is a programming language, which is very successful in system programming situations[84,250,314].

CE The time notation *Common Era* is a non-religious but chronological equivalent alternative to the traditional *Anno Domini (AD)* notation, which refers to the years *after* the birth of Jesus Christ[52]. The years CE are counted upwards, i.e., the smaller they are, the farther they are in the past. The year 1 CE comes directly after the year 1 before Common Era (BCE)[265,344].

client In a client-server architecture, the client is a device or process that requests a service from the server. It initiates the communication with the server, sends a request, and receives the response with the result of the request. Typical examples of clients are web browsers in the internet as well as clients for database management systems (DBMSes), such as psql.

client-server architecture is a system design where a central server receives requests from one or multiple clients[30,201,234,253,258]. These requests and responses are usually sent over network connections. A typical example of such a system is the World Wide Web (WWW), where web servers host websites and make them available to web browsers, the clients. Another typical example is the structure of database (DB) software, where a central server, the DBMS, offers access to the DB to the different clients. Here, the client can be some terminal software shipping with the DBMS, such as psql, or the different applications that access the DBs.

Concorde is an exact solver for the TSP[5,66,67], which could even solve the largest instance of TSPLib (with 85 900 cities) to optimality.

CSP The *Cutting Stock Problem* is about cutting shapes of a larger shape while reducing waste[197,205]. It is closely related to the BP.

Glossary (in English) V



DB A *database* is an organized collection of structured information or data, typically stored electronically in a computer system. Databases are discussed in our book *Databases*[311].

DBMS A *database management system* is the software layer located between the user or application and the DB. The DBMS allows the user/application to create, read, write, update, delete, and otherwise manipulate the data in the DB[336].

debugger A debugger is a tool that lets you execute a program step-by-step while observing the current values of variables. This allows you to find errors in the code more easily[2,259,331]. Learn more about debugging in [315].

denominator The number b of a fraction $\frac{a}{b} \in \mathbb{Q}$ is called the *denominator*.

docstring Docstrings are special string constants in Python that contain documentation for modules or functions[130]. They must be delimited by `"""..."""`[130,306].

doctest *doctests* are unit tests in the form of as small pieces of code in the docstrings that look like interactive Python sessions. The first line of a statement in such a Python snippet is indented with Python»> and the following lines by `....`. These snippets can be executed by modules like `doctest`[85] or tools such as pytest[175]. Their output is the compared to the text following the snippet in the docstring. If the output matches this text, the test succeeds. Otherwise it fails.

EA An *evolutionary algorithm* is a metaheuristic optimization method that maintains a population of candidate solutions, which undergo selection (where better solutions are chosen with higher probability) and reproduction (where mutation and recombination create a new candidate solution from one or two existing ones, repectively)[12,312].

FFA *Frequency Fitness Assignment* (中文: 频率适应度分配) is a algorithm plugin for optimization methods applied to discrete or combinatorial problems with not-too-many different possible objective values[318,320,321]. It replaces the objective values in all comparisons with their absolute encounter frequency so far during the search. FFA has successfully been applied to the Maximum Satisfiability (MaxSAT)[320,321], the TSP[189,190], and the QAP[59,297], amongst others. Its properties have been intensely researched on discrete benchmark functions such as OneMax, LeadingOnes, Trap, TwoMax, Jump, and Plateau[320,321,340]. The most common realizations of FFA so far are the $(1+1)$ FEA and FRLS.

FSSP The *Flow Shop Scheduling Problem* is a variant of a JSSP where all products go through the same sequence of machines[133,162].

Glossary (in English) VI



GA A *Genetic Algorithm* is an EA usually for bit-string based search spaces[12,76,126,214,220,312] (although many researchers use the terms EA and GA synonymously). It is a population-based metaheuristic applying a unary and a binary search operater as well as a selection scheme. The original GA is the Standard Genetic Algorithm (SGA), which today is considered outdated. Modern variants are the Greedy $(2+1)$ GA and the Self-Adjusting $(1+(\lambda,\lambda))$ Genetic Algorithm (Self-Adjusting $(1+(\lambda,\lambda))$ GA) as well as bit-string based $(\mu+\lambda)$ EAs.

Gantt chart Gantt charts are two-dimensional diagrams illustrating the steps of a process[168,330]. Their horizontal axis represents the time, the vertical axis represents work packages, machines, or workers. In the diagram area, bars located at certain vertical locations indicate that the work package, machine, or worker is processed or works at the time range covered by the bar as projected to the horizontal axis.

GECCO The *Genetic and Evolutionary Computation Conference* is one of the largest conferences in the field of EAs. It has been established in 1999 and its proceedings are published by ACM. Learn more at https://sig.sigevo.org/GECCOs.

Git is a distributed Version Control System (VCS) which allows multiple users to work on the same code while preserving the history of the code changes[272,301]. Learn more at https://git-scm.com.

GitHub is a website where software projects can be hosted and managed via the Git VCS[241,301]. Learn more at https://github.com.

GRASP A *Greedy Randomized Adaptive Search Procedure* is a metaheuristic where each cycle consists of two main components[106,108,109]: First, a starting point is generated using a heuristic method (as opposed to being randomly sampled). Then, a LS is applied to refine the solution. Then, another cycle starts and proceeds in the same way. The overall best solution is retained as result. Many examples of applications of GRASPs are listed in [109]. GRASP in combination with VND has been applied to the 2BP|O|F in [239].

Greedy $(2+1)$ GA The Greedy $(2+1)$ GA[50,51,284,285] is an EA for bit-string based search spaces, i.e., a Genetic Algorithm (GA). It was shown to be twice as fast as any mutation-only EA on OneMax[284,285].

Glossary (in English) VII

IBL
: The *improved bottom left* heuristic is a one-shot algorithm constructing a solution for 2BP[194]. For the 2BP|O|F problem variant, the input of the IBL algorithm is a permutation of the first n natural numbers, each number identifying one rectangle to be packed. For the 2BP|R|F problem flavor, the permutation elements are signed, where a negative sign indicates a rotation by 90°[194]. In[342,343] it is adapted to a multi-bin situation as follows: The algorithm iterates over the (potentially signed) permutation x and places the rectangles into the packing plan in the prescribed order. It first places the rectangle outside on top of the current bin b, with its bottom-right corner onto the top-right corner of the current bin b. It then moves the rectangle downwards and leftwards as far as possible, prioritizing the downward movement whenever possible. If the rectangle cannot be moved any further, it checks if it is completely contained in the bin b. If yes, it can remain there. Otherwise, a new bin is opened ($b \leftarrow b + 1$) and the rectangle is placed at its bottom-left corner. Once a new bin is opened, this bin is used for all further insertions. IBL is suitable as a decoding function γ inside an EA[194]. Historically, it improves upon the BL algorithm[157]: That algorithm cannot represent all possible optimal packings using a permutation, but the IBL can[194].

ILS
: An *iterated local search* is a variant of the LS which applies cycles of local search and perturbation[283]. First, an initial solution is generated and refined by the local search. Then the cycle begins: A perturbation, i.e., a large search move that likely destroys a good part of the solution, is applied. The resulting new solution is refined by local search. Then, an acceptance criterion decides whether we should use this or the previous solution to continue in the next cycle. ILS has successfully been applied to the QAP.

irrational
: An *irrational* number z is a real number that cannot be expressed as fraction $\frac{p}{q}$ of two integers $p \in \mathbb{Z}$ and $q \in \mathbb{N}_1$[25,323]. In other words, it holds that $z \in \mathbb{R}$ and $z \notin \mathbb{Q}$. The set of irrational numbers could be denoted as $\mathbb{R} \setminus \mathbb{Q}$. Irrational numbers have decimal expansions that neither terminate nor become periodic. Examples of irrational numbers are $\sqrt{2}$, ϕ, π, and e.

Ising Model
: The Ising models are benchmark problems for bit-string based black-box search[89,111,112,340]. Here, a neighborhood is defined around each bit. The bit needs to have the same value as all bits in the neighborhood structure T. Thus, we try to achieve that all bits have the same value. The objective function $f(x)$ of the minimization variant of this problem family can be defined as $f(x, T) = \sum_{(i,j)\in T} |x_i - x_j|$, where T contains the index pairs of neighboring bits and is defined differently for the Ising model instances Ising 1D and Ising 2D. The problems are highly epistatic, as all variables interact with some other variables.

Glossary (in English) VIII

Ising 1D The 1D is a benchmark problem for bit-string based black-box search[89,112,340]. The bits are arranged in a one-dimensional ring and we try to minimize the number of bits that have different values than their neighbors. Hence, the neighborhood structure T is defined as $T_{\mathbf{1}D} = \{(i+1, (i+1 \bmod n)+1) \mid i \in [0..n-1]\}$. The problems are highly epistatic, as all variables interact with some other variables. The $(1+1)$ EA has an expected runtime in $\mathcal{O}(n^{\mathbf{3}})$ on 1-Dimensional Ising Model (Ising 1D)[112].

Ising 2D The 2D is a benchmark problem for bit-string based black-box search[89,111,340]. The bits are arranged in a two-dimensional torus and we try to minimize the number of bits that have different values than their neighbors. Hence, the neighborhood structure $T = T_{\mathbf{2}D}$ is defined as the set of all $\{(\alpha+\beta N, \gamma+\delta N)\}$ pairs with $\alpha, \beta, \gamma, \delta \in 0..N-1$ and either ()$\gamma = (\alpha+1) \bmod N$ and $\delta = \beta$) or ($\gamma = \alpha$ and $\delta = (\beta+1) \bmod N$) with $n = N^{\mathbf{2}}$. The problems are highly epistatic, as all variables interact with some other variables. On a similar class of problems, the $(1+1)$ EA has an exponential expected runtime[111].

IT information technology

Java is another very successful programming language, with roots in the C family of languages[35,202].

JSSP The *Job Shop Scheduling Problem*[34,61,133,183] is one of the most prominent and well-studied scheduling tasks. In a JSSP instance, there are m machines and k jobs[286]. Each job must be processed once by each machine in a job-specific sequence and has a job-specific processing time on each machine. The goal is to find an assignment of jobs to machines that results in an overall shortest makespan, i.e., the schedule (Gantt chart) which can complete all the jobs in the shortest time. The JSSP is $\mathcal{NP}$-complete[57,182,183]. The 8 most common sets of JSSP benchmark instances are *abz*[1], *dmu*[80], *ft*[113], *la*[185], *orb*[6], *swv*[280], *ta*[286], and *yn*[335].Their data can be found (sometimes partially) in several repositories, such as the OR-Library[21,22], [269], and [313], the latter of which also contains many results from existing works. Encodings based on permutations with repetition are proposed in [32, 33, 120, 268] and used in [320]. The SOTA of the JSSP includes[128,143,238,240,270,310,341].

Glossary (in English) IX



Jump The Jump problem is a benchmark problem for bit-string based black-box search[95,117,340]. Like the OneMax, it tries to maximize the number of 1 bits in a bit string of chosen length n. However, the solutions representing the last ω bits around the optimum to flip are deceptive, meaning that they are actually worse and getting worse than surrounding solutions. The objective function $f(x)$ of the minimization variant is $n - \|x\|_1$ if $\|x\|_1 = n$ or $\|x\|_1 \leq n - \omega$ hold, and $\omega + \|x\|_1$ otherwise, where $\|x\|_1 = \sum_{i=1}^{n} x_i$. This means that, the larger ω, the larger the deceptive basin around the optimum that an algorithm needs to cross. An $(1+1)$ EA will converge to the resulting local optimum and thus have an expected runtime of $\Theta(n^\omega + n \ln n)$[95].In experiments, the $(1+1)$ FEA achieves polynomial runtime on Jump, seemingly in $\mathcal{O}(n^2 \ln n)$[320,321,340].

LABS *Low-Autocorrelation Binary Sequences* are bit strings of length n whose autocorrelation is as low as possible[37,38,42,82,83,118,212,215,222,235,247]. This means that sub-sequences of any length $1 \leq k < n$ should ideally never repeat. The objective function $f : \{0,1\}^n \mapsto \mathbb{N}_0$ represents the energy of a bit string x and is subject to minimization. We can define it as $f(x) = \sum_{k=1}^{n-1} \left(\sum_{i=0}^{n-1-k} s_i \cdot s_{i+k}\right)^2$ with $s_j = 2x_j - 1$. Such sequences are needed for communication engineering[38,42,222], cryptography[82,222], as well as modulation pulses in radar and sonar ranging[235]. The LABS is $\mathcal{NP}$-hard[299]. The SOTA on LABS contains[37,42,82,83,222].

LAMP Stack A system setup for web applications: Linux, Apache (a web server), MySQL, and the server-side scripting language PHP[47,140].

LeadingOnes The LeadingOnes problem is a benchmark problem for bit-string based black-box search[260,328,340]. It tries to maximize the number of *leading* 1 bits in the bit string of a chosen dimension n. The objective function $f(x)$ of the minimization variant of this problem can be defined as $f(x) = n - \sum_{i=1}^{n} \prod_{j=1}^{i} x_j$. Different from OneMax, the contribution to the objective value of the bit at index i thus depends on the state of all the bits *before* this index. The problem is unimodal, i.e., has only a single global optimum and no local optimum. Different from OneMax, it is not separable, i.e., we can optimize each variable separately. The expected runtime of $(1+1)$ EA on LeadingOnes is in $\Theta(n^2)$[95]. The average runtime $(1+1)$ FEA on LeadingOnes seems to be slower by factor proportional to n, i.e., seems to be in $\mathcal{O}(n^3)$[320].

Linux is the leading open source operating system, i.e., a free alternative to Microsoft Windows[13,139,271,300,307]. We recommend using it for this course, for software development, and for research. Learn more at https://www.linux.org. Its variant Ubuntu is particularly easy to use and install.

Glossary (in English) X



LK The *Lin-Kernighan* (LK) algorithm[192] is a SLS method for the TSP. The LK algorithm performs iterative k-opt moves, i.e., in each step, it constructs the two sets X and Y of k edges such that removing the edges in X from the current tour T and then adding the edges from Y to T results in a better tour.The LK algorithm is one of the most classical approaches to the TSP.

LKH The *Lin-Kernighan-Helsgaun* algorithm[67,141,142] is a SLS method for the TSP based on the Lin-Kernighan (LK) algorithm[192]. LKH-1 was proposed in [142] and LKH-2 in [141]. It holds the record in the *World TSP*[68] as of 2025.

LS A *local search* is an algorithm where a single „current" candidate solution is maintained and new solutions are selected from its neighborhood in each step. Most LSes are randomized, i.e., stochastic local search (SLS).

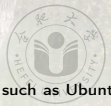
MA A *Memetic Algorithm* is an EA where each new solution is refined by a LS before it enters the population[312]. As LS, we can use, e.g., RLS, but TS[80,114] is maybe a better choise. Some SOTA algorithms in some domains may be classified as MAs, e.g., Lin-Kernighan Helsgaun (LKH) basically uses a population that is refined via a local search. MAs also worked well on the JSSP[80,143] and the TSP[114].

MariaDB An open source relational database management system that has forked off from MySQL[7,8,14,99,208,254]. See `https://mariadb.org` for more information.

MaxSAT The goal of satisfiability problems is to find an assignment for n Boolean variables that make a given Boolean formula $F : \{0,1\}^n \mapsto \{0,1\}$ become true. In the *Maximum Satisfiability* problem (MaxSAT)[146], F is given in conjunctive normal form, i.e., the variables appear either directly or negated in m „or" clauses, which are all combined into one „and." The objective function $f(x)$, subject to minimization, computes the number of clauses which are false under the variable setting x. If $f(x) = 0$, then all clauses of F are true, which solves the problem. The MaxSAT problem is $\mathcal{NP}$-complete[65] and exhibits low epistasis but deceptiveness[252], making it hard for local searches. Typical benchmark instances can be found in SATLIB, which contains smaller-scale instances, and the up-to-date MaxSAT Evaluation (MSE), providing SOTA benchmarks, algorithms, and results.

metaheuristic A *metaheuristic* is an optimization algorithm which performs a cycle of trial and error during which candidate solutions are sampled from the solution space $\mathbb{Y}$ and evaluated using an objective function $f : \mathbb{Y} \mapsto \mathbb{R}$[123,312]. The results of the objective function are used to decide which solution is promising. The neighborhoods around promising solutions *or* similarly-structured solutions are then sampled in the next cycle. Prominent metaheuristics are SLSes, SA, TS, and EAs.

Glossary (in English) XI



Microsoft Windows is a commercial proprietary operating system[40]. It is widely spread, but we recommend using a Linux variant such as Ubuntu for software development and for our course. Learn more at https://www.microsoft.com/windows.

ML Machine Learning, see, e.g., [266]

moptipy is the *Metaheuristic Optimization in Python* library[319]. It has been used in several different research works, including[59,189–191,297,333,342,343]. Algorithms implemented by moptipy include EAs, $(1+1)$ EA, RLS, SA, and the $(1+1)$ FEA, amongst others. Learn more at https://thomasweise.github.io/moptipy and https://thomasweise.github.io/moptipyapps.

MSE The *MaxSAT Evaluation* is a regularly updated set of benchmark instances for the MaxSAT[154]. It also includes solver software and benchmark results[308].

Mypy is a static type checking tool for Python[187] that makes use of type hints. Learn more at https://github.com/python/mypy and in [315].

MySQL An open source relational database management system[39,99,255,291,329]. MySQL is famous for its use in the LAMP Stack. See https://www.mysql.com for more information.

N-Queens The N-Queens Problem is a benchmark problem for bit-string based black-box search[89,340]. The goal is to arrange N queens on an $N \times N$ chessboard so that none can attack the other. The search space has thus dimension $n = N^2$, with each bit standing for a field and encoding whether or not a queen is placed there. The objective function $f(x)$ of the minimization variant is $N - \left(\sum_{i=\mathbf{1}}^{n} x_i\right) - N\sum_{\xi} \max\left\{0, Q_\xi(x) - 1\right\}$ where $Q_\xi(x)$ is the number of queens in the horizontal, vertical, or diagonale ξ. The problem is quite epistatic.

numerator The number a of a fraction $\frac{a}{b} \in \mathbb{Q}$ is called the *numerator*.

Glossary (in English) XII



OneMax OneMax is the simplest possible benchmark problem for bit-string based black-box search[96,223,340]. The goal is to maximize the number of 1 bits in the bit string of a chosen dimension n. The objective function $f(x)$ of the minimization variant of this problem can be defined as $f(x) = n - \sum_{i=1}^{n} x_i$. The problem is unimodal, i.e., has only a single global optimum and no local optimum. It is fully separable, i.e., we can optimize each variable separately. The expected runtime of $(1+1)$ EA on OneMax is in $\Theta(n \ln n)$[87,223]. The average runtime $(1+1)$ FEA on OneMax seems to be slower by factor proportional to n, i.e., seems to be in $\mathcal{O}(n^2 \ln n)$[320].

OR Operations Research (or Operational Research) is the application of sciences such as mathematics and computer science to the management and organization of systems, organizations, enterprises, factories, or projects. It encompasses the development and application of problem-solving methods and techniques (such as mathematical optimization, simulation, queueing theory and other stochastic models) with the goal to improve decision-making and efficiency[77].

OR-Library is a library of benchmark instances for various OR problems[21,22] located at https://people.brunel.ac.uk/~mastjjb/jeb/info.html. It contains instances of the JSSP, 1D- and 2D-Packing, Knapsack, periodic TSPs, amongst others.

Plateau The Plateau problem is a benchmark problem for bit-string based black-box search[4,340]. It is structured exactly like Jump, but instead of being deceptive, the last ω bits to flip around the optimum are neutral. The objective function $f(x)$ of the minimization variant is $n - \|x\|_1$ if $\|x\|_1 = n$ or $\|x\|_1 \leq n - \omega$ hold, and ω otherwise, where $\|x\|_1 = \sum_{i=1}^{n} x_i$. This means that, the larger ω, the larger the neutral plane around the optimum that an algorithm needs to cross. An $(1+1)$ EA has an expected runtime of $\Theta(n^\omega)$ on plateaus[4]. The average runtime of the $(1+1)$ FEA on Plateaus is slower by a factor seemingly proportional in n.

PostgreSQL An open source object-relational DBMS[107,230,246,291]. See https://postgresql.org for more information.

psql is the client program used to access the PostgreSQL DBMS server.

pytest is a framework for writing and executing unit tests in Python[81,176,232,236,331]. Learn more at https://pytest.org.

Python The Python programming language[149,186,204,315], i.e., what you will learn about in our book[315]. Learn more at https://python.org.

Glossary (in English) XIII

QAP The *Quadratic Assignment Problem* is an optimization problem where the goal is to assign a set of n facilities to a set of n locations[44,59,173,198]. Such an assignment can be represented as a permutation x of the first n natural numbers, where x_i specifies the location where facility i should be placed. For each QAP instance, a distance matrix A is given, where $A[a, b]$ specifies the distance from location a to location b, as well as a flow matrix B, where $B[i, j]$ is the amount of material flowing (e.g., daily) from facility i to facility j. The objective function $f(x)$ then rates a permutation x as $f(x) = \sum_{i=1}^{n} \sum_{j=1}^{n} A[x_i, x_j] * B[i, j]$. The QAP is $\mathcal{NP}$-complete[94,263] and even among this problem class, is considered as particularly hard for metaheuristics[24,44,198,283]. The QAP has applications such as building layout[70,101,174], keyboard layout[46], circuit design[102], wiring[277], and scheduling[275]. Typical benchmark instances can be found in QAPLIB. SOTA algorithms for the QAP include[24,114,213,219,288,289].

QAPLIB is a library of 136 benchmark instances for the Quadratic Assignment Problem (QAP)[43,45], 28 of which have not yet been solved to optimality[3] (at least by 2023).

Random Keys encoding is a decoding procedure $\gamma : \mathbb{X} \mapsto \mathbb{Y}$ that maps a continuous search space $\mathbb{X} \subset [0, 1]^n$ which is a subset of the n-dimensional real numbers to the space $\mathbb{Y}$ of permutations of the first n natural numbers[19,20]. For example, on $n = 5$, the vector $x = (0.49, 0.91, 0.33, 0.75, 0.51)$ is translated to the permutation $y = (3, 1, 5, 4, 2)$, because $x_3 = 0.33$ has the smallest value, putting 3 at the beginning, $x_1 = 0.49$ the second-smallest, placing 1 second, ..., and $x_2 = 0.91$ being the largest value meaning that the last number in y will be 2[20]. The advantage of the encoding is that we can search in $\mathbb{X}$ applying arbitrary mathematical operators, crossover, and mutation procedures, but will always get correct permutations. The disadvantage is that $|\mathbb{X}| \gg |\mathbb{Y}|$ and that there actually is no meaning underpinning moves in $\mathbb{X}$ in relation to their projected result in $\mathbb{Y}$.

relational database A relational DB is a database that organizes data into rows (tuples, records) and columns (attributes), which collectively form tables (relations) where the data points are related to each other[64,136,138,274,290,311,327].

RLS Randomized local search retains the best solution x_c discovered so far during the search and, in each step, it applies a unary search operator to this best-so-far solution x_c and derives a new solution x_n. If the new solution x_n is *better or equally good* when compared with x_c, i.e., not worse, then it replaces it, i.e., is stored as the new x_c. If the search space are bit strings of length n, then RLS uses a unary search operator that flips exactly one bit. This operator is the main difference to $(1 + 1)$ EA.

Glossary (in English) XIV

FRLS The RLS with FFA plugged in [340]. On the discrete bit-string domains, this algorithm has a very specific structure: Its unary search operator flips exactly one bit, distinguishing it from the unary operator in the $(1+1)$ FEA, which flips each bit with probability $1/n$. On other domains, such as permutations, we use the term synonymously with $(1+1)$ FEA.

RTS *Reactive Tabu Search* is a variant of TS where the length of the tabu list is adapted to the problem and to the current evolution of the search, and an escape strategy for diversifying the search is used when that is not sufficient[15–18]. It has successfully been applied to a wide range of problems, from the QAP[16,17], to Knapsack problems[15,17], the N-K landscape[15] to the training of neural networks[18].

SA *Simulated Annealing* is a SLS that sometimes accepts a worse solution[53,156,166,243,312]. The probability to do so decreases over time and with the difference in objective values, i.e.,is the lower the worse the new solution is. SA performed well on a variety of problems, including the QAP[217,298] and the JSSP[238].

SATLIB is a library of benchmark instances for the Maximum Satisfiability (MaxSAT) available at https://www.cs.ubc.ca/~hoos/SATLIB/index-ubc.html[147,148].

Self-Adjusting $(1+(\lambda,\lambda))$ GA The Self-Adjusting $(1+(\lambda,\lambda))$ GA is a theory-inspired EA for bit string-based search spaces[51]. It has linear runtime on the OneMax problem[86].

server In a client-server architecture, the server is a process that fulfills the requests of the clients. It usually waits for incoming communication carring the requests from the clients. For each request, it takes the necessary actions, performs the required computations, and then sends a response with the result of the request. Typical examples of servers are web servers[47] in the internet as well as DBMSes. It is also common to refer to the computer running the server processes as server as well, i.e., to call it the „server computer“[178].

SGA The Standard Genetic Algorithm[12,76,126,214,220,312] was the first EA. It maintains a population of solutions and applies mutation and crossover to generate offspring solutions. It uses fitness proportionate selection to choose which solutions should „survive“ into the next generation, which today is considered a very bad design choice, see, e.g., [328].

significand The significand is the part of a floating point number that stores the digits of the number (in binary representation). In the 64 bit double precision IEEE Standard 754 floating point number layout[145,150], the exponent is 52 bits.

Glossary (in English) XV



SLS *Stochastic local search* is a local search (LS), one of the simplest metaheuristic search paradigms[146]: A single „current" candidate solution is maintained and new solutions are *stochastically* sampled from its neighborhood, i.e., in a way that involves random numbers. If these solutions are deemed interesting based on their objective value, they replace the „current". The cycle continues. Basic examples are RLS and $(1+1)$ EA. More complex examples that add mechanisms to escape local optima on top of the SLS paradigm are TS and SA.

SOTA State-of-the-Art, i.e., the best-performing methods for a field or problem type

SQL The *Structured Query Language* is basically a programming language for querying and manipulating relational databases[55,72,74,75,151,211,276,278,279,290]. It is understood by many DBMSes. You find the Structured Query Language (SQL) commands supported by PostgreSQL in the reference[276].

STSP A *symmetric Traveling Salesperson Problem* is a special case of the TSP, where the $n \times n$ distance matrix A is symmetric, i.e., $A[a,b] = A[b,a] \forall a,b \in 1..n$.

terminal A terminal is a text-based window where you can enter commands and execute them[13,63]. Knowing what a terminal is and how to use it is very essential in any programming- or system administration-related task. If you want to open a terminal under Microsoft Windows, you can Druck auf ⊞+R, dann Schreiben von `cmd`, dann Druck auf ↵. Under Ubuntu Linux, Ctrl+Alt+T opens a terminal, which then runs a Bash shell inside.

transcendental A *transcendental* number is a number that is not the root of any integer polynomial, meaning that it is not an algebraic number of any degree[325]. Every real transcendental number must also be irrational, since a rational number is, by definition, an algebraic number of degree one[325]. Examples of transcendental numbers are π and e.

Trap The Trap problem is a benchmark problem for bit-string based black-box search[95,226,340]. It is structured exactly like OneMax, however the worst and the best solution are swapped. This makes the problem highly deceptive, as following the „gradient" in the search space leads the search away from the global optimum. For bit strings of length n, the objective function $f(x)$ of the minimization variant is 0 if $\|x\|_1 = 0$ and $n - \|x\|_1 + 1$ otherwise, where $\|x\|_1 = \sum_{i=1}^{n} x_i$. An $(1+1)$ EA will converge to the resulting local optimum and thus have an expected runtime of $\Theta(n^n)$[95]. In experiments, the $(1+1)$ FEA achieves polynomial runtime on Trap, seemingly in $\mathcal{O}(n^2 \ln n)$[320,321,340].

Glossary (in English) XVI



TS *Tabu Search* is one of the most successful SLSes[121,122,124,158]. In order to avoid returning to previous solutions, it declares certain solutions or search moves as „tabu.“ For example, if a bit in a discrete problem has been flipped by the search operator, it may be forbidden to flip the bit again for some time, thus preventing the algorithm from getting trapped in a cycle. This rule may be overridden if a new best-so-far solution is discovered that outperforms all previous solutions, in which case it would be accepted (this is the so-called aspiration criterion). TS has performed well on the QAP[16,17,216,219,287,288] and on the JSSP[228,240,242,270,341].

TSP In an instance of the *Traveling Salesperson Problem*, also known as *Traveling Salesman Problem*, a set of n cities or locations as well as the distances between them are given[5,135,184,189,316,322]. The goal is to find the shortest round-trip tour that starts at one city, visits all the other cities one time each, and returns to the origin. The TSP is one of the most well-known $\mathcal{NP}$-hard combinatorial optimization problems[119,135,177]. The TSP has many applications[5], including in transportation, logistics, (circuit) plotting and drilling problems[134], telescope aiming, and genome sequencing. Solutions are often encoded as permutations x of the first n natural numbers, where each city gets a number and the order of numbers in the permutation corresponds to the order of cities in the tour. The distances between the n cities are usually given in a $n \times n$ distance matrix A, where $A[a, b]$ is the distance from city a to city b with $a, b \in 1..n$. Then, the objective function subject to minimization is $f(x) = A[x_n, x_{\mathbf{1}}] + \sum_{i=\mathbf{1}}^{n-\mathbf{1}} A[x_i, x_{i+\mathbf{1}}]$. Typical benchmark instances can be found in TSPLib[256,257], the 8th DIMACS challenge[159–161], and the *World TSP* repository[68].

TSPLib is a library of benchmark instances for the Traveling Salesperson Problem (TSP)[256,257]. Its original URL was available until 2025 and then became defunct. The repository https://github.com/shredderzwj/TSPLIB contains the data. Additionally, you may also check https://github.com/thomasWeise/tspSuite, which contains all the TSPLib instances as well as Java code to load them. The TSPLib contains 110 instances of the symmetric Traveling Salesperson Problems (STSPs) with scales n ranging from 14 to 85 900, but 93 instances have less than 2000 nodes. There also are 19 asymmetric Traveling Salesperson Problems (ATSPs) with scales $n \in 17..443$.

Glossary (in English) XVII



TTP The *Traveling Tournament Problem* is the combinatorial optimization problem of both efficiently and fairly organizing a tournament of n teams that play against each other in a pairwise fashion[100,333]. The efficient part boils down to arranging the games such that the total travel length is short, which is somewhat similar to the classical TSP. Initially, each team is at its home location. On each day, a team needs to travel if its scheduled game is not at its present location. On the last day, each team may need to travel back home unless their last game is a home game. The total travel length sums up the lengths of all travels over all teams. The fair part is represented in several constraints, such as doubleRoundRobin, compactness, maxStreak, and noRepeat. The TTP is $\mathcal{NP}$-hard[309].

TwoMax The TwoMax problem is a benchmark problem for bit-string based black-box search[117,304,340]. It has a global and a local optimum, both of about the same size, at opposite ends of the search space. For bit strings of length n, the objective function $f(x)$ of the minimization variant is 0 if $\|x\|_1 = n$ and $1 + n - \max\{\|x\|_1, n - \|x\|_1\}$ otherwise, where $\|x\|_1 = \sum_{i=1}^{n} x_i$. An $(1+1)$ EA has a 50/50 chance to converge to the local or global optimum and thus has an expected runtime in $\Theta(n^n)$[116,117]. In experiments, the $(1+1)$ FEA achieves polynomial runtime seemingly in $\mathcal{O}(n^2 \ln n)$ on TwoMax[320,321,340].

type hint are annotations that help programmers and static code analysis tools such as Mypy to better understand what type a variable or function parameter is supposed to be[181,305]. Python is a dynamically typed programming language where you do not need to specify the type of, e.g., a variable. This creates problems for code analysis, both automated as well as manual: For example, it may not always be clear whether a variable or function parameter should be an integer or floating point number. The annotations allow us to explicitly state which type is expected. They are *ignored* during the program execution. They are a basically a piece of documentation.

Ubuntu is a variant of the open source operating system Linux[63,140]. We recommend that you use this operating system to follow this class, for software development, and for research. Learn more at https://ubuntu.com. If you are in China, you can download it from https://mirrors.ustc.edu.cn/ubuntu-releases.

UCS Universal Coded Character Set, see Unicode

Glossary (in English) XVIII



Unicode A standard for assigning characters to numbers[152,294,302]. The Unicode standard supports basically all characters from all languages that are currently in use, as well as many special symbols. It is the predominantly used way to represent characters in computers and is regularly updated and improved.

unit test Software development is centered around creating the program code of an application, library, or otherwise useful system. A *unit test* is an *additional* code fragment that is not part of that productive code. It exists to execute (a part of) the productive code in a certain scenario (e.g., with specific parameters), to observe the behavior of that code, and to compare whether this behavior meets the specification[23,231,233,236,261,296]. If not, the unit test fails. The use of unit tests is at least threefold: First, they help us to detect errors in the code. Second, program code is usually not developed only once and, from then on, used without change indefinitely. Instead, programs are often updated, improved, extended, and maintained over a long time. Unit tests can help us to detect whether such changes in the program code, maybe after years, violate the specification or, maybe, cause another, depending, module of the program to violate its specification. Third, they are part of the documentation or even specification of a program.

URI A *Uniform Resource Identifier* is an identifier for an abstract or physical resource in the internet[337]. It can be a URL, a name, or both. URIs are supersets of URLs. The connection strings of the PostgreSQL DBMS are examples of Uniform Resource Identifiers (URIs).

URL A *Uniform Resource Locator* identifies a resource in the WWW and a way to obtain it by describing a network access mechanism. The most notable example of URLs is the text you write into web browsers to visit websites[29]. URLs are subsets of URIs.

UTF-8 The *UCS Transformation Format 8* is one standard for encoding Unicode characters into a binary format that can be stored in files[152,337]. It is the world wide web's most commonly used character encoding, where each character is represented by one to four bytes. It is backwards compatible with ASCII.

VCS A *Version Control System* is a software which allows you to manage and preserve the historical development of your program code[301]. A distributed VCS allows multiple users to work on the same code and upload their changes to the server, which then preserves the change history. The most popular distributed VCS is Git.

VND *Variable Neighborhood Descend*; see VNS. VND in combination with GRASP has been applied to the 2BP|O|F in [239].

Glossary (in English) XIX



VNS *Variable Neighborhood Search* is a metaheuristic framework combining a systematic change of neighborhood with LS[137,221]. A *neighborhood* structure in LS defines which solutions can be reached from a given solution within one search step. Usually, one uses a single unary operator and, hence, has a single neighborhood structure. However, for many problems, we can easily design many operators. For permutations, for example, one operator may swap two elements, another one may reverse a section of the permutation, and yet another one, while yet another one may rotate a subsequence one step to the left or right[316]. In the VNS framework, a multiple neighborhoods are investigated in a systematic fashion. Assume that we have a list of neighborhood structures. Variable Neighborhood Descend (VND) works in a deterministic fashion. It finds the best solution that can be reached in one step from the current solution under the current neighborhood. If this is better than the current solution, we accept it and continue with the first neighborhood. Otherwise, we continue with the next neighborhood in the list. In Basic VNS, instead of picking the best move from the current neighborhood, the first improving move is used. Also, at the beginning of each cycle, a random point from the current neighborhood around the current solution is chosen as starting point.

WWW World Wide Web[28,78]

$i!$ The factorial $a!$ of a natural number $a \in \mathbb{N}_1$ is the product of all positive natural numbers less than or equal to a, i.e., $a! = 1 * 2 * 3 * 4 * \cdots * (a-1) * a$[64,97,203].

ϕ The golden ratio (or golden section) ϕ is the irrational number $\frac{1+\sqrt{5}}{2}$. It is the ratio of a line segment cut into two pieces of different lengths such that the ratio of the whole segment to that of the longer segment is equal to the ratio of the longer segment to the shorter segment[49,103]. The golden ratio is approximately $\phi \approx 1.618\,033\,988\,749\,894\,848\,204\,586\,834$[273]. Represented as `float` in Python, its value is `1.618033988749895`.

π is the ratio of the circumference U of a circle and its diameter d, i.e., $\pi = U/d$. $\pi \in \mathbb{R}$ is an irrational and transcendental number[110,163,227], which is approximately $\pi \approx 3.141\,592\,653\,589\,793\,238\,462\,643$. An early method to approximate π was developed by the Chinese mathematician LIU Hui (刘徽) based on inscribing regular e-gons into a circle somewhere in the third century CE[229,338] in his commentary to the famous Chinese mathematics book *Jiu Zhang Suanshu* (九章算术)[71,73,165,229,281]. In Python, π is provided by the `math` module as variable `pi` with value `3.141592653589793`. In PostgreSQL, it is provided by the SQL function `pi()` with value `3.141592653589793`[210].

Glossary (in English) XX

$i..j$ with $i, j \in \mathbb{Z}$ and $i \leq j$ is the set that contains all integer numbers in the inclusive range from i to j. It always holds that $i..j \subset \mathbb{Z}$. For example, 5..9 is equivalent to $\{5, 6, 7, 8, 9\}$

A the $n \times n$ distance matrix. $A[a, b]$ indicates the distance between locations a and b with $a, b \in 1..n$. A distance matrix can be symmetric, in which case $A[a, b] = A[b, a]\ \forall a, b \in 1..n$. Distance matrices represent the problem instance data in TSPs and part of the instance data of QAPs.

B the $n \times n$ flow matrix. $B[a, b]$ is the flow of goods between locations a and b with $a, b \in 1..n$. Flow matrices are one half of the problem instance data of QAPs.

e is Euler's number[105], the base of the natural logarithm. $e \in \mathbb{R}$ is an irrational and transcendental number[110,163], which is approximately $e \approx 2.718\,281\,828\,459\,045\,235\,360$. In Python, it is provided by the `math` module as variable `e` with value `2.718281828459045`. In PostgreSQL, you can obtain it via the SQL function `exp(1)` as value `2.718281828459045`[210].

$f(y)$ The *objective function* $f : \mathbb{Y} \mapsto \mathbb{R}$ can compute a cost or rating for each candidate solution y in the solution space $\mathbb{Y}$. The smaller this value, the better the solution, i.e., in the context of this work, objective functions are subject to minimization. Objective functions are not necessarily continuous and it is also quite common that they have integer or natural numbers as results, e.g., we often have $f : \mathbb{Y} \mapsto \mathbb{N}_0$ instead of $f : \mathbb{Y} \mapsto \mathbb{R}$.

$\gamma(x)$ The decoding function $\gamma : \mathbb{X} \mapsto \mathbb{Y}$ translates point from the search space $\mathbb{X}$ to one point y in the solution space $\mathbb{Y}$. It is a very common case that $\mathbb{X} = \mathbb{Y}$ and that the decoding function γ is the identity mapping. Examples for encodings that translate from a simple search space $\mathbb{X}$ to a complex solution data structure $\mathbb{Y}$ are the IBL algorithm that translates (potentially signed) permutations to two-dimensional packings on the 2BP, the permutation-based encoding of Gantt charts for the JSSP from[32,33,120,268,317], or the permutation-based encoding of game plans for the Traveling Tournament Problem (TTP) from[333]. Encodings can also translate between types of search spaces. The Random Keys encoding, for example, translates from $[0, 1]^n$ to permutations of the first n natural numbers..

Glossary (in English) XXI

k the number of jobs in an instance of the JSSP.

m the number of machines in an instance of the JSSP.

$\mathrm{mean}(A)$ The *arithmetic mean* $\mathrm{mean}(A)$ is an estimate of the expected value of a distribution from which a data sample A was, well, sampled. Its is computed on data sample $A = (a_0, a_1, \ldots, a_{n-1})$ as the sum of all n elements a_i in the sample data A divided by the total number n of values, i.e., $\mathrm{mean}(A) = \frac{1}{n}\sum_{i=0}^{n-1} a_i$. In Python, you can use function `mean` from the module `statistics` to compute the arithmetic mean.

n We usually use the value $n \in \mathbb{N}_0$ to denote the *scale*, i.e., the size of the problem to solve. This notion is very general in computer science and not limited to optimization. For example, if we sort a list or search in a list, the scale n is the length of the list, i.e., the number of elements in it. In a TSP, this may be the number of cities. In a discrete benchmark problem such as OneMax or Trap, this may be the number of bits over which the problem is defined. In a QAP, this may be the number of facilities and locations. In a MaxSAT problem, this may be the number of variables that are input to the Boolean formula that we want to become true. There are also problems where the scale is not defined straightforwardly like this. For example, in a JSSP, we have a number of machines and a number of jobs, which together define the size of the problem. But even then, the scale n relates to the size of the solution space $\mathbb{Y}$, and, hence, also of the search space $\mathbb{X}$, which usually grow exponentially with n. And this is already the main issue: The larger n, the (exponentially) larger the number of possible solutions and thus, the larger the runtime of algorithms. This is known as the „curse of dimensionality“[26,58]. Scales n are also often used in conjunction with the $\mathcal{O}$-notation to denote algorithm complexity. For example, comparison-based sorting a list of n items takes $\Theta(n \log n)$ steps[172] (because they need $\lceil \log_2 n! \rceil$ comparisons in the worst case).

$\mathbb{N}_0$ the set of the natural numbers *including* 0, i.e., 0, 1, 2, 3, and so on. It holds that $\mathbb{N}_0 \subset \mathbb{Z}$ and that $\mathbb{N}_0 = \mathbb{N}_1 \cup \{0\}$.

$\mathbb{N}_1$ the set of the natural numbers *excluding* 0, i.e., 1, 2, 3, 4, and so on. It holds that $\mathbb{N}_1 \subset \mathbb{Z}$ and that $\mathbb{N}_1 = \mathbb{N}_0 \setminus \{0\}$.

$\mathcal{NP}$ is the class of computational problems that can be solved in polynomial time by a non-deterministic machine and can be verified in polynomial time by a deterministic machine (such as a normal computer)[131].

Glossary (in English) XXII



$\mathcal{NP}$-complete A decision problem is $\mathcal{NP}$-complete if it is in $\mathcal{NP}$ and all problems in $\mathcal{NP}$ are reducible to it in polynomial time[131,251]. A problem is $\mathcal{NP}$-complete if it is $\mathcal{NP}$-hard and if it is in $\mathcal{NP}$.

$\mathcal{NP}$-hard A problem is $\mathcal{NP}$-hard if all problems in $\mathcal{NP}$ are reducible to it in polynomial time[131]. Algorithms that guarantee to find the correct solutions of $\mathcal{NP}$-hard problems[57,65,183] have a worst-case runtime that grows exponentially with the problem scale.

$\mathbf{\Omega}(g(x))$ If $f(x) = \mathbf{\Omega}(g(x))$, then there exist positive numbers $x_\mathbf{0} \in \mathbb{R}^+$ and $c \in \mathbb{R}^+$ such that $f(x) \geq c * g(x) \geq 0$ for all $x \geq x_\mathbf{0}$[69,169,170]. In other words, $\mathbf{\Omega}(g(x))$ describes a lower bound for function growth.

$\mathcal{O}(g(x))$ If $f(x) = \mathcal{O}(g(x))$, then there exist positive numbers $x_\mathbf{0} \in \mathbb{R}^+$ and $c \in \mathbb{R}^+$ such that $0 \leq f(x) \leq c * g(x)$ for all $x \geq x_\mathbf{0}$[11,69,169,170,180]. In other words, $\mathcal{O}(g(x))$ describes an upper bound for function growth.

$\mathbf{\Theta}(g(x))$ If $f(x) = \mathbf{\Theta}(g(x))$, then $f(x) = \mathcal{O}(g(x))$ and $f(x) = \mathbf{\Omega}(g(x))$[169,170], meaning that there exist positive constants $c_\mathbf{1}, c_\mathbf{2}, x_\mathbf{0} \in \mathbb{R}^+$ such that $0 \leq c_\mathbf{1} g(x) \leq f(x) \leq c_\mathbf{2} g(x)$ for all $x \geq x_\mathbf{0}$[69]. In other words, $\mathbf{\Theta}(g(x))$ describes an exact order of function growth.

$\mathbf{o}(g(x))$ If $f(x) = \mathbf{o}(g(x))$, then there exist positive numbers $x_\mathbf{0} \in \mathbb{R}^+$ such that $0 \leq f(x) \leq c * g(x)$ for all $x \geq x_\mathbf{0}$ *all* $c \in \mathbb{R}^+$[69]. Different from $\mathcal{O}(g(x))$, $\mathbf{o}(g(x))$ holds for *all* constants $c > 0$, whereas $\mathcal{O}(g(x))$ implies the existence of *some* constant $c > 0$. This basically means that $f(x)$ becomes insignificant compared to $g(x)$ as $x_\mathbf{0}$ grows toward infinity. It is often written that $\lim_{x_\mathbf{0} \to \infty} \frac{f(x)}{g(x)} = 0$. In other words, $\mathbf{o}(g(x))$ describes an upper bound for function growth that is not asymptotically tight, i.e., an upper bound that grows faster than $f(x)$.

$\mathbb{Q}$ the set of the rational numbers, i.e., the set of all numbers that can be the result of $\frac{a}{b}$ with $a, b \in \mathbb{Z}$ and $b \neq 0$. a is called the numerator and b is called the denominator. It holds that $\mathbb{Z} \subset \mathbb{Q}$ and $\mathbb{Q} \subset \mathbb{R}$.

$\mathbb{R}$ the set of the real numbers.

$\mathbb{R}^+$ the set of the positive real numbers, i.e., $\mathbb{R}^+ = \{x \in \mathbb{R} : x > 0\}$.

Glossary (in English) XXIII

$\mathrm{var}(A)$ The *variance* of a distribution is the expectation of the squared deviation of the underlying random variable from its mean. The variance $\mathrm{var}(A)$ of a data sample $A = (a_0, a_1, \ldots, a_{n-1})$ with n observations can be estimated as $\mathrm{var}(A) = \frac{1}{n-1}\sum_{i=0}^{n-1}(a_i - \mathrm{mean}(A))^2$. It can also be computed using $\mathrm{var}(A) = \left(\sum_{i=0}^{n-1} a_i^2\right) - \frac{1}{n}\left(\sum_{i=0}^{n-1} a_i\right)^2$, which affords computing it in a single pass, but this method may lead to disastrous rounding errors[56]. A better choice is Welford's algorithm[171,326]: It begins by setting its running mean $m \leftarrow 0$, running variance $v \leftarrow 0$, and its running counter $i \leftarrow 0$. It then iterates over A with the current value being a_i. In each step, it sets $delta \leftarrow a_i - m$, then updates $m \leftarrow m + \delta/(i+1)$, $v \leftarrow v + \delta * (a_i - m)$, and finally $i \leftarrow i + 1$. Once all the elements of A have been processed, the sample variance is $\frac{1}{n-1}v$.

$\omega(g(x))$ If $f(x) = \omega(g(x))$, then for every positive constant $c > 0$, there exists a number $x_0 \in \mathbb{R}^+$ such that $0 \leq c * g(x) < f(x)$ for all $x \geq x_0$[69]. In other words, $f(x) = \omega(g(x))$ if and only if $g(x) = o(f(x))$. In other words, ω describes a lower bound that is not asymptotically tight.

$\mathbb{X}$ The search space $\mathbb{X}$ is the space through which an optimization algorithm navigates in order to find good solutions. The search operators of metaheuristics are used to sample points $x \in \mathbb{X}$, for example. On many benchmark problems, the objective functions f directly evaluate the elements $x \in \mathbb{X}$, i.e., are defined as $f : \mathbb{X} \mapsto \mathbb{R}$. However, very often these elements x are not the same data structures that the user is interested in and/or that the objective functions evaluate: It is common that the search space $\mathbb{X}$ is a simple and plain data structure that can be tackled with standard operations or mathematical tools, such as real vectors $\mathbb{R}^n$, permutations, or bit strings, whereas the solution space $\mathbb{Y}$ can be something more complicated. Then, the user only cares about the solutions $y \in \mathbb{Y}$ and not about the search space $\mathbb{X}$. A decoding $\gamma : \mathbb{X} \mapsto \mathbb{Y}$ is employed to translate the $x \in \mathbb{X}$ to $y = \gamma(x)$ with $y \in \mathbb{Y}$.

x a point in the search space $\mathbb{X}$.

$\mathbb{Y}$ the set of candidate solutions of an optimization problem is called the *solution space* $\mathbb{Y}$. The solution space corresponds to the data structure holding the candidate solutions y. These are the elements that are passed to the user after the optimization process is completed. They are also the parameters of the objective function(s) $f : \mathbb{Y} \mapsto \mathbb{R}$.

Glossary (in English) XXIV



y a candidate solution $y \in \mathbb{Y}$ is one possible solution to an optimization problem. After the optimization process is completed, one or multiple such solutions are passed to the user.

$\mathbb{Z}$ the set of the integers numbers including positive and negative numbers and 0, i.e., ..., -3, -2, -1, 0, 1, 2, 3, ..., and so on. It holds that $\mathbb{Z} \subset \mathbb{R}$.